

Asset Pricing with Discrete-Time Affine processes

Jean-Paul Renne, Alain Monfort, Fulvio Pegoraro, and Guillaume Roussellet

2026-07-09

Contents

About	7
1 Affine processes	9
1.1 A simple Gaussian AR(1) generates nonlinear transforms such as squared states	9
1.2 Standard ARG and ARG0 processes	10
1.3 The jump-like dynamics generated by a compound Poisson process	11
1.4 Recursive Gaussian-ARG model with crisis and GDP-growth components	12
1.5 GDP forecasts under different crisis states	13
1.6 Laplace transforms can recover the full conditional distribution of future GDP growth	15
2 No-arbitrage pricing and risk-neutral dynamics	19
3 Term structure of interest rates: definitions, instruments, and yield-curve conventions	21
3.1 Several standard yield objects can be derived from a single discount curve	21
3.2 Clean and dirty prices	24
3.3 Treasury issuance and outstanding debt by maturity	26
4 Stylized facts and a first simple model	31
4.1 A few observed US yield curves with different shapes	31
4.2 A small number of principal components summarize most yield-curve variation	32
4.3 The level and slope of the yield curve over time	33
4.4 Overlays yields of several maturities to visualize their joint time-series behavior	34
4.5 The distributions of yield levels and monthly yield changes at short and long maturities	36
4.6 How average yields and yield volatilities vary with maturity . . .	37

4.7	The behavior of US and euro-area yield curves around the zero lower bound period	39
4.8	Time-varying yield volatility across maturities	41
4.9	The covariance and correlation structure of the yield curve	42
4.10	A simple HAC-robust inference exercise for the average term spread	46
4.11	Bond excess returns across holding periods and maturities	46
4.12	The term structure of average excess returns, volatility, and Sharpe ratios	49
4.13	A few principal components also summarize excess-return dynamics	51
4.14	Campbell-Shiller regression coefficients	53
4.15	The unconditional mean and volatility of the short rate	55
4.16	Two widely used term-premium estimates across maturities	55
4.17	Market-based and survey-based expectations for short rates, long rates, and inflation	58
4.18	A simple AR(1) short-rate model can generate average yield curves and term premia	61
4.19	Short-rate simulations and term premia in the AR(1) model	64
4.20	Nominal, real, and breakeven curves across states	65
4.21	Nominal and real yield curves with and without risk premia	69
4.22	Time variation in nominal, real, and inflation risk premia	73
5	Gaussian affine term structure models	79
5.1	Macro-spanning regressions	79
5.2	ACM-style regression estimation	82
5.3	Jensen adjustment in the log-inflation component	89
6	Non-Gaussian affine term structure models	91
6.1	A VARG term-structure model can generate movements	91
6.2	A three-regime Gaussian regime-switching short-rate	96
7	Shadow-rate framework and extensions	101
7.1	Shadow-rate and unconstrained Gaussian term structures in the simple Wu-Xia setting	101
7.2	Shadow-rate bond pricing and Monte Carlo validation	103
7.3	The lower bound changes conditional rate volatility across maturities	105
7.4	The shadow-rate idea in a small New Keynesian model with nominal and real curves	107
7.5	Shadow-rate pricing of defaultable bond spreads	110
7.6	Shadow default intensity and credit spreads in a fiscal-limits setting	113
7.7	Non-Gaussian shadow-rate model with jump-like factors	117
7.8	Shadow rate with ARG stochastic volatility	120
7.9	Affine-transform methods to a simple climate-finance model	124
8	Forward, futures, dividends, commodity pricing, and convenience yields	137

8.1	Persistent seasonal factors can generate recurring convenience-yield patterns	137
8.2	Observed futures term structures at a few selected dates	139
8.3	Gaussian state-space model for commodity futures	140
9	Options	147
9.1	Stock call and put prices in a two-factor affine model with stochastic volatility	147
9.2	The pricing of interest-rate caps and floors in a two-factor VARG term-structure model	152
9.3	Kim-style swaption exercise-boundary approximations in a non-Gaussian affine setting	156
9.4	Kim-style swaption boundary approximations in a Gaussian affine model	162
9.5	Prices inflation caps and floors in a two-factor affine inflation model	168
9.6	Inflation cap prices vary across states and strike levels	173
9.7	The price and yield effect of a deflation floor across states and horizons	175
10	Credit and liquidity risks	179
10.1	Standard framework	179
10.2	Credit ratings	186
10.3	VARG	189
10.4	Top-down approach	195
11	Consumption Capital Asset Pricing Model	203
11.1	Risk aversion and intertemporal substitution in the CCAPM	203
11.2	Asset-pricing implications of risk-aversion and EIS calibrations	205
11.3	Smoothing preferences affect consumption and pricing dynamics	209
11.4	Empirical correlations between key macro-financial variables	211
11.5	Term structures under a CCAPM-based pricing kernel	213
11.6	Decomposes the real term premium in the baseline CCAPM setting	215
11.7	Model-implied yield-curve moments with CCAPM restrictions	217
11.8	Separates trend and cycle contributions to the real term premium	219
12	Recursive preferences	221
12.1	External habits amplify variation in marginal utility	221
12.2	Exponentially smoothed consumption growth enters recursive preferences	223
12.3	The fixed-point resolution behind Epstein-Zin preferences	226
12.4	Epstein-Zin pricing implications vary with preference parameters	228
12.5	The Bansal-Yaron long-run-risk environment and its state variables	229
12.6	Yields, returns, and premia in the Bansal-Yaron model	232
12.7	The Bansal-Shaliastovich specification for bond and currency risk premia	238

13 Preferred-habitat model	243
13.1 The preferred-habitat specification and the maturity-demand function	243
13.2 Factor loadings in the preferred-habitat model	245
13.3 How bond supply shocks affect the yield curve under preferred habitat	246
13.4 The excess-return implications of the preferred-habitat model . .	248
13.5 The preferred-habitat setup to exchange-rate implications	250
13.6 Nominal and real term structures in the preferred-habitat framework	252
13.7 How preferred habitat shapes inflation compensation and term premia	255
13.8 Additional nominal-real yield-curve implications of habitat demand	257
13.9 Preferred-habitat bond demand to stock-market valuation effects	259
14 Filtering	265
14.1 Modified Kalman filter	265
14.2 Kalman filter	269
14.3 Quadratic Kalman Filter	280
14.4 Kitagawa-Hamilton filter	285
15 Estimation	291
15.1 Maximum likelihood estimation with observed variables	291
15.2 Maximum Likelihood with the Kalman filter	301
15.3 Kitagawa-Hamilton	308
15.4 Inversion technique plus regime switching	317
16 Imperfect information and rational expectations	337
16.1 Imperfect information	337
16.2 Rational Expectations	350
16.3 Imperfect information and rational expectations	362

About

This Bookdown project gathers the R code used to reproduce the tables, figures, and numerical illustrations of *Asset Pricing with Discrete-Time Affine Processes*.

The chapters are organized around the themes of the book and illustrate how the theoretical objects introduced in the text can be implemented, simulated, and visualized in practice. Whenever possible, the examples rely on the companion package `DTAM`, which provides the core routines for simulation, affine transforms, filtering, shadow-rate models, credit-risk applications, and related asset-pricing exercises.

The aim of this material is twofold. First, it documents how the main figures and quantitative examples of the book are produced. Second, it provides a practical entry point for readers who want to adapt the code to their own applications.

To run the examples, users should first install the companion package `DTAM`. The package can be installed either from the local source folder or from GitHub, for instance with `devtools::install_github("jrenne/DTAM")`, and then loaded in the usual way with `library(DTAM)`. Most chapters assume that `DTAM` is already available in the R session before the code chunks are executed.

Chapter 1

Affine processes

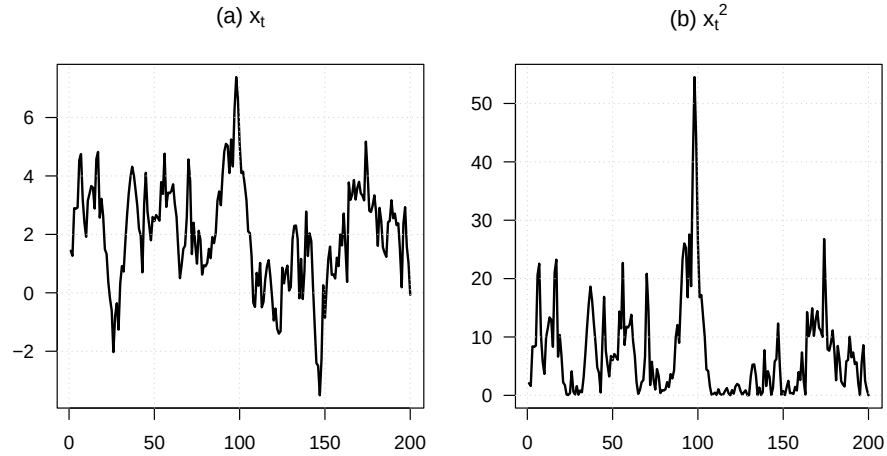
The following code produces a figure showing simulated path of an AR(1) process x_t , and x_t^2 .

1.1 A simple Gaussian AR(1) generates nonlinear transforms such as squared states

This example shows how a simple Gaussian AR(1) generates nonlinear transforms such as squared states.

```
# Purpose: This example shows how a simple Gaussian AR(1) generates nonlinear transforms such as  
# squared states  
set.seed(123)  
  
T <- 200  
  
phi <- .9  
mu <- 2 * (1 - phi)  
sigma <- 1  
  
# Start from the unconditional mean.  
x.t <- mu / (1 - phi)  
x <- NULL  
  
for (t in 1:T) {  
  x.t <- mu + phi * x.t + sigma * rnorm(1)  
  x <- c(x, x.t)  
}  
  
par(mfrow = c(1, 2), plt = c(.15, .95, .1, .85))
```

```
plot(x, type = "l", xlab = "", ylab = "", las = 1,
     main = expression(paste("(a) ", x[t], sep = "")), lwd = 2)
grid()
plot(x^2, type = "l", xlab = "", ylab = "", las = 1,
     main = expression(paste("(b) ", {x[t]}^2, sep = "")), lwd = 2)
grid()
```



The following code produces a figure illustrating ARG processes.

1.2 Standard ARG and ARG0 processes

This example contrasts a standard ARG process with its ARG0 counterpart.

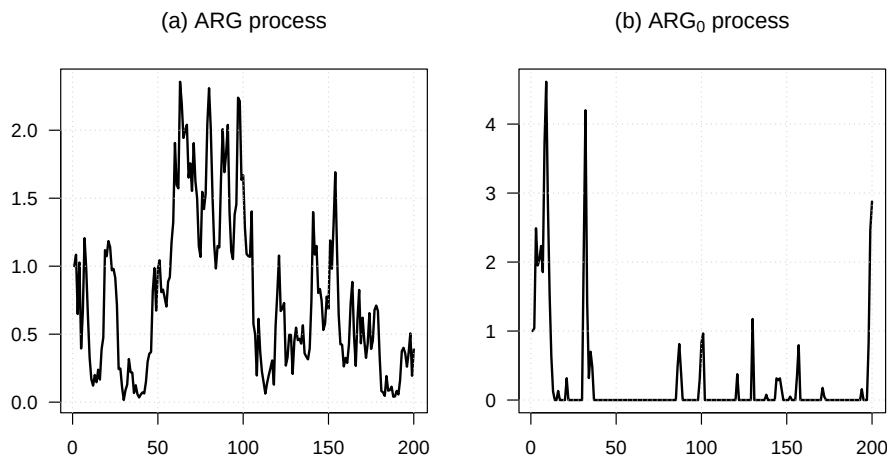
```
# Purpose: This example contrasts a standard ARG process with its ARG0 counterpart
library(DTAM)
set.seed(123)

# Simulate two ARG specifications and compare their paths.
par(mfrow = c(1, 2), plt = c(.15, .95, .1, .85))

W <- simul_ARG(200, mu = .05, nu = 1, rho = .95, alpha = 0)
plot(W, type = "l", xlab = "", ylab = "", las = 1,
     main = expression(paste("(a) ", ARG, " process", sep = "")), lwd = 2)
grid()

W <- simul_ARG(200, mu = .5, nu = 0, rho = .95, alpha = .1)
plot(W, type = "l", xlab = "", ylab = "", las = 1,
     main = expression(paste("(b) ", ARG[0], " process", sep = "")), lwd = 2)
grid()
```

1.3. THE JUMP-LIKE DYNAMICS GENERATED BY A COMPOUND POISSON PROCESS11

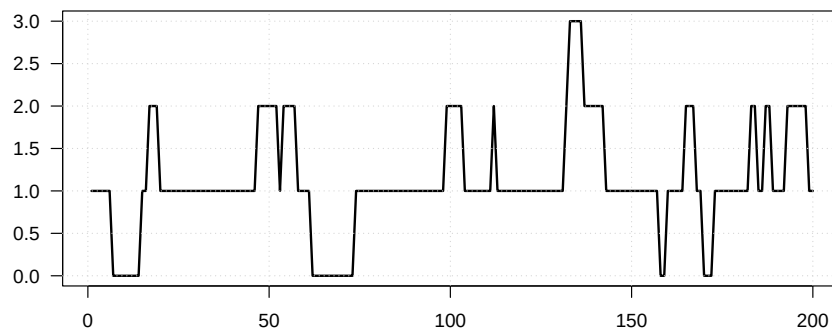


The following code simulates a Poisson process.

1.3 The jump-like dynamics generated by a compound Poisson process

This example illustrates the jump-like dynamics generated by a compound Poisson process.

```
# Purpose: This example illustrates the jump-like dynamics generated by a compound Poisson  
# process  
library(DTAM)  
set.seed(123)  
  
# Simulate and plot a compound Poisson path.  
par(mfrow = c(1, 1), plt = c(.1, .95, .15, .95))  
W <- simul_compound_poisson(200, Gamma = 1, Pi = 0.95, lambda = .1)  
plot(W, type = "l", las = 1, lwd = 2, xlab = "", ylab = "")  
grid()
```



The following code simulate a recursive Gaussian-ARG process.

1.4 Recursive Gaussian-ARG model with crisis and GDP-growth components

This example builds a two-factor recursive Gaussian-ARG system with a crisis component and GDP-growth component.

```
# Purpose: This example builds a two-factor recursive Gaussian-ARG system with a crisis
# component and GDP-growth component
library(DTAM)
set.seed(123)

mu <- .01
beta <- .8 / mu
alpha <- .05
rho <- .6
sigma <- .005
kappa <- .03 * (1 - rho)

# Calibrate the recursive Gaussian-ARG system.
# Save the parameterization for the next examples.
model <- list(
  mu = mu, beta = beta, alpha = alpha, rho = rho, sigma = sigma, kappa = kappa
)

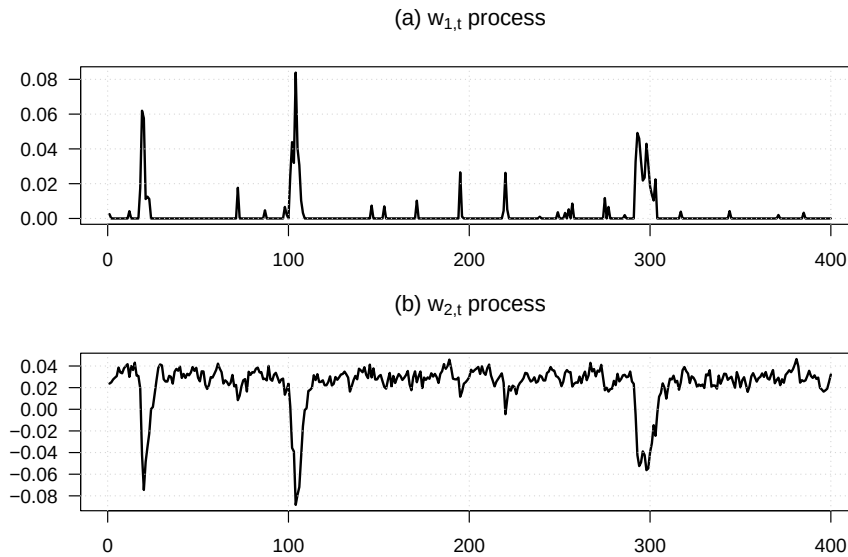
par(mfrow = c(2, 1), plt = c(.1, .95, .15, .7))
nb_periods <- 400

# Simulate and plot the nonnegative crisis factor.
W1 <- simul_ARG(nb_periods, mu = mu, nu = 0, rho = beta * mu, alpha = alpha)
plot(W1, type = "l", xlab = "", ylab = "", las = 1,
     main = expression(paste("(a) ", w[1*','*t], " process", sep = "")), lwd = 2)
grid()

mean_W1 <- alpha * mu / (1 - beta * mu)
mean_W2 <- (-mean_W1 + kappa) / (1 - rho)

# Simulate GDP growth conditional on the crisis factor.
W2 <- mean_W2
for (t in 2:nb_periods) {
  W2 <- c(W2, -W1[t] + kappa + rho * W2[t - 1] + sigma * rnorm(1))
}
plot(W2, type = "l", xlab = "", ylab = "", las = 1,
     main = expression(paste("(b) ", w[2*','*t], " process", sep = "")), lwd = 2)
```

```
grid()
```



The following code uses the previous model to compute forecasts of GDP (GDP growth is $w_{2,t}$ of the previous process, and $w_{1,t}$ is a crisis regime).

1.5 GDP forecasts under different crisis states

This example shows how the previous model can be used to produce horizon-by-horizon GDP forecasts under different crisis states.

```
# Purpose: This example shows how the previous model can be used to produce horizon-by-horizon
# GDP forecasts under different crisis states
psi <- function(U, model) {
  # U is a 2 x K matrix collecting the transform arguments.
  mu <- model$mu
  beta <- model$beta
  alpha <- model$alpha
  rho <- model$rho
  sigma <- model$sigma
  kappa <- model$kappa

  u1 <- U[1, ]
  u2 <- U[2, ]

  a1 <- mu * (u1 - u2) / (1 - mu * (u1 - u2)) * beta
  a2 <- u2 * rho
  b <- mu * (u1 - u2) / (1 - mu * (u1 - u2)) * alpha + u2 * kappa + u2^2 * sigma^2 / 2
```

```

    list(a = rbind(a1, a2), b = b)
}

# Three cases: current crisis only, current + future crises, and no future crises.
U <- matrix(c(0, 1,
              -10000, 1,
              -10000, 0), 2, 3)

# Set the forecast horizon and the two initial states.
H <- 10
X <- matrix(c(0, mean_W2,
              .01, mean_W2), 2, 2)
AB <- reverse_MHLT(psi, U, H = H, psi.parameterization = model)

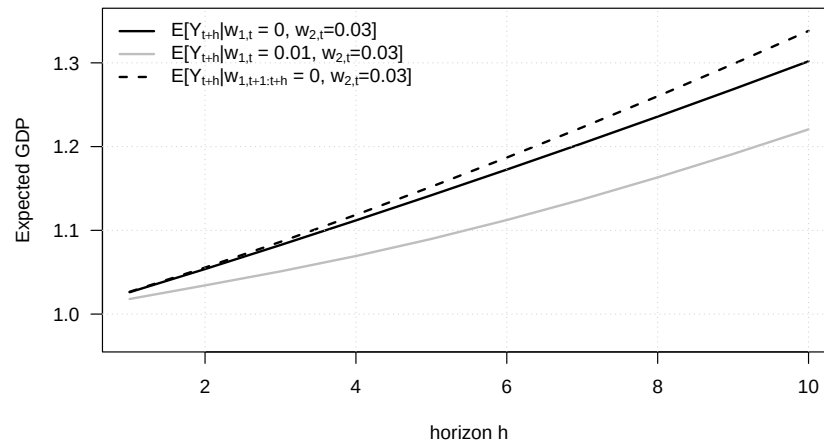
# Recover the three conditional GDP forecasts from the transform coefficients.
GDP_fcst_1 <- c(AB$B[, 1, ]) + t(matrix(AB$A[, 1, ], 2, H)) %*% X[, 1]
GDP_fcst_2 <- c(AB$B[, 1, ]) + t(matrix(AB$A[, 1, ], 2, H)) %*% X[, 2]
GDP_fcst_3 <- c(AB$B[, 2, ] - AB$B[, 3, ]) +
  t(matrix(AB$A[, 2, ] - AB$A[, 3, ], 2, H)) %*% X[, 1]

# Plot the forecast profiles.
par(plt = c(.15, .95, .2, .95))
plot(exp(GDP_fcst_1), type = "l", lwd = 2, las = 1,
     xlab = "horizon h", ylab = "Expected GDP", ylim = c(.97, 1.35))
lines(exp(GDP_fcst_2), col = "grey", lwd = 2)
lines(exp(GDP_fcst_3), lwd = 2, lty = 2)
grid()

legend("topleft",
      legend = c(
        expression(paste(E, "[", Y[t+h], "|",
                          w[1*','*t], " = 0, ",
                          w[2*','*t], "=0.03]", sep = "")),
        expression(paste(E, "[", Y[t+h], "|",
                          w[1*','*t], " = 0.01, ",
                          w[2*','*t], "=0.03]", sep = "")),
        expression(paste(E, "[", Y[t+h], "|",
                          w[1*','*t+1:t+h], " = 0, ",
                          w[2*','*t], "=0.03]", sep = ""))
      ),
      lty = c(1, 1, 2),
      col = c("black", "grey", "black"),
      lwd = c(2, 2, 2), bty = "n")

```

1.6. LAPLACE TRANSFORMS CAN RECOVER THE FULL CONDITIONAL DISTRIBUTION OF FUTURE GDP



The following code displays the conditional pdf of GDP at different horizons.

1.6 Laplace transforms can recover the full conditional distribution of future GDP growth

This example illustrates how Laplace transforms can recover the full conditional distribution of future GDP growth.

Purpose: This example illustrates how Laplace transforms can recover the full conditional distribution of future GDP growth

```
library(DTAM)
```

Here the variable of interest is cumulated GDP growth.

If indic_no_crisis is TRUE, future crisis realizations are shut down.

```
psi_logGDP <- function(u, model) {
```

```
  H <- model$H
```

```
  X <- model$X
```

```
  if (model$indic_no_crisis) {
```

```
    U <- rbind(-10000, 1 * c(u))
```

```
  } else {
```

```
    U <- rbind(0, 1 * c(u))
```

```
  }
```

```
  AB <- reverse_MHLT(psi, U, H = H, psi.parameterization = model)
```

```
  if (model$indic_no_crisis) {
```

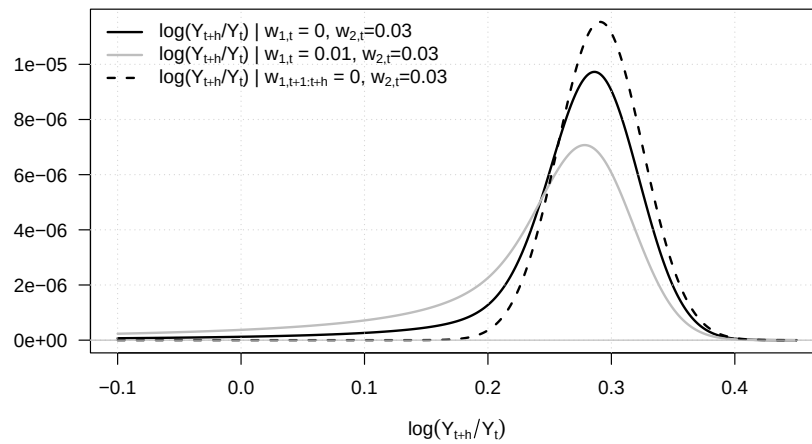
```
    U <- matrix(c(-10000, 0), 2, 1)
```

```
    AB_1w0 <- reverse_MHLT(psi, U, H = H, psi.parameterization = model)
```

```
  }
```


1.6. LAPLACE TRANSFORMS CAN RECOVER THE FULL CONDITIONAL DISTRIBUTION OF FUTURE GD

```
expression(paste("log(", Y[t+h], "/", Y[t], ") | ",
                  w[1*','*t+1:t+h], " = 0, ",
                  w[2*','*t], "=0.03", sep = ""))
),
lty = c(1, 1, 2),
col = c("black", "grey", "black"),
lwd = c(2, 2, 2), bty = "n")
```



Chapter 2

No-arbitrage pricing and risk-neutral dynamics

This chapter is mainly theoretical and does not have dedicated replication code.

Chapter 3

Term structure of interest rates: definitions, instruments, and yield-curve conventions

3.1 Several standard yield objects can be derived from a single discount curve

This example shows how several standard yield objects can be derived from a single discount curve.

Purpose: This example shows how several standard yield objects can be derived from a single discount curve

Nelson-Siegel-Svensson primitives and derived yield curves.

```
nss_yield <- function(h, theta) {  
  b0 <- theta["beta0"]; b1 <- theta["beta1"]; b2 <- theta["beta2"]; b3 <- theta["beta3"]  
  l1 <- theta["lambda1"]; l2 <- theta["lambda2"]  
  
  x1 <- (1 - exp(-l1 * h)) / (l1 * h)  
  x2 <- x1 - exp(-l1 * h)  
  x3 <- (1 - exp(-l2 * h)) / (l2 * h) - exp(-l2 * h)  
  
  b0 + b1 * x1 + b2 * x2 + b3 * x3  
}
```

```

disc <- function(h, theta) exp(-h * nss_yield(h, theta))

# Curves derived from the discount function.
zc_yield <- function(h, theta) nss_yield(h, theta)

# simple forward rate between h and h+dh under continuous compounding
fwd_rate <- function(h, dh, theta) {
  -(log(disc(h + dh, theta)) - log(disc(h, theta))) / dh
}

# Par yield for maturity h with p coupons per year (cashflows at i/p)
par_yield <- function(h, theta, p) {
  n <- round(h * p)           # number of coupon periods
  times <- (1:n) / p
  df <- disc(times, theta)
  dfT <- disc(h, theta)
  p * (1 - dfT) / sum(df)
}

# Swap rate: same as par yield when fixed leg schedule matches
swap_rate <- function(h, theta, p) {
  par_yield(h, theta, p)
}

# YTM of a coupon bond priced off the same discount curve
ytm_from_price <- function(price, c, h, p) {
  n <- round(h * p)
  times <- (1:n) / p

  f <- function(y) {
    dfy <- exp(-y * times)
    sum((c / p) * dfy) + exp(-y * h) - price
  }
  uniroot(f, c(-0.05, 0.30))$root
}

coupon_bond_price <- function(c, h, theta, p) {
  n <- round(h * p)
  times <- (1:n) / p
  sum((c / p) * disc(times, theta)) + disc(h, theta)
}

# Choose parameters and maturity grids.
theta <- c(

```

3.1. SEVERAL STANDARD YIELD OBJECTS CAN BE DERIVED FROM A SINGLE DISCOUNT CURVE23

```
beta0 = 0.03, beta1 = -0.015, beta2 = 0.02, beta3 = -0.005,
lambda1 = 1.2, lambda2 = 3.5
)

p <- 2 # number of payments per period -- 2 = semiannual payments

Hmax <- 20

# Dense grid for ZC + forward (exists for all maturities)
h_dense <- seq(0.25, Hmax, by = 0.25)

# Payment-date grid for par/swap (must have h*p integer)
h_pay <- seq(1/p, Hmax, by = 1/p)

# Compute the implied curves.
y_zc <- sapply(h_dense, zc_yield, theta = theta)
y_fwd <- sapply(h_dense, fwd_rate, dh = 1/12, theta = theta) # 1-month fwd

y_par <- sapply(h_pay, par_yield, theta = theta, p = p)
y_swap <- sapply(h_pay, swap_rate, theta = theta, p = p)

# Coupon-bond yields for two different coupon levels.

mats <- seq(1, Hmax, by=1)
coupons <- rep(.1, length(mats))
ytm_pts <- mapply(function(h, cpn) {
  pr <- coupon_bond_price(cpn, h, theta, p)
  ytm_from_price(pr, cpn, h, p)
}, mats, coupons)

mats2 <- seq(1, Hmax, by=1)
coupons2 <- rep(0.01, length(mats2))
ytm_pts2 <- mapply(function(h, cpn) {
  pr <- coupon_bond_price(cpn, h, theta, p)
  ytm_from_price(pr, cpn, h, p)
}, mats2, coupons2)

# Plot the resulting term-structure objects together.

par(plt=c(.14,.98,.18,.97))

plot(h_dense, 100*y_zc, type = "l", lwd = 2, las=1,
      ylim = 100 * c(.025, .034),
```

```

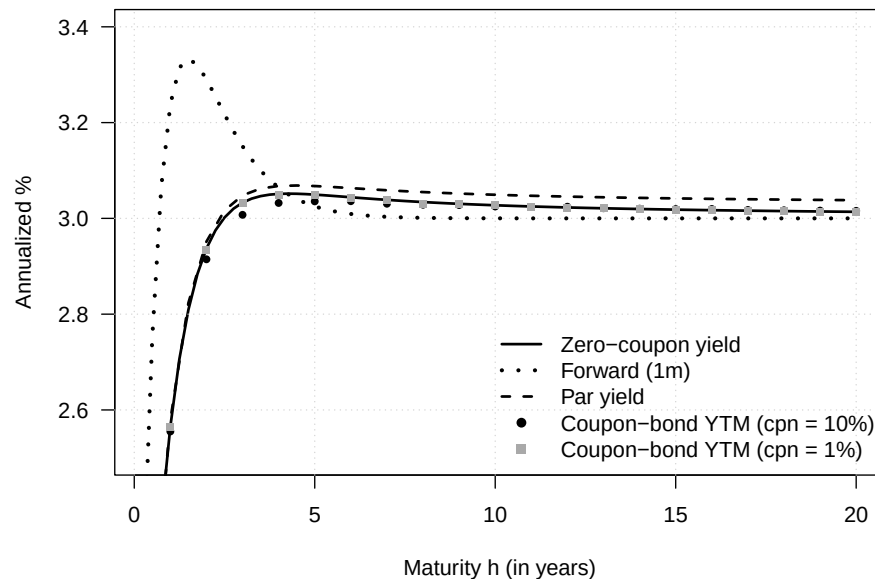
      xlab = "Maturity h (in years)", ylab = "Annualized %",
      main = "Multiple yield curves from one discount curve")
grid()

lines(h_dense, 100*y_fwd, lwd = 3, lty = 3)
lines(h_pay, 100*y_par, lwd = 2, lty = 2)
# lines(h_pay, y_swap, lwd = 2, lty = 4)

points(mats, 100*ytm_pts, pch = 16, cex = 0.8)
points(mats2, 100*ytm_pts2, pch = 15, cex = 0.8, col="darkgrey")

legend("bottomright",
      legend = c("Zero-coupon yield", "Forward (1m)",
                  "Par yield", "Coupon-bond YTM (cpn = 10%)",
                  "Coupon-bond YTM (cpn = 1%)"),
      col=c(rep("black",4),"darkgrey"),
      lty = c(1, 3, 2, NA, NA),
      lwd = c(2, 3, 2, NA, NA),
      pch = c(NA, NA, NA, 16, 15), bty = "n")

```



3.2 Clean and dirty prices

This example illustrates the distinction between clean and dirty prices for a coupon bond.


```

# Purpose: This example illustrates the distinction between clean and dirty prices for a coupon
# bond.

# Bond and discount-curve assumptions.
r_flat <- 0.03
coupon_rate <- 0.04
coupon_freq <- 2
coupon_period <- 1 / coupon_freq
coupon_payment <- coupon_rate / coupon_freq
maturity <- 5

# Monthly settlement dates over the life of the bond.
settlement_grid <- seq(0, maturity - 1 / 12, by = 1 / 12)
coupon_dates <- seq(coupon_period, maturity, by = coupon_period)

dirty_price <- clean_price <- accrued_interest <- numeric(length(settlement_grid))

for (i in seq_along(settlement_grid)) {
  s <- settlement_grid[i]
  remaining_dates <- coupon_dates[coupon_dates > s]
  cash_flows <- rep(coupon_payment, length(remaining_dates))
  cash_flows[remaining_dates == maturity] <- cash_flows[remaining_dates == maturity] + 1

  dirty_price[i] <- sum(cash_flows * exp(-r_flat * (remaining_dates - s)))

  previous_coupon <- ifelse(any(coupon_dates <= s), max(coupon_dates[coupon_dates <= s]), 0)
  next_coupon <- min(coupon_dates[coupon_dates > s])
  accrued_interest[i] <- coupon_payment * (s - previous_coupon) / (next_coupon - previous_coupon)
  clean_price[i] <- dirty_price[i] - accrued_interest[i]
}

# Plot clean and dirty prices.
op <- par(mar = c(4.1, 4.1, 0.8, 0.8), las = 1, cex.axis = 1.05)
ylim <- range(c(dirty_price, clean_price))

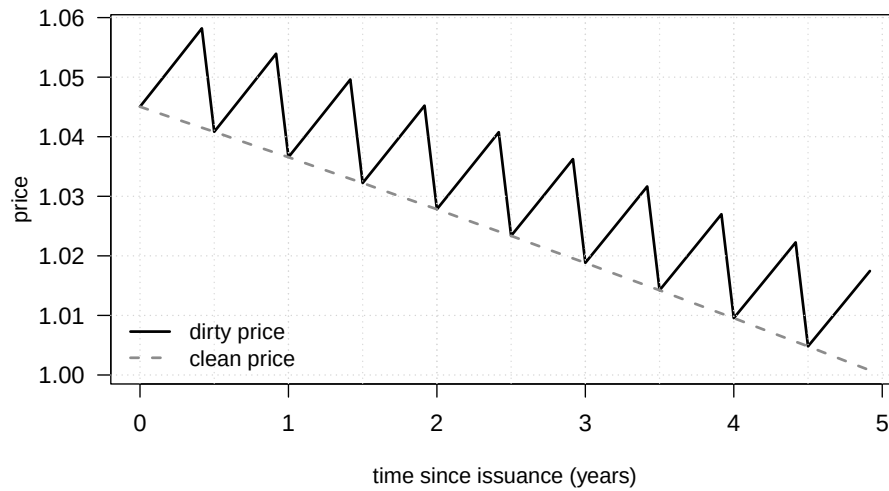
plot(
  settlement_grid, dirty_price,
  type = "l", lwd = 2, col = "black",
  xlab = "time since issuance (years)", ylab = "price",
  ylim = ylim
)
grid()
abline(v = coupon_dates[coupon_dates < maturity], col = "grey85", lty = 3)
lines(settlement_grid, clean_price, col = "grey55", lwd = 2, lty = 2)
legend(

```

```

"bottomleft",
legend = c("dirty price", "clean price"),
col = c("black", "grey55"),
lty = c(1, 2), lwd = 2, bty = "n",
cex = 1
)

```



```
par(op)
```

3.3 Treasury issuance and outstanding debt by maturity

This example documents the maturity composition of Treasury issuance and the stock of debt outstanding.

```

# Purpose: This example documents the maturity composition of Treasury issuance and the
# debt outstanding
library(tidyverse)
library(zoo)
library(patchwork)
library(readxl)

theme_baseR <- function() {
  theme_classic(base_size = 12) +
  theme(
    panel.grid.major.y = element_line(color = "grey60", linewidth = 0.4,
                                       linetype = 3),
    panel.grid.minor.y = element_line(color = "grey60", linewidth = 0.2,

```

3.3. TREASURY ISSUANCE AND OUTSTANDING DEBT BY MATURITY²⁷

```
        linetype = 3),
    panel.grid.major.x = element_line(color = "grey60", linewidth = 0.3,
        linetype = 3),
    panel.grid.minor.x = element_line(color = "grey60", linewidth = 0.3,
        linetype = 3),
    axis.ticks = element_line(linewidth = 0.6),
    panel.grid = element_line(),
    strip.text = element_text(face = "bold", size = 14),
    strip.background = element_blank(),
    panel.border = element_rect(color = "black", fill = NA, linewidth = 0.6),
    plot.margin = margin(10, 10, 10, 10)
  )
}

# Load and reshape Treasury issuance data.
ta_us_treasury_sifma <- read_excel("data/ta-us-treasury-sifma.xlsx", sheet = "Issuance", skip = 5)
ta_us_treasury_sifma <- ta_us_treasury_sifma %>%
  as_tibble() %>%
  dplyr::select(-...5, -...9, -...13)

names(ta_us_treasury_sifma) <-
  c("date", "Bills_Gross Issuance", "Bills_Gross Retirement", "Bills_net",
    "Notes_Gross Issuance", "Notes_Gross Retirement", "Notes_net",
    "Bonds_Gross Issuance", "Bonds_Gross Retirement", "Bonds_net",
    "Total_Gross Issuance", "Total_Gross Retirement", "Total_net"
  )

ta_us_treasury_sifma <- ta_us_treasury_sifma %>%
  mutate(date = as.yearmon(as.Date(date))) %>%
  pivot_longer(cols = 2:13)

ta_us_treasury_sifma <- ta_us_treasury_sifma %>%
  mutate(type = str_extract(name, "[^_]+"),
    variable = str_replace(name, "[^_]*_", ""),
    year = year(date))

# Load and reshape the outstanding-debt data.
ta_us_treasury_sifma_OS <- read_excel(
  "data/ta-us-treasury-sifma.xlsx",
  sheet = "Outstanding",
  skip = 3
)
ta_us_treasury_sifma_OS <- (ta_us_treasury_sifma_OS[1:25,]) %>%
  mutate(TIPS = as.numeric(str_replace(TIPS, "-", "NA")),
    FRN = as.numeric(str_replace(FRN, "-", "NA"))) %>%
```

```

    rename(year = ...1)

ta_us_treasury_sifma_OS <- ta_us_treasury_sifma_OS %>%
  mutate(variable = "outstanding") %>%
  pivot_longer(cols = 2:7) %>%
  rename(type = name)

# Plot net issuance by instrument type.
p <- ggplot(ta_us_treasury_sifma_OS %>%
  filter(variable == "net", type != "Total") %>%
  group_by(type) %>%
  mutate(y = rollmean(value, k=12, align = "right", fill = NA),
    type = factor(type, levels = c("Bonds", "Notes", "Bills"))) ,
  aes(x = date, y = y, col = type, group = type, fill = type)) +
  geom_area() +
  theme_baseR() +
  labs(x = "", y = "USD Billions", title = "(a) Net issuance") +
  scale_color_manual(values = rev(c("black", "grey60", "grey80"))) +
  scale_fill_manual(values = rev(c("black", "grey60", "grey80"))) +
  theme(axis.text.x = element_text(angle = 45, hjust = 1),
    strip.text = element_text(size = 11),
    plot.title = element_text(hjust = .5, face="bold"),
    legend.position = "bottom",
    legend.title = element_blank()
  )

ta_us_treasury_sifma_OS <- ta_us_treasury_sifma_OS %>%
  mutate(date = as.Date(paste0(year, "-01-01")))

# Plot outstanding amounts and composition.
q <- ggplot() +
  geom_area(data = ta_us_treasury_sifma_OS %>% filter(type != "Total") %>%
    mutate(type = factor(type, levels = rev(c("TIPS", "FRN", "Bonds", "Bills")))
    aes(x = date, y = value, col = type, group = type, fill = type),
    position = "fill", alpha = .6
  ) +
  geom_line(data = ta_us_treasury_sifma_OS %>% filter(type == "Total") %>%
    mutate(type = "Total (right axis)"),
    aes(x = date, y = value/max(value), col = type, group = type, fill = type)
    lwd = 2) +
  theme_baseR() +
  scale_y_continuous(
    name = "Share", # left axis
    sec.axis = sec_axis(

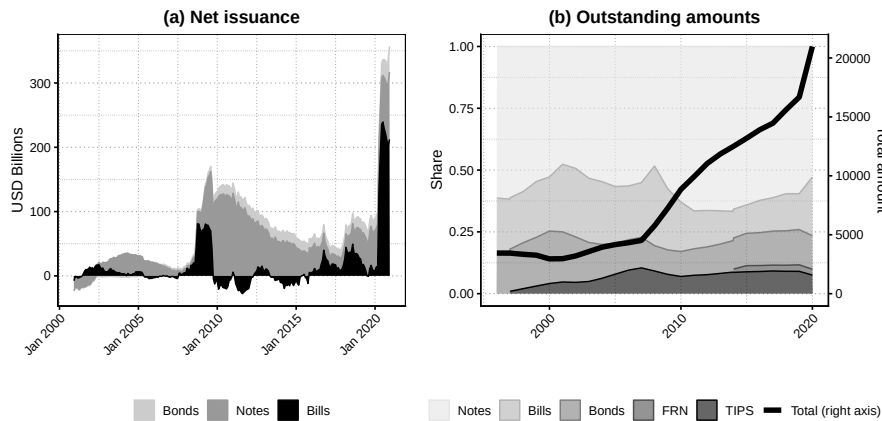
```

3.3. TREASURY ISSUANCE AND OUTSTANDING DEBT BY MATURITY²⁹

```

~ . * 20973.13,
name = "Total amount"
)
) +
scale_color_manual(values = c( "grey90", "grey70", "grey50", "grey30", "black", "black")) +
scale_fill_manual(values = c( "grey90", "grey70", "grey50", "grey30", "black", "black")) +
labs(x = "", title = "(b) Outstanding amounts") +
theme(axis.text.x = element_text(angle = 45, hjust = 1),
      strip.text = element_text(size = 11),
      plot.title = element_text(hjust = .5, face="bold"),
      legend.position = "bottom",
      legend.title = element_blank(),
      legend.box = "horizontal"
) +
guides(color = guide_legend(nrow = 1))
p + q

```



Chapter 4

Stylized facts and a first simple model

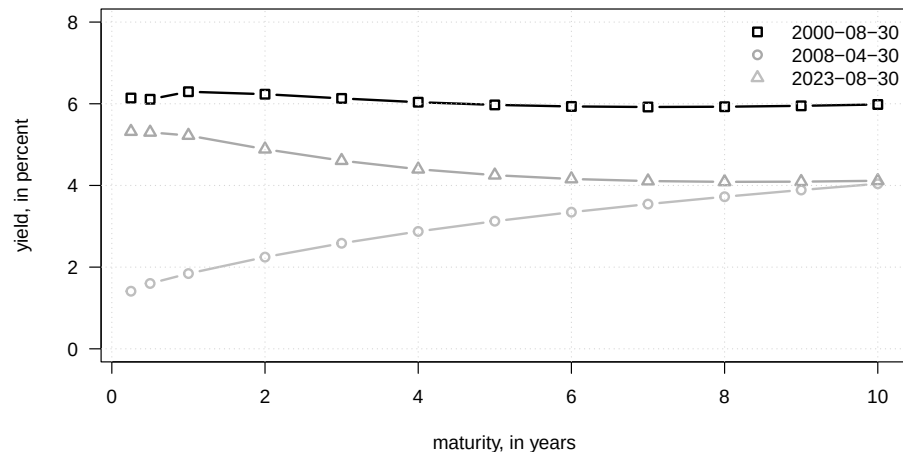
4.1 A few observed US yield curves with different shapes

This example plots a few observed US yield curves with different shapes.

Purpose: This example plots a few observed US yield curves with different shapes

```
library(DTAM)
t <- c(8000,10000,14000)
maturities <- c(.25,.5,1:10)
par(plt=c(.1,.98,.2,.97))
plot(maturities,YC_US[t[1],3:14],
     ylim=c(0,8),las=1,
     type="b",lwd=2,pch=0,col="black",
     xlab="maturity, in years",ylab="yield, in percent")
lines(maturities,YC_US[t[2],3:14],
      type="b",lwd=2,pch=1,col="grey")
lines(maturities,YC_US[t[3],3:14],
      type="b",lwd=2,pch=2,col="dark grey")
legend("topright",
      legend=c(YC_US$date[t]),
      lty=c(NaN,NaN),
      col=c("black","dark grey","grey"),
      pch=c(0,1,2),
      # cex=1.5,
      lwd=c(2,2),bty = "n")
```

```
grid()
```



4.2 A small number of principal components summarize most yield-curve variation

This example shows that a small number of principal components summarize most yield-curve variation.

Purpose: This example shows that a small number of principal components summarize most yield-curve variation

```
library(DTAM)
library(stringr)
# Detect the range of available maturities in the Liu-Wu data.
indic10y <- which(str_detect(names(YC_LW), "10"))
indic1m  <- which(str_detect(names(YC_LW), "1m"))
selected_dates <- complete.cases(YC_LW$yld_10y)
yds <- YC_LW[selected_dates, indic1m:indic10y]
PCA <- prcomp(yds)

# Report the cumulative variance explained by the principal components.
print(cumsum(PCA$sdev^2/sum(PCA$sdev^2)))
```

```
## [1] 0.9747138 0.9980741 0.9995896 0.9998753 0.9999268 0.9999500 0.9999657
## [8] 0.9999777 0.9999875 0.9999935 0.9999974 0.9999993 0.9999999 1.0000000
```

```
# Plot the first principal-component loading by maturity.
par(mfrow=c(1,2))
par(plt=c(.2,.95,.2,.8))
```

```
detect_reverse_PC1 <- -1 * (mean(PCA$rotation[,1]) < 0)
```

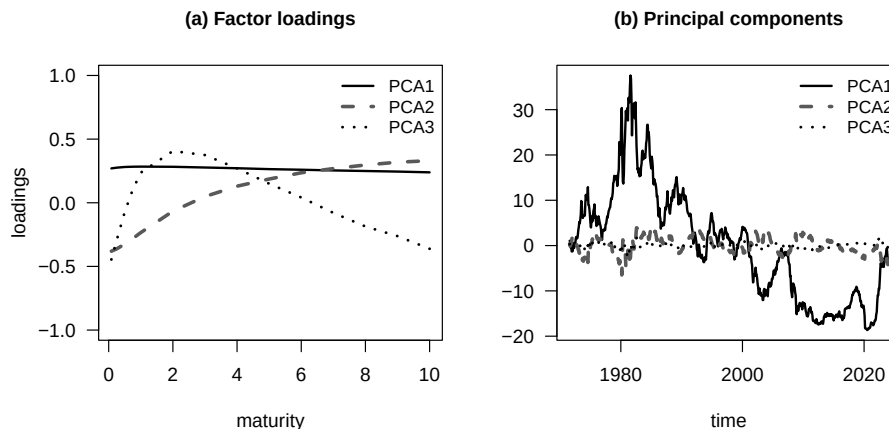


```

maturities <- c(1/12,3/12,6/12,9/12,1:10)
plot(c(maturities),detect_reverse_PC1*(PCA$rotation[,1]),type="l",lwd=2,ylim=c(-1,1),
     xlab="maturity",ylab="loadings",
     main="(a) Factor loadings",cex.main=1,las=1)
lines(maturities,PCA$rotation[,2],lwd=3,col="grey35",lty=2)
lines(maturities,PCA$rotation[,3],lwd=2.4,col="black",lty=3)
legend("topright",
      legend=c("PCA1", "PCA2", "PCA3"),
      lty=1:3,
      col=c("black", "grey35", "black"),
      cex=.9,
      lwd=c(2,3,2.4),bty = "n")

plot(YC_LW$date[selected_dates],detect_reverse_PC1*PCA$x[,1],type="l",lwd=2,
     las=1,xlab="time",ylab="",main="(b) Principal components",cex.main=1)
lines(YC_LW$date[selected_dates],PCA$x[,2],col="grey35",lwd=3,lty=2)
lines(YC_LW$date[selected_dates],PCA$x[,3],col="black",lwd=2.4,lty=3)
legend("topright",
      legend=c("PCA1", "PCA2", "PCA3"),
      lty=1:3,
      col=c("black", "grey35", "black"),
      cex=.9,
      lwd=c(2,3,2.4),bty = "n")

```



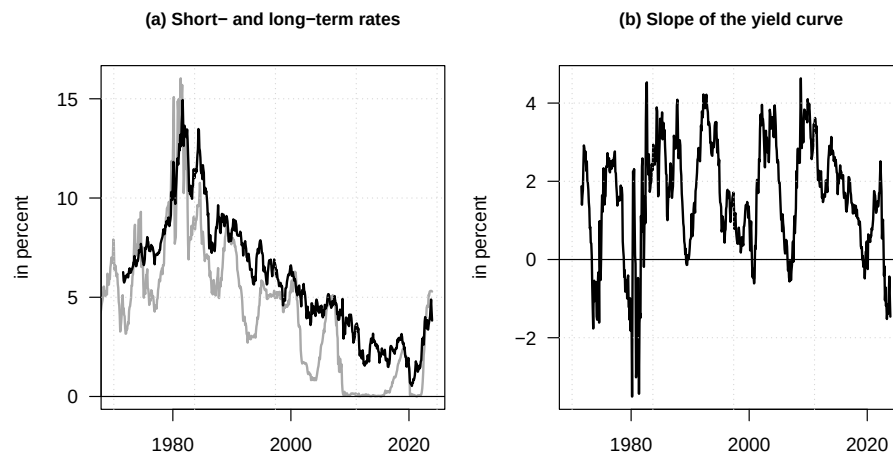
4.3 The level and slope of the yield curve over time

This example displays the level and slope of the yield curve over time.

```

# Purpose: This example displays the level and slope of the yield curve over time
library(DTAM)
# Compute the slope and plot short and long yields beside it.
par(mfrow=c(1,2))
par(plt=c(.2,.95,.1,.85))
slope <- YC_LW$yld_10y - YC_LW$yld_1m
plot(YC_LW$date, YC_LW$yld_1m, type="l", lwd=2, col="dark grey",
     main="(a) Short- and long-term rates", xlab="", ylab="in percent", las=1,
     xlim=c(as.Date("1970-01-01"), tail(YC_LW$date, 1)),
     cex.main=.9)
lines(YC_LW$date, YC_LW$yld_10y, lty=1, lwd=2)
grid()
abline(h=0)
plot(YC_LW$date, slope, type="l", lwd=2, col="black",
     main="(b) Slope of the yield curve", xlab="", ylab="in percent", las=1,
     xlim=c(as.Date("1970-01-01"), tail(YC_LW$date, 1)),
     cex.main=.9)
grid()
abline(h=0)

```



4.4 Overlays yields of several maturities to visualize their joint time-series behavior

This example overlays yields of several maturities to visualize their joint time-series behavior.

```

# Purpose: This example overlays yields of several maturities to visualize their joint
# time-series behavior
library(tidyverse)

```

4.4. OVERLAYS YIELDS OF SEVERAL MATURITIES TO VISUALIZE THEIR JOINT TIME-SERIES BEHAVIOR

```
library(zoo)

YC_LW_tibb <- YC_LW %>% as_tibble() %>%
  rename_with(~ c("1m", "3m", "6m", "9m", paste0(1:30, "y")), yld_1m:yld_30y) %>%
  relocate(date, .before = `1m`) %>%
  mutate(across(`1m`:`30y`, as.numeric)) %>%
  pivot_longer(cols = c(2:35)) %>%
  rename(maturity = name) %>%
  mutate(maturity = factor(maturity, levels = c("1m", "3m", "6m", "9m", paste0(1:30, "y"))),
         date = as.yearmon(date))

theme_baseR <- function() {
  theme_classic(base_size = 12) +
    theme(
      # Add light grid lines (only horizontal by default)
      panel.grid.major.y = element_line(color = "grey60", linewidth = 0.4,
                                         linetype = 3),
      panel.grid.minor.y = element_line(color = "grey60", linewidth = 0.2,
                                         linetype = 3),

      # Optionally add vertical grid lines (comment out if not wanted)
      panel.grid.major.x = element_line(color = "grey60", linewidth = 0.3,
                                         linetype = 3),
      panel.grid.minor.x = element_line(color = "grey60", linewidth = 0.3,
                                         linetype = 3),

      # Axis ticks more prominent
      axis.ticks = element_line(linewidth = 0.6),

      # Keep axis lines (theme_classic already does this)
      panel.grid = element_line(),

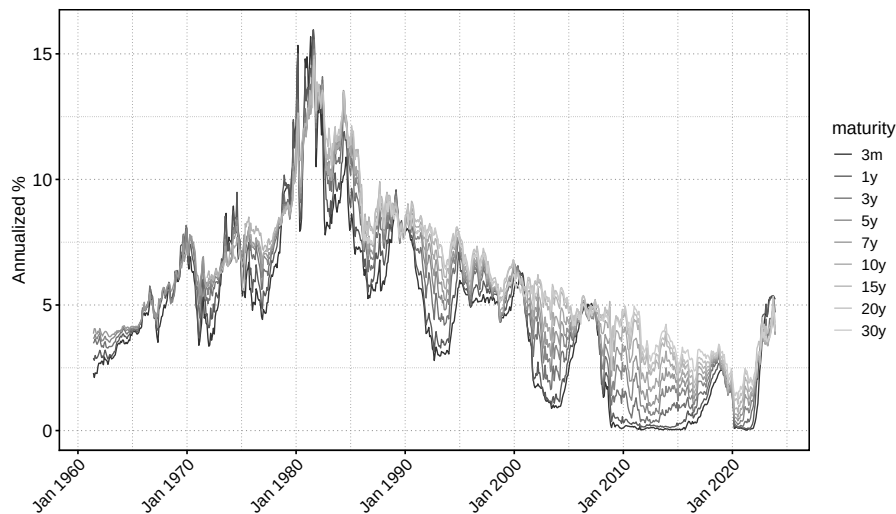
      # --- Facet labels bold, no box ---
      strip.text = element_text(face = "bold", size = 14),
      strip.background = element_blank(),

      # --- Box around the whole plotting panel ---
      panel.border = element_rect(color = "black", fill = NA, linewidth = 0.6),

      # Optional: small margin like base R
      plot.margin = margin(10, 10, 10, 10)
    )
}

library(ggplot2)
```

```
ggplot(YC_LW_tibb %>%
  filter(maturity %in% c("3m", paste0(c(1,3,5,7,10,15,20,30), "y"))),
  aes(x = date, y = value, col = maturity)) +
  geom_line(alpha = 1, lwd = .5) +
  theme_baseR() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1, size = 13),
        axis.text.y = element_text(size = 14),
        axis.title.y = element_text(size = 14),
        legend.text = element_text(size = 12),
        legend.title = element_text(size = 14)
  ) +
  scale_color_grey(start = 0.2, end = 0.8) +
  labs(y = "Annualized %", x = "")
```



4.5 The distributions of yield levels and monthly yield changes at short and long maturities

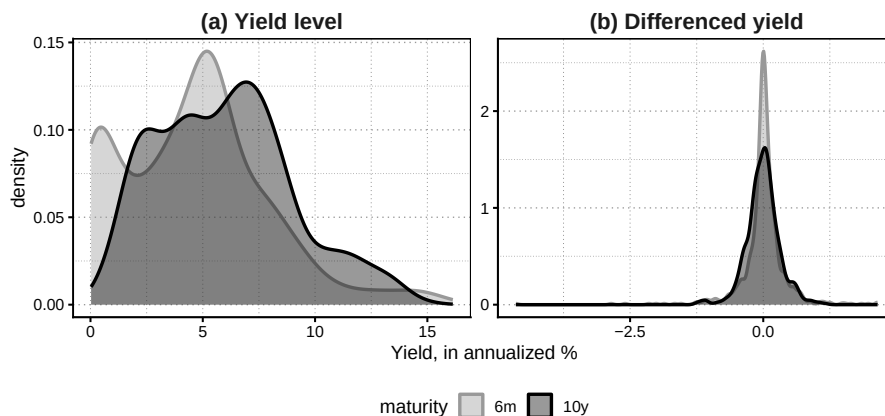
This example compares the distributions of yield levels and monthly yield changes at short and long maturities.

```
# Purpose: This example compares the distributions of yield levels and monthly yield c
# short and long maturities
library(ggplot2)
# Build yield-level and yield-change samples for two maturities.
YC_to_plot <- YC_LW_tibb %>%
  filter(maturity %in% c("6m", "10y")) %>%
```

4.6. HOW AVERAGE YIELDS AND YIELD VOLATILITIES VARY WITH MATURITY³⁷

```
group_by(maturity) %>%
  arrange(date) %>%
  mutate(d_value = value - dplyr::lag(value)) %>%
  pivot_longer(cols = 3:4) %>%
  rename(FD = name) %>%
  mutate(FD = factor(FD, levels = c("value", "d_value"),
                    labels = c("(a) Yield level", "(b) Differenced yield"))))

# Plot the two kernel-density comparisons.
ggplot(YC_to_plot,
       aes(x = value, group = maturity, col = maturity, bg = maturity)) +
  geom_density(alpha = .4, lwd = 1) +
  theme_baseR() +
  facet_wrap(~ FD, scales = "free",
            labeller = labeller(.multi_line = FALSE)) +
  labs(x = "Yield, in annualized %") +
  scale_color_manual(values = c("6m" = "grey60", "10y" = "black")) +
  scale_fill_manual(values = c("6m" = "grey60", "10y" = "black")) +
  theme(legend.position = "bottom")
```



4.6 How average yields and yield volatilities vary with maturity

This example summarizes how average yields and yield volatilities vary with maturity.

```
# Purpose: This example summarizes how average yields and yield volatilities vary with maturity
# Compute average yields and volatilities maturity by maturity.
YC_moments <- YC_LW_tibb %>%
```

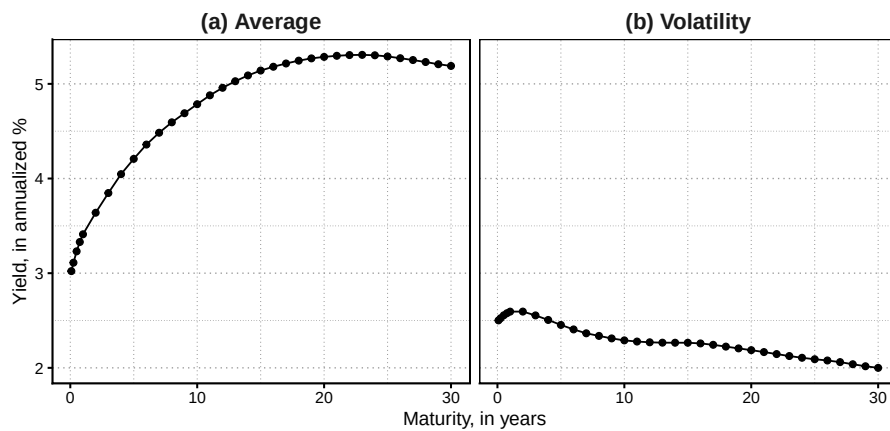
```

filter(if_all(everything(), ~ !is.nan(.x))) %>%
pivot_wider(names_from = maturity) %>%
drop_na() %>%
pivot_longer(cols = `1m`:`30y`) %>%
rename(maturity = name) %>%
mutate(maturity_nb = if_else(str_detect(maturity, "y"),
                             12 * as.numeric(str_remove(maturity, "y")),
                             as.numeric(str_remove(maturity, "m")))) %>%
mutate(maturity = factor(maturity,
                        levels = c(paste0(c(1,3,6,9),"m"),
                                   paste0(1:30,"y")))) %>%

group_by(maturity_nb) %>%
summarize(`(a) Average` = mean(value),
          `(b) Volatility` = sd(value, na.rm = T))

library(ggplot2)
# Plot the two maturity profiles.
ggplot(YC_moments %>% pivot_longer(2:3),
       aes(x = maturity_nb/12, y = value, group = name)) +
  geom_line() +
  geom_point() +
  facet_grid(~ name, scales = "free") +
  theme_baseR() +
  labs(y = "Yield, in annualized %", x = "Maturity, in years")

```



4.7 The behavior of US and euro-area yield curves around the zero lower bound period

This example highlights the behavior of US and euro-area yield curves around the zero lower bound period.

```
# Purpose: This example highlights the behavior of US and euro-area yield curves around the zero
# lower bound period
library(ggplot2)
# Build the US yield-curve panel around the lower-bound period.
p_US_ZLB <- ggplot(YC_LW_tibb %>%
  filter(maturity %in% c("3m", paste0(c(1,3,5,7,10,15,20,30), "y")),
    date > "Jun 2007", date <= "Jun 2022"),
  aes(x = date, y = value, col = maturity)) +
  geom_line(alpha = 1, lwd = .5) +
  theme_baseR() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1, size = 12),
    plot.title = element_text(
      hjust = 0.5,
      size = 13,
      face = "bold")) +
  scale_color_grey(start = 0.2, end = 0.8) +
  labs(y = "Annualized %", x = "", title = "(a) US Treasury Curve") +
  geom_hline(aes(yintercept = 0), col = "black") +
  geom_vline(xintercept = as.yearmon(c("Sep 2008", "Mar 2020")),
    lty = 5, lwd = 1, col = "black") +
  annotate(
    "text",
    x = as.yearmon("Nov 2008"), y = 5,
    label = "Lehman failure", size = 3,
    fontface = "italic", color = "black", hjust = 0, vjust = 0
  ) +
  annotate(
    "text",
    x = as.yearmon("Apr 2020"), y = 5,
    label = "COVID", size = 3,
    fontface = "italic", color = "black", hjust = 0, vjust = 0
  )
)

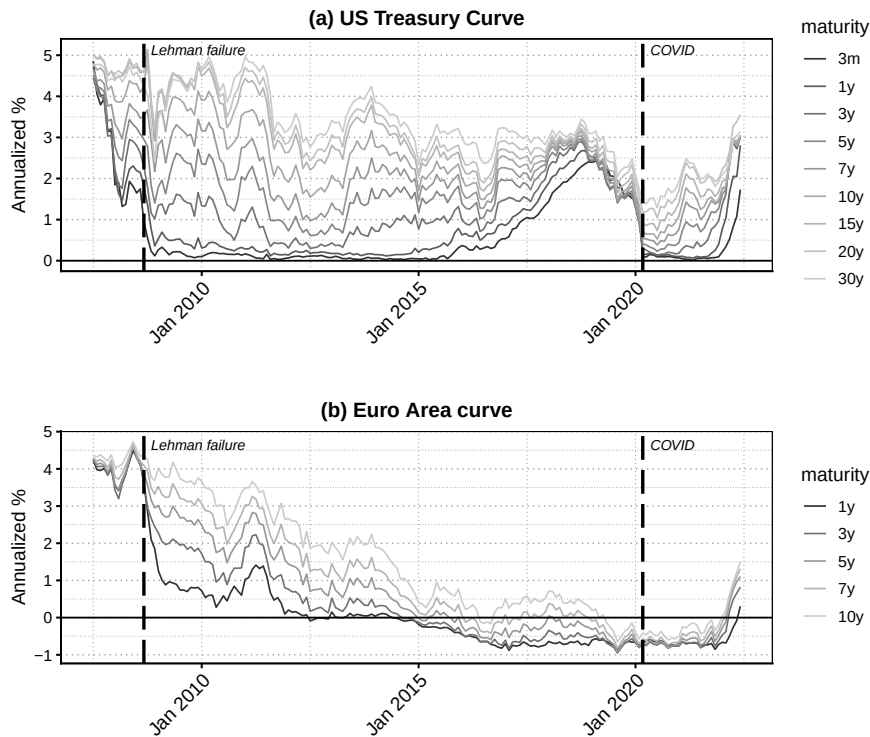
# Add the same zero and crisis markers to the euro-area panel.
# Build the euro-area panel on the same sample.
p_EUR_ZLB <- ggplot(
  YC_Euro %>%
  mutate(yearmon = as.yearmon(date)) %>%
  group_by(yearmon) %>%
  slice_tail(n=1) %>%
```

```

ungroup() %>%
relocate(yearmon, .before = date) %>%
dplyr::select(-date) %>%
pivot_longer(Y1:Y10) %>%
rename(maturity = name, date = yearmon) %>%
mutate(maturity = factor(paste0(parse_number(maturity), "y"),
                          levels = paste0(1:10, "y"))) %>%
filter(maturity %in% c(paste0(c(1,3,5,7,10), "y")),
       date > "Jun 2007", date <= "Jun 2022"),
aes(x = date, y = value, col = maturity)) +
geom_line(alpha = 1, lwd = .5) +
theme_baseR() +
theme(axis.text.x = element_text(angle = 45, hjust = 1, size = 12),
      plot.title = element_text(
        hjust = 0.5,
        size = 13,
        face = "bold"
      )) +
scale_color_grey(start = 0.2, end = 0.8) +
labs(y = "Annualized %", x = "", title = "(b) Euro Area curve") +
geom_hline(aes(yintercept = 0), col = "black") +
geom_vline(xintercept = as.yearmon(c("Sep 2008", "Mar 2020")),
           lty = 5, lwd = 1, col = "black") +
annotate(
  "text",
  x = as.yearmon("Nov 2008"), y = 4.5,
  label = "Lehman failure", size = 3,
  fontface = "italic", color = "black", hjust = 0, vjust = 0
) +
annotate(
  "text",
  x = as.yearmon("Apr 2020"), y = 4.5,
  label = "COVID", size = 3,
  fontface = "italic", color = "black", hjust = 0, vjust = 0
)

library(patchwork)
# Stack the two regional panels.
p_US_ZLB/p_EUR_ZLB

```

4.8 Time-varying yield volatility across maturities

This example tracks time-varying yield volatility across maturities.

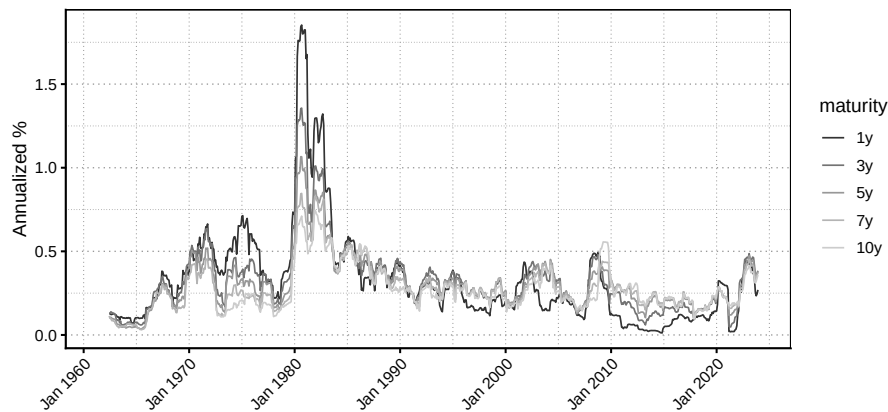
```
# Purpose: This example tracks time-varying yield volatility across maturities
library(zoo)
# Use rolling squared yield changes as a volatility proxy.
YC_vol <- YC_LW_tibb %>%
  filter(if_all(everything(), ~ !is.nan(.x))) %>%
  group_by(maturity) %>%
  arrange(maturity, date) %>%
  drop_na() %>%
  mutate(d_value = value - dplyr::lag(value)) %>%
  mutate(var_proxy = rollmean(d_value^2, k = 12, fill = NA, align = "right")) %>%
  mutate(vol_proxy = sqrt(var_proxy))

# Plot the volatility proxy across selected maturities.
ggplot(YC_vol %>% filter(maturity %in% c(paste0(c(1,3,5,7,10), "y"))),
```

```

aes(x = date, y = vol_proxy, col = maturity, group = maturity)) +
geom_line(alpha = 1, lwd = .5) +
theme_baseR() +
theme(axis.text.x = element_text(angle = 45, hjust = 1)) +
scale_color_grey(start = 0.2, end = 0.8) +
labs(y = "Annualized %", x = "")

```



4.9 The covariance and correlation structure of the yield curve

This example computes the covariance and correlation structure of the yield curve.

Purpose: This example computes the covariance and correlation structure of the yield

Compute covariance and correlation matrices for selected maturities.

```

Cov_yields <- YC_LW_tibb %>%
  filter(if_all(everything(), ~ !is.nan(.x)),
    maturity %in% c("3m", paste0(c(1,3,5,7,10,15,20,30), "y")))) %>%
  pivot_wider(names_from = maturity) %>%
  drop_na() %>%
  dplyr::select(-date) %>%
  cov()

```

```

Cor_yields <- YC_LW_tibb %>%
  filter(if_all(everything(), ~ !is.nan(.x)),
    maturity %in% c("3m", paste0(c(1,3,5,7,10,15,20,30), "y")))) %>%
  pivot_wider(names_from = maturity) %>%
  drop_na() %>%

```

4.9. THE COVARIANCE AND CORRELATION STRUCTURE OF THE YIELD CURVE43

Table 4.1: (#tab:tab:covcorYC)Covariance and correlation of the yield curve

	3m	1y	3y	5y	7y	10y	15y	20y	30y
3m	6.372	0.994	0.965	0.934	0.905	0.867	0.829	0.821	0.794
1y	6.509	6.728	0.983	0.956	0.929	0.892	0.855	0.846	0.820
3y	6.221	6.510	6.523	0.993	0.978	0.952	0.924	0.918	0.896
5y	5.784	6.082	6.219	6.019	0.996	0.981	0.961	0.956	0.939
7y	5.401	5.696	5.905	5.776	5.593	0.995	0.981	0.977	0.964
10y	5.011	5.299	5.571	5.512	5.386	5.244	0.995	0.992	0.982
15y	4.742	5.022	5.346	5.341	5.256	5.162	5.130	0.998	0.991
20y	4.530	4.801	5.125	5.130	5.053	4.969	4.945	4.782	0.995
30y	4.009	4.252	4.576	4.608	4.558	4.499	4.491	4.350	3.999

```
dplyr::select(-date) %>%
cor()

CovCor_yields <- Cov_yields
CovCor_yields[upper.tri(CovCor_yields)] <- Cor_yields[upper.tri(Cor_yields)]

# Format the upper triangle as correlations and the lower triangle as covariances.
library(kableExtra)
library(tibble)

CovCor_yields <- as.data.frame(CovCor_yields)

CovCor_yields[,1] <- sprintf("%.3f", CovCor_yields[,1])

for(i in 2:ncol(CovCor_yields)){
  CovCor_yields[1:(i-1), i] <- paste0("\\textbf{", sprintf("%.3f", CovCor_yields[1:(i-1), i]), ")")
  CovCor_yields[i:nrow(CovCor_yields), i] <- sprintf(
    "%.3f",
    as.numeric(CovCor_yields[i:nrow(CovCor_yields), i])
  )
}

# Export the LaTeX table.
kable(CovCor_yields, format = "latex",
      digits = 3, align = c('l', rep('c', 9)),
      caption = "Covariance and correlation of the yield curve",
      labe = "tab:covcorYC",
      booktabs = TRUE, escape = FALSE) %>% print()
```

The next table complements the covariance evidence by summarizing yield per-

sistence across maturities.

```
# Purpose: The next table complements the covariance evidence by summarizing yield per.
# across maturities
library(dplyr)
library(stringr)

# Estimate persistence measures for selected maturities.
Persis_yields <- YC_LW_tibb %>%
  filter(if_all(everything(), ~ !is.nan(.x)),
    maturity %in% c("3m", paste0(c(1,3,5,7,10,15,20,30), "y")))) %>%
  group_by(maturity) %>%
  arrange(maturity, date) %>%
  summarise(
    `Acf(1)` = acf(value, plot = FALSE, lag.max = 12, na.action = na.pass)$acf[2],
    `Acf(12)` = acf(value, plot = FALSE, lag.max = 12, na.action = na.pass)$acf[13],
    `Pacf(2)` = pacf(value, plot = FALSE, lag.max = 12, na.action = na.pass)$acf[2],
    `Pacf(12)` = pacf(value, plot = FALSE, lag.max = 12, na.action = na.pass)$acf[12],
    .groups = "drop"
  )

# Prepare a display-ready table.
Persis_tab <- Persis_yields %>%
  mutate(
    Maturity = case_when(
      maturity == "3m" ~ "3-month",
      str_detect(maturity, "y") ~ paste0(str_remove(maturity, "y"), "-year"),
      TRUE ~ maturity
    )
  ) %>%
  dplyr::select(
    Maturity,
    `Acf(1)`, `Acf(12)`, `Pacf(2)`, `Pacf(12)`
  ) %>%
  mutate(across(-Maturity, ~ round(.x, 3)))

fmt3 <- function(x) sprintf("%.3f", as.numeric(x))

# Convert rows to LaTeX table lines.
rows_tex <- apply(Persis_tab, 1, function(r) {
  paste0(
    r[[1]], " & ",
    fmt3(r[[2]]), " & ",
    fmt3(r[[3]]), " & ",
    fmt3(r[[4]]), " & ",
    fmt3(r[[5]]),
  )
})
```

```

    " \\\\"
  )
})

# Assemble the LaTeX table.
latex_table <- paste(
  "\\begin{table}[t]
  \\centering
  \\caption{Persistence of zero-coupon yields across maturities}
  \\label{tab:persistence}

  \\begin{tabularx}{\\textwidth}{l *{4}{>{\\centering\\arraybackslash}X}}
  \\toprule
  Maturity
  & $\\mathrm{ACF}(1)$
  & $\\mathrm{ACF}(12)$
  & $\\mathrm{PACF}(2)$
  & $\\mathrm{PACF}(12)$ \\\\"
  \\midrule",
  paste(rows_tex, collapse = "\\n"),
  "\\bottomrule
  \\end{tabularx}

  \\vspace{0.3em}
  \\parbox{1.0\\textwidth}{\\footnotesize\\textit{Notes}:
  The table reports autocorrelations (ACF) and partial autocorrelations (PACF)
  of monthly zero-coupon yields at selected maturities.
  $\\mathrm{ACF}(1)$ and $\\mathrm{ACF}(12)$ correspond to lags of 1 and 12 months, respectively.
  $\\mathrm{PACF}(2)$ and $\\mathrm{PACF}(12)$ report partial autocorrelations at lags 2 and 12.
  Yields are constructed from zero-coupon curve estimates of \\citet{Liu_Wu_2021}.}

  \\end{table}
  ", sep = "\\n")

# Write the table to file.
out_file <- "tables/persistence.txt"
writeLines(latex_table, out_file)

```

The next lines of codes compute a confidence interval for the mean of the slope of the yield curve, using Newey-West corrected standard deviation.

4.10 A simple HAC-robust inference exercise for the average term spread

This example illustrates a simple HAC-robust inference exercise for the average term spread.

```
# Purpose: This example illustrates a simple HAC-robust inference exercise for the average
# spread
library(DTAM)
# Estimate the average slope and compute robust standard errors.
slope <- YC_LW$yld_10y - YC_LW$yld_1m
eq     <- lm(slope~1)
lmtest::coefTest(eq,vcov=sandwich::vcovHC(eq))

##
## t test of coefficients:
##
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  1.61463    0.05685   28.401 < 2.2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

The next block produces a figure showing bond excess returns.

4.11 Bond excess returns across holding periods and maturities

This example plots bond excess returns across holding periods and maturities.

```
# Purpose: This example plots bond excess returns across holding periods and maturities.
library(DTAM)
library(stringr)
library(sandwich)

first_date      <- as.Date("1970-01-01")
last_date       <- as.Date("2023-12-01")
indic_first_date <- which(YC_LW$date==first_date)
indic_last_date  <- which(YC_LW$date==last_date)

holding_periods <- c(1,3,6) # in months
# Determine the long-term maturities available in the Liu-Wu data.
maturities <- gsub("yld_", "", names(YC_LW))
detect_longterm <- which(str_detect(maturities,"y"))
nb_longterm     <- length(detect_longterm)
names_longterm  <- maturities[detect_longterm]
maturities_longterm <- as.numeric(gsub("y", "", names_longterm))
```

4.11. BOND EXCESS RETURNS ACROSS HOLDING PERIODS AND MATURITIES⁴⁷

```

matrixH <- t(matrix(maturities_longterm,
                    length(maturities_longterm),
                    dim(YC_LW)[1]))

# Compute excess returns:
r_t_h <- as.matrix(YC_LW[,detect_longterm])
xsr <- array(NA, c(dim(YC_LW)[1], length(maturities_longterm), length(holding_periods)))

all_alpha_coef <- matrix(NA, length(holding_periods), nb_longterm)
all_alpha_stdv <- matrix(NA, length(holding_periods), nb_longterm)

all_beta_coef <- matrix(NA, length(holding_periods), nb_longterm)
all_beta_stdv <- matrix(NA, length(holding_periods), nb_longterm)

count_k <- 0
for(k in holding_periods){# expressed in months
  count_k <- count_k + 1
  # Match each holding period with the corresponding short rate.
  detect_k <- which(str_detect(names(YC_LW), paste(k, "m", sep="")))
  r_t_k <- matrix(YC_LW[,detect_k], ncol=1) %>% matrix(1,1,nb_longterm)

  r_tplusk_h <- NA*r_t_h
  r_tplusk_h[1:(dim(r_t_h)[1]-k),] <- r_t_h[(k+1):dim(r_t_h)[1],]
  # Approximate the maturity h-k yield after the holding period.
  w <- k/12
  r_tplusk_hminus_k <- (1-w)*r_tplusk_h[,1:(nb_longterm-1)] +
    w*r_tplusk_h[,2:nb_longterm] # exploits that we have all years
  r_tplusk_hminus_k <- cbind(r_tplusk_hminus_k, r_tplusk_h[,dim(r_tplusk_h)[2]])

  # Express maturities and holding periods in the same units.
  M <- matrixH*12/k * r_t_h - (matrixH*12/k - 1) * r_tplusk_hminus_k - r_t_k
  xsr[, ,count_k] <- M

  # Run the Campbell-Shiller regression maturity by maturity.
  dep_var <- r_tplusk_hminus_k - r_t_h
  indep_var <- k/(matrixH*12 - k)*(r_t_h - r_t_k)

  dep_var <- dep_var[indic_frst_date:indic_last_date,]
  indep_var <- indep_var[indic_frst_date:indic_last_date,]

  for(h in 1:nb_longterm){
    y <- c(dep_var[,h])
    x <- c(indep_var[,h])
    eq <- lm(y ~ x)
    all_alpha_coef[count_k,h] <- eq$coef[1]
  }
}

```

```

    all_alpha_stdv[count_k,h] <- sqrt(NeweyWest(eq)[1,1])
    all_beta_coef[count_k,h] <- eq$coef[2]
    all_beta_stdv[count_k,h] <- sqrt(NeweyWest(eq)[2,2])
  }
}

xsr <- xsr[indic_first_date:indic_last_date,,]
vec_dates <- YC_LW$date[indic_first_date:indic_last_date]

par(mfrow=c(1,2))
h <- 2
k1 <- 3
k2 <- 6
indic_k1 <- which(holding_periods==k1)
indic_k2 <- which(holding_periods==k2)
plot(vec_dates,xsr[,h,indic_k1],lwd=2,type="l",
     xlab="",ylab="annualized return, in percent",las=1,
     ylim=c(min(xsr[,h,c(indic_k1,indic_k2)],na.rm=TRUE),
            1.2*max(xsr[,h,c(indic_k1,indic_k2)],na.rm=TRUE)),
     main="(a)",col="dark grey")
lines(vec_dates,xsr[,h,indic_k2],lwd=2)
grid()
legend("topright",
     paste("k = ", c(k1, k2), " month",
            ifelse(c(k1, k2) > 1, "s", ""),
            ", h = ", h, " year", ifelse(h > 1, "s", "")),
     sep = ""),
     lty = c(1, 1),
     lwd = c(2, 2),
     col = c("dark grey", "black"),
     bg="white",
     seg.len = 3,
     cex = .9
)

h1 <- 2
h2 <- 10
k <- 6
indic_k <- which(holding_periods==k)
plot(vec_dates,xsr[,h2,indic_k],lwd=2,type="l",
     ylim=c(min(xsr[,1:max(h1,h2),indic_k],na.rm=TRUE),
            1.2*max(xsr[,1:max(h1,h2),indic_k],na.rm=TRUE)),
     xlab="",ylab="annualized return, in percent",las=1,
     main="(b)",col="dark grey")

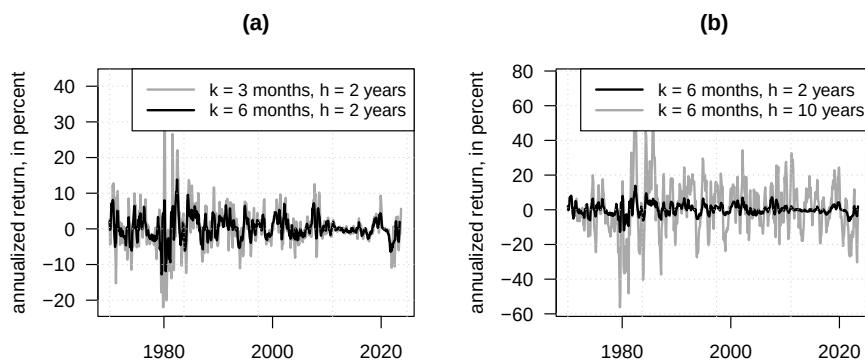
```



```

lines(vec_dates,xsr[,h1,indic_k],lwd=2)
grid()
legend("topright",
      paste("k = ",k," months, h = ",c(h1,h2)," year",ifelse(c(h1,h2)>1,"s",""),sep=""),
      lty = c(1, 1),
      lwd = c(2, 2),
      col = c("black", "dark grey"),
      bg="white",
      seg.len = 3,
      cex = .9
)

```



4.12 The term structure of average excess returns, volatility, and Sharpe ratios

This example summarizes the term structure of average excess returns, volatility, and Sharpe ratios.

```

# Purpose: This example summarizes the term structure of average excess returns, volatility, and
# Sharpe ratios
library(tidyverse)
library(zoo)
# Put excess returns for all holding periods in a single long data set.
xsr_tibb <- as_tibble(xsr[,1]) %>%
  mutate(date = as.yearmon(vec_dates),
         `holding period` = "1 month") %>%
  rename_with(~ paste0(1:30,"-year"), V1:V30) %>%
  relocate(date, `holding period`, .before = `1-year`)

```

```

for(slice in 2:dim(xsr)[3]){
  xsr_tibb <- xsr_tibb %>% bind_rows(
    ., as_tibble(xsr[, , slice]) %>%
      mutate(date = as.yearmon(vec_dates),
             `holding period` = paste0(holding_periods[slice], " months")) %>%
      rename_with(~ paste0(1:30, "-year"), V1:V30) %>%
      relocate(date, `holding period`, .before = `1-year`)
  )
}

xsr_tibb <- xsr_tibb %>% drop_na()

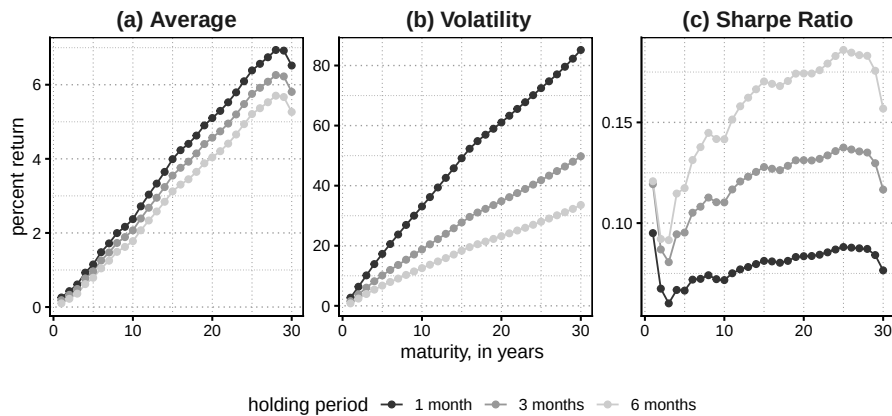
# Pivot maturities to long format.
xsr_tibb <- xsr_tibb %>%
  pivot_longer(cols = 3:32) %>%
  rename(maturity = name) %>%
  mutate(maturity = factor(maturity, levels = paste0(1:30, "-year")))

# Compute summary statistics XSR
summary_stats_XSR <- xsr_tibb %>%
  filter(if_all(everything(), ~ !is.nan(.x))) %>%
  filter(if_all(everything(), ~ !is.na(.x))) %>%
  group_by(maturity, `holding period`) %>%
  summarize(Average = mean(value, na.rm = T),
            Volatility = sd(value, na.rm = T)) %>%
  mutate(Sharpe = Average/Volatility)

library(ggplot2)

# Plot average returns, volatility, and Sharpe ratios.
ggplot(summary_stats_XSR %>% pivot_longer(cols = 3:5) %>%
  mutate(name = factor(name, levels = c("Average", "Volatility", "Sharpe"),
    labels = c("(a) Average", " (b) Volatility", "(c) Sharpe Ratio"))
  filter(parse_number(as.character(maturity)) <= 30),
  aes(x = parse_number(as.character(maturity)), y = value,
      col = `holding period`, group = `holding period`)) +
  geom_line() +
  geom_point() +
  facet_wrap(~ name, scales="free") +
  theme_baseR() +
  theme(legend.position = "bottom") +
  scale_color_grey(start = 0.2, end = 0.8) +
  labs(y = "percent return", x = "maturity, in years")

```



4.13 A few principal components also summarize excess-return dynamics

This example shows that a few principal components also summarize excess-return dynamics.

Purpose: This example shows that a few principal components also summarize excess-return dynamics
Run a PCA on six-month excess returns.

```
PCA_xsr <- xsr_tibb %>%
  filter(`holding period`=="6 months") %>%
  pivot_wider(names_from = maturity) %>%
  dplyr::select( - date, - `holding period`) %>%
  princomp()
```

```
print(cumsum(PCA_xsr$sdev^2/sum(PCA_xsr$sdev^2)))
```

```
##      Comp.1      Comp.2      Comp.3      Comp.4      Comp.5      Comp.6      Comp.7      Comp.8
## 0.9738747 0.9924074 0.9973369 0.9986190 0.9991932 0.9995290 0.9996769 0.9997768
##      Comp.9      Comp.10      Comp.11      Comp.12      Comp.13      Comp.14      Comp.15      Comp.16
## 0.9998322 0.9998736 0.9999030 0.9999244 0.9999414 0.9999550 0.9999655 0.9999739
##      Comp.17      Comp.18      Comp.19      Comp.20      Comp.21      Comp.22      Comp.23      Comp.24
## 0.9999809 0.9999858 0.9999892 0.9999919 0.9999938 0.9999954 0.9999967 0.9999978
##      Comp.25      Comp.26      Comp.27      Comp.28      Comp.29      Comp.30
## 0.9999986 0.9999990 0.9999994 0.9999997 0.9999999 1.0000000
```

Extract dates for the PCA score panel.
Plot the first loadings and the corresponding scores.
`par(mfrow=c(1,2))`
`par(plt=c(.2,.95,.2,.8))`

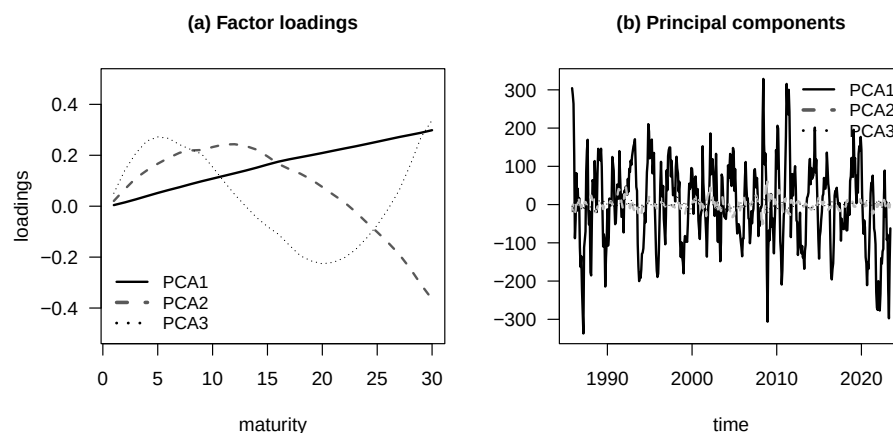
```

dates_plot_xsr <- xsr_tibb %>%
  filter(`holding period`=="6 months") %>%
  pivot_wider(names_from = maturity) %>%
  dplyr::select(date) %>% c() %>% unlist()

maturities <- c(1:30)
plot(c(maturities),PCA_xsr$loadings[,1],type="l",lwd=2,ylim=c(-.5,.5),
     xlab="maturity",ylab="loadings",
     main="(a) Factor loadings",cex.main=1,las=1)
lines(maturities,PCA_xsr$loadings[,2],lwd=2,col="grey35",lty=2)
lines(maturities,PCA_xsr$loadings[,3],lwd=1,col="black",lty=3)
legend("bottomleft",
      legend=c("PCA1", "PCA2", "PCA3"),
      lty=1:3,
      col=c("black", "grey35", "black"),
      cex=.9,
      lwd=c(2,3,2.4),bty = "n")

plot(dates_plot_xsr,PCA_xsr$scores[,1],type="l",lwd=2,
     las=1,xlab="time",ylab="",main="(b) Principal components",cex.main=1)
lines(dates_plot_xsr, PCA_xsr$scores[,2],col="grey",lwd=2,lty=2)
lines(dates_plot_xsr, PCA_xsr$scores[,3],col="black",lwd=1,lty=3)
legend("topright",
      legend=c("PCA1", "PCA2", "PCA3"),
      lty=1:3,
      col=c("black", "grey35", "black"),
      cex=.9,
      lwd=c(2,3,2.4),bty = "n")

```



The next block runs Campbell-Shiller regressions of excess returns.

4.14 Campbell-Shiller regression coefficients

This example visualizes the estimated slope and intercept patterns from Campbell-Shiller regressions.

```
# Purpose: This example visualizes the estimated slope and intercept patterns from
# Campbell-Shiller regressions
# Select two holding periods and build confidence bands.
par(mfrow=c(2,2))
k1 <- 3
k2 <- 6
indic_k1 <- which(holding_periods==k1)
indic_k2 <- which(holding_periods==k2)

lower_beta <- all_beta_coef - 1.64*all_beta_stdv
upper_beta <- all_beta_coef + 1.64*all_beta_stdv

lower_alpha <- all_alpha_coef - 1.64*all_alpha_stdv
upper_alpha <- all_alpha_coef + 1.64*all_alpha_stdv

# Use the same plotting scale across holding periods.
par(plt=c(.2,.95,.2,.85))

# Plot intercept and slope estimates across maturities.
plot(all_alpha_coef[indic_k1,],type="l",lwd=2,las=1,
     ylim=c(min(lower_alpha),max(upper_alpha)),
     xlab=expression(paste("maturity (",h,")",sep="")),ylab=expression(alpha[h*','*k]),
     main=paste("(a.1) Holding period k = ",k1," month",ifelse(k1>1,"s",""),sep=""),cex.main=1)
grid()
abline(h=0,col="grey")
lines(lower_alpha[indic_k1,],lty=2,lwd=2)
lines(upper_alpha[indic_k1,],lty=2,lwd=2)

plot(all_alpha_coef[indic_k2,],type="l",lwd=2,las=1,
     ylim=c(min(lower_alpha),max(upper_alpha)),
     xlab=expression(paste("maturity (",h,")",sep="")),ylab=expression(alpha[h*','*k]),
     main=paste("(b.1) Holding period k = ",k2," month",ifelse(k2>1,"s",""),sep=""),cex.main=1)
grid()
abline(h=0,col="grey")
lines(lower_alpha[indic_k2,],lty=2,lwd=2)
lines(upper_alpha[indic_k2,],lty=2,lwd=2)

plot(all_beta_coef[indic_k1,],type="l",lwd=2,las=1,
     ylim=c(min(lower_beta),max(upper_beta)),
     xlab=expression(paste("maturity (",h,")",sep="")),ylab=expression(beta[h*','*k]),
     main=paste("(a.2) Holding period k = ",k1," month",ifelse(k1>1,"s",""),sep=""),cex.main=1)
```

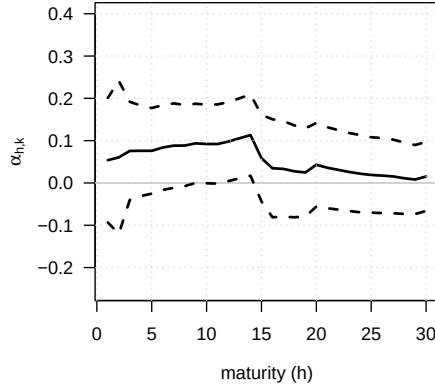
```

grid()
abline(h=1,col="grey")
lines(lower_beta[indic_k1,],lty=2,lwd=2)
lines(upper_beta[indic_k1,],lty=2,lwd=2)

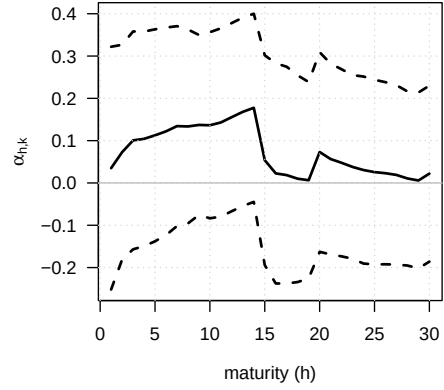
plot(all_beta_coef[indic_k2,],type="l",lwd=2,las=1,
     ylim=c(min(lower_beta),max(upper_beta)),
     xlab=expression(paste("maturity (",h,")",sep="")),ylab=expression(beta[h*','*',*k])),
     main=paste("(b.2) Holding period k = ",k2," month",ifelse(k2>1,"s",""),sep=""),ce
grid()
abline(h=1,col="grey")
lines(lower_beta[indic_k2,],lty=2,lwd=2)
lines(upper_beta[indic_k2,],lty=2,lwd=2)

```

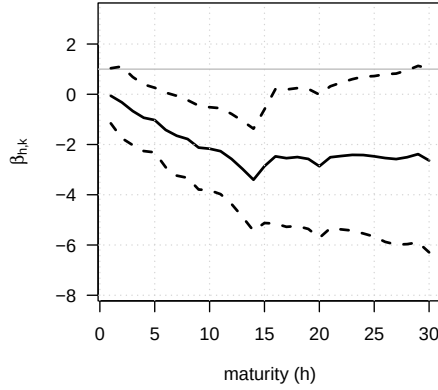
(a.1) Holding period k = 3 months



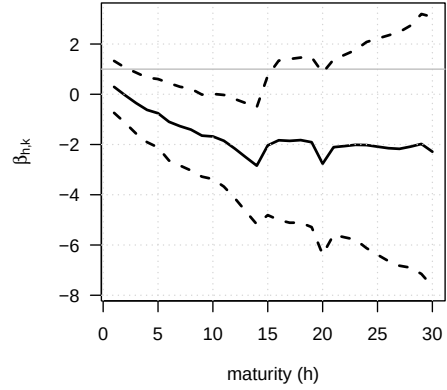
(b.1) Holding period k = 6 months



(a.2) Holding period k = 3 months



(b.2) Holding period k = 6 months



4.15 The unconditional mean and volatility of the short rate

This example reports the unconditional mean and volatility of the short rate.

```
# Purpose: This example reports the unconditional mean and volatility of the short rate
library(DTAM)
# Report the unconditional mean and volatility of the short rate.
mean(YC_US$DTB3, na.rm=TRUE)
```

```
## [1] 4.443374
```

```
sd(YC_US$DTB3, na.rm=TRUE)
```

```
## [1] 3.355053
```

4.16 Two widely used term-premium estimates across maturities

This example compares two widely used term-premium estimates across maturities.

```
# Purpose: This example compares two widely used term-premium estimates across maturities
# Merge macro data with ACM and Kim-Wright term-premium estimates.
Macro_data <- Data_Macro_US_monthly %>%
  mutate(date = as.yearmon(date))
```

```
TP_to_plot <- ACMTermPremium %>%
  mutate(date = as.yearmon(as.Date(DATE, format = "%d-%m-%Y"))) %>%
  dplyr::select(-DATE)
```

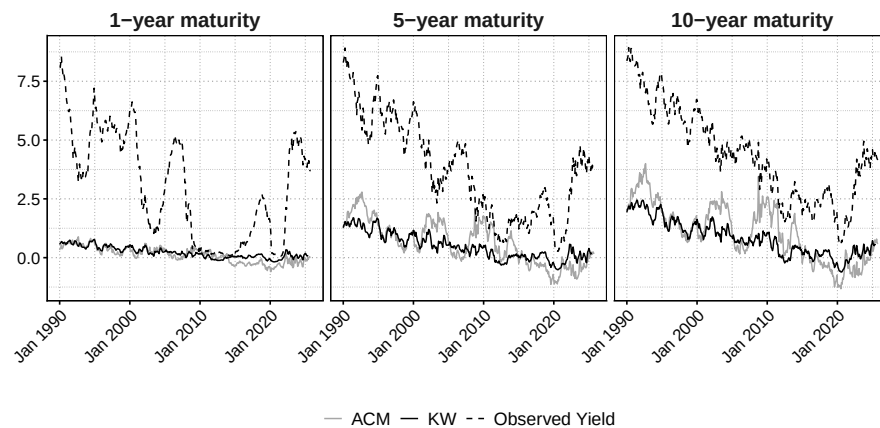
```
# Combine the two term-premium data sources.
```

```
total_TP_to_plot_KWACM <- Macro_data %>%
  full_join(TP_to_plot)
```

```
total_TP_to_plot_KWACM <- total_TP_to_plot_KWACM %>%
  dplyr::select(date, THREEFYTP1:THREEFYTP10, ACMY01:ACMTP10 ) %>%
  pivot_longer(cols = THREEFYTP1:ACMTP10) %>%
  mutate(maturity = parse_number(name),
         model = case_when(
           str_detect(name, "THREEFYTP") ~ "KW",
           str_detect(name, "ACMY") ~ "Observed Yield",
           str_detect(name, "ACMTP") ~ "ACM",
         ))
```

Plot observed yields and term-premium estimates for selected maturities.

```
ggplot(total_TP_to_plot_KWACM %>%
  filter(maturity %in% c(1,5,10),
         date >= "Jan 1990") %>%
  mutate(maturity = factor(
    paste0(maturity, "-year maturity"),
    levels = paste0(c(1,5,10), "-year maturity"))),
  aes(x = date, y = value, col = model, linetype = model)) +
  geom_line() +
  facet_wrap(~ maturity) +
  theme_baseR() +
  scale_color_manual(values = c("grey65", "black", "black")) +
  scale_linetype_manual(values = c(1,1,2)) +
  theme(axis.text.x = element_text(angle = 45, hjust = 1, size = 13),
        axis.text.y = element_text(size = 13),
        strip.text.x = element_text(size = 16),
        legend.text = element_text(size = 14),
        legend.position = "bottom",
        legend.title = element_blank()) +
  labs(x = "", y = "")
```



The next block turns to inflation-linked securities and prepares the nominal, real, and breakeven-rate series used in the corresponding stylized facts.

Purpose: Prepare nominal, real, and breakeven-rate series for inflation-linked securities.

```
library(zoo)
# Reshape nominal and TIPS data by maturity.
GSW_Data_tibb <- DAT_GSW %>% as_tibble() %>%
  mutate(date = as.yearmon(date)) %>%
  pivot_longer(cols = SVENY01:TIPSY20) %>%
```



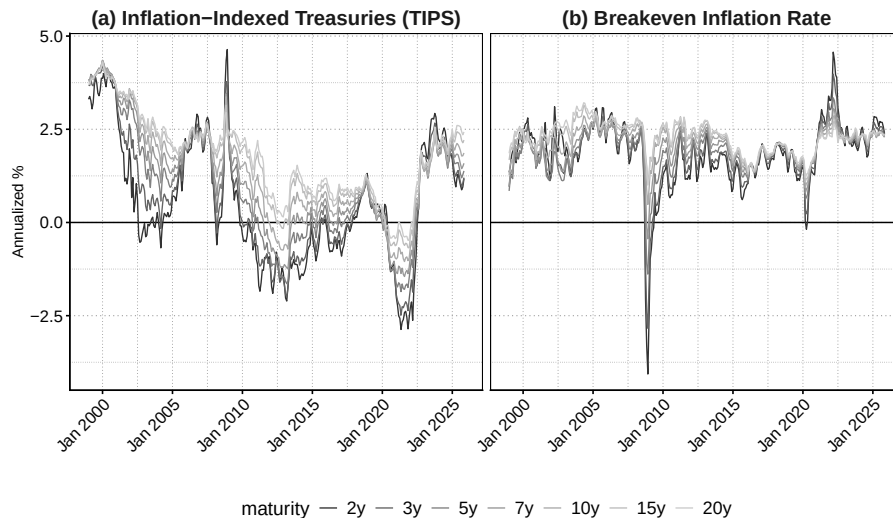
```

mutate(maturity = parse_number(name),
       `bond type` = str_extract(name, "[A-Za-z]+")) %>%
mutate(`bond type` = if_else(`bond type`=="SVENY", "Nominal", "TIPS")) %>%
dplyr::select(-name) %>%
relocate(value, .after = `bond type`)

# Add breakeven-inflation rates and put all series in long form.
GSW_Data_tibb <- GSW_Data_tibb %>%
  arrange(date, maturity, `bond type`) %>%
  group_by(date, maturity) %>%
  mutate(BEI = value - dplyr::lead(value)) %>%
  ungroup() %>% pivot_longer(cols = c(value, BEI),
                             names_to = "bond_type_new",
                             values_to = "value") %>%
  mutate(`bond type` = if_else(bond_type_new == "BEI", "BEI", `bond type`)) %>%
  dplyr::select(-bond_type_new) %>%
  drop_na()

library(ggplot2)
# Plot TIPS yields and breakeven-inflation rates.
ggplot(GSW_Data_tibb %>%
       filter(maturity %in% c(2,3,5,7,10,15,20),
              `bond type` != "Nominal") %>%
       mutate(`bond type` = if_else(
         `bond type` == "BEI",
         "(b) Breakeven Inflation Rate",
         "(a) Inflation-Indexed Treasuries (TIPS)"
       ),
              maturity = factor(maturity, labels = paste0(c(2,3,5,7,10,15,20),"y"))),
       aes(x = date, y = value, col = maturity, group = maturity)) +
  geom_hline(yintercept = 0) +
  geom_line(alpha = 1, lwd = .5) +
  theme_baseR() +
  facet_grid(~`bond type`) +
  theme(axis.text.x = element_text(angle = 45, hjust = 1, size = 13),
        axis.text.y = element_text(size = 13),
        strip.text = element_text(size = 16),
        legend.text = element_text(size = 14),
        legend.title = element_text(size = 15),
        legend.position = "bottom") +
  guides(color = guide_legend(nrow = 1)) +
  scale_color_grey(start = 0.2, end = 0.8) +
  labs(y = "Annualized %", x = "")

```



4.17 Market-based and survey-based expectations for short rates, long rates, and inflation

This example compares market-based and survey-based expectations for short rates, long rates, and inflation.

```
# Purpose: This example compares market-based and survey-based expectations for short
# long rates, and inflation
library(ggtext)
# Merge survey forecasts with market-implied forward rates.
SPF <- SPF %>% mutate(date = as.yearmon(date))

survey_plot <- YC_LW_FULL %>%
  mutate(date = as.yearmon(date)) %>%
  mutate(`3m_in_12m` = 1/3 * (15 * yld_15m - 12 * yld_12m)) %>%
  left_join(SPF)

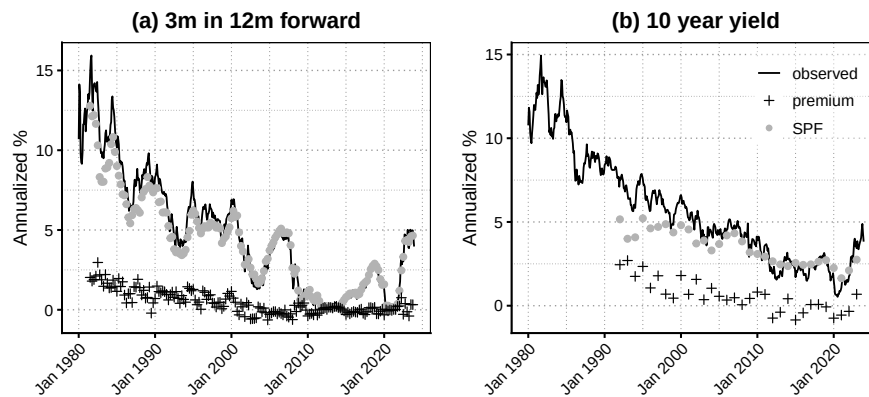
data_legend <- tibble(
  x = as.yearmon("Jan 2000"),
  y = NA,
  shape = c(0,3,1),
  linetype = c(1,0,0),
  name = c("observed", "SPF", "premium"),
  color = c("black", "grey70", "black")
)
```

4.17. MARKET-BASED AND SURVEY-BASED EXPECTATIONS FOR SHORT RATES, LONG RATES, AND IN

```
# Plot forward rates, survey forecasts, and their wedge.
p <- ggplot(survey_plot %>%
  filter(date >= "Jan 1980"), aes(x=date)) +
  geom_line(aes(y = `3m_in_12m`)) +
  geom_point(aes(y = BILL1), col = "grey70") +
  geom_point(aes(y = `3m_in_12m` - BILL1), col = "black", pch = 3) +
  theme_baseR() +
  labs(x = "", y = "Annualized %", title = "(a) 3m in 12m forward") +
  theme(axis.text.x = element_text(angle = 45, hjust = 1),
    strip.text = element_text(size = 11),
    plot.title = element_text(hjust = .5, face="bold"))

q <- ggplot(survey_plot %>%
  filter(date >= "Jan 1980"), aes(x=date)) +
  geom_line(aes(y = yld_120m)) +
  geom_point(aes(y = BILL10), col = "grey70") +
  geom_point(aes(y = yld_120m - BILL10), col = "black", pch = 3) +
  theme_baseR() +
  labs(x = "", y = "Annualized %", title = "(b) 10 year yield") +
  theme(axis.text.x = element_text(angle = 45, hjust = 1),
    strip.text = element_text(size = 11),
    plot.title = element_text(hjust = .5, face="bold")) +
  geom_line(data = data_legend, aes(x=x, y=y, colour = name, linetype= name, fill = name)) +
  geom_point(data = data_legend, aes(x=x, y=y, shape = name, color = name, fill = name)) +
  scale_color_manual(values = c("black", "black", "grey70")) +
  scale_fill_manual(values = c("black", "black", "grey70")) +
  scale_shape_manual(values = c(NA,3,16)) +
  scale_linetype_manual(values= c(1, NA,NA)) +
  theme(legend.title = element_blank(),
    legend.position = "inside",
    legend.position.inside = c(.8,.8),
    legend.background = element_rect(fill = alpha("white", 0.4)))

p + q
```



```
# Build the inflation-linked data set for the survey comparison.
```

```
inflation_survey_plots <- GSW_Data_tibb %>%
```

```
  filter(`bond type` != "Nominal") %>%
```

```
  pivot_wider(names_from = c(`bond type`, maturity)) %>%
```

```
  left_join(SPF)
```

```
inflation_survey_plots <- inflation_survey_plots %>%
```

```
  mutate(SPF_2_proxy = 9/10 * CPI1 + 1/10 * CPI10)
```

```
# Plot breakeven inflation, survey inflation, and their wedge.
```

```
p <- ggplot(inflation_survey_plots, aes(x=date)) +
```

```
  geom_line(aes(y = BEI_2)) +
```

```
  geom_point(aes(y = SPF_2_proxy), col = "grey70") +
```

```
  geom_point(aes(y = BEI_2 - SPF_2_proxy), col = "black", pch = 3) +
```

```
  theme_baseR() +
```

```
  labs(x = "", y = "Annualized %", title = "(a) 2-year BEI") +
```

```
  geom_hline(yintercept = 0, lty = 3) +
```

```
  theme(axis.text.x = element_text(angle = 45, hjust = 1),
```

```
        strip.text = element_text(size = 11),
```

```
        plot.title = element_text(hjust = .5, face="bold")) +
```

```
  geom_line(data = data_legend, aes(x=x, y=y, colour = name, linetype= name, fill = name,
```

```
  geom_point(data = data_legend, aes(x=x, y=y, shape = name, color = name, fill = name,
```

```
  scale_color_manual(values = c("black", "black", "grey70")) +
```

```
  scale_fill_manual(values = c("black", "black", "grey70")) +
```

```
  scale_shape_manual(values = c(NA,3,16)) +
```

```
  scale_linetype_manual(values= c(1, NA,NA)) +
```

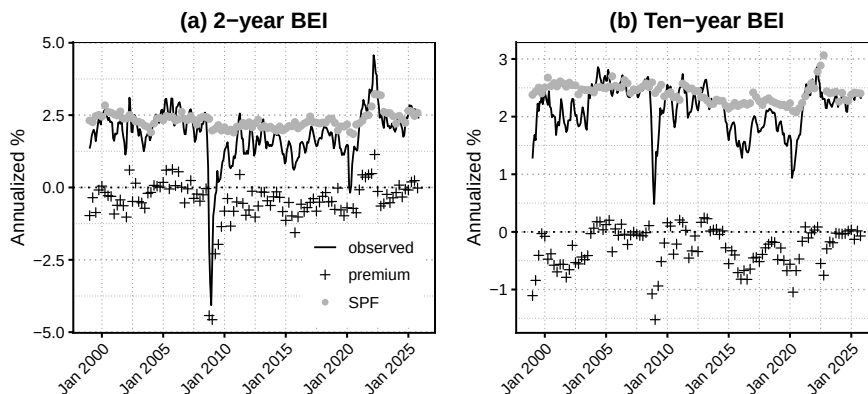
```
  theme(legend.title = element_blank(),
```

```
        legend.position = "inside",
```

```
        legend.position.inside = c(0.8,.2))
```

4.18. A SIMPLE AR(1) SHORT-RATE MODEL CAN GENERATE AVERAGE YIELD CURVES AND TERM PREMIA

```
q <- ggplot(inflation_survey_plots, aes(x=date)) +
  geom_line(aes(y = BEI_10 )) +
  geom_point(aes(y = CPI10), col = "grey70") +
  geom_point(aes(y = BEI_10 - CPI10), col = "black", pch = 3)+
  theme_baseR() +
  labs(x = "", y = "Annualized %", title = "(b) Ten-year BEI") +
  geom_hline(yintercept = 0, lty = 3)+
  theme(axis.text.x = element_text(angle = 45, hjust = 1),
        strip.text = element_text(size = 11),
        plot.title = element_text(hjust = .5, face="bold"),
        legend.background = element_rect(fill = alpha("white", 0.4))
  )
p + q
```



4.18 A simple AR(1) short-rate model can generate average yield curves and term premia

This example shows how a simple AR(1) short-rate model can generate average yield curves and term premia.

```
# Purpose: This example shows how a simple AR(1) short-rate model can generate average yield
# curves and term premia
# Calibrate the physical and risk-neutral AR(1) short-rate model.
r_bar <- .03
rho <- .9
sigma <- sqrt(1-rho^2) * 0.01
lambda0 <- - 0.01/sigma^2 * (1 - rho)
lambda1 <- (.97 - rho)/sigma^2
```

```

model <- list(r_bar = r_bar,
             rho = rho,
             sigma = sigma,
             lambda0 = lambda0,
             lambda1 = lambda1)

H <- 200

# Compute affine loadings for nominal zero-coupon bond prices.
make_ab <- function(model,H){
  all_a <- NULL
  all_b <- NULL
  a <- 0
  b <- 0
  for(i in 1:H){
    b <- b + a*((1-model$rho)*model$r_bar + model$lambda0 * model$sigma^2) +
      a^2*model$sigma^2/2
    a <- a*(model$rho + model$lambda1 * model$sigma^2) - 1

    all_a <- c(all_a,a)
    all_b <- c(all_b,b)
  }
  return(list(all_a = all_a,
             all_b = all_b))
}

ab <- make_ab(model,H)
# Recompute the same objects under the expectations hypothesis.
model_EH <- model
model_EH$lambda0 <- 0
model_EH$lambda1 <- 0
ab_EH <- make_ab(model_EH,H)

Er <- r_bar
Vr <- sigma^2/(1-rho^2)

ErQ <- ((1-rho)*r_bar + lambda0*sigma)/(1 - (rho + lambda1*sigma))

avg_r <- -(ab$all_a*Er + ab$all_b)/1:H
std_r <- abs(ab$all_a)/1:H * sqrt(Vr)
lower <- avg_r - 2*std_r
upper <- avg_r + 2*std_r

# Plot average yield curves and unconditional dispersion bands.
par(plt=c(.1,.95,.2,.95))

```

4.18. A SIMPLE AR(1) SHORT-RATE MODEL CAN GENERATE AVERAGE YIELD CURVES AND TERM PRE

```

plot(avg_r,type="l",col="white",
      ylim=c(.0,.06),xlab="maturity",ylab="",las=1)
polygon(c(1:H,H:1),c(lower,rev(upper)),col="#CCCCCBB",border = NaN)

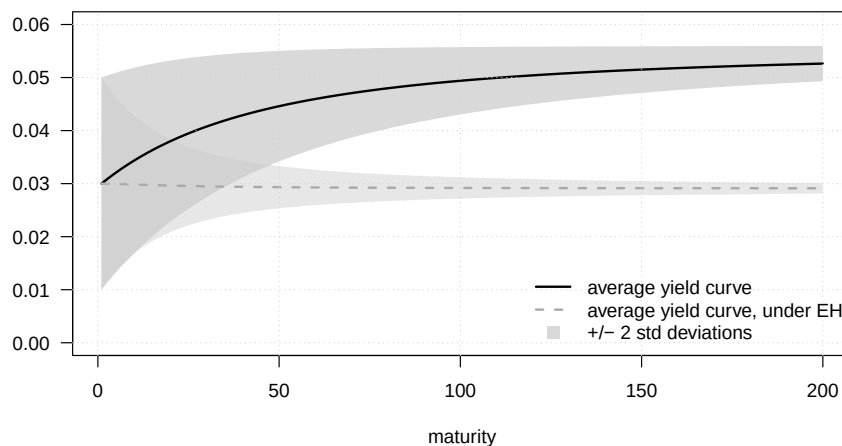
avg_r_EH <- -(ab_EH$all_a*Er + ab_EH$all_b)/1:H
std_r_EH <- abs(ab_EH$all_a)/1:H * sqrt(Vr)
lower <- avg_r_EH - 2*std_r_EH
upper <- avg_r_EH + 2*std_r_EH
polygon(c(1:H,H:1),c(lower,rev(upper)),col="#CCCCC77",border = NaN)

lines(-(ab$all_a*Er + ab$all_b)/1:H,col="black",lwd=2)
lines(-(ab_EH$all_a*Er + ab_EH$all_b)/1:H,col="dark grey",lwd=2,lty=2)

grid()

legend("bottomright",
      legend=c("average yield curve",
               "average yield curve, under EH",
               "+/- 2 std deviations"),
      lty=c(1,2,NaN),
      col=c("black","dark grey","#CCCCCBB"),
      pch=c(NaN,NaN,15),
      pt.cex = c(NaN,NaN,1.5),
      # cex=1.5,
      lwd=c(2,2),
      bty = "n")

```



```
# abline(h=ErQ,lty=3)
```

4.19 Short-rate simulations and term premia in the AR(1) model

This example simulates short-rate paths and the associated term premium in the AR(1) model.

```
# Purpose: This example simulates short-rate paths and the associated term premium in
# model
# Simulate a short-rate path under the calibrated AR(1) model.
T <- 150
h <- 10
set.seed(1111)

model_lambda1_0 <- model
model_lambda1_0$lambda0 <- model$lambda0 + model$lambda1*r_bar
model_lambda1_0$lambda1 <- 0
ab_lambda1_0 <- make_ab(model_lambda1_0,H)

# Helper used to simulate physical short-rate dynamics.
simul_r <- function(model,T){
  r <- r_bar
  all_r <- NULL
  for(t in 1:T){
    r <- (1 - model$rho)*model$r_bar + model$rho*r + model$sigma*rnorm(1)
    all_r <- c(all_r,r)
  }
  return(all_r)
}

sim_r <- simul_r(model,T)

# Compute term premia with and without state-dependent prices of risk.
TP <- (matrix(ab$all_a, ncol = 1) %*% matrix(sim_r, nrow = 1) + ab$all_b) -
  (matrix(ab_EH$all_a, ncol = 1) %*% matrix(sim_r, nrow = 1) + ab_EH$all_b)

TP_lambda1_0 <- (
  matrix(ab_lambda1_0$all_a, ncol = 1) %*% matrix(sim_r, nrow = 1) +
  ab_lambda1_0$all_b
) -
  (matrix(ab_EH$all_a, ncol = 1) %*% matrix(sim_r, nrow = 1) + ab_EH$all_b)

# Plot the simulated short rate and the induced term premium.
par(mfrow=c(1,2))
par(plt=c(.2,.95,.15,.85))
plot(sim_r*100,type="l",lwd=2,las=1,xlab="",ylab="in percent",
      main=expression(paste("(a) Short term rate ",r[t],sep="")))
```



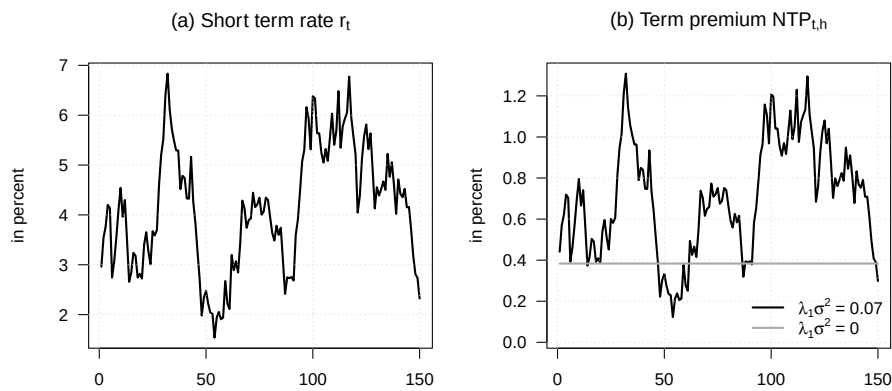
```

grid()

plot(-TP[h,]*100/h,type="l",lwd=2,las=1,xlab="",ylab="in percent",
     ylim=c(min(-TP[h,]*100/h)-.1,max(-TP[h,]*100/h)),
     main=expression(paste("(b) Term premium ",NTP[t*','*h],sep="")))
grid()
lines(-TP_lambda1_0[h,]*100/h,col="dark grey",lwd=2)

legend("bottomright",
      legend=c(expression(paste(lambda[1] * sigma^2," = 0.07" ,sep="")),
                expression(paste(lambda[1] * sigma^2," = 0" , ,sep="))),
      lty=c(1,1),
      col=c("black","dark grey"),
      pch=NaN,
      # cex=1.5,
      lwd=c(2,2),bty = "n",bg="white")

```



4.20 Nominal, real, and breakeven curves across states

This example compares nominal, real, and breakeven-inflation curves under different inflation and short-rate states.

Purpose: This example compares nominal, real, and breakeven-inflation curves under different inflation and short-rate states

Enumerate inflation-persistence and current-state configurations.

```

parameters <- expand.grid(
  r_bar = .03,
  rho   = .9,
  sigma = sigma,

```

```

pi_bar = .025,
phi     = c(.8,.95),
current_r = c(0.02, 0.04),
current_pi = c(0.01, 0.025),
lambda0 = 0,
lambda1 = 0
)

parameters <- parameters %>%
  mutate(omega = sqrt(1 - phi^2) * 0.01)

H <- 100

# Compute nominal, real, and breakeven curves for one configuration.
make_ab <- function(model,H){

  all_a_nom <- NULL
  all_b_nom <- NULL
  a_nom <- 0
  b_nom <- 0

  all_a_real <- NULL
  all_b_real <- NULL
  a_real <- c(0,0)
  b_real <- 0

  for(i in 1:H){
    b_nom <- b_nom + a_nom * ((1 - model$rho) * model$r_bar + model$lambda0 * model$sigma^2/2)
    a_nom <- a_nom * (model$rho + model$lambda1 * model$sigma^2) - 1

    all_a_nom <- c(all_a_nom, a_nom)
    all_b_nom <- c(all_b_nom, b_nom)

    b_real <- b_real + a_real[1] * (1 - model$rho) * model$r_bar +
      (a_real[2] + 1) * model$pi_bar * (1 - model$phi) +
      a_real[1]^2 * model$sigma^2/2 + (a_real[2] + 1)^2 * model$omega^2/2
    a_real <- c(
      a_real[1] * model$rho - 1,
      (a_real[2] + 1) * model$phi
    )

    all_a_real <- cbind(all_a_real, a_real)
    all_b_real <- c(all_b_real, b_real)
  }
}

```

4.20. NOMINAL, REAL, AND BREAK-EVEN CURVES ACROSS STATES⁶⁷

```

}

# Construct yield curves
nominal_YC <- - 1/c(1:H) * (all_b_nom + all_a_nom * model$current_r)
real_YC    <- - 1/c(1:H) * (all_b_real +
                           c(t(all_a_real) %*% c(model$current_r,
                                                  model$current_pi)))

return(list(all_a_nom = all_a_nom,
            all_b_nom = all_b_nom,
            all_a_real = all_a_real,
            all_b_real = all_b_real,
            nominal_YC = nominal_YC,
            real_YC    = real_YC,
            BEI        = nominal_YC - real_YC))
}

# Apply the curve calculation to all configurations.
loadings <- parameters %>%
  mutate(case = paste0("phi = ", phi, ", r(t) = ", current_r, ", pi(t) = ", current_pi)) %>%
  relocate(case, .before = r_bar) %>%
  group_by(case) %>%
  nest() %>%
  mutate(
    models = map(data, ~ make_ab(., H))
  ) %>% ungroup()

# Reshape curves and build facet labels.
yield_curves <- loadings %>% unnest() %>%
  mutate(element = names(models)) %>%
  relocate(element, .before = r_bar) %>%
  dplyr::select(case, element, models) %>%
  separate(col = case,
           into = c("case_phi", "case_current"),
           sep = ",\\s*", # split at the first comma
           extra = "merge") # keep the remaining commas)

YC_example_to_plot <- yield_curves %>%
  filter(element %in% c("nominal_YC", "real_YC", "BEI")) %>%
  mutate(x_axis = map(models, seq_along)) %>%
  unnest(c(models, x_axis)) %>%
  mutate(
    label_phi = paste0("phi == ", readr::parse_number(case_phi)),

```

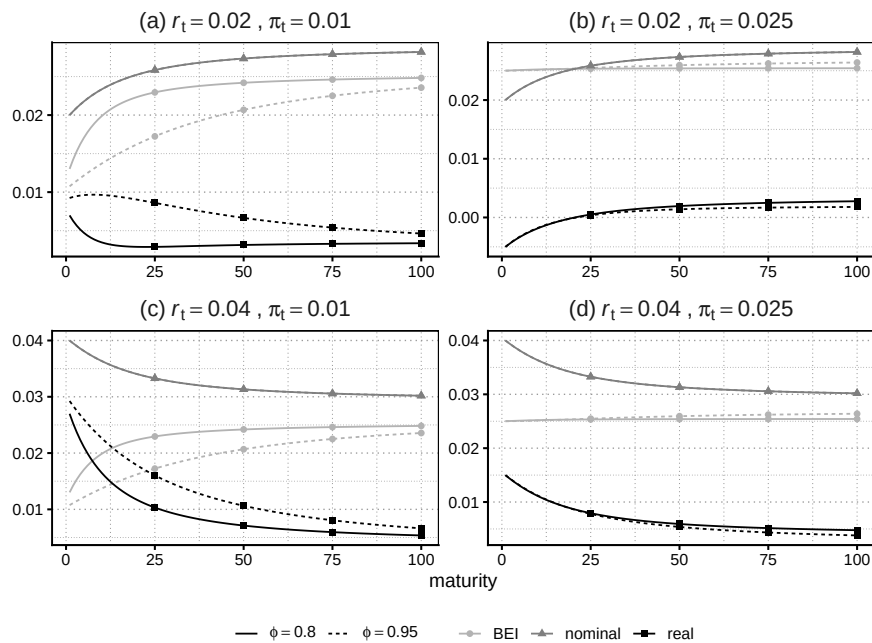
```

label_math = str_replace_all(case_current, " = ", " == "),
label_math = str_replace(label_math, "r\\(t\\)", "italic(r)[t]"),
label_math = str_replace(label_math, "pi\\(t\\)", "pi[t]"),
label_math = str_replace(label_math, ", ", " ~ ',' ~ "),
label_math = if_else(case_current == "r(t) = 0.02, pi(t) = 0.01",
  paste("'(a)' ~", label_math), label_math),
label_math = if_else(case_current == "r(t) = 0.02, pi(t) = 0.025",
  paste("'(b)' ~", label_math), label_math),
label_math = if_else(case_current == "r(t) = 0.04, pi(t) = 0.01",
  paste("'(c)' ~", label_math), label_math),
label_math = if_else(case_current == "r(t) = 0.04, pi(t) = 0.025",
  paste("'(d)' ~", label_math), label_math),
)

# Plot nominal, real, and breakeven curves across configurations.
ggplot(YC_example_to_plot,
  aes(x = x_axis, y = models, col = element,
    linetype = label_phi)) +
  geom_line() +
  geom_point(data =
    YC_example_to_plot %>%
    group_by(label_math, label_phi, element) %>%
    slice(seq(0, n(), 25)) %>%
    ungroup(),
    aes(x = x_axis, y = models, col = element, shape = element)) +
  facet_wrap(. ~ label_math, scales = "free",
    labeller = label_parsed) +
  labs(x = "maturity", y = "", col = "Curve", linetype = "") +
  theme_baseR() +
  scale_color_manual(values = c("grey70", "grey50", "black"),
    labels = function(x) sub("_.*", "", x)) +
  scale_shape_discrete(name = "Curve") +
  scale_linetype_discrete(labels = function(x) parse(text = x)) +
  theme(legend.position = "bottom",
    legend.title = element_blank()) +
  guides(
    color = guide_legend(override.aes = list(shape = c(16,17,15))),
    shape = "none" # hide separate shape legend
  )

```

4.21. NOMINAL AND REAL YIELD CURVES WITH AND WITHOUT RISK PREMIA69



```
# abline(h=ErQ,lty=3)
```

4.21 Nominal and real yield curves with and without risk premia

This example contrasts average nominal and real yield curves with and without risk premia.

```
# Purpose: This example contrasts average nominal and real yield curves with and without risk
# premia
# Calibrate nominal and real prices of risk.
parameters <- list(
  r_bar = .03,
  rho = .9,
  sigma = sigma,
  pi_bar = .025,
  phi = .95,
  omega = sqrt(1 - .95^2) * 0.01,
  lambda0_n = - 0.01/sigma^2 * (1 - rho),
  lambda1_n = (.97 - rho)/sigma^2 ,
  lambda0_r = -(1 - .95) * 0.015 / (sqrt(1 - .95^2) * 0.01)^2,
  lambda1_r = c(-0.02,0.04)/c(sigma^2, (sqrt(1 - .95^2) * 0.01)^2),
```

```

    current_r = 0.03,
    current_pi = .025
  )

H <- 200

# Compute nominal and real affine loadings for this calibration.
make_ab <- function(model,H){

  all_a_nom <- NULL
  all_b_nom <- NULL
  a_nom      <- 0
  b_nom      <- 0

  all_a_real <- NULL
  all_b_real <- NULL
  a_real     <- c(0,0)
  b_real     <- 0

  for(i in 1:H){
    b_nom <- b_nom + a_nom * ((1 - model$rho) * model$r_bar + model$lambda0_n * model$
      a_nom^2 * model$sigma^2/2
    a_nom <- a_nom * (model$rho + model$lambda1_n * model$sigma^2) - 1

    all_a_nom <- c(all_a_nom, a_nom)
    all_b_nom <- c(all_b_nom, b_nom)

    b_real <- b_real +
      a_real[1] * ((1 - model$rho) * model$r_bar +
        model$lambda0_n * model$sigma^2) +
      (a_real[2] + 1) * (model$pi_bar * (1 - model$phi) + model$omega^2 * model$lambda
      a_real[1]^2 * model$sigma^2/2 + (a_real[2] + 1)^2 * model$omega^2/2

    a_real <- c(
      a_real[1] * (model$rho + model$sigma^2 * model$lambda1_n) - 1 +
      (a_real[2] + 1) * model$omega^2 * model$lambda1_r[1],
      (a_real[2] + 1) * (model$phi + model$omega^2 * model$lambda1_r[2])
    )

    all_a_real <- cbind(all_a_real, a_real)
    all_b_real <- c(all_b_real, b_real)
  }

  # Construct yield curves
  nominal_YC <- - 1/c(1:H) * (all_b_nom + all_a_nom * model$current_r)

```

```

real_YC      <- - 1/c(1:H) * (all_b_real +
                             c(t(all_a_real) %*% c(model$current_r,
                                                    model$current_pi)))

return(list(all_a_nom = all_a_nom,
            all_b_nom = all_b_nom,
            all_a_real = all_a_real,
            all_b_real = all_b_real,
            nominal_YC = nominal_YC,
            real_YC    = real_YC,
            BEI        = nominal_YC - real_YC))
}

example_curves <- make_ab(parameters, H)

# Recompute curves under the expectations hypothesis.
parameters_EH <- parameters
parameters_EH$lambda0_n <- 0
parameters_EH$lambda1_n <- 0
parameters_EH$lambda0_r <- 0
parameters_EH$lambda1_r <- c(0,0)

example_curves_EH <- make_ab(parameters_EH, H)

# Construct dispersion bands from the unconditional factor variances.
uncondi_var_facs <- diag(c(parameters$sigma^2/(1-parameters$rho^2),
                           parameters$omega^2/(1-parameters$phi^2)))
std_r    <- abs(example_curves$all_a_nom)/1:H * sqrt(uncondi_var_facs[1,1])
std_pi   <- apply(sqrt(uncondi_var_facs) %*%
                  abs(example_curves$all_a_real)/matrix(1:H, nrow = 2, ncol = H, byrow = T) ,2,s

std_r_EH  <- abs(example_curves_EH$all_a_nom)/1:H * sqrt(uncondi_var_facs[1,1])
std_pi_EH <- apply(sqrt(uncondi_var_facs) %*%
                  abs(example_curves_EH$all_a_real) /
                  matrix(1:H, nrow = 2, ncol = H, byrow = T),
                  2, sum)

# Put curves and confidence bands in long format for plotting.
examples_total <- data.frame(
  type      = rep(c(rep("(a) Nominal yield curve", H), rep("(b) Real yield curve",H)),2),
  premium   = c(rep("average yield curve", 2*H), rep("average yield curve, under EH", 2*H)),
  maturity  = rep(1:H,4),

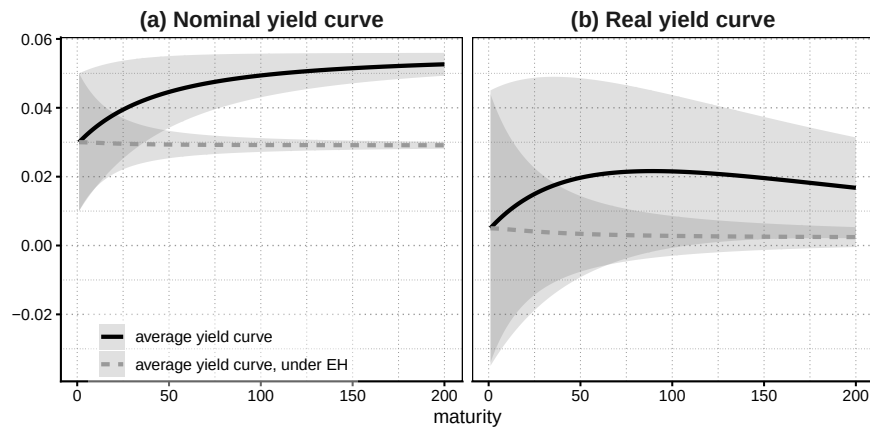
```

```

value      = c(example_curves$nominal_YC,
                example_curves$real_YC,
                example_curves_EH$nominal_YC,
                example_curves_EH$real_YC),
ci_low     = c(example_curves$nominal_YC - 2 * std_r,
                example_curves$real_YC - 2 * std_pi,
                example_curves_EH$nominal_YC - 2 * std_r_EH,
                example_curves_EH$real_YC - 2 * std_pi_EH),
ci_high    = c(example_curves$nominal_YC + 2 * std_r,
                example_curves$real_YC + 2 * std_pi,
                example_curves_EH$nominal_YC + 2 * std_r_EH,
                example_curves_EH$real_YC + 2 * std_pi_EH)
) %>% as_tibble()

# Plot average nominal and real curves with and without risk premia.
ggplot(examples_total,
       aes(x = maturity, y = value, col = premium,
           group = premium, linetype = premium)) +
  geom_ribbon(aes(ymin = ci_low, ymax = ci_high), alpha = 0.2, fill = "grey40", col = "black") +
  geom_line(lwd = 1.25) +
  facet_grid(~type) +
  scale_color_manual(values = c("black", "grey60")) +
  theme_baseR() +
  labs(y = "") +
  theme(legend.title = element_blank(),
        legend.direction = "vertical",
        legend.position = c(0.2, 0.09),
        legend.background = element_rect(fill = alpha("white", 0.4)))

```



Purpose: This example simulates time variation in nominal, real, and inflation risk premia

[illegible]

```
# Simulate short-rate and inflation paths.
```

```
set.seed(2234)
```

```
chosen mat <- 50
```

```
# Compute nominal, real, breakeven, and premium series at one maturity.
```

[illegible]

```

                                c(example_curves_sim$all_a_real[,chosen_mat]))
TIPS_EH_sim      <- -1/chosen_mat * c((example_curves_sim_EH$all_b_real[chosen_mat] +
                                cbind(rt_sim, pit_sim) %*%
                                c(example_curves_sim_EH$all_a_real[,chosen_mat]

sim_to_plot <- tibble(x = 1:length_sim,
                      "short-rate" = rt_sim,
                      "inflation rate" = pit_sim,
                      "ex post real rate" = rt_sim - pit_sim,
                      "nominal curve" = nom_YC_sim,
                      "nominal EH" = nom_YC_EH_sim,
                      "TIPS curve" = TIPS_sim,
                      "TIPS EH" = TIPS_EH_sim,
                      "nominal term premium" = nom_YC_sim - nom_YC_EH_sim,
                      "real term premium" = TIPS_sim - TIPS_EH_sim,
                      "BEI" = nom_YC_sim - TIPS_sim,
                      "BEI EH" = nom_YC_EH_sim - TIPS_EH_sim,
                      "IRP" = nom_YC_sim - TIPS_sim - (nom_YC_EH_sim - TIPS_EH_sim)
)

# Plot the associated curves and term-premium components.
op <- par(mfrow = c(1, 3), mar = c(3.4, 3.8, 2.7, 0.8), las = 1,
          cex.axis = 1.15)

plot_component <- function(yield_series, premium_series, main_title,
                           yield_label, premium_label,
                           y_min = NULL) {
  ylim <- range(c(yield_series, premium_series), na.rm = TRUE)
  if (!is.null(y_min)) {
    ylim[1] <- y_min
  }
  plot(
    sim_to_plot$x, yield_series,
    type = "n", xlab = "", ylab = "", ylim = ylim,
    main = main_title, cex.main = 1.25
  )
  grid()
  abline(h = 0, col = "grey80", lwd = 0.8)
  lines(sim_to_plot$x, premium_series, col = "grey70", lwd = 2)
  lines(sim_to_plot$x, yield_series, col = "black", lwd = 2)
  legend(
    "bottomleft",
    legend = c(premium_label, yield_label),
    col = c("grey70", "black"),
    lty = 1, lwd = 2, bty = "n",

```

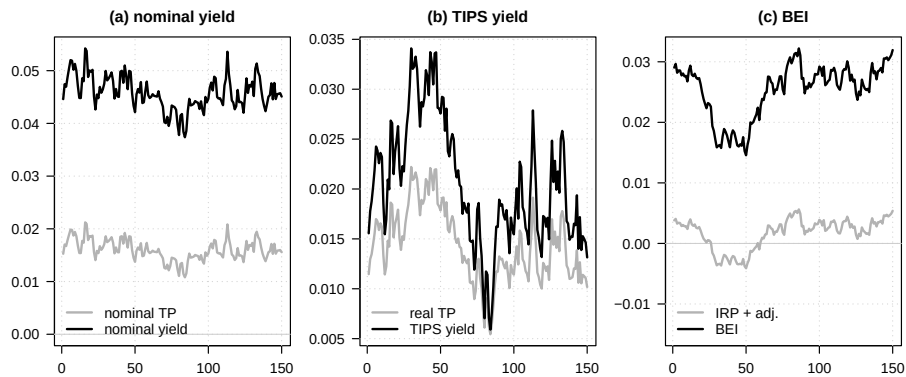
4.22. TIME VARIATION IN NOMINAL, REAL, AND INFLATION RISK PREMIA75

```

    cex = 1.1
  )
}

plot_component(
  sim_to_plot$`nominal curve`,
  sim_to_plot$`nominal term premium`,
  "(a) nominal yield",
  "nominal yield",
  "nominal TP",
  y_min = 0
)
plot_component(
  sim_to_plot$`TIPS curve`,
  sim_to_plot$`real term premium`,
  "(b) TIPS yield",
  "TIPS yield",
  "real TP"
)
plot_component(
  sim_to_plot$BEI,
  sim_to_plot$IRP,
  "(c) BEI",
  "BEI",
  "IRP + adj.",
  y_min = -0.015
)

```



```
par(op)
```

*# Purpose: Check the exact decomposition of the nominal term premium in the
simulated nominal-real example.*

```

state_sim <- cbind(rt_sim, pit_sim)

state_dynamics <- function(model, measure = c("P", "Q")) {
  measure <- match.arg(measure)
  mu <- c((1 - model$rho) * model$r_bar,
        (1 - model$phi) * model$pi_bar)
  Phi <- matrix(c(model$rho, 0,
                 0, model$phi),
               nrow = 2, byrow = TRUE)
  if (measure == "Q") {
    mu <- mu + c(model$sigma^2 * model$lambda0_n,
                model$omega^2 * model$lambda0_r)
    Phi <- Phi + matrix(c(model$sigma^2 * model$lambda1_n, 0,
                        model$omega^2 * model$lambda1_r[1],
                        model$omega^2 * model$lambda1_r[2]),
                      nrow = 2, byrow = TRUE)
  }
  list(mu = mu, Phi = Phi,
       Sigma = diag(c(model$sigma^2, model$omega^2)))
}

log_inflation_component <- function(model, h, states, measure = c("P", "Q")) {
  dyn <- state_dynamics(model, measure)
  c_h <- c(0, 0)
  d_h <- 0
  pi_loading <- c(0, 1)
  for (i in seq_len(h)) {
    loading <- pi_loading + c_h
    d_h <- d_h +
      sum(loading * dyn$mu) +
      0.5 * drop(t(loading) %*% dyn$Sigma %*% loading)
    c_h <- drop(t(dyn$Phi) %*% loading)
  }
  drop(states %*% c_h + d_h) / h
}

eh_yield_component <- function(model, h, states, delta0, delta1) {
  dyn <- state_dynamics(model, "P")
  a_h <- c(0, 0)
  b_h <- 0
  for (i in seq_len(h)) {
    b_h <- -delta0 +
      sum(a_h * dyn$mu) +
      0.5 * drop(t(a_h) %*% dyn$Sigma %*% a_h) +
      b_h
  }
}

```

4.22. TIME VARIATION IN NOMINAL, REAL, AND INFLATION RISK PREMIA77

```

    a_h <- -delta1 + drop(t(dyn$Phi) %*% a_h)
  }
  -drop(states %*% a_h + b_h) / h
}

lambda_pi_t <- parameters_sim$lambda0_r +
  parameters_sim$lambda1_r[1] * rt_sim +
  parameters_sim$lambda1_r[2] * pit_sim
real_short_sim <- rt_sim -
  ((1 - parameters_sim$phi) * parameters_sim$pi_bar +
   parameters_sim$phi * pit_sim) -
  parameters_sim$omega^2 * lambda_pi_t -
  0.5 * parameters_sim$omega^2

real_short_delta0 <- - (1 - parameters_sim$phi) * parameters_sim$pi_bar -
  parameters_sim$omega^2 * parameters_sim$lambda0_r -
  0.5 * parameters_sim$omega^2
real_short_delta1 <- c(1 - parameters_sim$omega^2 * parameters_sim$lambda1_r[1],
  -parameters_sim$phi -
  parameters_sim$omega^2 * parameters_sim$lambda1_r[2])

nom_EH_check <- eh_yield_component(parameters_sim, chosen_mat, state_sim,
  delta0 = 0, delta1 = c(1, 0))
real_EH_check <- eh_yield_component(parameters_sim, chosen_mat, state_sim,
  delta0 = real_short_delta0,
  delta1 = real_short_delta1)
PiP_check <- log_inflation_component(parameters_sim, chosen_mat, state_sim, "P")
PiQ_check <- log_inflation_component(parameters_sim, chosen_mat, state_sim, "Q")

NTP_check <- nom_YC_sim - nom_EH_check
RTP_check <- TIPS_sim - real_EH_check
IRP_check <- PiQ_check - PiP_check
CA_check <- (nom_YC_sim - TIPS_sim) - PiQ_check
RES_check <- real_EH_check + PiP_check - nom_EH_check
identity_error <- NTP_check - (RTP_check + IRP_check + CA_check + RES_check)

decomp_check_table <- data.frame(
  component = c("NTP", "RTP", "IRP", "CA", "RES", "identity error"),
  mean_bps = 10000 * c(mean(NTP_check), mean(RTP_check), mean(IRP_check),
    mean(CA_check), mean(RES_check), mean(identity_error)),
  sd_bps = 10000 * c(sd(NTP_check), sd(RTP_check), sd(IRP_check),
    sd(CA_check), sd(RES_check), sd(identity_error)),
  min_bps = 10000 * c(min(NTP_check), min(RTP_check), min(IRP_check),
    min(CA_check), min(RES_check), min(identity_error)),
  max_bps = 10000 * c(max(NTP_check), max(RTP_check), max(IRP_check),

```

```

max(CA_check), max(RES_check), max(identity_error))
)
decomp_check_table[, -1] <- round(decomp_check_table[, -1], 3)
decomp_check_table

##           component mean_bps sd_bps min_bps max_bps
## 1             NTP  160.103 19.612 108.027 212.248
## 2             RTP  144.179 35.829  56.485 221.263
## 3             IRP   14.074 25.194 -43.830  53.440
## 4              CA    2.789  0.000   2.789   2.789
## 5             RES   -0.939  0.850  -2.241   1.093
## 6 identity error    0.000  0.000   0.000   0.000
max(abs(nom_EH_check - nom_YC_EH_sim))

## [1] 1.040834e-17
range(real_short_sim - (real_short_delta0 + state_sim %*% real_short_delta1))

## [1] -8.673617e-19  8.673617e-18

```

Chapter 5

Gaussian affine term structure models

5.1 Macro-spanning regressions

This example illustrates a simple empirical check of the macro-spanning implication of Gaussian macro-finance term-structure models. We regress quarterly macroeconomic variables on the first three principal components of the yield curve. The exercise is deliberately descriptive: low R^2 values do not, by themselves, reject a structural term-structure model, but they show why empirical work often introduces unspanned macroeconomic variables.

*# Purpose: This example checks how much macroeconomic variation is spanned by
the first three empirical yield-curve principal components in the U.S. and euro area.*
`library(DTAM)`

```
data("YC_US", package = "DTAM")
data("YC_Euro", package = "DTAM")
data("Data_Macro_US_quarterly", package = "DTAM")
data("Data_Macro_EA_quarterly", package = "DTAM")

quarter_id <- function(d) {
  paste0(format(d, "%Y"), "Q", ((as.integer(format(d, "%m")) - 1) %/% 3) + 1)
}

quarterly_average <- function(x, yield_cols) {
  x$q <- quarter_id(as.Date(x$date))
  out <- aggregate(x[, yield_cols], list(q = x$q), mean, na.rm = TRUE)
  for (cc in yield_cols) {
    out[[cc]][!is.finite(out[[cc]])] <- NA
  }
}
```

```

    }
    out
  }

pc_regressions <- function(yields, macro, yield_cols, macro_vars, area) {
  yq <- quarterly_average(yields[, c("date", yield_cols)], yield_cols)
  dat <- merge(macro[, c("q", macro_vars)], yq, by = "q")
  dat <- dat[complete.cases(dat[, c(macro_vars, yield_cols)]), ]
  pcs <- prcomp(dat[, yield_cols], center = TRUE, scale. = TRUE)$x[, 1:5, drop = FALSE]

  out <- data.frame()
  for (m in macro_vars) {
    for (k in c(1, 3, 5)) {
      fit <- lm(dat[[m]] ~ pcs[, 1:k])
      out <- rbind(out, data.frame(
        area = area,
        variable = m,
        pcs = k,
        r2 = summary(fit)$r.squared,
        n = nrow(dat),
        start = dat$q[1],
        end = tail(dat$q, 1)
      ))
    }
  }
  out
}

macro_us <- Data_Macro_US_quarterly
macro_us$q <- quarter_id(macro_us$date)
macro_us$inflation <- 400 * macro_us$pi
macro_us$gdp_growth <- 400 * macro_us$dy

us_yields <- c(
  "DTB3", "DTB6", "THREEFY1", "THREEFY2", "THREEFY3", "THREEFY4",
  "THREEFY5", "THREEFY6", "THREEFY7", "THREEFY8", "THREEFY9", "THREEFY10"
)

macro_ea <- Data_Macro_EA_quarterly
macro_ea$inflation <- 400 * macro_ea$pi
macro_ea$gdp_growth <- 400 * macro_ea$dy

ea_yields <- paste0("Y", 1:10)

results <- rbind(

```



```

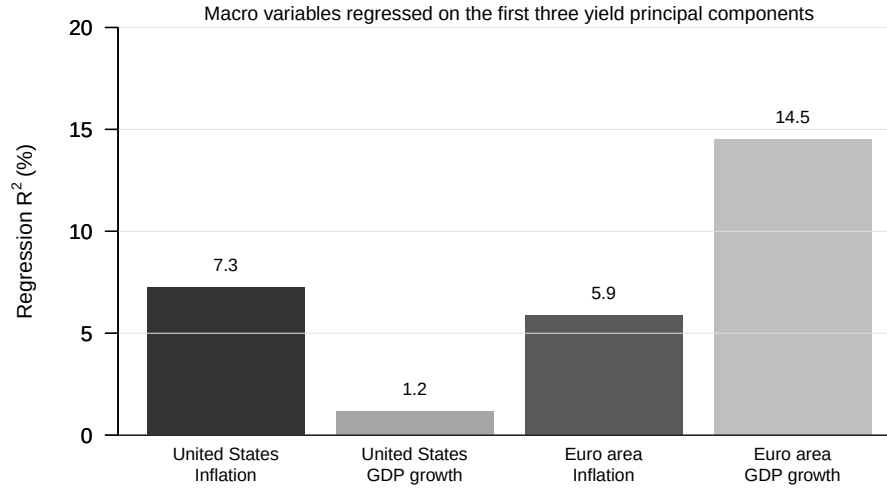
pc_regressions(YC_US, macro_us, us_yields,
               c("inflation", "gdp_growth"), "United States"),
pc_regressions(YC_Euro, macro_ea, ea_yields,
               c("inflation", "gdp_growth"), "Euro area")
)

results$variable_label <- ifelse(results$variable == "inflation", "Inflation", "GDP growth")
results$area_var <- paste(results$area, results$variable_label, sep = "\n")

plot_dat <- subset(results, pcs == 3)
plot_dat <- plot_dat[match(
  c("United States\nInflation", "United States\nGDP growth",
    "Euro area\nInflation", "Euro area\nGDP growth"),
  plot_dat$area_var
), ]

op <- par(mar = c(4.8, 4.5, 1.2, 0.8), las = 1)
bp <- barplot(
  100 * plot_dat$r2,
  names.arg = plot_dat$area_var,
  col = c("grey20", "grey65", "grey35", "grey75"),
  border = NA,
  ylim = c(0, 20),
  ylab = expression(paste("Regression ", R^2, " (%)")),
  cex.names = 0.82
)
axis(2, las = 1)
abline(h = seq(0, 20, 5), col = "grey88", lwd = 0.7)
box(bty = "l")
text(bp, 100 * plot_dat$r2 + 1.1, sprintf("%.1f", 100 * plot_dat$r2), cex = 0.86)
mtext("Macro variables regressed on the first three yield principal components",
      side = 3, line = 0.1, cex = 0.95)

```



```
par(op)
```

```
subset(results, pcs == 3)
```

```
##           area  variable pcs           r2    n  start    end variable_label
## 2  United States  inflation  3 0.07256503 142 1990Q1 2025Q2      Inflation
## 5  United States  gdp_growth 3 0.01189909 142 1990Q1 2025Q2      GDP growth
## 8    Euro area  inflation  3 0.05892463  54 2004Q3 2017Q4      Inflation
## 11   Euro area  gdp_growth  3 0.14525661  54 2004Q3 2017Q4      GDP growth
##           area_var
## 2  United States\nInflation
## 5  United States\nGDP growth
## 8    Euro area\nInflation
## 11   Euro area\nGDP growth
```

The first three yield principal components explain only a small share of quarterly inflation and real GDP growth in this simple exercise. This is the empirical tension behind the macro-spanning puzzle: standard Gaussian macro-finance term-structure models imply that a sufficiently rich cross section of yields spans the macroeconomic state, whereas low-dimensional empirical yield factors often leave substantial macroeconomic variation unexplained.

5.2 ACM-style regression estimation

This example implements a simplified version of the regression-based Gaussian affine estimation approach of Adrian, Crump, and Moench (2013). We use the dense monthly zero-coupon yield panel of Liu and Wu distributed with DTAM, extract five yield principal components, estimate the ACM regressions, and

recover the model-implied term premia. The one-month yield in the same panel is used as the short rate, and the monthly maturity grid lets us construct one-month holding returns without interpolation. We estimate the model over the complete sample for which yields are available at all maturities up to 10 years.

Purpose: This example estimates an ACM-style Gaussian affine model and plots the resulting model-implied Treasury term premia.

```
library(DTAM)

month_id <- function(d) format(as.Date(d), "%Y-%m")

maturities <- 1:120
yield_cols <- sprintf("yld_%dm", maturities)
yields <- YC_LW_FULL[, c("date", yield_cols)]
for (cc in yield_cols) {
  yields[[cc]][!is.finite(yields[[cc]])] <- NA
}

dat <- yields[complete.cases(yields[, yield_cols]), ]
Y <- as.matrix(dat[, yield_cols]) / 100
short_m <- dat$yld_1m / 100 / 12
Tobs <- nrow(Y)

pc <- prcomp(Y, center = TRUE, scale. = FALSE)
K <- 5
X <- pc$x[, 1:K, drop = FALSE]

# Step 1: factor VAR.
X_lag <- X[-Tobs, , drop = FALSE]
X_lead <- X[-1, , drop = FALSE]
var_fit <- lm(X_lead ~ X_lag)
mu <- coef(var_fit)[1, ]
Phi <- t(coef(var_fit)[-1, , drop = FALSE])
V <- residuals(var_fit)
Sigma <- crossprod(V) / nrow(V)

# Step 2: one-month excess-return regressions.
ret_mats <- seq(24, 120, by = 12)
rx <- matrix(NA_real_, Tobs - 1, length(ret_mats))
for (i in seq_along(ret_mats)) {
  n <- ret_mats[i]
  p_t <- -(n / 12) * Y[-Tobs, n]
  p_next <- -((n - 1) / 12) * Y[-1, n - 1]
  rx[, i] <- p_next - p_t - short_m[-Tobs]
}
```

```

Z <- cbind(1, X_lag, V)
coef_rx <- matrix(NA_real_, ncol(Z), length(ret_mats))
resid_rx <- matrix(NA_real_, nrow(rx), length(ret_mats))
for (i in seq_along(ret_mats)) {
  fit <- lm(rx[, i] ~ Z[, -1])
  coef_rx[, i] <- coef(fit)
  resid_rx[, i] <- resid(fit)
}

a <- coef_rx[1, ]
D <- t(coef_rx[2:(K + 1), , drop = FALSE])
Beta <- t(coef_rx[(K + 2):(2 * K + 1), , drop = FALSE])
s2 <- colMeans(resid_rx^2)
quad <- rowSums((Beta %*% Sigma) * Beta)

# Step 3: market prices of risk.
lambda0 <- as.vector(solve(crossprod(Beta), crossprod(Beta, a + 0.5 * (quad + s2))))
lambda1 <- solve(crossprod(Beta), crossprod(Beta, D))

short_fit <- lm(short_m ~ X)
delta0 <- coef(short_fit)[1]
delta1 <- as.vector(coef(short_fit)[-1])

compute_yields <- function(mu_use, Phi_use, H = 120) {
  ab <- compute_AB_classical(
    xi0 = delta0,
    xi1 = matrix(delta1, ncol = 1),
    H = H,
    psi = psi_GaussianVAR,
    psi.parameterization = list(
      mu = matrix(mu_use, ncol = 1),
      Phi = Phi_use,
      Sigma = Sigma
    )
  )
  sapply(1:H, function(h) {
    12 * as.vector(ab$b[1, 1, h] + X %*% ab$a[, 1, h])
  })
}

yield_fit <- compute_yields(mu - lambda0, Phi - lambda1, H = 120)
yield_eh <- compute_yields(mu, Phi, H = 120)

horizons <- c(24, 60, 120)
tp_est <- sapply(horizons, function(h) {

```

```

    100 * (yield_fit[, h] - yield_eh[, h])
  })
  colnames(tp_est) <- c("2-year", "5-year", "10-year")

  out <- data.frame(date = dat$date, tp_est)
  names(out)[2:4] <- c("TP2", "TP5", "TP10")
  out$m <- month_id(out$date)

  lambda_t <- sweep(X %*% t(lambda1), 2, lambda0, "+")
  max_sr_monthly <- sqrt(exp(rowSums((lambda_t %*% solve(Sigma)) * lambda_t)) - 1)
  out$MaxSR_annualized <- sqrt(12) * max_sr_monthly

  op <- par(mfrow = c(2, 1), mar = c(3.0, 4.4, 2.0, 0.8), las = 1)
  matplot(
    out$date,
    out[, c("TP2", "TP5", "TP10")],
    type = "n",
    lty = 1,
    lwd = c(1.5, 1.7, 2.2),
    col = c("grey65", "grey35", "black"),
    xlab = "",
    ylab = "Term premium (%)"
  )
  title(main = "(a) Estimated term premiums", font.main = 1)
  grid()
  abline(h = 0, col = "grey80", lwd = 0.8)
  matlines(
    out$date,
    out[, c("TP2", "TP5", "TP10")],
    lty = 1,
    lwd = c(1.5, 1.7, 2.2),
    col = c("grey65", "grey35", "black")
  )
  legend(
    "topright",
    legend = c("2-year", "5-year", "10-year"),
    col = c("grey65", "grey35", "black"),
    lty = 1,
    lwd = c(1.5, 1.7, 2.2),
    bty = "n"
  )

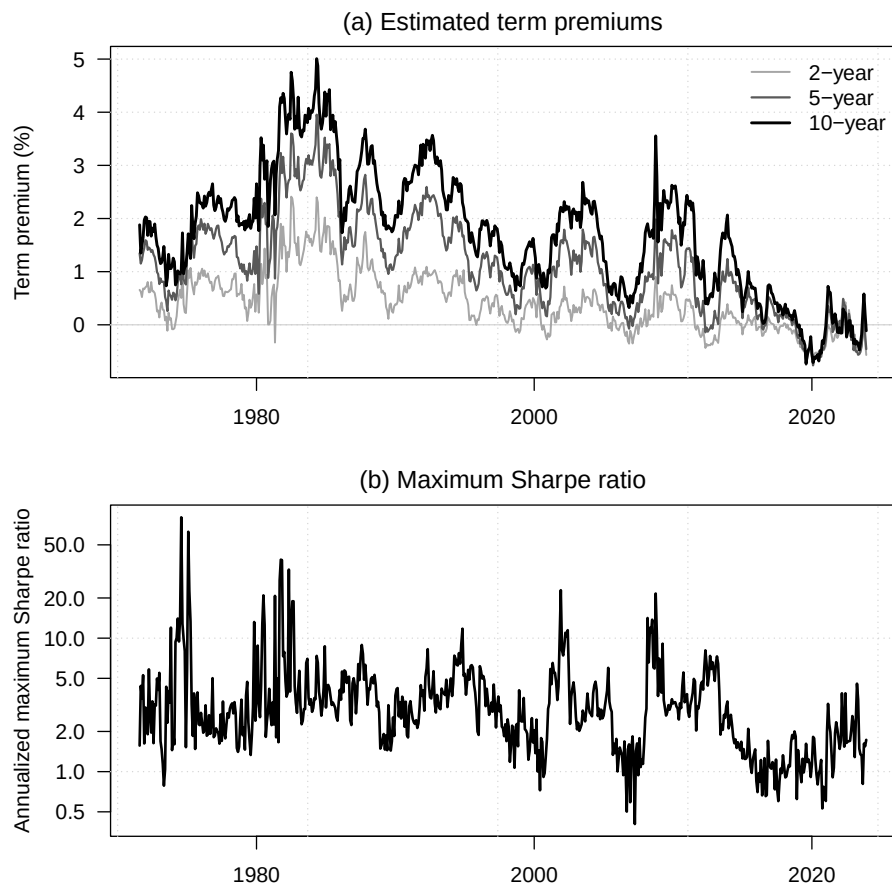
  plot(
    out$date,
    out$MaxSR_annualized,

```

```

type = "n",
log = "y",
xlab = "",
ylab = "Annualized maximum Sharpe ratio"
)
title(main = "(b) Maximum Sharpe ratio", font.main = 1)
grid()
lines(out$date, out$MaxSR_annualized, col = "black", lwd = 2)

```



```

par(op)

sample_table <- data.frame(
  sample = "Estimation and figure",
  start = min(dat$date),
  end = max(dat$date),
  observations = nrow(dat)
)

```

```

)
sample_table

##           sample      start      end observations
## 1 Estimation and figure 1971-08-01 2023-12-01      629
summary_table <- data.frame(
  maturity = c("2-year", "5-year", "10-year"),
  mean = c(mean(out$TP2), mean(out$TP5), mean(out$TP10)),
  min = c(min(out$TP2), min(out$TP5), min(out$TP10)),
  max = c(max(out$TP2), max(out$TP5), max(out$TP10))
)
summary_table[, -1] <- round(summary_table[, -1], 2)
summary_table

## maturity mean  min  max
## 1  2-year 0.42 -0.73 2.41
## 2  5-year 1.14 -0.76 3.95
## 3 10-year 1.75 -0.73 5.01

```

The complete sample starts only once all yields from 1 month to 120 months are available. This is why the first observations of `YC_LW_FULL`, for which the longest maturities are missing, are not used in the estimation.

As a diagnostic, the next figure compares the recomputed 10-year term premium with the official ACM 10-year term premium. This comparison is useful for checking the order of magnitude of the replication, but it is not meant as a standalone figure for the TeX version of the book.

```

# Purpose: This diagnostic compares the recomputed 10-year term premium with
# the official ACM 10-year term premium.
data("ACMTermPremium", package = "DTAM")

benchmark <- ACMTermPremium[, c("date", "ACMTP10")]
benchmark$m <- month_id(benchmark$date)
check_out <- merge(out[, c("date", "m", "TP10")], benchmark[, c("m", "ACMTP10")], by = "m")
check_out <- check_out[order(check_out$date), ]

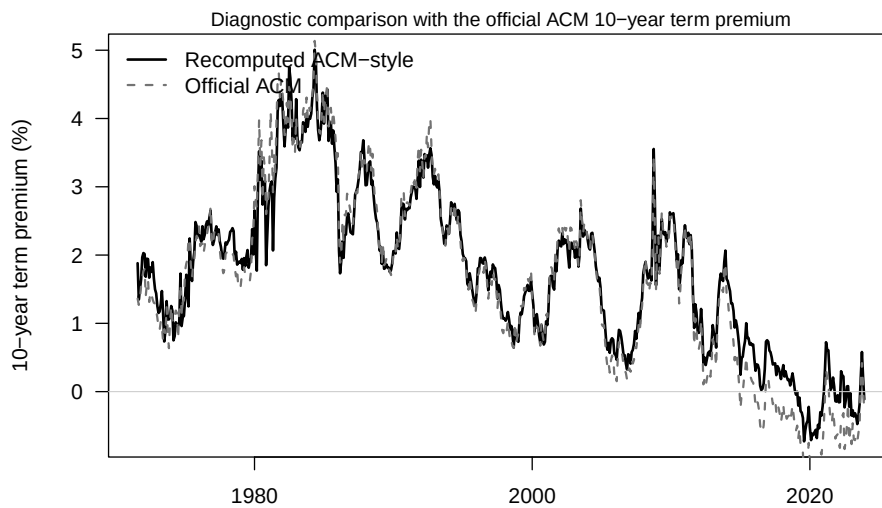
op <- par(mar = c(4.2, 4.4, 1.2, 0.8), las = 1)
plot(
  check_out$date,
  check_out$TP10,
  type = "l",
  lwd = 2,
  col = "black",
  xlab = "",
  ylab = "10-year term premium (%)"
)

```

```

)
lines(check_out$date, check_out$ACMTP10, col = "grey45", lty = 2, lwd = 1.7)
abline(h = 0, col = "grey80", lwd = 0.8)
legend(
  "topleft",
  legend = c("Recomputed ACM-style", "Official ACM"),
  col = c("black", "grey45"),
  lty = c(1, 2),
  lwd = c(2, 1.7),
  bty = "n"
)
mtext("Diagnostic comparison with the official ACM 10-year term premium", side = 3, lin

```



```

par(op)

check_table <- data.frame(
  series = c("Recomputed ACM-style", "Official ACM"),
  mean = c(mean(check_out$TP10), mean(check_out$ACMTP10)),
  min = c(min(check_out$TP10), min(check_out$ACMTP10)),
  max = c(max(check_out$TP10), max(check_out$ACMTP10)),
  correlation_with_ACM = c(cor(check_out$TP10, check_out$ACMTP10), 1)
)
check_table[, -1] <- round(check_table[, -1], 2)
check_table

```

```

##           series mean  min  max correlation_with_ACM
## 1 Recomputed ACM-style 1.75 -0.73 5.01                0.99

```



```
## 2          Official ACM 1.65 -1.32 5.14          1.00
```

5.3 Jensen adjustment in the log-inflation component

The log-moment inflation component, $h^{-1} \log E_t[\exp(\pi_{t,t+h})]$, coincides with expected average inflation plus a Jensen term, $(2h)^{-1} \text{Var}(\pi_{t,t+h})$, when cumulative log inflation is conditionally Gaussian. The following calculation uses the U.S. CPI series in DTAM to give a simple historical order of magnitude for this term at a five-year horizon. It is only a historical benchmark, not an estimate of the corresponding conditional physical or risk-neutral variance.

Purpose: Estimate the empirical order of magnitude of the Jensen term in the # five-year log-inflation component using U.S. CPI data from DTAM.

```
data("Data_Macro_US_monthly", package = "DTAM")

h_months <- 60
h_years <- 5
cpi_us <- Data_Macro_US_monthly$CPIAUCSL
dates_us <- Data_Macro_US_monthly$date

pi_5y <- log(
  cpi_us[(h_months + 1):length(cpi_us)] /
  cpi_us[1:(length(cpi_us) - h_months)]
)
pi_5y_dates <- dates_us[(h_months + 1):length(dates_us)]

jensen_summary <- function(start_date = NULL) {
  keep <- is.finite(pi_5y)
  if (!is.null(start_date)) {
    keep <- keep & pi_5y_dates >= as.Date(start_date)
  }
  jensen_bps <- 10000 * 0.5 * var(pi_5y[keep]) / h_years
  data.frame(
    sample_start = min(pi_5y_dates[keep]),
    sample_end = max(pi_5y_dates[keep]),
    observations = sum(keep),
    sd_average_inflation = 100 * sd(pi_5y[keep] / h_years),
    jensen_term_bps = jensen_bps
  )
}

jensen_table <- rbind(
  `Full sample` = jensen_summary(),
  `Since 1990` = jensen_summary("1990-01-01")
)
```

```

)

jensen_table$sd_average_inflation <- round(jensen_table$sd_average_inflation, 2)
jensen_table$jensen_term_bps <- round(jensen_table$jensen_term_bps, 1)
jensen_table

##           sample_start sample_end observations sd_average_inflation
## Full sample  1964-01-01 2025-06-01          738                2.15
## Since 1990   1990-01-01 2025-06-01          426                0.88
##           jensen_term_bps
## Full sample              11.6
## Since 1990               1.9

```

Chapter 6

Non-Gaussian affine term structure models

6.1 A VARG term-structure model can generate movements

This example illustrates how a VARG term-structure model can generate movements in long-term yields even when the short-term rate is stuck at zero. We consider a two-factor specification in which the first factor is the short-term rate, $r_t = w_{1,t}$, with $\nu_1 = 0$, while the second factor is a persistent nonnegative factor that Granger-causes the first one through a positive $\beta_{1,2}$ coefficient. As a result, two dates may feature the same current short-term rate $r_t = 0$, but different values of $w_{2,t}$, and hence different yield curves because expected future short rates differ.

```
# Purpose: This example illustrates how a VARG term-structure model can generate movements
library(DTAM)

set.seed(123)

TT <- 150
H  <- 10

# Two-factor VARG:
# - factor 1 is the short-term rate  $r_t = w_{\{1,t\}}$  and has  $\nu_1 = 0$ ,
#   so its conditional distribution admits a Dirac mass at zero;
# - factor 2 is persistent and causes factor 1.
model <- list(
  alpha = matrix(c(0.20, 0.05), ncol = 1),
```

```

nu      = matrix(c(0, 0.5), ncol = 1),
mu      = matrix(c(0.004, 0.0025), ncol = 1),
beta    = matrix(c(225, 14,
                    0, 380), 2, 2, byrow = TRUE),
alpha_w = matrix(c(0, 0), ncol = 1),
n_w     = 2
)
# In particular, beta_11 * mu_1 = 0.9, so the short-term rate is highly persistent.

xi0 <- 0
xi1 <- matrix(c(1, 0), ncol = 1)

# Risk-neutral dynamics and affine bond-pricing coefficients.
modelQ <- make_VARG_Q(model)
AB <- compute_AB_classical(
  xi0 = xi0,
  xi1 = xi1,
  H = H,
  psi = psi_VARG,
  psi.parameterization = modelQ
)

# Simulate the physical dynamics from the unconditional mean.
w0 <- c(compute_uncondmean_VARG(model))
simX <- t(simul_VARG(model, nb_periods = TT - 1, w0 = w0))

# Compute the whole yield curve at each date.
yields <- matrix(1, TT, 1) %%% matrix(AB$b, nrow = 1) +
  simX %%% matrix(AB$a, dim(AB$a)[1], dim(AB$a)[3])

# One-step-ahead probability that the short-term rate stays at zero.
p_stay_zero <- exp(-(model$alpha[1] + simX %%% matrix(model$beta[1, ], ncol = 1)))

# Select two dates such that  $r_t = 0$  but  $w_{\{2,t\}}$  differs sharply.
idx_zero <- which(simX[, 1] < 1e-12)
stopifnot(length(idx_zero) >= 2)
i_low <- idx_zero[which.min(simX[idx_zero, 2])]
i_high <- idx_zero[which.max(simX[idx_zero, 2])]

par(mfrow = c(3, 1), mar = c(3.2, 4.2, 2.8, 1.2))

# Panel (a): short rate and long yield through time.
plot(
  1:TT, 100 * yields[, H],
  type = "l", lwd = 2, col = "black", las = 1,

```

```

xlab = "time", ylab = "in percent",
ylim = 120 * range(c(simX[, 1], yields[, H])),
main = expression(paste("(a) Short-term rate and 10-period yield"))
)
grid()
lines(1:TT, 100 * simX[, 1], col = "darkgrey", lwd = 2)
abline(v = i_low, col = "grey40", lty = 3)
abline(v = i_high, col = "black", lty = 3)
legend(
  "topleft",
  legend = c(
    expression(paste("Short-term rate (", r[t], ") ", sep="")),
    expression(paste("Long-term rate (", r[t*10], ") ", sep=""))
  ),
  col = c("darkgrey", "black"),
  lwd = c(2, 2),
  lty = c(1, 1),
  seg.len = 3,
  bty = "n"
)

# Panel (b): yield curves at two dates with the same current short rate r_t = 0.
plot(
  1:H, 100 * yields[i_low, ],
  type = "l", lwd = 2, col = "grey70", las = 1, lty = 2,
  xlab = "maturity", ylab = "in percent",
  ylim = 120 * range(yields[c(i_low, i_high), ]),
  main = expression(paste("(b) Yield curves at two dates with ", r[t], " = 0", sep=""))
)
grid()
lines(1:H, 100 * yields[i_high, ], lwd = 2, col = "black")
legend(
  "topleft",
  legend = c(
    bquote(paste("date ", .(i_low), ": ", r[t], " = 0, ",
      w[2*"], "*t], " = ",
      .(format(round(simX[i_low, 2], 3), nsmall = 3)))),
    bquote(paste("date ", .(i_high), ": ", r[t], " = 0, ",
      w[2*"], "*t], " = ",
      .(format(round(simX[i_high, 2], 3), nsmall = 3))))
  ),
  col = c("grey70", "black"),
  lwd = c(2, 2),
  lty = c(2, 1),
  seg.len = 3,

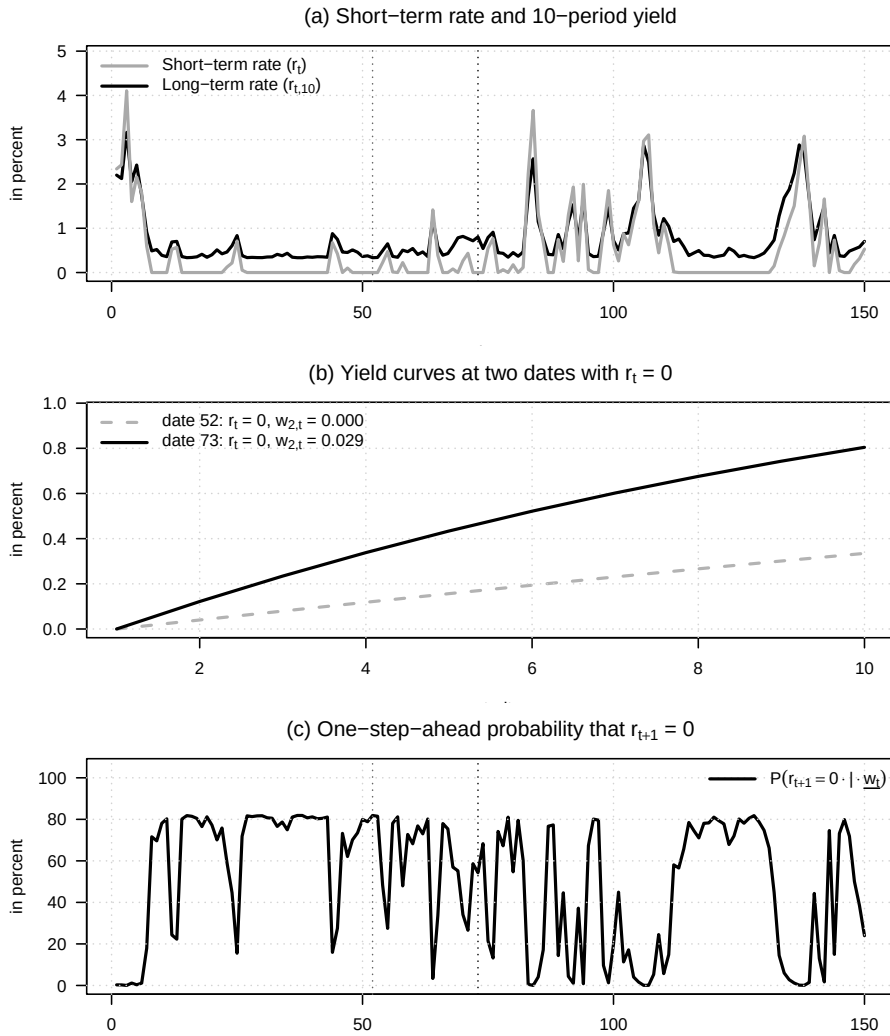
```

```

    cex = 1.0,
    bty = "n"
)

# Panel (c): one-step-ahead probability that the short rate remains at zero.
plot(
  1:TT, 100 * c(p_stay_zero),
  type = "l", lwd = 2, col = "black", las = 1,
  xlab = "time", ylab = "in percent",
  ylim = c(0, 105),
  main = expression(paste("(c) One-step-ahead probability that ", r[t+1], " = 0"))
)
grid()
abline(v = i_low, col = "grey40", lty = 3)
abline(v = i_high, col = "black", lty = 3)
legend(
  "topright",
  legend = c(expression(P(r[t+1] == 0 %.% "|" %.% underline(w[t])))),
  col = "black",
  lwd = 2,
  lty = 1,
  seg.len = 3,
  bty = "n"
)

```



Panel (a) shows that the 10-period yield keeps moving while the short-term rate repeatedly returns to zero. Panel (b) makes the mechanism transparent: two dates may share the same current short-term rate $r_t = 0$, yet display markedly different yield curves because the second factor changes the expected future path of short-term rates. Panel (c) reports the one-step-ahead probability that the short-term rate remains at zero; this probability is high when the second factor is low and falls when the second factor signals a return of positive short-term rates.

6.2 A three-regime Gaussian regime-switching short-rate

This example illustrates a three-regime Gaussian regime-switching short-rate model. The regimes are interpreted as normal times, small crises, and extreme crises. In the extreme-crisis regime, the mean-reversion level of the short rate is zero, capturing the idea that the central bank pushes the short rate toward the lower bound. We then add priced regime risk by tilting risk-neutral transition probabilities toward the extreme-crisis regime.

```
# Purpose: This example illustrates a three-regime Gaussian regime-switching short-rate model.
library(DTAM)

# set.seed(123)

TT <- 350
H <- 10

rho <- 0.7

# Regime-specific long-run means of the short rate.
mu_bar <- c(0.03, 0.01, 0.00)
mu <- matrix((1 - rho) * mu_bar, 1, 3)

# Regime-specific volatilities.
sigma <- c(0.003, 0.00001, 0.00001)

# Regime-transition matrix under the physical measure.
Pi <- matrix(c(
  0.95, 0.05, 0.00,
  0.05, 0.90, 0.05,
  0.05, 0.00, 0.95
), 3, 3, byrow = TRUE)

# Regime-risk prices: the SDF is high in the extreme-crisis regime.
gamma <- c(0, 0, 1.25)
Pi_star <- sweep(Pi, 2, exp(gamma), `*`)
Pi_star <- Pi_star / rowSums(Pi_star)

regime_names <- c("Normal times", "Small crisis", "Extreme crisis")

model_P <- list(
  Phi = matrix(rho, 1, 1),
  Pi = Pi,
  mu = mu,
  Sigma = array(sigma^2, c(1, 1, 3))
)
```


6.2. A THREE-REGIME GAUSSIAN REGIME-SWITCHING SHORT-RATE⁹⁷

```
)
model_Q <- model_P
model_Q$Pi <- Pi_star

xi0 <- 0
xi1 <- matrix(c(1, rep(0, 3)), ncol = 1)
AB_P <- compute_AB_classical(
  xi0 = xi0,
  xi1 = xi1,
  H = H,
  psi = psi_RSVAR,
  psi.parameterization = model_P
)
AB_Q <- compute_AB_classical(
  xi0 = xi0,
  xi1 = xi1,
  H = H,
  psi = psi_RSVAR,
  psi.parameterization = model_Q
)

# Simulate regimes and short rates under the physical dynamics.
sim <- simul_RSVAR(model_P, nb.sim = TT, y0 = mu_bar[1], z0 = 1)
r <- matrix(sim$y[, 1], TT, 1)
z <- sim$z
regime <- sim$regime
W <- sim$w
yields_P <- matrix(1, TT, 1) %*% matrix(AB_P$b, nrow = 1) +
  W %*% matrix(AB_P$a, dim(AB_P$a)[1], dim(AB_P$a)[3])
yields_Q <- matrix(1, TT, 1) %*% matrix(AB_Q$b, nrow = 1) +
  W %*% matrix(AB_Q$a, dim(AB_Q$a)[1], dim(AB_Q$a)[3])

par(mfrow = c(2, 1), mar = c(3.2, 4.2, 2.8, 1.2))

add_regime_shading <- function(regime, col_regime) {
  rr <- rle(regime)
  end <- cumsum(rr$lengths)
  start <- c(1, head(end, -1) + 1)
  usr <- par("usr")
  for (i in seq_along(rr$values)) {
    col_i <- col_regime[rr$values[i]]
    if (!is.na(col_i)) {
      rect(start[i] - 0.5, usr[3], end[i] + 0.5, usr[4],
          col = col_i, border = NA)
    }
  }
}
```

```

    }
  }

plot(
  1:TT, 100 * r,
  type = "n", lwd = 2, las = 1, col = "black",
  xlab = "time", ylab = "in percent",
  ylim = 100 * range(c(r, yields_P[, H], yields_Q[, H])),
  xlim=c(1,TT*(1.2)),
  main = expression(paste("(a) Regime-switching short rate"))
)
add_regime_shading(regime, c(NA, "grey90", "grey75"))
grid()
lines(1:TT, 100 * r, col = "black", lwd = 2)
lines(1:TT, 100 * yields_P[, H], col = "grey60", lwd = 2, lty = 2)
lines(1:TT, 100 * yields_Q[, H], col = "black", lwd = 2)
legend(
  "topright",
  legend = c(expression(r[t]), "EH yield", "RN yield"),
  col = c("black", "grey60", "black"),
  lty = c(1, 2, 1),
  lwd = 2,
  bg = "white",
  box.col = "white",
  bty = "o"
)
legend(
  "bottomright",
  legend = regime_names,
  fill = c("white", "grey90", "grey75"),
  border = c("grey60", NA, NA),
  bg = "white",
  box.col = "white",
  bty = "o"
)

curve_cols <- c("grey75", "grey40", "black")
band_cols <- grDevices::adjustcolor(curve_cols, alpha.f = 0.18)

regime_curve_summary <- lapply(1:3, function(j) {
  y_q <- yields_Q[regime == j, , drop = FALSE]
  y_p <- yields_P[regime == j, , drop = FALSE]
  list(
    mean_Q = apply(y_q, 2, mean),
    mean_P = apply(y_p, 2, mean),
  )
})

```

6.2. A THREE-REGIME GAUSSIAN REGIME-SWITCHING SHORT-RATE99

```

    q10_Q = apply(y_q, 2, quantile, probs = 0.10),
    q90_Q = apply(y_q, 2, quantile, probs = 0.90)
  )
})

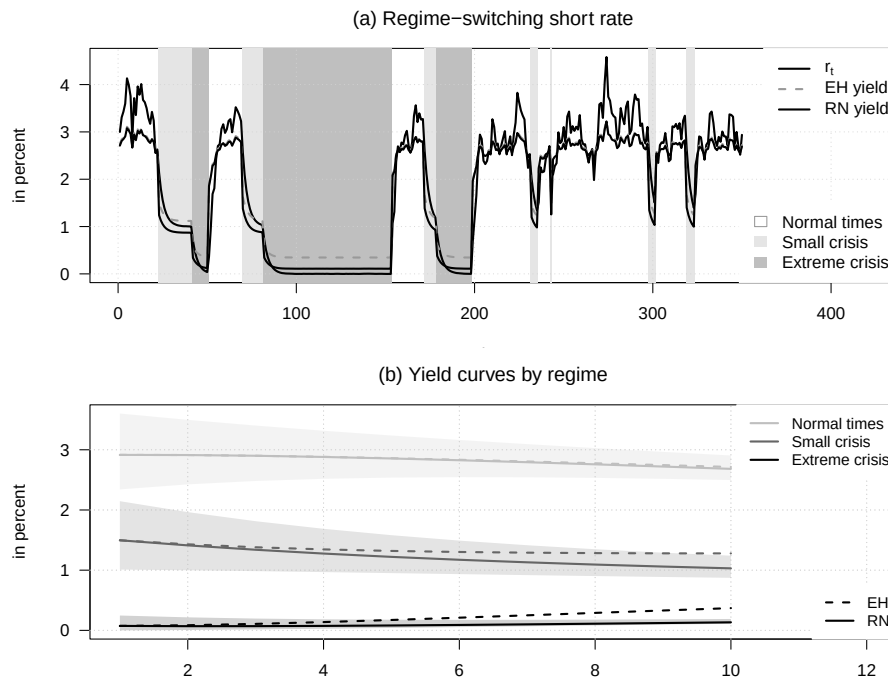
plot(
  1:H, 100 * regime_curve_summary[[1]]$mean_Q,
  type = "n", lwd = 2, las = 1, col = "grey75",
  xlab = "maturity", ylab = "in percent",
  ylim = 100 * range(unlist(lapply(
    regime_curve_summary,
    function(x) c(x$q10_Q, x$q90_Q, x$mean_P)
  ))),
  xlim=c(1,H*(1.2)),
  main = expression(paste("(b) Yield curves by regime"))
)
grid()
for (j in 1:3) {
  polygon(
    c(1:H, H:1),
    100 * c(regime_curve_summary[[j]]$q10_Q, rev(regime_curve_summary[[j]]$q90_Q)),
    col = band_cols[j],
    border = NA
  )
}
grid()
for (j in 1:3) {
  lines(1:H, 100 * regime_curve_summary[[j]]$mean_P,
        col = curve_cols[j], lwd = 2, lty = 2)
  lines(1:H, 100 * regime_curve_summary[[j]]$mean_Q,
        col = curve_cols[j], lwd = 2)
}
legend(
  "topright",
  legend = regime_names,
  col = c("grey75", "grey40", "black"),
  lwd = 2,
  bg = "white",
  box.col = "white",
  bty = "o",
  cex=.9
)
legend(
  "bottomright",
  legend = c("EH", "RN"),

```

```

col = "black",
lty = c(2, 1),
lwd = 2,
bg = "white",
box.col = "white",
bty = "o",
cex=.9
)

```



Panel (a) displays the simulated short rate together with the 10-period expectations-hypothesis yield and the risk-neutral yield. The background shading indicates the prevailing regime: white for normal times, light grey for small crises, and darker grey for extreme crises. Panel (b) reports average yield curves by regime. Dashed lines use the physical transition probabilities, while solid lines use the risk-neutral transition probabilities tilted toward the extreme-crisis regime. The 10–90% bands are computed from the risk-neutral yields and show that regime switching generates non-Gaussian yield distributions.

Chapter 7

Shadow-rate framework and extensions

7.1 Shadow-rate and unconstrained Gaussian term structures in the simple Wu-Xia setting

This example compares shadow-rate and unconstrained Gaussian term structures in the simple Wu-Xia setting.

```
# Purpose: This example compares shadow-rate and unconstrained Gaussian term structures in the  
# simple Wu-Xia setting  
library(DTAM)  
  
# Specify the Gaussian shadow-rate state dynamics.  
rho <- .96  
Phi <- as.matrix(rho)  
mu <- matrix(0.03*(1-rho),1,1)  
Sigma <- (1-rho^2) * (.02)^2 * matrix(1)  
model <- list(mu = mu, Phi = Phi, Sigma = Sigma,  
              n_w=1)  
  
# Simulate the latent shadow rate.  
TT <- 250  
set.seed(1234)  
s <- simul_GaussianVAR(model,nb.sim = TT)  
  
par(mfrow=c(1,1))  
par(plt=c(.1,.95,.13,.95))
```

```

plot(s,type="l",col="white",xlab="",ylab="",las=1)
# grid()
abline(h=0,col="grey",lty=2)
lines(s,lwd=2,col="lightgrey")

delta0 <- 0
delta1 <- matrix(1,1,1)

# Compute shadow-rate forward rates with the Wu-Xia approximation.
H <- 40 # maximum maturity
res <- compute_F_Shadow_Gaussian(s,
                                psi.parameterization = model,
                                ell_bar=0,
                                b=delta0,
                                a=delta1,
                                c=matrix(0,1,1),
                                H = H-1,
                                indic_WuXia = TRUE)

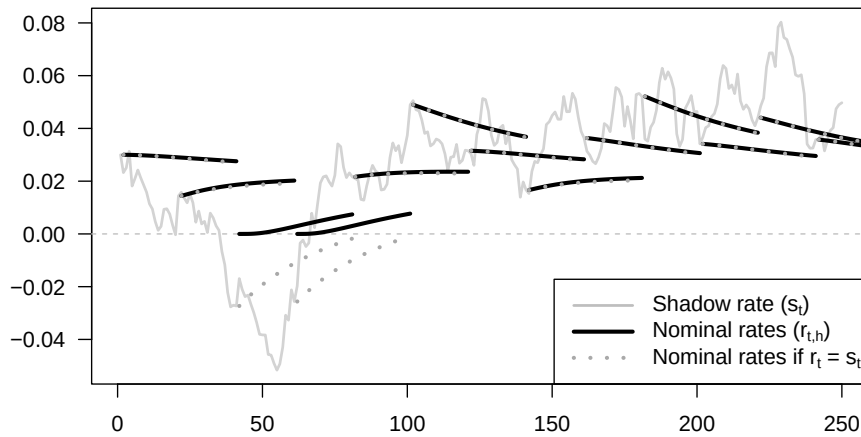
f <- cbind(pmax(0,s),res$F)
y <- t(apply(f,1,cumsum))/t(matrix(1:H,H,TT))

AB_GATSM <- compute_AB_classical(xi0=0,xi1=matrix(1),H = H,
                                psi = psi_GaussianVAR,psi.parameterization = model)
y_noSR <- t(matrix(AB_GATSM$b,H,TT)) + s %*% matrix(AB_GATSM$a,ncol=H)

all_t <- seq(1,TT, by=H/2)
for(t in all_t){
  lines((t+1):(t+H),y[t,],type="l",col="black",lwd=3)
  lines((t+1):(t+H),y_noSR[t,],type="l",col="darkgrey",lwd=3,lty=3)
}

legend("bottomright",
      c(expression(paste("Shadow rate (",s[t],")",sep="")),
        expression(paste("Nominal rates (",r[t*','*h],")",sep="")),
        expression(paste("Nominal rates if ",r[t], " = ",s[t],sep=")))),
      lty=c(1,1,3),
      lwd=c(2,3,3),
      pt.cex = 1,
      col= c("grey","black","darkgrey"),
      seg.len = 3)

```



7.2 Shadow-rate bond pricing and Monte Carlo validation

This example checks shadow-rate bond pricing formulas against Monte Carlo simulation.

Purpose: This example checks shadow-rate bond pricing formulas against Monte Carlo simulation

```
H <- 30

ell_bar <- 0

# Compute bond prices based on simulations:
selected_date <- 1
nb.replic <- 100000
model$Sigma12 <- matrix(sqrt(model$Sigma))
simul_s <- simul_GaussianVAR(model,nb.sim=H,x0=s[selected_date],nb.replic = nb.replic)
simul_s <- matrix(simul_s,H,nb.replic)
simul_r <- pmax(simul_s,ell_bar)
aux <- apply(simul_r,2,cumsum)
Bth <- exp(-aux)
aux <- apply(Bth,1,mean)
rth <- -log(aux)/(1:H)

res_WX <- compute_F_Shadow_Gaussian(s,
                                   psi.parameterization = model,
                                   ell_bar=ell_bar,
                                   b=0,
                                   a=matrix(1,1,1),
```

```

                                c=matrix(0,1,1),
                                H = H-1,
                                indic_WuXia = TRUE)
res_PR <- compute_F_Shadow_Gaussian(s,
                                psi.parameterization = model,
                                ell_bar=ell_bar,
                                b=0,
                                a=matrix(1,1,1),
                                c=matrix(0,1,1),
                                H = H-1,
                                indic_WuXia = FALSE)
res_affine <- compute_F_Shadow_affine(s,psi_GaussianVAR,
                                psi.parameterization = model,
                                ell_bar=ell_bar,
                                b=0,
                                a=matrix(1,1,1),
                                c=matrix(0,1,1),
                                H = H-1)

r <- pmax(ell_bar,s)

f_WX <- cbind(r,res_WX$F)
y_WX <- t(apply(f_WX,1,cumsum))/t(matrix(1:H,H,TT))

f_PR <- cbind(r,res_PR$F)
y_PR <- t(apply(f_PR,1,cumsum))/t(matrix(1:H,H,TT))

f_affine <- cbind(r,res_affine$F)
y_affine <- t(apply(f_affine,1,cumsum))/t(matrix(1:H,H,TT))

res_noLB <- compute_AB_classical(xi0=0,xi1=matrix(1),H = H,
                                psi = psi_GaussianVAR,
                                psi.parameterization = model)
y_noLB <- matrix(1,TT,1) %*% matrix(res_noLB$b,nrow=1) +
  s %*% matrix(res_noLB$a,nrow=1)

plot(rth,lwd=3,col="grey",type="l",
     ylim=c(min(rth,
                 y_WX[selected_date,],
                 y_PR[selected_date,],
                 y_affine[selected_date,]),
            max(rth,
                 y_WX[selected_date,],
                 y_PR[selected_date,],
                 y_affine[selected_date,]))))

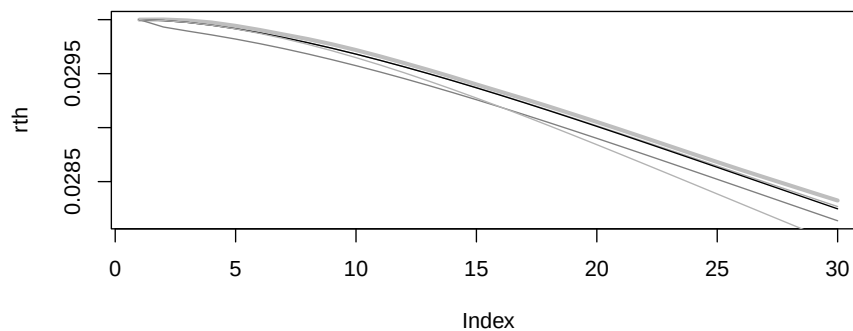
```



```

lines(y_WX[selected_date,],col="darkgrey")
lines(y_PR[selected_date,],col="black")
lines(y_affine[selected_date,],col="grey50")
lines(y_noLB[selected_date,],col="grey70")

```



```

# cbind(rth[H],
#       y_WX[selected_date,H],
#       y_PR[selected_date,H],
#       y_affine[selected_date,H],
#       y_noLB[selected_date,H])

# plot(abs(res_WX$all_b_bar_n - res_WX$all_b_n)/res_WX$all_sigma_n)

```

7.3 The lower bound changes conditional rate volatility across maturities

This example shows how the lower bound changes conditional rate volatility across maturities.

```

# Purpose: This example shows how the lower bound changes conditional rate volatility across
# maturities
library(expm)

# Set the maturities and helper function used in the truncated-normal moments.
all_h <- c(1,3,6)

g <- function(x){
  x*pnorm(x) + dnorm(x)
}

```

```

max_y <- 100*res$all_sigma_n[max(all_h)]

# Compute and plot the conditional volatility of observed rates.
par(mfrow=c(1,1))
par(plt=c(.15,.95,.15,.95))
plot(s,type="l",
      ylim=c(0,max_y),
      col="white",xlab="",ylab="Conditional volat., in percent",
      las=1)

grid()
count <- 0
for(h in rev(all_h)){
  count <- count + 1

  mult <- res$all_b_bar_n[h] + s %*% t(model$Phi%^h) %*% delta1
  sigma1 <- res$all_sigma_n[h]
  ratio <- mult/sigma1
  var_rt <- (mult^2 + sigma1^2)*pnorm(ratio) -
    mult*sigma1*dnorm(ratio) - sigma1^2*g(ratio)^2
  vol_rt <- 100*sqrt(var_rt)

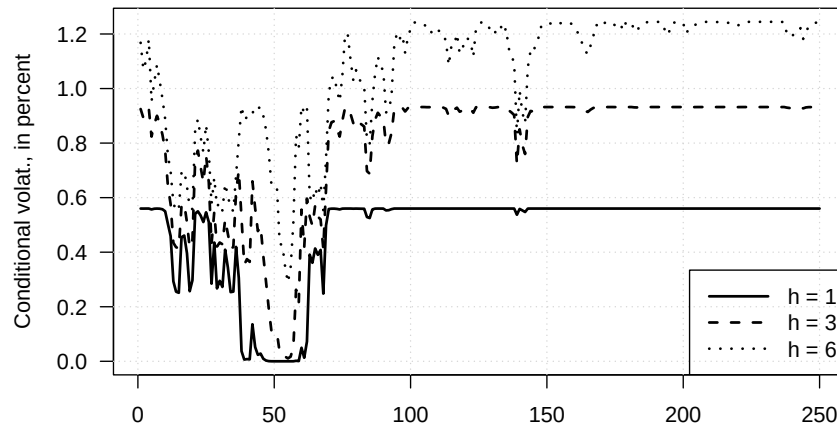
  lines(vol_rt,lwd=2,lty=length(all_h) + 1 - count)
}

# Add maturity labels.
legend_labels <- paste("h =", all_h)

legend("bottomright",legend = legend_labels,
      lty=1:length(all_h),
      lwd=2,
      col= "black",
      seg.len = 3)

```

7.4. THE SHADOW-RATE IDEA IN A SMALL NEW KEYNESIAN MODEL WITH NOMINAL AND REAL CURVES



7.4 The shadow-rate idea in a small New Keynesian model with nominal and real curves

This example embeds the shadow-rate idea in a small New Keynesian model with nominal and real curves.

*# Purpose: This example embeds the shadow-rate idea in a small New Keynesian model with nominal
and real curves*

```
library(DTAM)
```

```
r_star <- .03  
pi_star <- .02  
rho <- .5  
phi <- .8  
beta <- .2  
alpha <- 1.5  
gamma <- 0.5  
sigma_r <- .005  
sigma_pi <- .005  
sigma_y <- .005  
kappa <- .2
```

```
A0 <- diag(4)  
A0[1,3] <- (rho-1)*alpha  
A0[1,4] <- (rho-1)*gamma  
# A0[4,3] <- -beta
```

```
A0_1 <- solve(A0)
```


7.4. THE SHADOW-RATE IDEA IN A SMALL NEW KEYNESIAN MODEL WITH NOMINAL AND REAL CURVES

```

H = H)

f_nominal <- res$F_c_equal_0
f_real <- res$F
y_nominal <- t(apply(f_nominal,1,cumsum))/t(matrix(1:H,H,TT))
y_real <- t(apply(f_real,1,cumsum))/t(matrix(1:H,H,TT))

AB_GATSM <-
  compute_AB_classical(xi0=0,
    xi1=matrix(c(1,0,0,0),ncol=1),
    kappa0 = 0, kappa1 = matrix(c(0,0,-1,0),ncol=1),
    H = H,
    psi = psi_GaussianVAR,psi.parameterization = model)
y_noSR_nominal <- t(matrix(AB_GATSM$b[,1,],H,TT)) + W %%% matrix(AB_GATSM$a[,1,],ncol=H)
y_noSR_real <- t(matrix(AB_GATSM$b[,2,],H,TT)) + W %%% matrix(AB_GATSM$a[,2,],ncol=H)

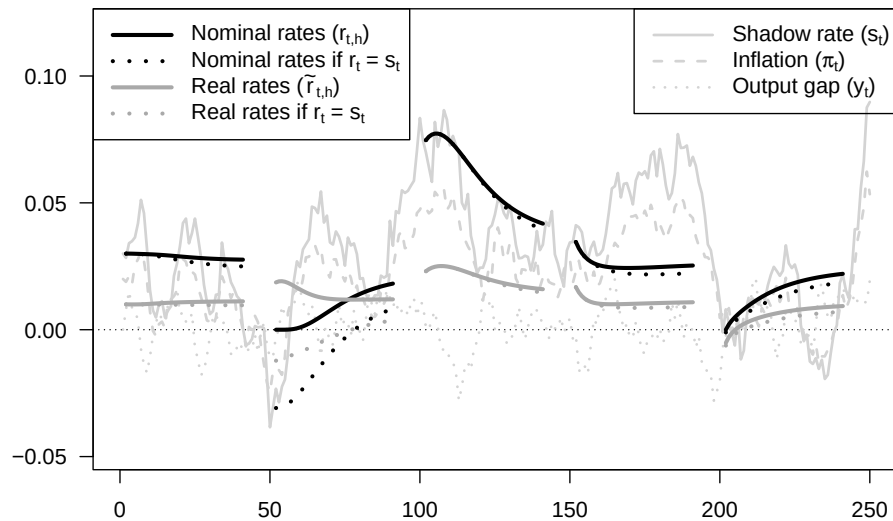
all_t <- seq(1,TT,by=H+10)

for(t in all_t){
  lines((t+1):(t+H),y_nominal[t,],type="l",col="black",lwd=3)
  lines((t+1):(t+H),y_real[t,],type="l",col="darkgrey",lwd=3)
  lines((t+1):(t+H),y_noSR_nominal[t,],type="l",col="black",lwd=3,lty=3)
  lines((t+1):(t+H),y_noSR_real[t,],type="l",col="darkgrey",lwd=3,lty=3)
}

legend("topright",
  c(expression(paste("Shadow rate (",s[t],")",sep="")),
    expression(paste("Inflation (",pi[t],")",sep="")),
    expression(paste("Output gap (",y[t],")",sep="))),
  lty=c(1,2,3),
  lwd=c(2,2,2),
  pt.cex = 1,
  col= "lightgrey",
  seg.len = 3)

legend("topleft",
  c(expression(paste("Nominal rates (",r[t*','*h],")",sep="")),
    expression(paste("Nominal rates if ",r[t], " = ",s[t],sep="")),
    expression(paste("Real rates (",tilde(r)[t*','*h],")",sep="")),
    expression(paste("Real rates if ",r[t], " = ",s[t],sep="))),
  lty=c(1,3,1,3),
  lwd=c(3,3,3,3),
  pt.cex = 1,
  col= c("black","black","darkgrey","darkgrey"),
  seg.len = 3)

```



7.5 Shadow-rate pricing of defaultable bond spreads

This example uses a shadow-rate pricing device to illustrate defaultable bond spreads in a structural credit model.

Purpose: This example uses a shadow-rate pricing device to illustrate defaultable bond spreads in a structural credit model

```
library(DTAM)
```

Specify output gap's (z_t) dynamics:

```
rho <- .8
```

```
sigma <- .02
```

```
gamma <- 2 # RRA coefficient
```

```
delta <- .995 # subjective time discount
```

```
mu <- .015 # average consumption growth
```

```
kappa <- 2 # sensitivity of default intensity to output gap
```

```
Phi <- matrix(c(rho,1,0,0),2,2)
```

```
Sigma12 <- matrix(c(sigma,0),2,1)
```

```
Sigma <- Sigma12 %*% t(Sigma12)
```

```
model <- list(mu = matrix(0,2,1), Phi = Phi,
              Sigma = Sigma, Sigma12 = Sigma12,
              n_w=2)
```

```

# Simulate z:
TT <- 100
set.seed(123)
W <- simul_GaussianVAR(model,nb.sim = TT)

# plot(W[,1],type="l")

mu_c1 <- matrix(c(1,-1),2,1)
dc <- mu + W %*% mu_c1

# plot(cumsum(dc),type="l")

H <- 10 # maximum maturity
res_default <- compute_F_Shadow_Gaussian(W,
                                           psi.parameterization = model,
                                           ell_bar=0,
                                           b=0,
                                           a=matrix(c(-kappa,0),2,1),
                                           c=matrix(c(-gamma,gamma),2,1),
                                           H = H)

res_rf <- compute_F_Shadow_Gaussian(W,
                                     psi.parameterization = model,
                                     ell_bar=0,
                                     b=0,
                                     a=.00001*matrix(c(-kappa,0),2,1),
                                     c=matrix(c(-gamma,gamma),2,1),
                                     H = H)

f_PD <- res_default$F_c_equal_0
f_bond <- res_default$F
f_rf <- res_rf$F
y_PD <- t(apply(f_PD,1,cumsum))/t(matrix(1:H,H,TT))
y_rf <- -log(delta) + gamma*mu + t(apply(f_rf,1,cumsum))/t(matrix(1:H,H,TT))
y_bond <- -log(delta) + gamma*mu + t(apply(f_bond,1,cumsum))/t(matrix(1:H,H,TT))

par(mfrow=c(2,1))
par(plt=c(.11,.98,.13,.8))

plot(W[,1],col="white",xlab="",ylab="",las=1,
      main = expression(paste("(a) Output gap (",z[t],")",sep="")))
grid()
abline(h=0,col="grey")
lines(W[,1],lwd=2)

plot(y_bond[,H],type="l",col="white",las=1,
      ylim=c(0,.13),xlab="",ylab="",

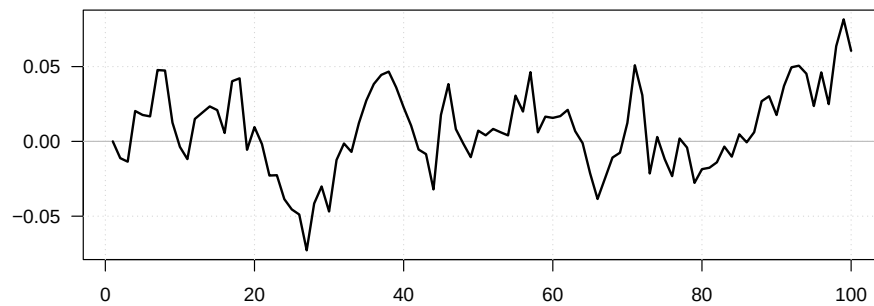
```

```

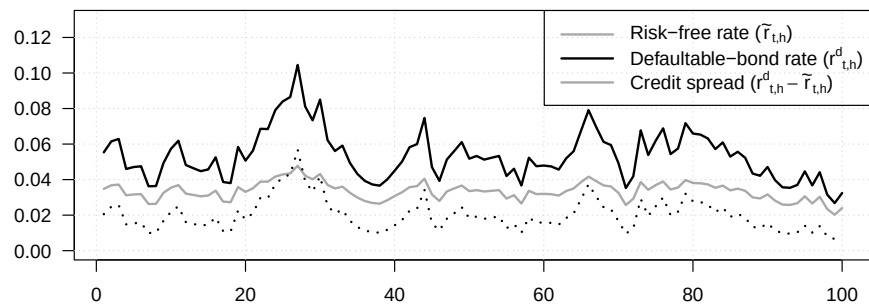
    main=expression(paste("(b) Bond yields",sep=""))
grid()
lines(y_rf[,H],col="darkgrey",lwd=2)
lines(y_bond[,H],col="black",lwd=2,lty=1)
lines(y_bond[,H]-y_rf[,H],col="black",lwd=2,lty=3)

legend("topright",
      c(expression(paste("Risk-free rate (",tilde(r)[t*','*h],")",sep="")),
        expression(paste("Defaultable-bond rate (",{r^d}[t*','*h],")",sep="")),
        expression(paste("Credit spread (",{r^d}[t*','*h]-tilde(r)[t*','*h],")",sep="")),
      lty=c(1,1),
      lwd=c(2,2),
      pt.cex = 1,
      col= c("darkgrey","black"),
      seg.len = 3)

```

(a) Output gap (z_t)

(b) Bond yields



```

# Compute probability of default:
PD <- 1 - exp(t(apply(f_PD,1,cumsum)))

```


7.6 Shadow default intensity and credit spreads in a fiscal-limits setting

This example studies shadow default intensity and credit spreads in a fiscal-limits setting.

```
# Purpose: This example studies shadow default intensity and credit spreads in a fiscal-limits
# setting
library(DTAM)

r_star <- .03
ell_star <- 1.1
d_star <- .9
sigma_r <- .005
sigma_ell <- .04
sigma_d <- .04
rho_r <- .9
rho_ell <- .98
rho_d <- .98
kappa <- .5

H <- 5 # bond maturity

# Specify state vector's dynamics ( $w_t = (r_t, ell_t, d_t, r_{t-1})'$ ):
Phi <- diag(c(rho_r, rho_ell, rho_d, 0))
Phi[4,1] <- 1
mu <- (diag(4) - Phi) %*% c(r_star,
                             ell_star, d_star, r_star)
Sigma12 <- diag(c(sigma_r, sigma_ell, sigma_d, 0))
Sigma <- Sigma12 %*% t(Sigma12)
model <- list(mu = mu, Phi = Phi,
              Sigma = Sigma, Sigma12 = Sigma12,
              n_w=4)

# Simulate model:
TT <- 100
set.seed(123)
W <- simul_GaussianVAR(model, nb.sim = TT)

par(mfrow=c(2,2))
par(plt=c(.15, .95, .15, .85))

plot(W[,1], type="l", col="white", ylim=c(.01, .07),
      xlab="", ylab="", las=1,
      main=expression(paste("(a) Short-term interest rate (", r[t], ")", sep="")))
```

```

grid()
lines(W[,1],lwd=2,col="black")

plot(W[,2],ylim=c(0,2),col="white",xlab="",ylab="",las=1,
      main=expression(paste("(b) Debt and fiscal limit",sep="")))
grid()
lines(W[,2],type="l",ylim=c(0,2),col="black",lwd=2,lty=1)
lines(W[,3],type="l",ylim=c(0,2),col="black",lwd=2,lty=3)
lines(W[,2]-W[,3],type="l",ylim=c(0,2),col="darkgrey",lwd=2,lty=1)

legend("topright",
      c(expression(paste("Debt-to-GDP (",d[t],")",sep="")),
        expression(paste("Fiscal limit (",l[t],")",sep="")),
        expression(paste("Fiscal space (",l[t],"-",d[t],")",sep="))),
      lty=c(3,1,1),
      lwd=c(2,2,2),
      pt.cex = 1,
      col= c("black","black","darkgrey"),
      seg.len = 3)

a <- matrix(c(0,-kappa,kappa,0),4,1)
c <- matrix(c(0,0,0,-1),4,1)

res_default <- compute_F_Shadow_Gaussian(W,
                                           psi.parameterization = model,
                                           ell_bar=0,
                                           b=0,
                                           a=a,
                                           c=c,
                                           H = H)

res_rf <- compute_F_Shadow_Gaussian(W,
                                     psi.parameterization = model,
                                     ell_bar=0,
                                     b=0,
                                     a=.00001*a,
                                     c=c,
                                     H = H)

# res_default <- compute_F_Shadow_affine(W,psi_GaussianVAR,
# #                                     psi.parameterization = model,
# #                                     ell_bar=0,
# #                                     b=0,
# #                                     a=.5*matrix(c(0,-1,1,1),4,1),
# #                                     c=matrix(c(-1,0,0,0),4,1),
# #                                     H = H)

```

```

# res_rf <- compute_F_Shadow_affine(W,psi_GaussianVAR,
#                                     psi.parameterization = model,
#                                     ell_bar=0,
#                                     b=0,
#                                     a=.00001*matrix(c(0,-1,1,1),4,1),
#                                     c=matrix(c(-1,0,0,0),4,1),
#                                     H = H)

f_PD <- res_default$F_c_equal_0
f_bond <- res_default$F
f_rf <- res_rf$F
y_PD <- t(apply(f_PD,1,cumsum))/t(matrix(1:H,H,TT))
y_rf <- t(apply(f_rf,1,cumsum))/t(matrix(1:H,H,TT))
y_bond <- t(apply(f_bond,1,cumsum))/t(matrix(1:H,H,TT))

plot(y_bond[,H],type="l",col="white",las=1,
      ylim=c(0.01,.1),xlab="",ylab="",
      main=expression(paste("(c) Long-term interest rates",sep="")))
grid()
lines(y_rf[,H],col="darkgrey",lwd=2)
lines(y_bond[,H],col="black",lwd=2,lty=3)

legend("topright",
      c(expression(paste("Risk-free (" ,tilde(r)[t*','*h] ,")",sep="")),
        expression(paste("Defaultable (" ,{r^d}[t*','*h] ,")",sep="))),
      lty=c(1,3),
      lwd=c(2,2),
      pt.cex = 1,
      col= c("darkgrey","black"),
      seg.len = 3)

# Check pricing formula -----
# n <- length(model$mu)
# nb.replic <- 10000
# all_P <- NULL
# for(i in 1:nb.replic){
#   sum_c <- 0
#   sum_max <- 0
#   w <- W
#   for(h in 1:H){
#     w <- t(matrix(model$mu,n,TT)) + w %*% t(model$Phi) +
#       matrix(rnorm(TT*n),TT,n) %*% t(model$Sigma12)
#     sum_c <- sum_c + w %*% c
#     sum_max <- sum_max + pmax(0,w %*% a)

```

```

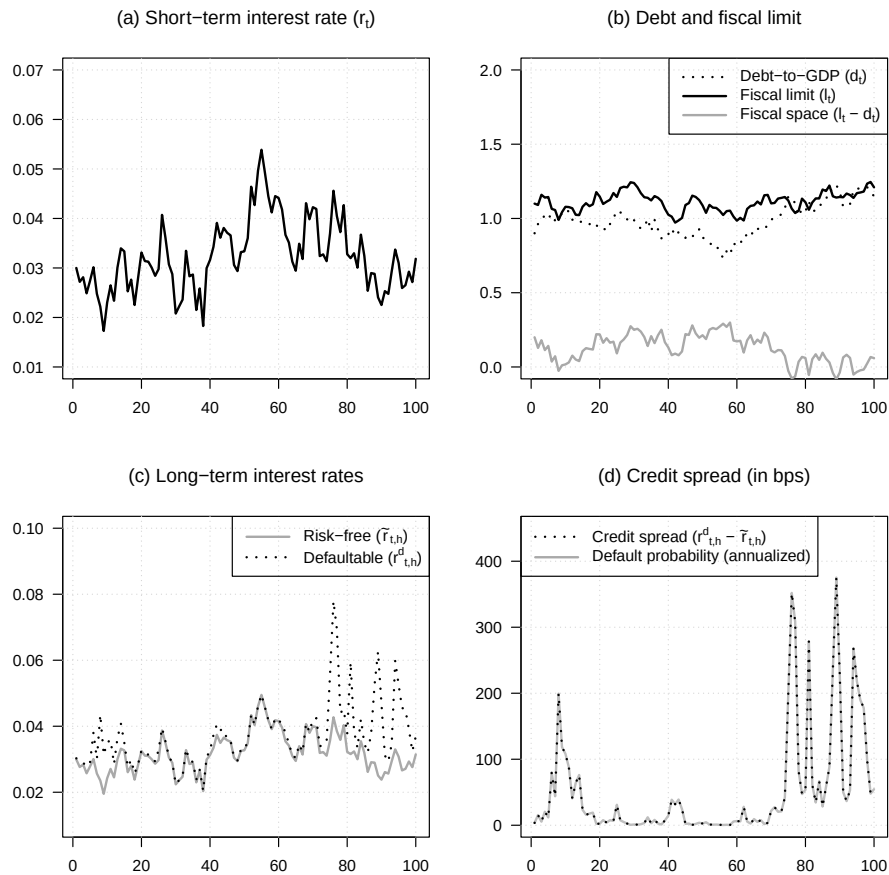
#   }
#   P <- exp(sum_c - sum_max)
#   all_P <- cbind(all_P,P)
# }
# P_simul <- apply(all_P,1,mean)
# y_simul <- -log(P_simul)/H
# lines(y_simul,col="red",lwd=2,lty=3)
# -----

# Probability of default:
PD <- 1 - exp(t(apply(f_PD,1,cumsum)))

plot(10000*(y_bond[,H]-y_rf[,H]),type="l",col="white",las=1,
      ylim=10000*c(0,.045),xlab="",ylab="",
      main=expression(paste("(d) Credit spread (in bps)",sep="")))
grid()
lines(10000*y_PD[,H],col="grey",lwd=2)
lines(10000*(y_bond[,H]-y_rf[,H]),col="black",lwd=2,lty=3)

legend("topleft",
      c(expression(paste("Credit spread (",{r^d}[t*','*h]," - ",tilde(r)[t*','*h],"")
        expression(paste("Default probability (annualized)",sep="")))),
      lty=c(3,1),
      lwd=c(2,2),
      pt.cex = 1,
      col= c("black","darkgrey"),
      seg.len = 3)

```



7.7 Non-Gaussian shadow-rate model with jump-like factors

This example illustrates a non-Gaussian shadow-rate model with jump-like factors and compares constrained and unconstrained curves.

Purpose: This example illustrates a non-Gaussian shadow-rate model with jump-like factors and compares constrained and unconstrained curves
`set.seed(7)`

```
model <- list(
  alpha = matrix(c(.0,.0),ncol=1),
  nu = matrix(c(.2,.2),ncol=1),
  mu = matrix(c(.01,.01),ncol=1),
  beta = NaN,
```

```

    n_w = 2
  ))
  model$beta <- .95*diag(1/c(model$mu))

  TT = 150

  simX <- t(simul_VARG(model,nb_periods = TT-1))
  DeltaNplus <- rpois(n = TT,simX[,1])
  DeltaNminus <- rpois(n = TT,simX[,2])
  Nplus <- cumsum(DeltaNplus)
  Nminus <- cumsum(DeltaNminus)

  par(mfrow=c(2,1))
  par(plt=c(.1,.9,.15,.85))

  scaled_simX <- simX * 10

  maxN <- max(Nplus,Nminus)+2

  plot(1:TT,rep(0,TT),las=1,col="white",
       ylim=c(0,maxN),xlab="",ylab="",
       main=expression(paste("(a) Simulated state variables",sep="")))
  grid()
  Nvalues <- 1:maxN
  ylabels_z <- seq(0,maxN/10,by=.2)
  axis(side=4,at=10*ylabels_z,labels=ylabels_z,las=1)
  lines(scaled_simX[,1],col="black",lwd=2)
  lines(scaled_simX[,2],col="darkgrey",lwd=2)

  lines(Nplus,col="black",lwd=3,lty=3)
  lines(Nminus,col="darkgrey",lwd=3,lty=3)

  legend("topleft",
        c(expression(paste(N[t]^{'+'}," (lhs)",sep="")),
          expression(paste(N[t]^{-'}," (lhs)",sep="")),
          expression(paste(z[t]^{'+'}," (rhs)",sep="")),
          expression(paste(z[t]^{-'}," (rhs)",sep="))),
        lty=c(3,3,1,1),
        lwd=c(3,3,2,2),
        pt.cex = 1,
        col= c("black","darkgrey","black","darkgrey"),
        seg.len = 3)

  # Determine state vector
  W <- cbind(simX,

```

```

      Nplus,Nminus)

# Shadow rate specification:
xi0 <- .0025
xi1 <- matrix(c(0,0,.25/100,-.25/100),ncol=1)

s <- xi0 + W %*% xi1
plot(0*s,col="white",type="l",lwd=2,las=1,ylim=c(-.01,.025),
     xlab="",ylab="",
     main=expression(paste("(b) Term structures of interest rates (w/ and w/o lower bound)")))
grid()
abline(h=0,col="black",lty=2)
lines(s,col="darkgrey",lwd=2)

model$n_w <- 4 # update number of factors.

HH <- 30 # Maximum bond maturity

# Compute bond prices without LB:
res <- compute_AB_classical(xi0, xi1, kappa0 = NaN, kappa1 = NaN,
                           H=HH, psi=psi_VARG_Poisson,
                           psi.parameterization=model)
yields <- matrix(1,TT,1) %*% matrix(res$b,nrow=1) +
  W %*% matrix(res$a,dim(res$a)[1],dim(res$a)[3])

# Compute bond prices without LB:
F_affine <- compute_F_Shadow_affine(W=W,psi_VARG_Poisson,
                                     psi.parameterization=model,
                                     ell_bar=0,b=xi0,a=xi1,c=0*xi1,
                                     H=HH,
                                     eps = 10^(-6), # to compute dG
                                     max_x = 1000,
                                     dx_statio = 1,
                                     min_dx = 1e-06,
                                     nb_x1 = 500)
f_bond <- cbind(pmax(0,s),F_affine$F[,1:(HH-1)])
yields_SR <- t(apply(f_bond,1,cumsum))/t(matrix(1:HH,HH,TT))

all_ts <- seq(5,TT-HH,by=HH+5)
for(t in all_ts){
  lines((t+1):(t+HH),yields[t,col="black",lwd=3,lty=3)
  lines((t+1):(t+HH),yields_SR[t,col="black",lwd=3)
}

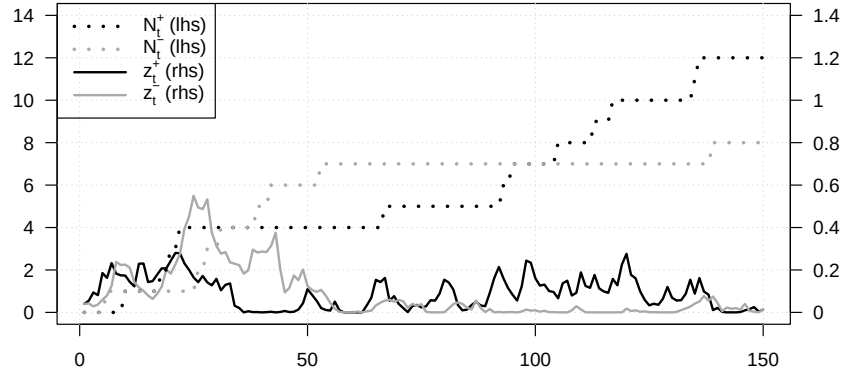
```

```

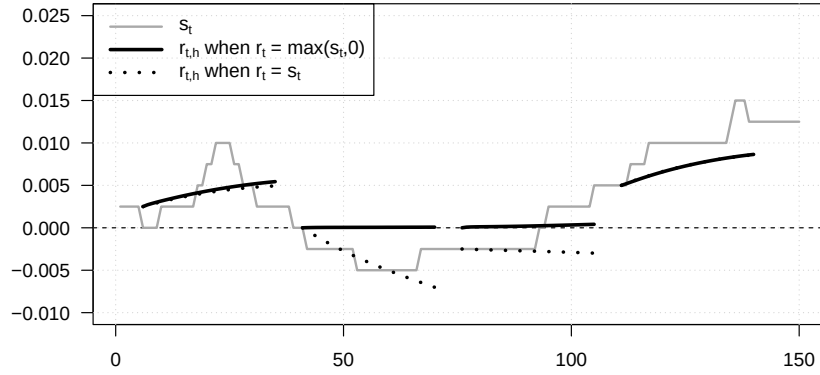
legend("topleft",
      c(expression(s[t]),
        expression(paste(r[t*'],'*h'," when ",r[t]," = max(",s[t],"",0)",sep="")),
        expression(paste(r[t*'],'*h'," when ",r[t]," = ",s[t],sep="))),
      lty=c(1,1,3),
      lwd=c(2,3,3),
      pt.cex = 1,
      col= c("darkgrey","black","black"),
      seg.len = 3)

```

(a) Simulated state variables



(b) Term structures of interest rates (w/ and w/o lower bound)



7.8 Shadow rate with ARG stochastic volatility

This example combines a shadow rate with stochastic volatility driven by an ARG factor.

Purpose: This example combines a shadow rate with stochastic volatility driven by an ARG factor

```
set.seed(1)

TT <- 200

psi_AR_CH <- function(u,psi.parameterization){
  # Laplace transform of a process defined as follows:
  # s_{t+1} = mu + phi*s_{t} + sqrt(z_t)*eps_{t+1}, with eps_t ~ N(0,1) and
  # z_t ARG(nu,beta,mu_z)

  mu <- psi.parameterization$mu
  phi <- psi.parameterization$phi

  nu <- psi.parameterization$nu
  mu_z <- psi.parameterization$mu_z
  beta <- psi.parameterization$beta

  u <- matrix(u,nrow=2)
  k <- dim(u)[2]

  u_s <- u[1,]
  u_z <- u[2,]

  b <- u_s*mu - nu*log(1 - mu_z*u_z)
  a <- matrix(NA,2,k)
  a[1,] <- u_s * phi
  a[2,] <- beta*mu_z*u_z/(1 - mu_z*u_z) + u_s^2/2

  return(list(a = a, b = b))
}

Es <- .02
phi <- .95
phi_z <- .95
mu_z <- .000015
nu <- .3
model <- list(mu = Es*(1-phi), phi = phi,
             nu = nu, mu_z = mu_z, beta = phi_z/mu_z,
             n_w = 2)

sim_z <- simul_ARG(nb.sim=TT,
                  mu = model$mu_z,
                  nu = model$nu,
```

```

        rho = phi_z)

Es <- model$mu/(1 - model$phi)
s <- Es
sim_s <- s
for(t in 2:TT){
  s <- model$mu + model$phi * s + sqrt(sim_z[t])*rnorm(1)
  sim_s <- c(sim_s,s)
}
W <- cbind(sim_s,sim_z)
s <- W[,1]

# Shadow rate specification:
xi0 <- 0
xi1 <- matrix(c(1,0),ncol=1)

HH <- 20 # Maximum bond maturity

# Compute bond prices without LB:
res <- compute_AB_classical(xi0, xi1, kappa0 = NaN, kappa1 = NaN,
                           H=HH, psi=psi_AR_CH,
                           psi.parameterization=model)
yields <- matrix(1,TT,1) %%% matrix(res$b,nrow=1) +
  W %%% matrix(res$a,dim(res$a)[1],dim(res$a)[3])

# Compute bond prices with LB:
F_affine <- compute_F_Shadow_affine(W=W,psi_AR_CH,
                                   psi.parameterization=model,
                                   ell_bar=0,b=xi0,a=xi1,c=0*xi1,
                                   H=HH,
                                   eps = 10^(-6), # to compute dG
                                   max_x = 1000,
                                   dx_statio = 1,
                                   min_dx = 1e-06,
                                   nb_x1 = 500)
f_bond <- cbind(pmax(0,s),F_affine$F[,1:(HH-1)])
yields_SR <- t(apply(f_bond,1,cumsum))/t(matrix(1:HH,HH,TT))

par(mfrow=c(2,1))
par(plt=c(.1,.95,.15,.85))
plot(sim_s,type="l",las=1,col="white",xlab="",ylab="",
     main=expression(paste("(a) State variables (",s[t], " and ",z[t],")",sep="")))
grid()
abline(h=0,col="grey")

```

```

lines(sim_s,col="grey",lwd=2,lty=1)
lines(3*sqrt(sim_z),col="black",lwd=1)

# Compute conditional variance of  $r_t$ :
mult <- model$mu + model$phi * sim_s
sigma1 <- sqrt(sim_z)
ratio <- mult/sigma1
var_rt <- (mult^2 + sigma1^2)*pnorm(ratio) -
  mult*sigma1*dnorm(ratio) - sigma1^2*g(ratio)^2
lines(3*sqrt(var_rt),lwd=2,lty=3,col="black")

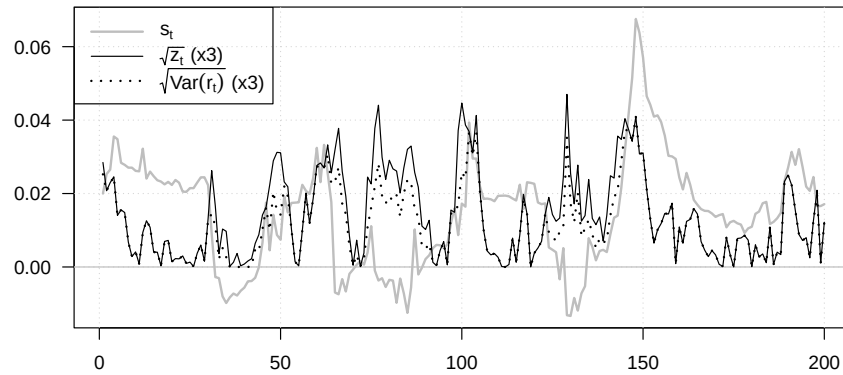
legend("topleft",
  c(expression(s[t]),
    expression(paste(sqrt(z[t]), " (x3)", sep="")),
    expression(paste(sqrt(Var(r[t])), " (x3)", sep=""))
  ),
  lty=c(1,1,3),
  lwd=c(2,1,2),
  pt.cex = 1,
  col= c("grey","black","black"),
  seg.len = 3)

plot(0*s,col="white",type="l",lwd=2,las=1,ylim=c(-.03,.08),
  xlab="",ylab="",
  main=expression(paste("(b) Term structures of interest rates (w/ and w/o lower bound)")))
grid()
abline(h=0,col="grey")
lines(s,col="darkgrey",lwd=2)

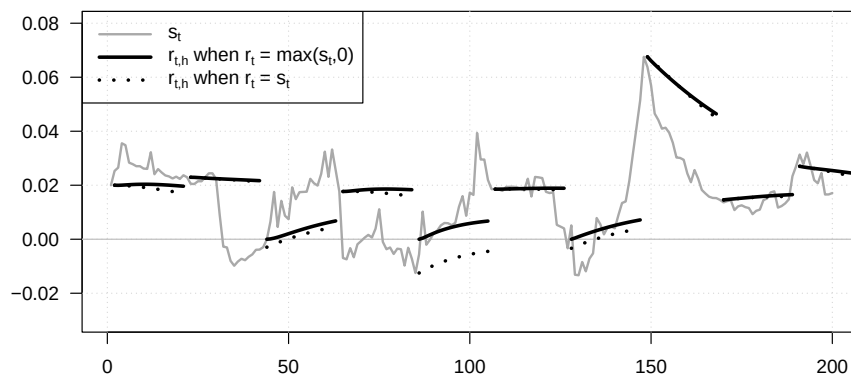
all_ts <- seq(1,TT,by=HH+1)
for(t in all_ts){
  lines((t+1):(t+HH),yields[t,],col="black",lwd=3,lty=3)
  lines((t+1):(t+HH),yields_SR[t,],col="black",lwd=3)
}

legend("topleft",
  c(expression(s[t]),
    expression(paste(r[t*','*h], " when ", r[t], " = max(", s[t], ", 0)", sep="")),
    expression(paste(r[t*','*h], " when ", r[t], " = ", s[t], sep=""))
  ),
  lty=c(1,1,3),
  lwd=c(2,3,3),
  pt.cex = 1,
  col= c("darkgrey","black","black"),
  seg.len = 3)

```

(a) State variables (s_t and z_t)

(b) Term structures of interest rates (w/ and w/o lower bound)



7.9 Affine-transform methods to a simple climate-finance model

This example applies affine-transform methods to a simple climate-finance model.

Purpose: This example applies affine-transform methods to a simple climate-finance model.

```
library(DTAM)

time_step_yrs <- 5
HH <- 200 # max maturity used to evaluate inter-temporal utility

model <- list(mu_c = .015*time_step_yrs,
              gamma = 1.45,
```

```

        et = .0075*time_step_yrs, # initial growth rate of carbon concentrations
        E0 = 2170,
        Et = 3315,
        delta = 0.95,
        mu_e = 0.001,
        rho_e = .8,
        rho = .99999,
        S = 3,
        alpha = .0018*time_step_yrs*1,
        beta = .0018*time_step_yrs*1,
        tau_star = 3,
        rho_star = .9,
        stdv_tau_star = .5,
        Ct = 60*10^12, # Global consumption in 2025, in USD
        n_w = 3)

psi_IAM <- function(u, psi.parameterization){
  # This function returns the conditional Laplace transform of
  # the state vector of the macro-climate model.
  # The state vector is  $w_t = (e_t, \tau_t, \tau_t^*)'$ .

  mu_e <- psi.parameterization$mu_e
  rho_e <- psi.parameterization$rho_e
  rho <- psi.parameterization$rho
  S <- psi.parameterization$S
  alpha <- psi.parameterization$alpha
  beta <- psi.parameterization$beta
  tau_star <- psi.parameterization$tau_star
  rho_star <- psi.parameterization$rho_star
  sigma_star <- sqrt(1-rho_star^2)*psi.parameterization$stdv_tau_star

  u <- matrix(u,nrow=3)
  k <- dim(u)[2]

  u_e <- matrix(u[1,],nrow=1)
  u_tau <- matrix(u[2,],nrow=1)
  u_star <- matrix(u[3,],nrow=1)

  a <- matrix(0,3,k)
  a[1,] <- rho_e*u_e/(1 - mu_e*u_e) + S/log(2)*u_tau
  a[2,] <- rho*u_tau
  a[3,] <- rho_star*u_star
  b <- (u_star*tau_star*(1-rho_star) + u_star^2*sigma_star^2/2)

  return(list(a = a, b = b))

```

```

}

# Compute distribution of the date of net-zero emissions -----
u1 <- matrix(c(-1000,0,0),ncol=1)
res <- reverse_MHLT(psi_IAM,u1=u1,u2=0*u1,H=40,psi.parameterization = model)
A <- matrix(res$A,nrow=3)
B <- c(res$B)
P_net_zero <- exp(B + model$et*A[1,])

# Compute distribution of atmospheric carbon concentration in 2100 -----
all_h <- c(2060,2080,2100)
step <- .025

values_tau <- seq(1.8,4.5,by=step)
PDF <- matrix(0,length(values_tau),length(all_h))

psi_tau <- function(u,model){
  # This function returns  $E_t[\exp(u \cdot \tau_{t+h})]$ 
  # The horizon h is one of the 'model' entries.
  U <- rbind(0,1*c(u),0)
  res <- reverse_MHLT(psi_IAM,
                      u1 = U,u2 = 0*U,
                      H = h,model)
  Xt <- matrix(c(model$et,model$S/log(2)*log(model$Et/model$E0),model$tau_star),ncol=1)
  psi <- exp(res$B[1,1,h] + t(res$A[, ,model$h]) %*% Xt)
  return(psi)
}

count_h <- 0
for(h in all_h){
  count_h <- count_h + 1
  model$h <- (h - 2025)/time_step_yrs
  PDF[,count_h] <- make_pdf(model,values.of.variable = values_tau,
                           psi=psi_tau, max_x = 500, min_x = 10^(-5),
                           nb_x = 1000)/step
}

par(mfrow=c(1,2))
par(plt=c(.12,.95,.2,.85))
plot(2025+time_step_yrs*(1:(length(P_net_zero)-1)),diff(P_net_zero),
     col="white",xlab="Year",ylab="",las=1,
     main=expression(paste("(a) Pdf of date of net-zero emissions (" ,e[t] , " = 0)",sep=
grid()
lines(2025+time_step_yrs*(1:(length(P_net_zero)-1)),diff(P_net_zero),
     type="l",lwd=2,xlab="Year",ylab="")

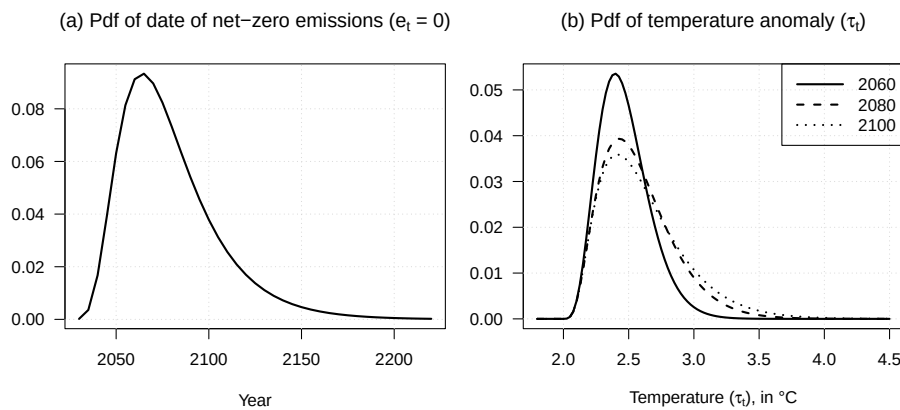
```

```

plot(values_tau, PDF[,1], type = "l", col = "white",
      xlab = expression(paste("Temperature (", tau[t], "), in °C", sep = "")),
      ylab = "", las = 1,
      main=expression(paste("(b) Pdf of temperature anomaly (",tau[t],")",sep="")))
grid()
for(i in 1:length(all_h)){
  lines(values_tau,PDF[,i],lty=i,lwd=2)
}

legend("topright",legend=all_h,
      lty=c(1,2,3),
      lwd=2,
      pt.cex = 1,
      col= "black",
      seg.len = 3)

```



The next chunk computes the social cost of carbon in the affine climate-finance model, first with and then without the risk-premium component.

Purpose: Compute the social cost of carbon and separate the risk-premium component.

```

compute_SCC <- function(model,HH=200,
  indic_compute_riskpremium = FALSE,
  eps = 10^(-6), # to compute dG
  max_x = 1000000,
  dx_statio = 1000,
  min_dx = 1e-06,
  nb_x1 = 1000){
  # This function computes the social of carbon (SCC) in the context of a small
  # Integrated Assessment Model (IAM) featuring a form of stochastic tipping point.

  W <- t(matrix(c(model$et,

```

```

        model$S/log(2)*log(model$Et/model$E0),
        model$tau_star),3,2))
# Use two values (i.e., two rows) to compute dU/dE

# Perturb emissions to approximate the marginal effect on utility.
epsE <- 1 # Delta E_t, in GtCO2
W[2,2] <- model$S/log(2)*log((model$Et + epsE)/model$E0)

a <- (model$gamma - 1) * matrix(c(0,model$beta,-model$beta),ncol=1)
c <- (model$gamma - 1) * matrix(c(0,model$alpha,0),ncol=1)
# Compute expected discounted marginal utility under the affine model.
F_affine <- compute_F_Shadow_affine(W=W,psi_IAM,
                                   psi.parameterization=model,
                                   ell_bar=0,
                                   b=0,a=a,c=c,chi=1,
                                   H=HH,eps = eps, # to compute dG
                                   max_x = max_x,dx_statio = dx_statio,
                                   min_dx = min_dx,nb_x1 = nb_x1)
f_bond <- F_affine$F - log(model$delta) - (1 - model$gamma)*model$mu_c
aux <- exp(t(apply(-cbind(0,f_bond),1,cumsum)))

Ct <- model$Ct*time_step_yrs # $tr, 2024
U <- Ct^(1 - model$gamma)/(1 - model$gamma) * apply(aux,1,sum)
dU <- (U[2] - U[1])/epsE

SCC <- -Ct^(model$gamma) * dU / 10^9 # /10^9 because eps is in GtCO2

if(indic_compute_riskpremium){
  # Compute the SDF discount factors used to isolate the risk-premium component.
  a <- model$gamma * matrix(c(0,model$beta,-model$beta),ncol=1)
  c <- model$gamma * matrix(c(0,model$alpha,0),ncol=1)
  F_affine_SDF <- compute_F_Shadow_affine(W=W,psi_IAM,
                                           psi.parameterization=model,
                                           ell_bar=0,
                                           b=0,a=a,c=c,chi=1,
                                           H=HH,eps = eps, # to compute dG
                                           max_x = max_x,dx_statio = dx_statio,
                                           min_dx = min_dx,nb_x1 = nb_x1)
  f_bond <- F_affine_SDF$F - log(model$delta) + model$gamma*model$mu_c
  DF <- exp(t(apply(-cbind(0,f_bond),1,cumsum))) # Discount factors
  DF <- DF[1,]

  # Compute the consumption response to the emissions perturbation.
  a <- matrix(c(0,model$beta,-model$beta),ncol=1)
  c <- - matrix(c(0,model$alpha,0),ncol=1)

```



```

F_affine_C <- compute_F_Shadow_affine(W=W,psi_IAM,
                                     psi.parameterization=model,
                                     ell_bar=0,
                                     b=0,a=a,c=c,chi=-1,
                                     H=HH,eps = eps, # to compute dG
                                     max_x = max_x,dx_statio = dx_statio,
                                     min_dx = min_dx,nb_x1 = nb_x1)

f_bond <- F_affine_C$F - model$mu_c
C      <- Ct * exp(t(apply(-cbind(0,f_bond),1,cumsum)))
dc_h   <- (C[2,] - C[1,])/epsE
}else{
  DF    <- NaN
  dc_h  <- NaN
  F_affine_SDF <- NaN
  F_affine_C   <- NaN
}

SCC_noRP <- sum(-dc_h * DF)/10^9

return(list(SCC=SCC,SCC_noRP=SCC_noRP,
            U = U,
            F_affine=F_affine,
            F_affine_SDF=F_affine_SDF,
            F_affine_C=F_affine_C,
            DF=DF,C=C,dc_h=dc_h))
}

res_SCC <- compute_SCC(model,HH=HH,indic_compute_riskpremium=TRUE)
SCC <- res_SCC$SCC

# Print the SCC with and without the risk-premium component.
print(SCC)

## [1] 746.4317
print(res_SCC$SCC_noRP)

## [1] 747.1965

```

The following table records the calibration used in the social-cost-of-carbon exercise.

```

# Purpose: The following table records the calibration used in the social-cost-of-carbon
# exercise

# Load required package
library(xtable)

```

```

# ---- Parameters to exclude ----
exclude_params <- c("rho", "alpha", "beta", "n_w", "h", "tau_star") # add any names you want to exclude

# Filter model list
model_filtered <- model[!names(model) %in% exclude_params]

# ---- Mapping between R names and LaTeX names ----
names_R <- c("mu_c", "gamma", "et", "E0", "Et", "delta", "mu_e", "rho_e", "rho", "S",
             "alpha", "beta", "tau_star", "rho_star", "stdv_tau_star", "Ct")

names_Latex <- c(
  "\\mu_c",      # Mean consumption growth rate
  "\\gamma",     # Risk aversion
  "e_t",        # Carbon growth rate
  "\\mathcal{E}_0", # Pre-industrial carbon stock
  "\\mathcal{E}_t", # Current carbon stock
  "\\delta",     # Discount factor
  "\\mu_e",      # Mean carbon growth rate
  "\\rho_e",     # Persistence of carbon growth
  "\\rho",       # Persistence of carbon stock
  "$S$",        # ECS
  "\\alpha",     # Damage slope
  "\\beta",      # High-damage slope
  "\\tau_*",     # Tipping threshold
  "\\rho_*",     # Persistence of threshold
  "\\sigma(\\tau_t^*)", # Volatility of threshold
  "C_t"         # Global consumption
)

# Create a lookup table
latex_map <- setNames(names_Latex, names_R)

# ---- Convert to data frame ----
param_df <- data.frame(
  Parameter = names(model_filtered),
  Value = unlist(model_filtered)
)

# Replace R names by LaTeX names
param_df$Parameter <- latex_map[param_df$Parameter]

# ---- Format numbers ----
param_df$Value <- formatC(param_df$Value, format = "g", digits = 5)

# ---- Optional descriptions ----

```

```

descriptions <- c(
  "$\\mu_c$" = "Mean consumption growth rate (5 years)",
  "$\\gamma$" = "Risk aversion coefficient",
  "$e_t$" = "Initial carbon growth rate (5 years)",
  "$\\mathcal{E}_0$" = "Pre-industrial carbon stock (GtCO$_2$)",
  "$\\mathcal{E}_t$" = "Current carbon stock (GtCO$_2$)",
  "$\\delta$" = "Time discount factor (5 years time step)",
  "$\\mu_e$" = "Scale of carbon growth rate shocks",
  "$\\rho_e$" = "Persistence of carbon growth rate",
  "$S$" = "Equilibrium climate sensitivity ( $^{\circ}$ C per doubling)",
  "$\\alpha$" = "Baseline damage slope (change in 5-year consumption growth per  $^{\circ}$ C)",
  "$\\beta$" = "High-damage slope (change in 5-year consumption growth per  $^{\circ}$ C)",
  "$\\tau_*$" = "Expected tipping threshold ( $^{\circ}$ C)",
  "$\\rho_*$" = "Persistence of tipping threshold",
  "$\\sigma(\\tau_t^*)$" = paste0(
    "Unconditional volatility of tipping threshold ",
    "($\\sqrt{\\mathbb{V}ar(\\tau_t^*)}$, in  $^{\circ}$ C)"
  ),
  "$C_t$" = "Global consumption (USD)"
)

param_df$Description <- descriptions[param_df$Parameter]

# ---- Generate LaTeX table ----
latex_table <- c(
  "\\begin{table}[ht]",
  "\\centering",
  "\\caption{This table reports the parameterization of the model used in Example~\\ref{exm.SCC}.",
  "\\label{tab:model_parameters}",
  "\\begin{tabularx}{\\textwidth}{l r r X}",
  "\\hline",
  "Parameter & Value & Description & \\",
  "\\hline",
  apply(param_df, 1, function(row) {
    paste(row['Parameter'], '&', row['Value'], '&&', row['Description'], '\\\\')
  }),
  "\\hline",
  "\\end{tabularx}",
  "% ---- Add explanatory note below ----",
  "\\vspace{0.3em}",
  paste0(
    "\\parbox{1.0\\textwidth}{\\footnotesize\\textit{Notes}: ",
    "Calibration values are based on standard IAM assumptions for ",
    "consumption growth and equilibrium climate sensitivity. Carbon stock ",
    "and temperature parameters are consistent with historical data. The ",

```

```

        "unconditional volatility of the threshold satisfies ",
        "$\\sigma(\\tau_t^*) = \\sqrt{1-\\rho_*^2}\\sigma_*$."
    ),
    "\\end{table}"
)

# ---- Save to file ----
output_dir <- "tables"
output_file <- file.path(output_dir, "table_param_SCC.txt")
if (!dir.exists(output_dir)) dir.create(output_dir, recursive = TRUE)
writeLines(latex_table, con = output_file)

```

This figure shows how the social cost of carbon responds to the average tipping threshold and to the curvature of damages.

Purpose: This figure shows how the social cost of carbon responds to the average tipping threshold and to the curvature of damages

Define the baseline damage slopes and threshold grid.

```
alpha_baseline <- .001 * time_step_yrs * 1
```

```
model$alpha <- .001 * time_step_yrs * 1
```

```
model$beta <- .001 * time_step_yrs * 2
```

```
model$tau_star <- 2.5
```

```
all_alpha <- c(.5*alpha_baseline,alpha_baseline,alpha_baseline)
```

```
all_beta <- c(2*alpha_baseline,2*alpha_baseline,4*alpha_baseline)
```

```
all_tau_star <- rev(seq(2.25,4.25,by=.5))
```

```
SCC_matrix <- matrix(NA,length(all_alpha),length(all_tau_star))
```

Recompute the SCC over the damage and threshold scenarios.

```

for(i in 1:length(all_alpha)){
  for(j in 1:length(all_tau_star)){
    model$alpha <- all_alpha[i]
    model$beta <- all_beta[i]
    model$tau_star <- all_tau_star[j]
    res_SCC <- compute_SCC(model,HH=HH,
                          indic_compute_riskpremium=TRUE,
                          eps = 10^(-6), # to compute dG
                          max_x = 10000,
                          dx_statio = 200,
                          min_dx = 1e-05,
                          nb_x1 = 500)
    SCC_matrix[i,j] <- res_SCC$SCC
  }
}

```

```

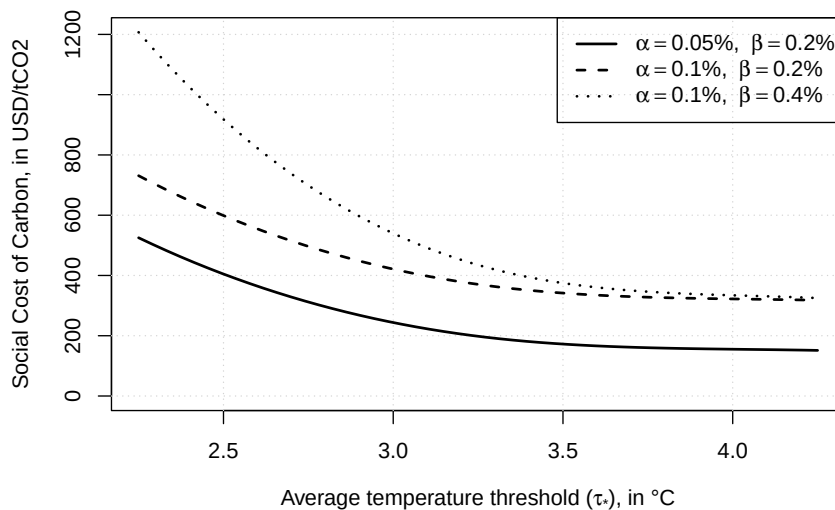
}

# Plot the SCC response curves.
par(mfrow=c(1,1))
par(plt=c(.15,.95,.2,.95))
plot(all_tau_star, 0 * all_tau_star, ylim = c(0, max(SCC_matrix)),
     col = "white",
     xlab = expression(paste("Average temperature threshold (",
                             tau['*'], ")", in "°C", sep = "")),
     ylab = "Social Cost of Carbon, in USD/tCO2")
grid()
for(i in 1:length(all_alpha)){
  spline_points <- spline(all_tau_star, SCC_matrix[i,], n = 50)
  lines(spline_points$x,spline_points$y,lwd=2,lty=i)
  # points(all_tau_star, SCC_matrix[i,],col="red")
}

legend_labels <- mapply(function(a, b) {
  bquote(alpha == .(100*a/time_step_yrs)*"%", " ~ beta == .(100*b/time_step_yrs)*"%")
}, all_alpha, all_beta, SIMPLIFY = FALSE)

# Add legend
legend("topright", legend = legend_labels, lty = 1:3, lwd = 2)

```



The final block checks the analytical formula against simulated paths of the climate-economy system.

```

# Purpose: Check the analytical formula against simulated climate-economy paths.
# Check formulas using simulations.

```

```

nb.replic <- 40000
TT <- 200

# Compute the analytical benchmark.
res_SCC <- compute_SCC(model,HH=TT,indic_compute_riskpremium=TRUE,
  eps = 10^(-5), # to compute dG
  max_x = 100000,
  dx_statio =100,
  min_dx = 1e-8,
  nb_x1 = 10000)

model$sigma_star <- model$stdv_tau_star*sqrt(1-model$rho_star^2)

# Simulate the tipping threshold and the carbon stock.
res <-
  simul_GaussianVAR(list(mu=model$tau_star*(1-model$rho_star),Phi = matrix(model$rho_star,
    Sigma12 = matrix(model$sigma_star)),
    nb.sim=TT,nb.replic = nb.replic)
sim_tau_star <- matrix(res,nrow=dim(res)[1])

sim_e <- simul_ARG(nb.sim = TT,
  mu=model$mu_e,
  alpha=0,
  nu = 0,
  rho = model$rho_e,
  w0 = model$et,
  nb.replic=nb.replic)
sim_logE <- log(model$Et) + apply(sim_e,2,cumsum)
epsE <- 1
sim_logE_dE <- log(model$Et + epsE) + apply(sim_e,2,cumsum)
sim_tau <- model$S/log(2)*(sim_logE - log(model$E0))
sim_tau_dE <- model$S/log(2)*(sim_logE_dE - log(model$E0))

sim_dc <- model$mu_c - model$alpha*sim_tau - model$beta*pmax(0,sim_tau - sim_tau_star)
sim_dc_demeaned <- - model$alpha*sim_tau - model$beta*pmax(0,sim_tau - sim_tau_star)

sim_dc_dE <- model$mu_c - model$alpha*sim_tau_dE - model$beta*pmax(0,sim_tau_dE - sim_tau_star)

# Compare simulated transform moments with the affine formula.
par(mfrow=c(1,2))
par(plt=c(.15,.95,.1,.9))

sim_exp_sum_dc <- exp(apply(log(model$delta)+(1-model$gamma)*sim_dc,2,cumsum))
sim_exp_sum_dc_dE <- exp(apply(log(model$delta)+(1-model$gamma)*sim_dc_dE,2,cumsum))
E_exp_sum_dc_simul <- apply(sim_exp_sum_dc,1,mean)

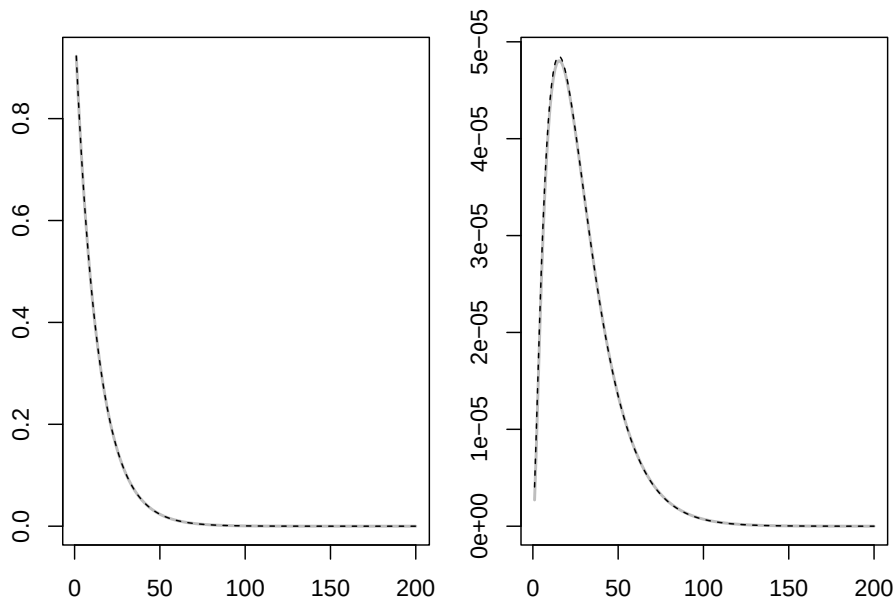
```

```

E_exp_sum_dc_dE_simul <- apply(sim_exp_sum_dc_dE,1,mean)
f <- res_SCC$F_affine$F - log(model$delta) - (1-model$gamma)*model$mu_c
E_exp_sum_dc_formula <- exp(t(apply(-f,1,cumsum)))
plot(E_exp_sum_dc_simul,type="l",lwd=2,col="grey")
lines(E_exp_sum_dc_formula[1,],col="black",lty=2)

plot(E_exp_sum_dc_dE_simul-E_exp_sum_dc_simul,type="l",lwd=2,col="grey",
      ylim=c(min(E_exp_sum_dc_formula[2,]-E_exp_sum_dc_formula[1,]),
              max(E_exp_sum_dc_formula[2,]-E_exp_sum_dc_formula[1,])))
lines(E_exp_sum_dc_formula[2,]-E_exp_sum_dc_formula[1,],col="black",lty=2)

```



```

SCC_simul <- model$Ct*time_step_yrs/(model$gamma-1)*
  (sum(E_exp_sum_dc_dE_simul) - sum(E_exp_sum_dc_simul))/epsE/10^9

# Print analytical and simulation-based SCC estimates.
print(cbind(res_SCC$SCC,SCC_simul))

##                SCC_simul
## [1,] 1211.177 1200.441

```


Chapter 8

Forward, futures, dividends, commodity pricing, and convenience yields

8.1 Persistent seasonal factors can generate recurring convenience-yield patterns

This example illustrates how persistent seasonal factors can generate recurring convenience-yield patterns.

```
# Purpose: This example illustrates how persistent seasonal factors can generate recurring  
# convenience-yield patterns  
library(DTAM)  
  
ell <- 12  
nb_j <- trunc(ell/2)  
  
# Build the trigonometric transition matrix for seasonal cycles.  
rho <- rep(.995,nb_j)  
# sigma_xi <- rep(.01,nb_j)  
sigma_xi <- c(.01,.01,rep(0,nb_j-2))  
  
Theta <- matrix(0,2*nb_j,2*nb_j)  
for(j in 1:nb_j){  
  lambda_j <- 2*pi*j/ell  
  Theta[(2*(j-1)+1):(2*j),(2*(j-1)+1):(2*j)] <- rho[j] * matrix(c(+cos(lambda_j),  
                                                                -sin(lambda_j),  
                                                                +sin(lambda_j),
```

```

}
Omega <- diag(sigma_xi %x% c(1,1))

nb.sim <- 100*ell
start_set <- 92*ell

set.seed(123)

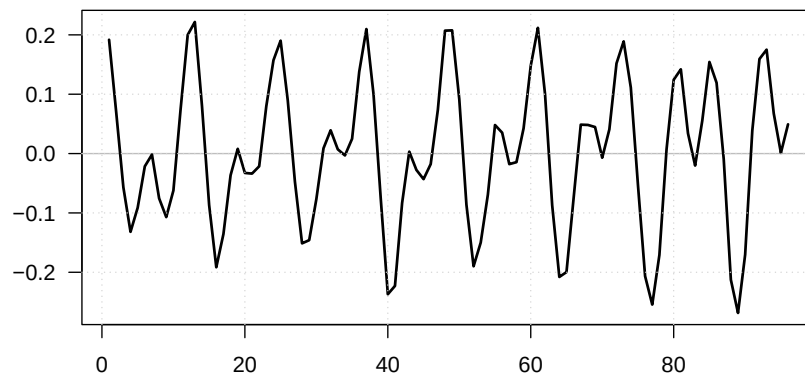
# Simulate the seasonal state vector.
model <- list(Phi = Theta,
              mu = matrix(0,2*nb_j,1),
              Sigma12 = Omega)

x0 <- matrix(rep(1,2*nb_j),ncol=1)
x0 <- rnorm(n=2*nb_j)

simulated_gamma <- simul_GaussianVAR(model,x0 = x0,nb.sim = nb.sim)
gamma <- simulated_gamma %*% (matrix(1,nb_j,1) %x% matrix(c(1,0),2,1))

# Plot the final part of the simulated seasonal component.
par(mfrow=c(1,1))
par(plt=c(.15,.95,.15,.95))
plot(gamma[(start_set+1):nb.sim],col="white",
     type="l",las=1,
     xlab="",ylab="")
abline(h=0,col="grey")
lines(gamma[(start_set+1):nb.sim],lwd=2)
grid()
+cos(lambda_j)),2,2)

```



8.2 Observed futures term structures at a few selected dates

This example plots observed futures term structures at a few selected dates. The futures panel contains NYMEX natural-gas futures settlement prices. The rows are monthly observation dates from January 2010 to January 2025, and the columns are contract-specific settlement-price series for monthly maturities from the February 2010 contract to the December 2030 contract. The data were extracted from licensed Refinitiv/Datastream-style queries: the Refinitiv sheet uses RIC-style futures tickers such as NGG25, while the Datastream sheet uses mnemonics such as NNG0210 and series names of the form NYM-NATURAL GAS FEB 2010 - SETT. PRICE.¹

```
# Purpose: This example plots observed futures term structures at a few selected dates
library(DTAM)
library(stringr)

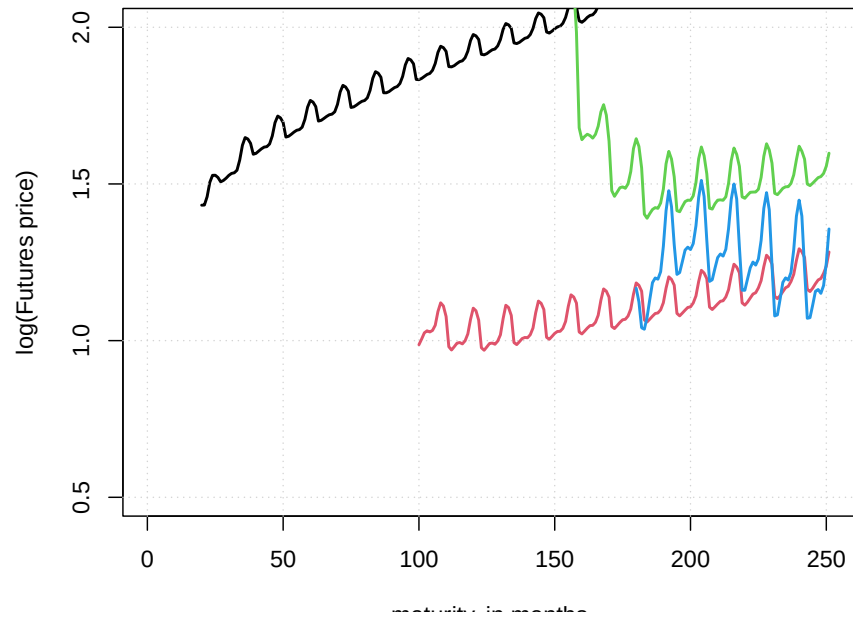
private_futures_file <- "private_data/Futures.rda"
precomputed_figure <- if (knitr::is_latex_output()) {
  "_main_files/figure-latex/fig_futures-1.pdf"
} else {
  "figure/fig_futures-1.png"
}

if (!file.exists(private_futures_file) && file.exists(precomputed_figure)) {
  knitr::include_graphics(precomputed_figure)
} else {
  if (!file.exists(private_futures_file)) {
    stop("Private futures data not found: ", private_futures_file, call. = FALSE)
  }
  load(private_futures_file)

  # Extract selected futures curves from the private local data set.
  par(plt=c(.15,.95,.15,.95))
  futures <- as.matrix(Futures[,2:dim(Futures)[2]])
  dates <- c(20,100,150,180)
  plot(log(futures[dates[1],]),type="l",ylim=c(0.5,2),
        ylab="log(Futures price)",xlab="maturity, in months",lwd=2)
  grid()
  # Overlay the remaining selected dates.
  for(i in 2:length(dates)){
    lines(log(futures[dates[i],]),col=i,lwd=2)
```

¹The underlying settlement-price observations are not distributed with DTAM or with the public version of this Rbookdown project. A public blank extraction workbook, with the observation dates, contract headers, and contract metadata but no price observations, is provided here: [data/Futures_extraction_template.xlsx](#).

```
}
}
```



8.3 Gaussian state-space model for commodity futures

This example fits a Gaussian state-space model to commodity futures and compares fitted and observed curves.

```
# Purpose: This example fits a Gaussian state-space model to commodity futures and compares
# fitted and observed curves
library(DTAM)
library(stringr)

private_futures_file <- "private_data/Futures.rda"
precomputed_figure <- if (knitr::is_latex_output()) {
  "_main_files/figure-latex/fig_futures_KF-1.pdf"
} else {
  "figure/fig_futures_KF-1.png"
}

if (!file.exists(private_futures_file) && file.exists(precomputed_figure)) {
  knitr::include_graphics(precomputed_figure)
} else {
```

```

if (!file.exists(private_futures_file)) {
  stop("Private futures data not found: ", private_futures_file, call. = FALSE)
}
load(private_futures_file)

# Prepare the panel of futures prices by pricing date and time to maturity.

tickers <- names(Futures)[2:dim(Futures)[2]]
nb_maturities <- length(tickers)

month_letters <- c("JAN", "FEB", "MAR", "APR", "MAY", "JUN",
                  "JUL", "AUG", "SEP", "OCT", "NOV", "DEC")

months_letter_in_ticker <- str_sub(tickers, 17, 19)
months <- apply(matrix(1:length(months_letter_in_ticker), ncol=1), 1,
                function(i){which(months_letter_in_ticker[i]==month_letters)})
years <- as.numeric(str_sub(tickers, 21, 24))
tickers_maturities <- as.Date(paste(years, "-", months, "-01", sep=""))

nb_dates <- 60
N <- dim(Futures)[1]
list_smpl_dates <- sort(Futures$Date[(N-nb_dates+1):N])

max_maturity <- 60 # in months

data <- matrix(NA, nb_dates, max_maturity)
tempo <- NULL
for(count_date in 1:nb_dates){
  ddate <- list_smpl_dates[count_date]
  indic_date <- which(Futures$Date==ddate)
  for(iii in 1:nb_maturities){
    p <- Futures[indic_date, 1+iii]
    # Compute maturity, in months.
    diff_years <- as.numeric(format(as.Date(tickers_maturities[iii]), "%Y")) -
      as.numeric(format(ddate, "%Y"))
    diff_month <- as.numeric(format(as.Date(tickers_maturities[iii]), "%m")) -
      as.numeric(format(ddate, "%m"))
    maturity_in_months <- 12*diff_years + diff_month

    if((maturity_in_months >= 1)&(maturity_in_months <= max_maturity)){
      data[count_date, maturity_in_months] <- p
    }
  }
}
Y <- log(data)

```

```

# Build the Gaussian state-space representation used in the Kalman filter.

make_model <- function(phi,rho,sigma_xi,sigma_mu,sigma_z){

  Omega    <- diag(sigma_xi %x% c(1,1))

  Theta    <- matrix(0,2*nb_j,2*nb_j)
  for(j in 1:nb_j){
    lambda_j <- 2*pi*j/ell
    Theta[(2*(j-1)+1):(2*j),(2*(j-1)+1):(2*j)] <- rho[j] * matrix(c(+cos(lambda_j),
                                                                    -sin(lambda_j),
                                                                    +sin(lambda_j),
                                                                    +cos(lambda_j)),2,2)

  }

  Phi <- matrix(0,2+nb_j*2,2+nb_j*2)
  Phi[3:(2+nb_j*2),3:(2+nb_j*2)] <- Theta
  Phi[1,1] <- 1
  Phi[2,2] <- phi

  Sigma <- matrix(0,2+nb_j*2,2+nb_j*2)
  Sigma[1,1] <- sigma_mu
  Sigma[2,2] <- sigma_z
  Sigma[3:(2+nb_j*2),3:(2+nb_j*2)] <- Omega

  # Pricing formulas:
  psi.parameterization <- list(
    mu = matrix(0,2+2*nb_j,1),
    Phi = Phi,
    Sigma12 = Sigma,
    Sigma = Sigma %%% t(Sigma))

  return(psi.parameterization)
}

estim_model <- function(param){

  phi <- exp(param[1])/(1+exp(param[1]))
  rho <- exp(param[2:(1+nb_j)])/(1+exp(param[2:(1+nb_j)]))
  # sigma_xi <- param[(1+nb_j+1):(1+2*nb_j)]
  # sigma_mu <- param[2+2*nb_j]
  # sigma_z <- param[3+2*nb_j]

  # Pricing formulas:
  model <- make_model(phi,rho,sigma_xi,sigma_mu,sigma_z)
}

```

8.3. GAUSSIAN STATE-SPACE MODEL FOR COMMODITY FUTURES 143

```

# Use the reverse-order multi-horizon transform to price futures across maturities.
h = max_maturity # maximum maturity
u1 <- matrix(c(1,1,rep(c(1,0),nb_j)),ncol=1)
u2 <- 0 * u1
AB <- reverse_MHLT(psi_GaussianVAR,u1,u2,H=h,psi.parameterization=model)

# Define matrices needed in the Kalman_filter procedures:
nu_t <- matrix(0,dim(Y)[1],2+2*nb_j) # intercept in transition equation
H <- model$Phi # autoregressive matrix in transition equation
G <- t(matrix(AB$A,2+2*nb_j,max_maturity)) # matrix of loadings
mu_t <- t(matrix(AB$B,max_maturity,nb_dates))
N <- model$Sigma12 # volatility matrix in transition equation
M <- diag(rep(.1,max_maturity)) # matrix of std dev of measurement errors
Sigma_0 <- 2*diag(2+2*nb_j) # unconditional variance of w
rho_0 <- rep(0,2+2*nb_j) # unconditional mean of w
filter.res <- Kalman_filter(Y,nu_t,H,N,mu_t,G,M,Sigma_0,rho_0)
# smoother.res <- Kalman_smoother(Y,nu_t,H,N,mu_t,G,M,Sigma_0,rho_0)

return(filter.res)
}

loglik <- function(param){
  # print(param)
  filter.res <- estim_model(param)
  return(-filter.res$loglik)
}

# Set model specifications:
ell <- 12
nb_j <- trunc(ell/2)

# Model parameterization:
rho <- rep(.995,nb_j)
phi <- .98 # autocorrelation of z
sigma_xi <- rep(0.01,nb_j)
sigma_mu <- .03
sigma_z <- .10

param <- c(log(phi/(1-phi)),log(rho/(1-rho)),sigma_xi,sigma_mu,sigma_z)

# res.optim <- optim(par=param,
# fn=log-likelihood,
# method="Nelder-Mead",
# control=list(trace=FALSE,maxit=50),hessian=FALSE)

```

```

# param <- res.optim$par

filter.res <- estim_model(param)

Y_fitted <- filter.res$fitted.obs

# Produce the chart comparing fitted and observed futures curves.

indic_selected_dates <- c(9,33,60)
selected_dates <- list_smpl_dates[indic_selected_dates]

cols <- gray.colors(n=length(indic_selected_dates), start = .0, end = .7)

nf <- layout(
  matrix(c(1,2), nrow=2, ncol=1, byrow=TRUE),
  widths=c(1,1),
  heights=c(1,2)
)

par(plt=c(.1,.95,.2,.8))
# par(mfrow=c(2,1))

# sigma_mu <- sd(diff(filter.res$r[,1]))
# sigma_z <- sd(diff(filter.res$r[,2]))
# plot(list_smpl_dates,filter.res$r[,1],type="l")
# lines(list_smpl_dates,filter.res$r[,2],type="b")

gamma <- filter.res$r[,3:(2+2*nb_j)] %*% matrix(rep(c(1,0),nb_j),ncol=1)
plot(list_smpl_dates,gamma,type="l",lwd=2,las=1,
     xlab="",ylab="",
     main=expression(paste("(a) Estimated seasonality component ",gamma[t],sep="")))
abline(h=0,col="black")
grid()

plot(tickers_maturities,rep(0,length(tickers_maturities)),
     xaxt="n",yaxt="n",las=1,
     xlab = "maturity",ylab = "",
     ylim=c(-.5 + min(exp(Y[indic_selected_dates,]),na.rm = TRUE),
            .0 + max(exp(Y[indic_selected_dates,]),na.rm = TRUE)),
     xlim=c(as.Date("2020-06-01"),as.Date("2030-03-01")),
     main=expression(paste("(b) Term structure of futures prices ",F[t*','*',*T],sep="")))
col="white")

xlab <- seq(as.Date("2020-01-01"), by="+6 month", length.out=60)

```



```

axis.Date(1, at=xlab, format="%m.%y",las=2)
for(i in 1:length(xlab)){
  abline(v=xlab[i],col="grey",lty=3)
}
ylab <- seq(trunc(exp(min(Y[indic_selected_dates,],na.rm = TRUE))),
            trunc(exp(max(Y[indic_selected_dates,],na.rm = TRUE))+1),by=.5)
axis(2,at=ylab,labels=ylab,las=1)
for(i in 1:length(ylab)){
  abline(h=ylab[i],col="grey",lty=3)
}

count_date <- 0
for(ddate in selected_dates){
  count_date <- count_date + 1
  ddate <- as.Date(ddate)
  indic_date <- which(ddate == list_smpl_dates)

  # x-axis
  x_axis <- seq(ddate, by="+1 month", length.out=max_maturity+1)
  x_axis <- x_axis[2:length(x_axis)]

  # y-axis
  y_axis_fitted <- exp(Y_fitted[indic_date,])*complete.cases(Y[indic_date,])
  y_axis_data <- exp(Y[indic_date,])

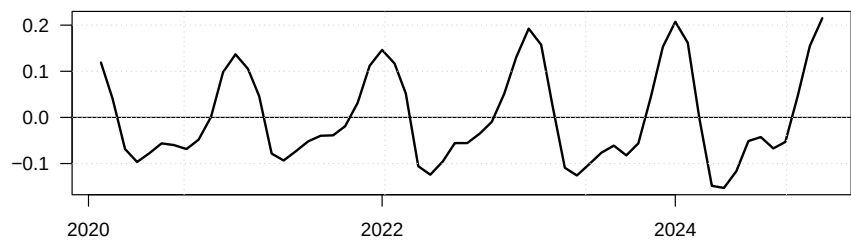
  points(x_axis,y_axis_fitted,pch=3,col=cols[count_date],lwd=2)
  lines(x_axis,y_axis_data,col=cols[count_date],lwd=2)
}

legend("topleft",
       title="Pricing date:",
       legend=selected_dates,
       lwd=c(2),lty=c(1),col=cols,seg.len = 2)

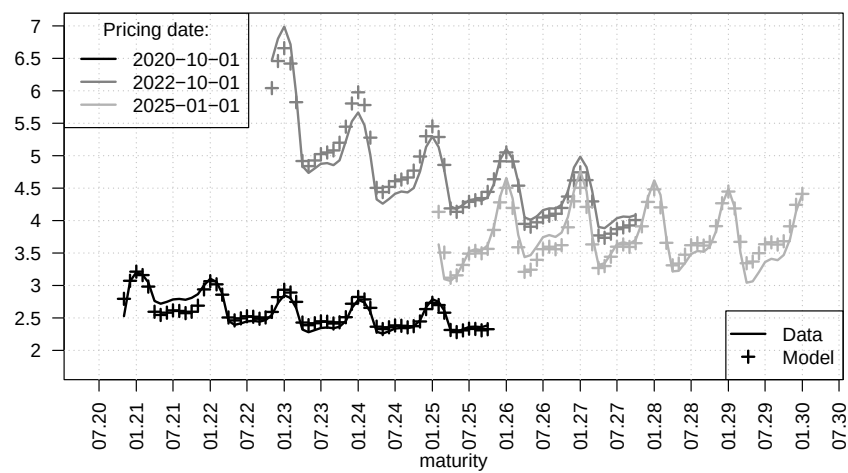
legend("bottomright",
       legend=c("Data","Model"),
       lwd=c(2),lty=c(1,NaN),col="black",pch=c(NaN,3),seg.len = 2)
}

```

(a) Estimated seasonality component γ_t



(b) Term structure of futures prices $F_{t,T}$



Chapter 9

Options

9.1 Stock call and put prices in a two-factor affine model with stochastic volatility

This example studies stock call and put prices in a two-factor affine model with stochastic volatility.

Purpose: This example studies stock call and put prices in a two-factor affine model with stochastic volatility

```
library(DTAM)
```

Define the joint transform used in the option-pricing recursion.

```
psi_stocks2fact <- function(u, psi.parameterization){  
  alpha <- psi.parameterization$alpha  
  beta  <- psi.parameterization$beta  
  phi   <- psi.parameterization$phi  
  sigma <- psi.parameterization$sigma  
  chi   <- psi.parameterization$chi  
  
  u1 <- c(u[1,])  
  u2 <- c(u[2,])  
  
  a <- rbind(u1*phi,  
             chi*u1^2/2 + u2*mu*beta/(1-mu*u2))  
  
  b <- matrix(1/2*u1^2*sigma^2 + u2*mu*alpha/(1-mu*u2),ncol=1)  
  
  return(list(  
    a = a, b = b  
  ))  
}
```

```

}

# Simulate the two-factor state vector entering returns and volatility.
simul_w <- function(model, nb.sim, w1_ini = 0, w2_ini = 0){

  all_w2 <- simul_ARG(nb.sim = nb.sim,
                      mu = model$mu, rho = model$mu*model$beta, nu = 0,
                      alpha = model$alpha)

  all_w1 <- NULL
  w1 <- w1_ini
  w2 <- w2_ini

  for(t in 1:nb.sim){
    w1 <- model$phi * w1 + sqrt(model$sigma^2 + model$chi*w2)*rnorm(1)
    w2 <- all_w2[t]

    all_w1 <- c(all_w1, w1)
  }

  return(cbind(all_w1, all_w2))
}

# Calibrate the illustrative two-factor model.
alpha <- .05
rho <- .9
mu <- 2
beta <- rho/mu
sigma <- .01
chi <- .0002
phi <- .2
kappa <- .003
delta <- .004

model <- list(
  alpha = alpha,
  mu = mu,
  beta = beta,
  sigma = sigma,
  chi = chi,
  phi = phi,
  kappa = kappa,
  delta = delta,
  n_w = 2
)

```

9.1. STOCK CALL AND PUT PRICES IN A TWO-FACTOR AFFINE MODEL WITH STOCHASTIC VOLATILITY

```

set.seed(2)

# Simulate the state vector and construct the stock-price path.
nb.sim <- 200
sim.w <- simul_w(model, nb.sim = nb.sim, w1_ini = 0, w2_ini = 0)

w1 <- sim.w[,1]
w2 <- sim.w[,2]

dS <- model$delta + w1 - model$kappa * w2
S <- exp(cumsum(dS))

K_over_S <- seq(.9,1.1,by=.1)

parameterization = list()
parameterization$model <- model
parameterization$xi0 = 0
parameterization$xi1 = matrix(0,2,1)

H <- 5

# Price calls and puts for several strikes and maturities.
res <- price_Stock_calls_puts(W = sim.w, # Values of state vector (T x n)
                             S = S, # ex-dividend stock price (T x 1)
                             H = H, # maximum maturity, in model periods
                             a = matrix(c(1,-kappa),ncol=1),
                             b = delta, # specif. of ex-dividend stock returns
                             K_over_S = K_over_S, # vector of strikes
                             psi = psi_stocks2fact, # Laplace transform of W
                             parameterization = parameterization,
                             max_x = 1000, # settings for Riemann sum comput.
                             dx_statio = 1,
                             min_dx = 1e-05,
                             nb_x1 = 10000)

# Plot the simulated factors, stock price, and option-price profiles.
par(mfrow=c(3,1))

par(plt=c(.10,.90,.15,.8))

plot(w2,type="l",las=1,xlab="",ylab="",col="white",
     main=expression(paste("(a) Crisis factor (",w[2*','*t],")",sep="")))
)
grid()
lines(w2,lwd=2)

```

```

plot(w1,type="l",col="white",xaxt="n",yaxt="n",
     ylim=c(-.15,.15),
     las=1,xlab="",ylab="",
     main=expression(paste("(b) Stock returns (grey, lhs) and stock prices (black, rhs)"))
)
grid()

ylim_dS <- seq(-.15,.15,by=.05)
axis(side=2,at=ylim_dS,labels = sprintf("%.2f", ylim_dS),las=1,
     col.axis = "darkgrey",col = "darkgrey")
lines(dS,col="darkgrey",lwd=2)

par(new=TRUE)
plot(S,type="l",col="white",xaxt="n",yaxt="n",
     ylim=c(.2,1.8),
     xlab="",ylab="")
lines(S,col="black",lwd=2)
ylim_S <- seq(0,2,by=.25)
axis(side=4,at=ylim_S,labels = sprintf("%.2f", ylim_S),las=1,
     col.axis = "black",col = "black")

mtext("Stock return", side = 2, line=3,cex=.8,col="darkgrey")
mtext("Stock price", side = 4, line=3,cex=.8)

Calls_over_S <- res$Calls / array(S,c(nb.sim,length(K_over_S),H))
Puts_over_S <- res$Puts / array(S,c(nb.sim,length(K_over_S),H))

plot(S,col="white",las=1,xlab="",ylab="",
     main=expression(paste("(c) Call prices (black) and put prices (grey)",sep="")),
     ylim=c(-max(Puts_over_S),max(Calls_over_S)),yaxt="n")
)
grid()
for(i in 1:length(K_over_S)){
  lines(Calls_over_S[,i,H],lwd=i)
  lines(-Puts_over_S[,i,H],lwd=i,col="darkgrey")
}

# ylim <- par("usr")[3:4]
ylim_calls <- seq(0,.5,by=.05)
axis(side=2,at=ylim_calls,labels = sprintf("%.2f", ylim_calls),las=1,
     col.axis = "black",col = "black")
ylim_puts <- seq(0,.5,by=.05)
axis(side=4,at=-ylim_puts,labels = sprintf("%.2f", ylim_puts),las=1,
     col.axis = "darkgrey",col = "darkgrey")

```

9.1. STOCK CALL AND PUT PRICES IN A TWO-FACTOR AFFINE MODEL WITH STOCHASTIC VOLATILITY

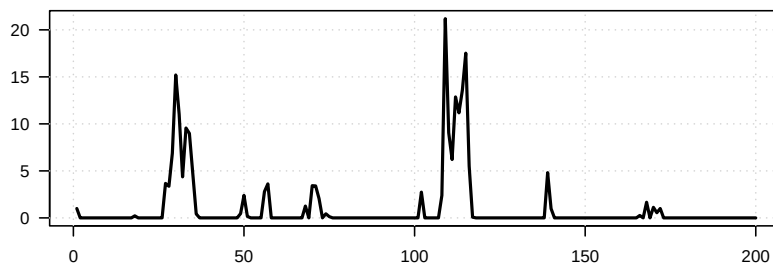
```

mtext(expression(paste("Call price / ", S[t], sep="")), side = 2, line=3, cex=.8)
mtext(expression(paste("Put price / ", S[t], sep="")), side = 4, line=3, cex=.8, col="darkgrey")

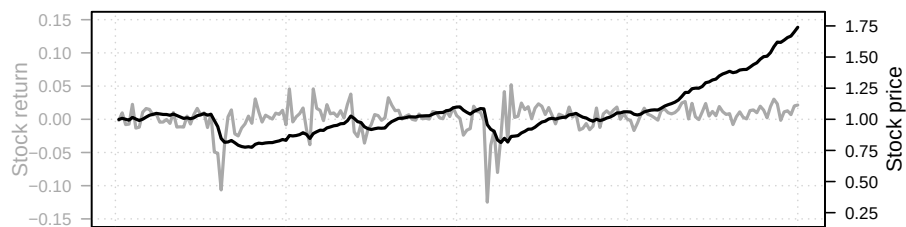
legend("bottomright",
      legend = sprintf("K/S = %.2f", K_over_S),
      lwd = seq_along(K_over_S),
      col = "black",
      bty = "n")

```

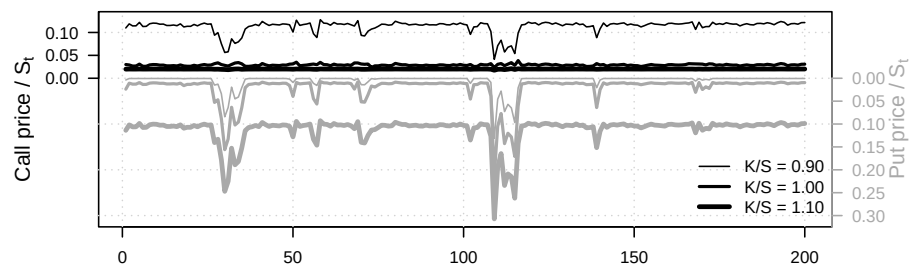
(a) Crisis factor ($w_{2,t}$)



(b) Stock returns (grey, lhs) and stock prices (black, rhs)



(c) Call prices (black) and put prices (grey)



9.2 The pricing of interest-rate caps and floors in a two-factor VARG term-structure model

This example illustrates the pricing of interest-rate caps and floors in a two-factor VARG term-structure model.

Purpose: This example illustrates the pricing of interest-rate caps and floors in a # VARG term-structure model

```
library(DTAM)

set.seed(4)

freq <- 4 # number of periods per year
tau  <- 2 # maturity of reference rate, in model periods
# alpha_tau <- tau/freq # year fraction corresponding to tau

# Specify VARG process:
r_bar <- .02/freq
alpha <- matrix(0,2,1)
nu    <- matrix(c(1,0),ncol=1)
rho1  <- .98
rho2  <- .9
mu    <- matrix(c((1-rho1)*r_bar/nu[1],
                  .0003),ncol=1)
beta  <- matrix(c(rho1/mu[1],0,
                  (1-rho2)/mu[2],rho2/mu[2]),2,2,byrow = TRUE)

Ew <- solve(diag(2) - diag(c(mu)) %*% beta) %*% c(nu*mu)

model <- list(
  alpha = alpha,
  nu    = nu,
  mu    = mu,
  beta  = beta,
  n_w   = 2
)

# Specification of short-term rate:
xi0 <- 0
xi1 <- matrix(c(0,1),2,1)

parameterization <- list(model = model,
                         xi0 = xi0, xi1 = xi1)

all_K <- c(.01, .03) # Strikes (annualized)
```


9.2. THE PRICING OF INTEREST-RATE CAPS AND FLOORS IN A TWO-FACTOR VARG TERM-STRUCTURE

```

H <- 1.5*freq # maximum maturity, in model periods

TT <- 150

sim_w <- simul_VARG(model,nb_periods = TT)

par(mfrow=c(3,1))
par(plt=c(.1,.9,.15,.85))

plot(0*sim_w[1,]*freq,las=1,
      ylim=c(0,5),col="white",
      main=paste("(a) State vector",sep=""),
      xlab="",ylab="")
mtext("Yield factors (in percent)", side = 2,
      line=2,cex=.8)

# grid()
lines(100*sim_w[1,]*freq,col="darkgrey",lwd=3,lty=2)
lines(100*sim_w[2,]*freq,col="black",lwd=2,lty=1)
abline(h=0,col="grey",lty=3)

for(k in 1:length(all_K)){
  abline(h=100*all_K[k],lty=2)
}

legend("topright",
      legend = c(expression(w[1*','*t]),
                  expression(paste(w[2*','*t]," ( = ",r[t],")",sep="")),
                  expression(paste("Strikes ",K,sep=""))),
      col = c("darkgrey", "black", "black"),
      lwd = c(3, 2, 1),
      lty = c(2, 1, 2),
      bg="white")

W <- t(sim_w)

res <- price_IR_caps_floors(W,
                             # Values of state vector (T x n)
                             H = H,      # maximum maturity, in model periods
                             tau = tau,   # maturity of ref. rate, in model periods
                             freq = freq, # number of model periods per year
                             all_K,      # vector of (annualized) strikes
                             psi = psi_VARG,
                             parameterization = parameterization,
                             max_x = 10000, # settings for Riemann sum comput.

```

```

dx_statio = 5,
min_dx = 1e-05,
nb_x1 = 1000)

for(k in 1:length(all_K)){
  plot(res$Caplet[,k,1],
       ylim=100*c(- max(res$Floors,na.rm = TRUE),
                  + max(res$Caps,na.rm = TRUE)),
       main=paste("(",letters[k+1],") Prices of caps & floors; strike = ",100*all_K[k],
       col="white",yaxt="n",xlab="",ylab="")
  grid()
  for(h in 1:length(res$maturities.in.model.period)){
    lines(100*res$Caps[,k,h],lwd=res$maturities.in.model.period[h]/2,col="black")
    lines(-100*res$Floors[,k,h],lwd=res$maturities.in.model.period[h]/2,col="darkgrey")
  }

  legend("topright",
        legend = paste("maturity: ",res$maturities.in.model.period/freq," years",sep=" ",
        col = c("black"),
        lwd = res$maturities.in.model.period/2,
        lty = 1,
        bg="white")

  legend("topleft",
        legend = c("Caps","Floors"),
        col = c("black","darkgrey"),
        lwd = NaN,
        pch= 15,
        pt.cex=1.5,
        lty = NaN,
        bg="white")

  axis_label <- 1:5
  axis(side = 2,at = axis_label,
       labels = axis_label,
       las=1)
  mtext("Cap price, in percent", side = 2,
       line=2,cex=.8)

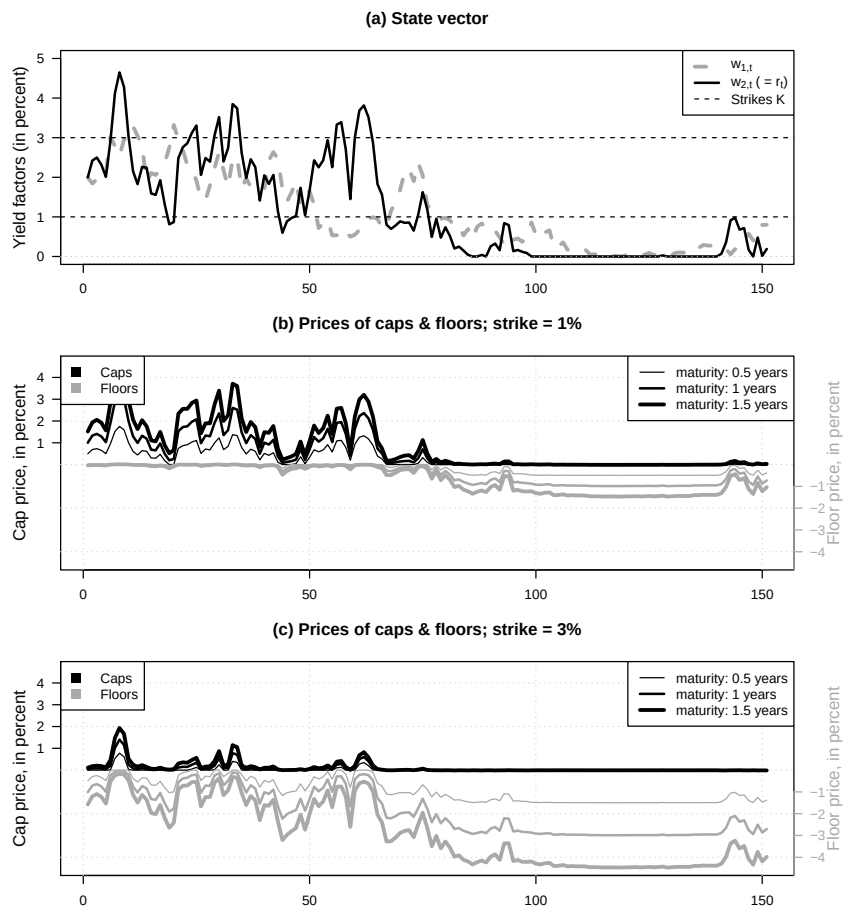
  axis(side = 4,at = -axis_label,
       labels = -axis_label,
       las=1,
       col="darkgrey",col.axis = "darkgrey")
  mtext("Floor price, in percent", side = 4,

```

9.2. THE PRICING OF INTEREST-RATE CAPS AND FLOORS IN A TWO-FACTOR VARG TERM-STRUCTURE

```
line=2,col="darkgrey",cex=.8)
```

```
}
```



```
# Check caplet formula using simulations:
```

```
# Select starting date:
```

```
# t <- 30
```

```
# State vector:
```

```
# wt <- matrix(W[t,],2,1)
```

```
# alpha_tau <- tau/freq
```

```

# Nb.replic <- 40000
# all_simul <- NULL
# for(i in 1:Nb.replic){
#   sim_w_i <- simul_VARG(model, nb_periods = H, w0 = wt)
#   L_tau <- 1/alpha_tau*(exp(- t(sim_w_i) %% res$A_tau - res$B_tau) - 1)
#   r <- sim_w_i[2,]
#   all_simul <- cbind(all_simul,
#                     exp(-sum(r[1:H]))*pmax(L_tau[H+1-tau]-all_K, 0)
#   )
# }
# Caplet_simul <- alpha_tau * apply(all_simul, 1, mean)
# cbind(res$Caplet[t,,H], Caplet_simul)

```

9.3 Kim-style swaption exercise-boundary approximations in a non-Gaussian affine setting

This example compares Kim-style swaption exercise-boundary approximations in a non-Gaussian affine setting.

```

# Purpose: This example compares Kim-style swaption exercise-boundary approximations in
# non-Gaussian affine setting

# =====
# Figure for Kim (2012) swaption boundary approximations
# Two-factor Gaussian VAR term-structure illustration
# -----
# Produces: contour plot of Gaussian density of  $w = (w_1, w_2)$ ,
#           true exercise boundary  $P(w)=1$  for several  $K$ ,
#           Approx A: local Taylor (tangent) boundary at  $w^*$ ,
#           Approx B: effective ZCB boundary ( $\beta w = \gamma$ ).
# =====

library(akima) # interp() to get smooth contour for  $P(w)=1$ 

# Unconditional mean and variance of  $w$ :
Ew <- compute_uncondmean_VARG(model)
Vw <- compute_uncondvar_VARG(model)

compute_DF <- function(w_mat, AB, H) {
  # w_mat: n_obs x 2
  # Returns: n_obs x H matrix with entries  $B(w; j)$ 
  A <- matrix(AB$A[, 1:H], 2, H)

```

9.3. KIM-STYLE SWAPTION EXERCISE-BOUNDARY APPROXIMATIONS IN A NON-GAUSSIAN AFFINE SE

```

B <- c(AB$B[, , 1:H]) # length H

# For each observation i and maturity j:
# log B_{i,j} = A[,j]' w_i + B[j]
logB <- sweep(w_mat %*% A, 2, B, "+")
exp(logB)
}

compute_coupon_bond <- function(w_mat, AB, K, H, tau, freq) {
  # Coupon-bond corresponding to fixed leg + principal at maturity:
  # c_j = alpha_tau*K for j=tau,2*tau,...,H and 1 + alpha_tau*K at j=H
  # K is the annualized coupon rate
  nb_coupons <- H / tau
  if (abs(nb_coupons - round(nb_coupons)) > 1e-12) {
    stop("H must be an integer multiple of tau.")
  }
  Bmat <- compute_DF(w_mat, AB, H)
  alpha_tau <- tau/freq
  c_j <- matrix(c(rep(alpha_tau*K, nb_coupons-1), 1 + alpha_tau*K), ncol=1)
  if(freq>1){
    c_j <- c_j %x% matrix(c(rep(0,tau-1),1),ncol=1)
  }
  drop(Bmat %*% c_j)
}

duration_FisherWeil <- function(w_mat, AB, K, H, tau, freq) {
  # Fisher--Weil duration of the coupon-bond, computed pointwise
  # K is the annualized coupon rate
  # Coupon payments:
  # c_j = alpha_tau*K for j=tau,2*tau,...,H and 1 + alpha_tau*K at j=H
  nb_coupons <- H / tau
  if (abs(nb_coupons - round(nb_coupons)) > 1e-12) {
    stop("H must be an integer multiple of tau.")
  }
  Bmat <- compute_DF(w_mat, AB, H)
  alpha_tau <- tau/freq
  c_j <- matrix(c(rep(alpha_tau*K, nb_coupons-1), 1 + alpha_tau*K), ncol=1)
  if(freq>1){
    c_j <- c_j %x% matrix(c(rep(0,tau-1),1),ncol=1)
  }
  numer <- Bmat %*% (c_j * (1:H))
  denom <- Bmat %*% c_j
  drop(numer / denom)
}

```

```

# Numerical gradient of P(w) at w (2D)
grad_coupon_bond <- function(w, AB, K, H, tau, freq, eps = 1e-6) {
  stopifnot(length(w) == 2)
  f0 <- compute_coupon_bond(matrix(w, nrow=1), AB, K, H, tau, freq)

  g <- numeric(2)
  for (i in 1:2) {
    wp <- w
    wp[i] <- wp[i] + eps
    fp <- compute_coupon_bond(matrix(wp, nrow=1), AB, K, H, tau, freq)
    g[i] <- (fp - f0) / eps
  }
  matrix(g, ncol = 1)
}

```

```

# Approach A: pick w* along the local normal direction from mu=E[w_T]
# by a 1D search: w*(x) = mu + x d, find P(w*(x)) = 1

```

```

run_Approach_A <- function(mu, AB, K, H, tau, freq,
                           x_range = c(-0.1, 0.1),
                           eps = 1e-6,
                           grid_n = 2000) {

  beta_mu <- grad_coupon_bond(mu, AB, K, H, tau, freq, eps = eps)
  d <- as.numeric(beta_mu)
  d <- d / sqrt(sum(d^2)) # unit direction

  # grid search to get a good bracket / good minimizer
  xgrid <- seq(x_range[1], x_range[2], length.out = grid_n)
  wgrid <- cbind(mu[1] + xgrid*d[1], mu[2] + xgrid*d[2])
  Pgrid <- compute_coupon_bond(wgrid, AB, K, H, tau, freq)

  # pick closest point to boundary (robust even if no sign change)
  idx <- which.min((Pgrid - 1)^2)
  w_star <- wgrid[idx, ]

  beta_star <- grad_coupon_bond(w_star, AB, K, H, tau, freq, eps = eps)

  list(w_star = w_star, beta = beta_star)
}

```

```

# Approach B: Fisher--Weil effective maturity tau, then affine boundary beta'w < gamma

```

```

run_Approach_B <- function(mu, AB, K, H, tau, freq) {
  # tau is generally non-integer; we linearly interpolate A_tau and B_tau
  tau_star <- duration_FisherWeil(matrix(mu, nrow=1), AB, K, H, tau, freq)
  tau_minus <- max(1, min(H, floor(tau_star)))
}

```

9.3. KIM-STYLE SWAPTION EXERCISE-BOUNDARY APPROXIMATIONS IN A NON-GAUSSIAN AFFINE SE

```

tau_plus  <- tau_minus + 1

w <- tau_star - tau_minus
weights <- c(1 - w, w)

# Interpolate A and B
A_tau <- AB$A[,tau_minus] * weights[1] + AB$A[,tau_plus] * weights[2]
B_tau <- AB$B[,tau_minus] * weights[1] + AB$B[,tau_plus] * weights[2]

beta <- matrix(A_tau, ncol = 1)

# Kim-style scaling term xi to improve the mapping (you had this structure)
# Keep your construction, but ensure it uses K consistently.
Bth_minus <- compute_DF(matrix(mu, nrow=1), AB, H)[tau_minus]
Bth_plus  <- compute_DF(matrix(mu, nrow=1), AB, H)[tau_plus]

P <- compute_coupon_bond(matrix(mu, nrow=1), AB, K, H, tau, freq)

xi <- P / (Bth_minus * weights[1] + Bth_plus * weights[2])

gamma <- -c(B_tau) - log(xi)

list(tau_star = tau_star, xi = xi, beta = beta, gamma = gamma)
}

# Utility: for a given boundary beta1*w1 + beta2*w2 = gamma, solve for w2 as function of w1
fy <- function(x, beta, gamma) {
  # returns y such that beta1*x + beta2*y = gamma
  -(beta[1] * x - gamma) / beta[2]
}
fx <- function(y, beta, gamma) {
  # returns x such that beta1*x + beta2*y = gamma
  (gamma - beta[2] * y) / beta[1]
}

swap_maturity_years <- 10

H <- freq*swap_maturity_years

tau <- 2
alpha_tau <- tau/freq

AB <- compute_AB_classical(xi0, xi1, H = H,
                           psi = psi_VARG,

```

```

                                psi.parameterization = model)

# Plot window and density contours.
min_w <- 0.00 # expressed in annualized terms
max_w <- 0.1 # expressed in annualized terms

par(mfrow=c(1,1))
par(plt=c(.15,.97,.2,.97))

# plot_isodensity_gaussian(Ew, Vw, min_w = min_w, max_w = max_w, maintitle = "")
plot(0,
      xlim=c(0,.015),
      ylim=c(0,.015),col="white",
      xaxs = "i", yaxs = "i",las = 1,
      xaxt="n",yaxt="n",
      xlab=expression(paste(w[1*'],'*t'," (annualized)",sep="")),
      ylab=expression(paste(w[2*'],'*t'," (annualized)",sep="")))
grid()
labels <- seq(0,5,by=1)
axis(side=1,at=labels/(100*freq),labels = paste0(labels,"%"),las=1)
axis(side=2,at=labels/(100*freq),labels = paste0(labels,"%"),las=1)

points(Ew[1], Ew[2], pch = 4, lwd = 2, cex=1.5)
text(Ew[1]-.0005, Ew[2], labels = expression(E[t](w[T])), pos = 1)

# Exercise-boundary illustration for several strikes K.
all_K <- c(0.01, 0.02, 0.03)

# Grid for "true delineation"  $P(w)=1$  via interpolation
nbw <- 100
w_values <- seq(min_w, max_w, length.out = nbw)
w1w2 <- cbind(w_values %x% rep(1, nbw),
              rep(1, nbw) %x% w_values)

for (K in all_K) {

  # --- True boundary: contour of  $P(w)=1$  ---
  Pvals <- compute_coupon_bond(w1w2, AB, K, H, tau, freq)

  interp_obj <- akima::interp(
    x = w1w2[,1], y = w1w2[,2], z = Pvals,
    xo = seq(min_w, max_w, length.out = 200),
    yo = seq(min_w, max_w, length.out = 200),
    linear = TRUE, extrap = FALSE
  )
}

```


9.3. KIM-STYLE SWAPTION EXERCISE-BOUNDARY APPROXIMATIONS IN A NON-GAUSSIAN AFFINE SE

```

contour(interp_obj$x, interp_obj$y, interp_obj$z,
        levels = 1, add = TRUE, drawlabels = FALSE,
        lwd = 3, col = "darkgrey")

# --- Approach A: Taylor boundary at w* ---
resA <- run_Approach_A(mu = Ew, AB = AB, K = K, H = H, tau, freq)
w_star <- resA$w_star
betaA <- resA$beta

points(w_star[1], w_star[2], pch = 3, lwd = 2, cex=1.5)

xline <- c(min_w, max_w)

# tangent line: betaA'(w - w*) = 0 -> beta1(x-x*) + beta2(y-y*) = 0
ylineA <- w_star[2] - (betaA[1] / betaA[2]) * (xline - w_star[1])
lines(xline, ylineA, col = "black", lty = 2, lwd = 2)

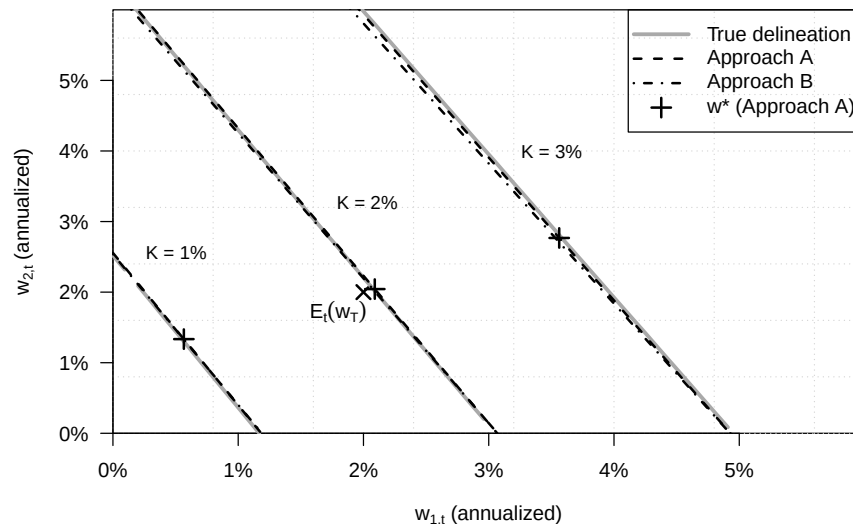
# --- Approach B: effective ZCB boundary beta'w = gamma ---
resB <- run_Approach_B(mu = Ew, AB = AB, K = K, H = H, tau, freq)
betaB <- resB$beta
gamma <- resB$gamma

ylineB <- fy(xline, beta = betaB, gamma = gamma)
lines(xline, ylineB, col = "black", lty = 4, lwd = 2)

# --- Label K near the lower part of the plot ---
text(w_star[1] - .001, w_star[2] + .003,
     labels = paste0("K = ", round(100*K), "%"),
     pos = 4, cex = 0.9)
}

legend("topright",
      legend = c("True delineation", "Approach A", "Approach B",
                 expression(paste(w*"", " (Approach A)", sep=""))),
      lty = c(1, 2, 4, NA),
      lwd = c(3, 2, 2, 2),
      pch = c(NA, NA, NA, 3),
      col = c("darkgrey", "black", "black", "black"),
      pt.cex = 1.5,
      seg.len = 3,
      bty = "o")

```



9.4 Kim-style swaption boundary approximations in a Gaussian affine model

This example revisits Kim-style swaption boundary approximations in a Gaussian affine model.

Purpose: This example revisits Kim-style swaption boundary approximations in a Gaussian affine model

```
# =====
# Figure for Kim (2012) swaption boundary approximations
# Two-factor Gaussian VAR term-structure illustration
# -----
# Produces: contour plot of Gaussian density of  $w = (w_1, w_2)$ ,
#           true exercise boundary  $P(w)=1$  for several  $K$ ,
#           Approx A: local Taylor (tangent) boundary at  $w^*$ ,
#           Approx B: effective ZCB boundary ( $\beta w = \gamma$ ).
# =====
```

```
library(DTAM) # compute_AB_classical, psi_GaussianVAR, simul_GaussianVAR, ...
library(akima) # interp() to get smooth contour for  $P(w)=1$ 
```

```
rho <- 0.7
phi <- 0.9
rbar <- 0.04
```

```
sigma_r <- 0.005
```

```

sigma_x <- 0.0025

# A0/A1 representation:
A1 <- matrix(c(rho, -rho,
               0,    phi), 2, 2, byrow = TRUE)
A0 <- matrix(c(1, -1,
               0,  1), 2, 2, byrow = TRUE)

Phi    <- solve(A0) %*% A1
Sigma12 <- solve(A0) %*% diag(c(sigma_r, sigma_x))
Sigma  <- Sigma12 %*% t(Sigma12)

mu <- solve(A0) %*% matrix(c(0, rbar * (1 - phi)), 2, 1)

model <- list(mu = mu, Phi = Phi, Sigma = Sigma, Sigma12 = Sigma12)

# Unconditional mean and variance of w:
Ew <- solve(diag(2) - Phi) %*% mu
Vw <- matrix(solve(diag(2^2) - Phi %x% Phi) %*% c(Sigma), 2, 2)

Ew <- c(Ew) # convenient as numeric vector

compute_DF <- function(w_mat, AB, H) {
  # w_mat: n_obs x 2
  # Returns: n_obs x H matrix with entries B(w; j)
  A <- matrix(AB$A[,1:H], 2, H)
  B <- c(AB$B[,1:H]) # length H

  # For each observation i and maturity j:
  # log B_{i,j} = A[,j]' w_i + B[j]
  logB <- sweep(w_mat %*% A, 2, B, "+")
  exp(logB)
}

compute_coupon_bond <- function(w_mat, AB, K, H) {
  # Coupon-bond corresponding to fixed leg + principal at maturity:
  # c_j = K/p for j=1,...,H-1 and 1+K/p at j=H.
  # In your figure you effectively set p=1 (annual payments).
  # If you later want p!=1, make it an argument.
  Bmat <- compute_DF(w_mat, AB, H)
  c_j <- c(rep(K, H-1), 1 + K) # p=1 convention here
  drop(Bmat %*% c_j)
}

duration_FisherWeil <- function(w_mat, AB, K, H) {

```

```

# Fisher--Weil duration of the coupon-bond, computed pointwise
Bmat <- compute_DF(w_mat, AB, H)
c_j <- c(rep(K, H-1), 1 + K)

numer <- Bmat %*% (c_j * (1:H))
denom <- Bmat %*% c_j
drop(numer / denom)
}

# Numerical gradient of P(w) at w (2D)
grad_coupon_bond <- function(w, AB, K, H, eps = 1e-6) {
  stopifnot(length(w) == 2)
  f0 <- compute_coupon_bond(matrix(w, nrow=1), AB, K, H)

  g <- numeric(2)
  for (i in 1:2) {
    wp <- w
    wp[i] <- wp[i] + eps
    fp <- compute_coupon_bond(matrix(wp, nrow=1), AB, K, H)
    g[i] <- (fp - f0) / eps
  }
  matrix(g, ncol = 1)
}

# Approach A: pick w* along the local normal direction from mu=E[w_T]
# by a 1D search: w*(x) = mu + x d, find P(w*(x)) = 1
run_Approach_A <- function(mu, AB, K, H,
                           x_range = c(-0.1, 0.1),
                           eps = 1e-6,
                           grid_n = 1000) {

  beta_mu <- grad_coupon_bond(mu, AB, K, H, eps = eps)
  d <- as.numeric(beta_mu)
  d <- d / sqrt(sum(d^2)) # unit direction

  # grid search to get a good bracket / good minimizer
  xgrid <- seq(x_range[1], x_range[2], length.out = grid_n)
  wgrid <- cbind(mu[1] + xgrid*d[1], mu[2] + xgrid*d[2])
  Pgrid <- compute_coupon_bond(wgrid, AB, K, H)

  # pick closest point to boundary (robust even if no sign change)
  idx <- which.min((Pgrid - 1)^2)
  w_star <- wgrid[idx, ]

  beta_star <- grad_coupon_bond(w_star, AB, K, H, eps = eps)

```

```

list(w_star = w_star, beta = beta_star)
}

# Approach B: Fisher--Weil effective maturity tau, then affine boundary beta'w < gamma
run_Approach_B <- function(mu, AB, K, H) {
  # tau is generally non-integer; we linearly interpolate A_tau and B_tau
  tau <- duration_FisherWeil(matrix(mu, nrow=1), AB, K, H)
  tau_minus <- max(1, min(H, floor(tau)))
  tau_plus <- max(1, min(H, tau_minus + 1))

  w <- tau - tau_minus
  weights <- c(1 - w, w)

  # Interpolate A and B
  A_tau <- AB$A[,tau_minus] * weights[1] + AB$A[,tau_plus] * weights[2]
  B_tau <- AB$B[,tau_minus] * weights[1] + AB$B[,tau_plus] * weights[2]

  beta <- matrix(A_tau, ncol = 1)

  # Kim-style scaling term xi to improve the mapping (you had this structure)
  # Keep your construction, but ensure it uses K consistently.
  Bth_minus <- compute_DF(matrix(mu, nrow=1), AB, H)[tau_minus]
  Bth_plus <- compute_DF(matrix(mu, nrow=1), AB, H)[tau_plus]

  P_minus <- compute_coupon_bond(matrix(mu, nrow=1), AB, K, tau_minus)
  P_plus <- compute_coupon_bond(matrix(mu, nrow=1), AB, K, tau_plus)

  xi <- (P_minus * weights[1] + P_plus * weights[2]) /
    (Bth_minus * weights[1] + Bth_plus * weights[2])

  gamma <- -c(B_tau) - log(xi)

  list(tau = tau, xi = xi, beta = beta, gamma = gamma)
}

# Utility: for a given boundary beta1*w1 + beta2*w2 = gamma, solve for w2 as function of w1
fy <- function(x, beta, gamma) {
  # returns y such that beta1*x + beta2*y = gamma
  -(beta[1] * x - gamma) / beta[2]
}
fx <- function(y, beta, gamma) {
  # returns x such that beta1*x + beta2*y = gamma
  (gamma - beta[2] * y) / beta[1]
}

```

```

xi0 <- 0
xi1 <- matrix(c(1, 0), 2, 1)  # short rate loading  $r_t = xi0 + xi1' w_t$ 

H <- 10
AB <- compute_AB_classical(xi0, xi1, H = H,
                           psi = psi_GaussianVAR,
                           psi.parameterization = model)

# Plot window and density contours.
min_w <- 0.01
max_w <- 0.075

par(mfrow=c(1,1))
par(plt=c(.15,.97,.2,.97))

plot_isodensity_gaussian(Ew, Vw, min_w = min_w, max_w = max_w, maintitle = "")

# Exercise-boundary illustration for several strikes K.
all_K <- c(0.03, 0.04, 0.05)

# Grid for "true delineation"  $P(w)=1$  via interpolation
nbw <- 100
w_values <- seq(min_w, max_w, length.out = nbw)
w1w2 <- cbind(w_values %x% rep(1, nbw),
              rep(1, nbw) %x% w_values)

for (K in all_K) {

  # --- True boundary: contour of  $P(w)=1$  ---
  Pvals <- compute_coupon_bond(w1w2, AB, K, H)

  interp_obj <- akima::interp(
    x = w1w2[,1], y = w1w2[,2], z = Pvals,
    xo = seq(min_w, max_w, length.out = 200),
    yo = seq(min_w, max_w, length.out = 200),
    linear = TRUE, extrap = FALSE
  )

  contour(interp_obj$x, interp_obj$y, interp_obj$z,
         levels = 1, add = TRUE, drawlabels = FALSE,
         lwd = 3, col = "darkgrey")

  # --- Approach A: Taylor boundary at  $w^*$  ---
  resA <- run_Approach_A(mu = Ew, AB = AB, K = K, H = H)
  w_star <- resA$w_star

```

```

betaA <- resA$beta

points(w_star[1], w_star[2], pch = 4, lwd = 2, cex=1.5)

xline <- c(min_w, max_w)
# tangent line: betaA'(w - w*) = 0 -> beta1(x-x*) + beta2(y-y*) = 0
ylineA <- w_star[2] - (betaA[1] / betaA[2]) * (xline - w_star[1])
lines(xline, ylineA, col = "black", lty = 2, lwd = 2)

# --- Approach B: effective ZCB boundary beta'w = gamma ---
resB <- run_Approach_B(mu = Ew, AB = AB, K = K, H = H)
betaB <- resB$beta
gamma <- resB$gamma

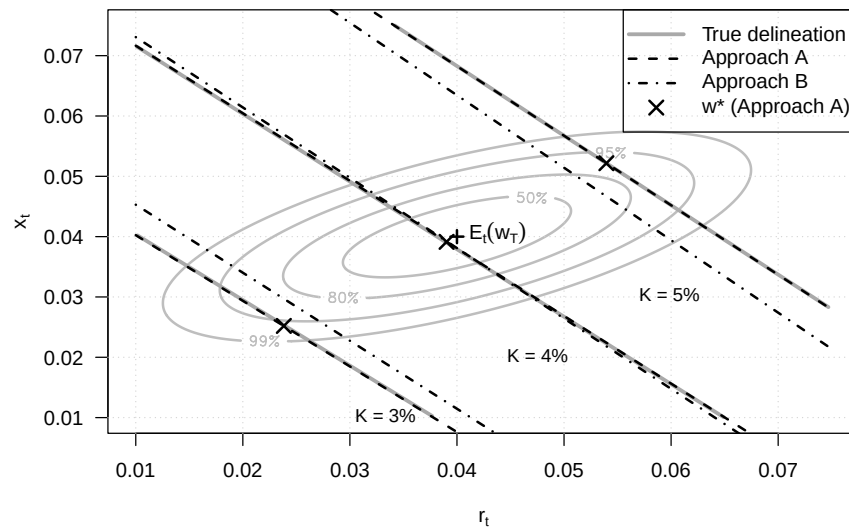
ylineB <- fy(xline, beta = betaB, gamma = gamma)
lines(xline, ylineB, col = "black", lty = 4, lwd = 2)

# --- Label K near the lower part of the plot (simple placement rule) ---
y_lab <- min_w - 0.03 + K
x_lab <- fx(y_lab, beta = betaB, gamma = gamma)

text(x_lab - 0.004, y_lab,
     labels = paste0("K = ", round(100*K), "%"),
     pos = 2, cex = 0.9)
}

legend("topright",
      legend = c("True delineation", "Approach A", "Approach B",
                  expression(paste(w*"%", " (Approach A)", sep=""))),
      lty = c(1, 2, 4, NA),
      lwd = c(3, 2, 2, 2),
      pch = c(NA, NA, NA, 4),
      col = c("darkgrey", "black", "black", "black"),
      pt.cex = 1.5,
      seg.len = 3,
      bty = "o")

```



9.5 Prices inflation caps and floors in a two-factor affine inflation model

This example prices inflation caps and floors in a two-factor affine inflation model.

Purpose: This example prices inflation caps and floors in a two-factor affine inflation model.

```
library(DTAM)
```

Define the transform used by the inflation option-pricing routine.

```
psi_stocks2fact <- function(u, psi.parameterization){
  alpha <- psi.parameterization$alpha
  beta  <- psi.parameterization$beta
  phi   <- psi.parameterization$phi
  sigma <- psi.parameterization$sigma
  chi   <- psi.parameterization$chi

  u1 <- c(u[1,])
  u2 <- c(u[2,])

  a <- rbind(u1*phi,
             chi*u1^2/2 + u2*mu*beta/(1-mu*u2))

  b <- matrix(1/2*u1^2*sigma^2 + u2*mu*alpha/(1-mu*u2), ncol=1)

  return(list(
```



```

    a = a, b = b
  ))
}

# Simulate the two-factor inflation state vector.
simul_w <- function(model, nb.sim, w1_ini = 0, w2_ini = 0){

  all_w2 <- simul_ARG(nb.sim = nb.sim,
                      mu = model$mu, rho = model$mu*model$beta, nu = 0,
                      alpha = model$alpha)

  all_w1 <- NULL
  w1 <- w1_ini
  w2 <- w2_ini

  for(t in 1:nb.sim){
    w1 <- model$phi * w1 + sqrt(model$sigma^2 + model$chi*w2)*rnorm(1)
    w2 <- all_w2[t]

    all_w1 <- c(all_w1, w1)
  }

  return(cbind(all_w1, all_w2))
}

freq <- 4 # quarterly data

# Calibrate the inflation process.
alpha <- .05
rho <- .9
mu <- 2
beta <- rho/mu
sigma <- .004/freq
chi <- .00004/freq
phi <- .8
kappa <- 0
mu_pi0 <- .03/freq

model <- list(
  alpha = alpha,
  mu = mu,
  beta = beta,
  sigma = sigma,
  chi = chi,

```

```

    phi    = phi,
    kappa  = kappa,
    mu_pi0 = mu_pi0,
    n_w    = 2
  )

  set.seed(2)

  # Simulate inflation and the volatility factor.
  nb.sim <- 200
  sim.w <- simul_w(model, nb.sim = nb.sim, w1_ini = 0, w2_ini = 0)

  w1 <- sim.w[,1]
  w2 <- sim.w[,2]

  pi_t <- model$mu_pi0 + w1 - model$kappa * w2
  annualized_pi_t <- freq * pi_t

  parameterization = list()
  parameterization$model <- model
  parameterization$xi0 = 0
  parameterization$xi1 = matrix(0,2,1)
  parameterization$mu_pi0 = model$mu_pi0
  parameterization$mu_pi1 = matrix(c(1,-model$kappa),2,1)

  all_K <- seq(0,.06,by=.03)/freq

  ell <- 0

  H <- 8

  Pi_t_minus_ell <- exp(ell*model$mu_pi0)

  # Price caps and floors over the simulated state path.
  res <-
    price_Inflation_caps_floors(W = sim.w, # Values of state vector (T x n)
                                H = H, # maximum maturity, in model periods
                                ell = ell, # Indexation lag
                                all_K = all_K, # vector of strikes, expressed at model frequency
                                # e.g.: quarter. data and K=.01 means annualized strike
                                # Indexation lag multiplicative factor
                                Pi_t_minus_ell = Pi_t_minus_ell,
                                psi = psi_stocks2fact, # Laplace transform of W
                                parameterization, # see details below
                                max_x = 10000, # settings for Riemann sum comput.

```

```

dx_statio = 10,
min_dx = 1e-05,
nb_x1 = 1000)

# Plot the factor, inflation, and option-price paths.
par(mfrow=c(3,1))
par(plt=c(.1,.9,.15,.85))

plot(w2,type="l",las=1,xlab="",ylab="",col="white",
      main=expression(paste("(a) Volatility factor ('w[2*','*t]',')",sep="")))
)
grid()
lines(w2,lwd=2)

ylim_pi <- seq(-.1,.15,by=.05)

plot(w1,type="l",col="white",
      ylim=c(-.05,.10),
      las=1,xlab="",ylab="",
      main=expression(paste("(b) Annualized inflation ('pi[t]',')",sep="")))
)
grid()
abline(h=0,lty=3)
lines(annualized_pi_t,col="black",lwd=2)

h <- H

plot(res$Caps[,1,h],type="l",las=1,
      ylim = 1.2*100*c(min(-res$Floors[, ,h],na.rm = TRUE),
                       max(res$Caps[, ,h],na.rm = TRUE)),
      col="white",
      xlab="", ylab="",
      yaxt="n",
      main=expression(paste("(c) Prices of caps & floors; maturity: ",h," quarters",sep="")))
grid()

for(i in 1:length(all_K)){
  lines(100*res$Caps[,i,h],lwd=i,col="black")
}
for(i in 1:length(all_K)){
  lines(-100*res$Floors[,i,h],lwd=i,col="darkgrey")
}

legend("bottomright",
      legend = paste("Strike: ",100*all_K*freq,"%",sep=""),

```

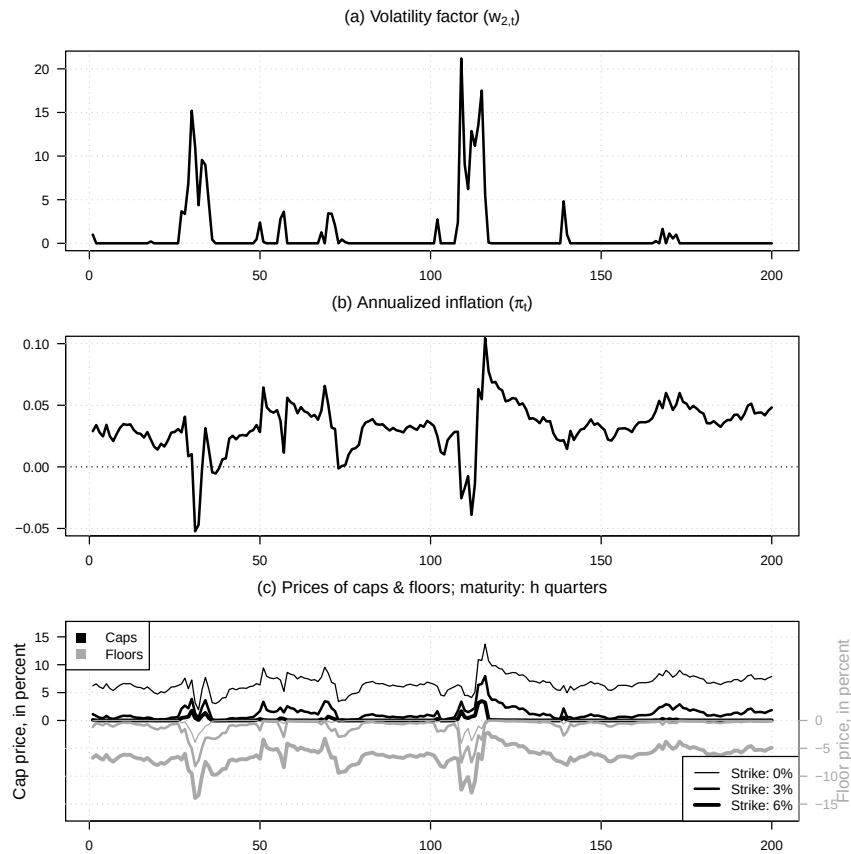
```
col = c("black"),
lwd = 1:length(all_K),
lty = 1,
bg="white")

legend("topleft",
      legend = c("Caps","Floors"),
      col = c("black","darkgrey"),
      lwd = NaN,
      pch= 15,
      pt.cex=1.5,
      lty = NaN,
      bg="white")

axis_label <- seq(0,20,by=5)
axis(side = 2,at = axis_label,
     labels = axis_label,
     las=1)
mtext("Cap price, in percent", side = 2,
     line=2.3,cex=.8)

axis(side = 4,at = -axis_label,
     labels = -axis_label,
     las=1,
     col="darkgrey",col.axis = "darkgrey")
mtext("Floor price, in percent", side = 4,
     line=2.3,col="darkgrey",cex=.8)
```

9.6. INFLATION CAP PRICES VARY ACROSS STATES AND STRIKE LEVELS173



9.6 Inflation cap prices vary across states and strike levels

This example shows how inflation cap prices vary across states and strike levels.

Purpose: This example shows how inflation cap prices vary across states and strike levels

```
# Build a grid of state values.
w1_values <- matrix(seq(-.03,.06,by=.01),ncol=1)
n1 <- length(w1_values)
w2_values <- matrix(seq(0,80,by=10),ncol=1)
n2 <- length(w2_values)

W <- cbind(
  matrix(1,n2,1) %x% w1_values,
  w2_values %x% matrix(1,n1,1)
```

```

)

all_K <- c(0,.05)/freq

# Price inflation caps on the state grid.
res <-
  price_inflation_caps_floors(W = W, # Values of state vector (T x n)
                              H = H, # maximum maturity, in model periods
                              ell = ell, # Indexation lag
                              all_K = all_K, # vector of strikes, expressed at model frequency
                              # e.g.: quarter. data and K=.01 means annualized strike of 1%
                              # Indexation lag multiplicative factor
                              Pi_t_minus_ell = Pi_t_minus_ell,
                              psi = psi_stocks2fact, # Laplace transform of W
                              parameterization, # see details below
                              max_x = 10000, # settings for Riemann sum comput.
                              dx_statio = 10,
                              min_dx = 1e-05,
                              nb_x1 = 1000)

h <- H

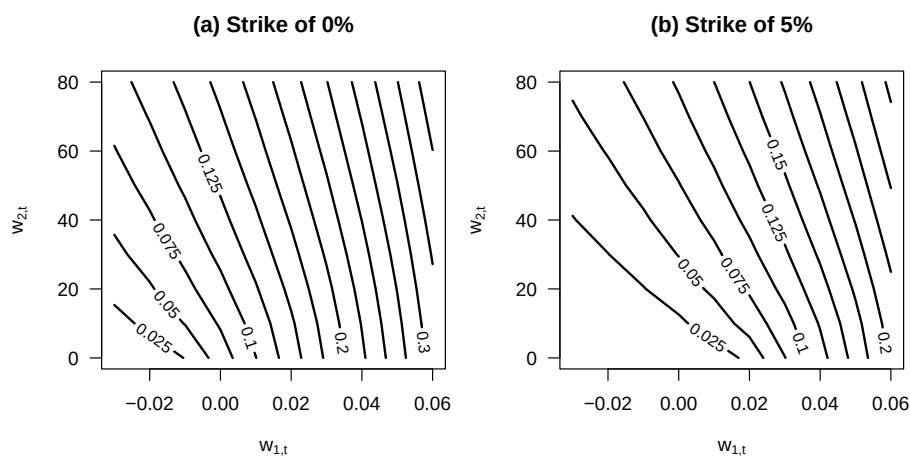
level <- .025

# Plot cap-price contours strike by strike.
nb_strikes <- length(all_K)
par(mfrow=c(1,nb_strikes))
par(plt=c(.2,.95,.2,.85))

for(k in 1:nb_strikes){
  M <- matrix(res$Caps[,k,h],n1,n2)

  contour(w1_values, w2_values, M,
          las=1,
          levels = seq(level,.4,by=level),
          xlab = expression(paste(w[1*'],'*t',sep="")),
          ylab = expression(paste(w[2*'],'*t',sep="")),
          main = paste("(",letters[k],") Strike of ",all_K[k]*freq*100,"%",sep=""),
          labcex = .8, lwd=2)
}

```



9.7 The price and yield effect of a deflation floor across states and horizons

This example studies the price and yield effect of a deflation floor across states and horizons.

```
# Purpose: This example studies the price and yield effect of a deflation floor across states
# and horizons
parameterization$mu_pi0 <- 0

# Uncond mean of w:
Ew1 <- 0
Ew2 <- model$alpha*model$mu/(1 - model$mu*model$beta)

W <- matrix(c(Ew1,Ew2,
              Ew1-.08/freq,Ew2),2,2,byrow=TRUE)

H <- 20
all_Pi_ult1 <- seq(.95,1.1,by=.005)
n_Pi <- length(all_Pi_ult1)
all_K <- - (matrix(1,n_Pi,1) %x% matrix(1/1:H,ncol=1))*
  (log(all_Pi_ult1) %x% matrix(1,H,1))

# Compute bond prices:

res <-
  price_inflation_caps_floors(W = W, # Values of state vector (T x n)
                              H = H, # maximum maturity, in model periods)
```

```

ell = ell, # Indexation lag
all_K = all_K, # vector of strikes, expressed at model frequency
# e.g.: quarter. data and K=.01 means annualized strike
# Indexation lag multiplicative factor
Pi_t_minus_ell = Pi_t_minus_ell,
psi = psi_stocks2fact, # Laplace transform of W
parameterization, # see details below
max_x = 10000, # settings for Riemann sum comput.
dx_statio = 10,
min_dx = 1e-05,
nb_x1 = 1000)

# Compute real rates:
resAB <- compute_AB_classical(xi0 = parameterization$xi0,
                             xi1 = parameterization$xi1,
                             kappa0 = - parameterization$mu_pi0,
                             kappa1 = - parameterization$mu_pi1,
                             psi = psi_stocks2fact,
                             psi.parameterization = parameterization$model,
                             H = H)

# plot(resAB$b[1,2,]*freq)
# Compute term structures of prices of real bonds (without deflation floor):
B <- exp(W %%% resAB$A[,2,] + matrix(1,dim(W)[1],1) %%% matrix(resAB$b[1,2,],nrow=1))

# plot(-freq*log(B[1,])/1:H)
# plot(-freq*log(B[2,])/1:H)

par(mfrow=c(2,dim(W)[1]))
par(plt=c(.2,.98,.2,.8))

for(i_state in 1:dim(W)[1]){

  M <- res$Floors[i_state,,]

  match_horizon <- NULL
  for(i in 1:n_Pi){
    aux <- M[(H*(i-1)+1):(H*i),]
    match_horizon <- c(match_horizon,diag(aux))
  }

  M <- matrix(match_horizon,H,n_Pi) * matrix(all_Pi_ult1,H,n_Pi,byrow = TRUE)

  levels <- c(.002,seq(.01,.10,by=.01))

  par(mfg=c(1,i_state))

```



```

contour(1:H, log(all_Pi_ult1), M,
       las=1,
       levels = levels,
       xlab = expression(paste("Residual maturity (",T-t,"), in quarters",sep="")),
       ylab = expression(log(Pi[u-1*','*t-1])),
       main = bquote("(a." * .(i_state) * ") Option price for " ~
                     w[1*","*t] == .(W[i_state,1])),
       labcex = .8, lwd=2)

# Compute change in yield:
B_without_option <- matrix(B[i_state,],H,n_Pi)
B_with_option <- B_without_option + M

yield_without_option <- - freq * log(B_without_option)/matrix(1:H,H,n_Pi)
yield_with_option    <- - freq * log(B_with_option)/matrix(1:H,H,n_Pi)

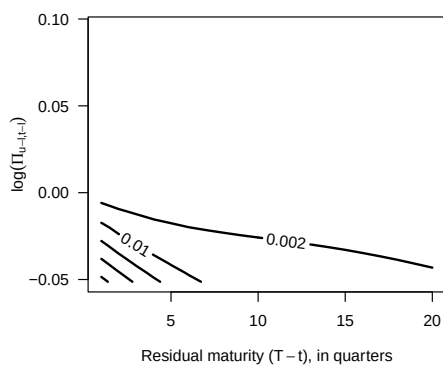
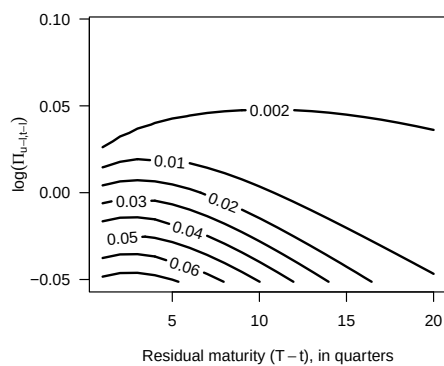
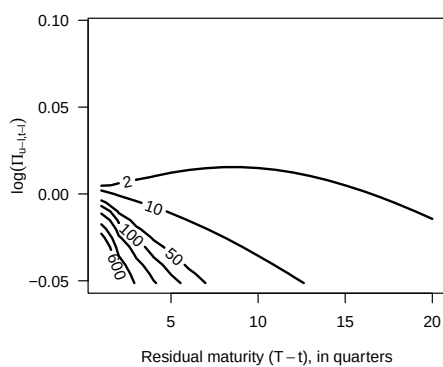
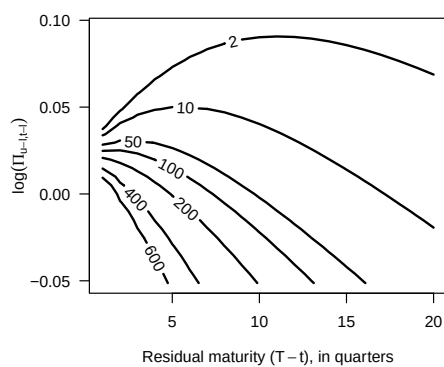
delta_yields <- 10000*(yield_without_option - yield_with_option)

par(mfg=c(2,i_state))

levels <- c(2,10,50,100,seq(200,600,by=200))

contour(1:H, log(all_Pi_ult1), delta_yields,
       las=1,
       levels = levels,
       xlab = expression(paste("Residual maturity (",T-t,"), in quarters",sep="")),
       ylab = expression(log(Pi[u-1*','*t-1])),
       main = bquote("(b." * .(i_state) * ") Yield effect (in bps) for " ~
                     w[1*","*t] == .(W[i_state,1])),
       labcex = .8, lwd=2)
}

```

(a.1) Option price for $w_{1,t} = 0$ (a.2) Option price for $w_{1,t} = -0.02$ (b.1) Yield effect (in bps) for $w_{1,t} = 0$ (b.2) Yield effect (in bps) for $w_{1,t} = -0.02$ 

Chapter 10

Credit and liquidity risks

10.1 Standard framework

This example compares standard structural and reduced-form views of credit spreads. The empirical series combine two ingredients: a credit-spread component and a default-rate component. The credit spread is computed from the Moody's Seasoned Aaa and Baa corporate bond yields available on FRED (DAAA and DBAA), using annual averages and the difference $DBAA - DAAA$. The default component contains global corporate default counts and default rates from S&P Global Ratings' "Global corporate default summary" table, sourced by S&P Global Ratings to S&P Global Ratings Credit Research & Insights and S&P Global Market Intelligence's CreditPro. The 2024 update corresponds to S&P Global Ratings, "Default, Transition, and Recovery: 2024 Annual Global Corporate Default And Rating Transition Study" (March 27, 2025); the same global summary table is also reproduced in related regional S&P default-transition studies.¹

Purpose: This example compares standard structural and reduced-form views of credit spreads
`library(DTAM)`

```
private_credit_file <- "private_data/Credit_spds.rda"
precomputed_figure <- if (knitr::is_latex_output()) {
  "_main_files/figure-latex/fig_standardCredit1-1.pdf"
} else {
  "_main_files/figure-latex/fig_standardCredit1-1.pdf"
}
```

¹The underlying observations are not distributed with DTAM or with the public version of this Rbookdown project. Readers wishing to reproduce them should consult the source pages for DAAA, DBAA, FRED terms, and the relevant S&P Global Ratings corporate default-transition study, subject to the applicable permissions.

```

if (!file.exists(private_credit_file) && file.exists(precomputed_figure)) {
  knitr::include_graphics(precomputed_figure)
} else {
  if (!file.exists(private_credit_file)) {
    stop("Private credit-spread data not found: ", private_credit_file, call. = FALSE)
  }
  load(private_credit_file)

RR <- .6
lambda <- (0.5*Credit_spds$Investment.grade.default.rate + 0.5*Credit_spds$Default.grade.default.rate)
spd_BBB_AAA <- Credit_spds$BBB_AAA_spd/100

# Spread's maturity:
H <- 20

make_models <- function(param){
  # Specify two-factor VARG model:
  mu_x <- exp(param[1])
  mu_lambda <- exp(param[2])
  rho_x <- exp(param[3])/(1+exp(param[3]))
  rho_lambda <- exp(param[4])/(1+exp(param[4]))
  nu <- exp(param[5])
  # prices of risk:
  alpha_x <- exp(param[6])
  alpha_lambda <- exp(param[7])
  alpha_SDF <- matrix(c(alpha_x,
                        alpha_lambda),ncol=1)

  alpha = matrix(0,2,1)
  mu = matrix(c(mu_x,mu_lambda),2,1)
  nu = matrix(c(nu,0),2,1)
  beta = matrix(c(rho_x/mu_x,(1-rho_lambda)/mu_lambda,0,rho_lambda/mu_lambda),2,2)

  model <- list(alpha=alpha,mu=mu,nu=nu,beta=beta)

  muQ <- mu / (1 - mu * alpha_SDF)
  alphaQ <- alpha * muQ/mu
  nuQ <- nu
  betaQ <- beta * (matrix(muQ/mu,ncol=1) %*% matrix(1,1,dim(beta)[2]))

  modelQ <- list(alpha=alphaQ,mu=muQ,nu=nuQ,beta=betaQ)

  return(list(model = model,
              modelQ = modelQ,
              mu_x=mu_x,mu_lambda=mu_lambda,

```

```

        rho_x=rho_x,rho_lambda=rho_lambda,
        nu=nu,alpha_x=alpha_x,alpha_lambda=alpha_lambda))
}

loss4optim <- function(param){

  Models <- make_models(param)
  model <- Models$model
  modelQ <- Models$modelQ

  xi0 <- 0
  xi1 <- matrix(0,length(model$alpha),1)
  kappa0 <- 0
  kappa1 <- matrix(c(0,1-RR),ncol=1)

  bond_prices <- compute_AB_classical(xi0=xi0, xi1=xi1,
                                     kappa0=kappa0, kappa1=kappa1,
                                     H=H, # maximum maturity
                                     psi=psi_VARG,psi.parameterization=modelQ)

  e <- 2 # e = 1 is for the risk-free yield
  b_spd <- bond_prices$b[1,e,H]
  a_spd <- matrix(bond_prices$a[,e,H],ncol=1)

  # Compute x to have a perfect fit of spreads:
  x <- (spd_BBB_AAA - b_spd - a_spd[2]*lambda)/a_spd[1]

  # Moments:
  mean_Y <- compute_uncondmean_VARG(model)
  Var_Y <- compute_uncondvar_VARG(model)

  # Penalize negative x values:
  penalty_neg_x <- 10000000*sum(abs(x[x<0]))

  # Penalty for distance between empirical and model-implied means:
  penalty_mean_distance_lambda <- 1000000000*abs(mean_Y[2] - mean(lambda))

  # penalty for average spread:
  avg_spd <- b_spd + sum(a_spd*mean_Y)
  penalty_mean_spd <- 1000000000*abs(avg_spd - mean(spd_BBB_AAA))

  # penalty for stdv of spreads:
  stdv_spd <- sqrt(t(a_spd) %*% Var_Y %*% a_spd)
  penalty_var_spd <- 10000*abs(stdv_spd - sd(spd_BBB_AAA))

```

```

# penalty for stdv of lambda:
a_lambda <- matrix(c(0,1),ncol=1)
stdv_lambda <- sqrt(t(a_lambda) %*% Var_Y %*% a_lambda)
penalty_var_lambda <- 10000*abs(stdv_lambda - sd(lambda))

Phi_Y <- model$beta * matrix(model$mu,2,2)
Var_Y_Y_1 <- rbind(cbind(Var_Y,Phi_Y%*%Var_Y),
                  cbind(Var_Y%*%t(Phi_Y),Var_Y))

# penalty for autocorrelation of lambda:
T <- length(spd_BBB_AAA)
Autocor_lambda <- cor(lambda[2:T],lambda[1:(T-1)])
a1 <- rbind(a_lambda,0*a_lambda)
a2 <- rbind(0*a_lambda,a_lambda)
cov_lambda_lambda_1 <- t(a1) %*% Var_Y_Y_1 %*% a2
AC_lambda_model <- cov_lambda_lambda_1/stdv_lambda^2
penalty_AC_lambda <- 1000*abs(AC_lambda_model - Autocor_lambda)

# penalty for autocorrelation of spd:
Autocor_spd <- cor(spd_BBB_AAA[2:T],spd_BBB_AAA[1:(T-1)])
a1 <- rbind(a_spd,0*a_spd)
a2 <- rbind(0*a_spd,a_spd)
cov_spd_spd_1 <- t(a1) %*% Var_Y_Y_1 %*% a2
AC_spd_model <- cov_spd_spd_1/stdv_spd^2
penalty_AC_spd <- 1000*abs(AC_spd_model - Autocor_spd)

# Compute loss:
loss <- penalty_neg_x +
  penalty_mean_distance_lambda +
  penalty_mean_spd + penalty_var_spd + penalty_var_lambda +
  penalty_AC_lambda + penalty_AC_spd

return(loss)
}

# Initial values:
rho_x <- .91
rho_lambda <- .2
mu_x <- .0005
mu_lambda <- .0025
nu <- 2.7
alpha_x <- 50
alpha_lambda <- 0.05
param <- log(c(mu_x,mu_lambda,rho_x,rho_lambda,nu,alpha_x,alpha_lambda))
param[3] <- log(rho_x/(1-rho_x))

```

```

param[4] <- log(rho_lambda/(1-rho_lambda))

for(i in 1:10){
  res.optim <- optim(par=param,
                    fn=loss4optim,
                    method="Nelder-Mead",
                    control=list(trace=FALSE,maxit=2000),hessian=FALSE)
}

param <- res.optim$par
Models <- make_models(param)
model <- Models$model
modelQ <- Models$modelQ

# Compute  $x_t$  -----
xi0 <- 0
xi1 <- matrix(0,length(model$alpha),1)
kappa0 <- 0
kappa1 <- matrix(c(0,1-RR),ncol=1)
bond_prices <- compute_AB_classical(xi0=xi0, xi1=xi1,
                                   kappa0=kappa0, kappa1=kappa1,
                                   H=H, # maximum maturity
                                   psi=psi_VARG,psi.parameterization=modelQ)

k <- 2 # k=1 is for the risk-free yield
b_spd <- bond_prices$b[1,k,H]
a_spd <- matrix(bond_prices$a[,k,H],ncol=1)
x <- (spd_BBB_AAA - b_spd - a_spd[2]*lambda)/a_spd[1]
Y <- cbind(x,lambda)

# Calculate Physical probabilities of default:
kappa1_lambda <- kappa1/(1-RR)
surviv_proba_P <- compute_AB_classical(xi0=xi0, xi1=xi1,
                                       kappa0=kappa0, kappa1=kappa1_lambda,
                                       H=H, # maximum maturity
                                       psi=psi_VARG,psi.parameterization=model)
surviv_proba_Q <- compute_AB_classical(xi0=xi0, xi1=xi1,
                                       kappa0=kappa0, kappa1=kappa1_lambda,
                                       H=H, # maximum maturity
                                       psi=psi_VARG,psi.parameterization=modelQ)

e <- 2 # e = 1 is for the risk-free entity
H1 <- 5
PD_P_H1 <- 1 - exp(surviv_proba_P$b[1,e,H1] + Y %*% surviv_proba_P$a[,e,H1])
PD_Q_H1 <- 1 - exp(surviv_proba_Q$b[1,e,H1] + Y %*% surviv_proba_Q$a[,e,H1])

```

```

H2 <- 10
PD_P_H2 <- 1 - exp(surviv_proba_P$B[1,e,H2] + Y %% surviv_proba_P$A[,e,H2])
PD_Q_H2 <- 1 - exp(surviv_proba_Q$B[1,e,H2] + Y %% surviv_proba_Q$A[,e,H2])

# nb_periods <- 200
# sim_Y <- simul_VARG(Models$model, nb_periods = nb_periods)
# plot(sim_Y[2,],type="l",lwd=2)
# lines(sim_Y[1,],col="red",lwd=2,lty=3)

par(mfrow=c(3,1))
par(plt=c(.1,.95,.2,.85))

plot(Credit_spds$year,Credit_spds$Default.rate/100,
     ylim=c(0,5),col="white",
     xlab="",ylab="in percent",las=1,
     main=expression(paste("(a) Default rate and credit spread",sep="")))
grid()
lines(Credit_spds$year,Credit_spds$Default.rate,lwd=2,lty=2)
lines(Credit_spds$year,Credit_spds$Investment.grade.default.rate,lwd=2,lty=1)
lines(Credit_spds$year,Credit_spds$BBB_AAA_spd,col="dark grey",type="b",lwd=2)

legend("topleft",
      legend=c(expression(paste("Default rate (average)",sep="")),
                expression(paste("Default rate (investment grade)",sep="")),
                expression(paste("Credit spread",sep="))),
      lty=c(2,1,NaN),
      pch=c(NaN,NaN,1),
      col=c("black","black","dark grey"),
      lwd = 2,bty = "n",seg.len = 3)

plot(Credit_spds$year,100*lambda,
     ylim=c(0,3),col="white",
     xlab="",ylab="in percent",las=1,
     main=expression(paste("(b) Factors ",x[t]," and ",lambda[t],sep="")))
grid()
lines(Credit_spds$year,100*lambda,lwd=2,type="b",pch=3)
lines(Credit_spds$year,100*x,col="dark grey",lwd=2,type="b",pch=0)

legend("topleft",
      legend=c(expression(x[t]),expression(lambda[t])),
      lty=c(NaN,NaN),
      pch=c(0,3),
      col=c("dark grey","black"),
      lwd = 2,bty = "n")

```



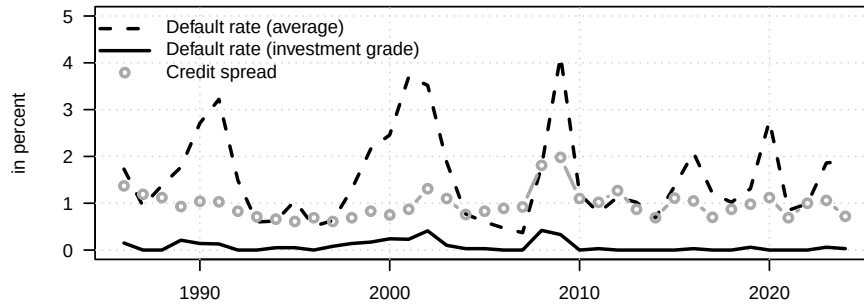
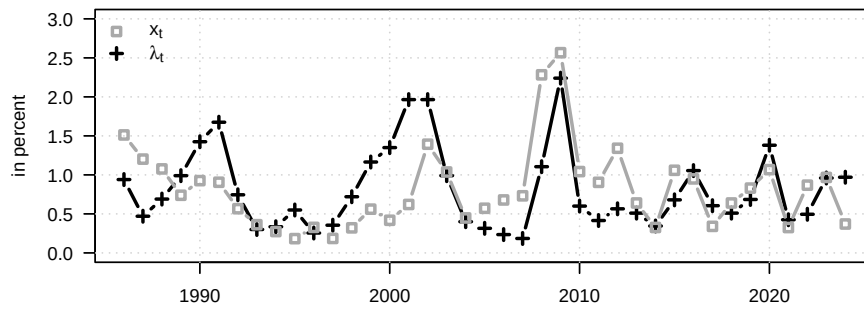
```

plot(Credit_spds$year,Credit_spds$Default.rate/100,
     ylim=c(0,120*max(PD_Q_H2)),col="white",
     xlab="",ylab="in percent",las=1,
     main=expression(paste("(c) Physical and risk-neutral probabilities of default (PD)",sep="")))
grid()
lines(Credit_spds$year,100*PD_P_H1,lwd=2,lty=1)
lines(Credit_spds$year,100*PD_Q_H1,lwd=2,lty=3)
lines(Credit_spds$year,100*PD_P_H2,lwd=2,lty=1,col="dark grey")
lines(Credit_spds$year,100*PD_Q_H2,lwd=2,lty=3,col="dark grey")

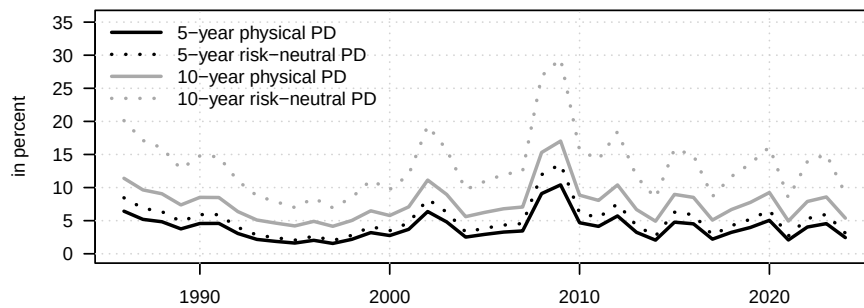
legend("topleft",
      legend=c("5-year physical PD",
               "5-year risk-neutral PD",
               "10-year physical PD",
               "10-year risk-neutral PD"),
      lty=c(1,3,1,3),
      col=c("black","black","dark grey","dark grey"),
      lwd = 2,bty = "n",seg.len = 3)
}

```

(a) Default rate and credit spread

(b) Factors x_t and λ_t 

(c) Physical and risk-neutral probabilities of default (PD)



10.2 Credit ratings

This example illustrates bond pricing and migration dynamics in a ratings-based framework.

```
# Purpose: This example illustrates bond pricing and migration dynamics in a ratings-based
# framework
library(DTAM)
```

```

nb.sim <- 150

# Baseline matrix of transition probabilities:
Pi <- t(matrix(c(.95,.03,.02,.0,
                 .03,.94,.025,.005,
                 .05,.09,.85,.01,
                 0,0,0,1),4,4))
K <- dim(Pi)[1] # number of ratings
eig_res <- eigen(Pi)
V <- eig_res$vectors
V <- V[,K:1]
eig_values <- eig_res$values[K:1] # ensure 1 is the last eigenvalue.

# Specify model -----

# Credit ratings:
model <- list()
model$xi0 <- .02
model$xi1 <- matrix(c(.01,0),ncol=1)
model$kappa0 <- matrix(-log(eig_values),nrow=1)
model$kappa1 <- rbind(0,.2*model$kappa0)
model$V <- V

# Specify y_t's dynamics:
mu <- matrix(0,2,1)
rho <- .9
Phi <- rho*diag(2)
Sigma <- (1-rho^2) * diag(2) # This ensures that Var(y_{i,t})=1.
model$psi.parameterization <- list(mu = mu, Phi = Phi, Sigma = Sigma)

# Simulate Y:
Y <- simul_GaussianVAR(model$psi.parameterization,
                      nb.sim = nb.sim)

# Price bonds: -----
res_ratings <- compute_bond_price_Ratings(model,Y,H=10,psi=psi_GaussianVAR)

# Simulate ratings: -----
# flag remains 1 as long as:
# - not all ratings visited
# - a default occur
# - short-lived rating (less than 2 periods)
# <=> max(tau) == K-1

```

```

flag <- 1
while(flag==1){
  tau1 <- simul_rating_migration(model,Y,tau_ini=1)
  if((max(tau1)==K-1)&( sum(abs(diff(tau1[1:(nb.sim-1)]))*diff(tau1[2:nb.sim]))==0 )){
    flag <- 0
  }
}
flag <- 1
while(flag==1){
  tau2 <- simul_rating_migration(model,Y,tau_ini=1)
  if((max(tau2)==K-1)&( sum(abs(diff(tau2[1:(nb.sim-1)]))*diff(tau2[2:nb.sim]))==0 )){
    flag <- 0
  }
}

# Prepare plot -----

h <- 5

spreads <- cbind(res_ratings$Risky_bond_spreads[, ,h],10)
spds1 <- matrix(NaN,dim(Y)[1],1)
spds2 <- matrix(NaN,dim(Y)[1],1)
for(t in 1:dim(Y)[1]){
  spds1[t] <- spreads[t,tau1[t]]
  spds2[t] <- spreads[t,tau2[t]]
}

par(mfrow=c(1,1))
par(plt=c(.1,.95,.2,.95))
plot(10000*res_ratings$Risky_bond_spreads[,1,h],type="l",col="grey",lty=1,lwd=3,
     ylim=10000*c(0,1.3*max(res_ratings$Risky_bond_spreads[, ,h])),
     las=1,xlab="time",ylab="spread (in basis points)")
lines(10000*res_ratings$Risky_bond_spreads[,2,h],type="l",col="grey",lty=2,lwd=3)
lines(10000*res_ratings$Risky_bond_spreads[,3,h],type="l",col="grey",lty=3,lwd=3)

lines(10000*spds1,lwd=3)
lines(10000*spds2,lwd=3,lty=3)

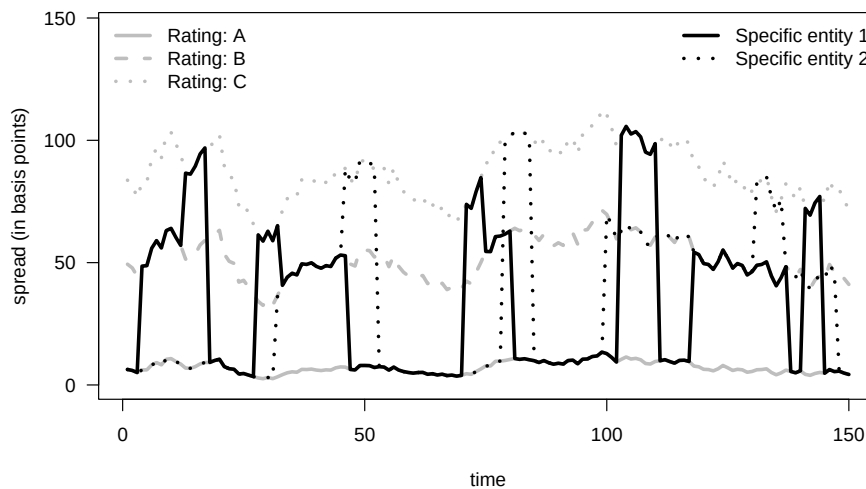
legend("topleft",
      legend=c(paste("Rating: ",LETTERS[1:(K-1)],sep="")),
      lty=c(1:(K-1)),
      col=c(rep("grey",K-1)),
      lwd = c(rep(3,K-1)),bty = "n")

```

```

legend("topright",
      legend=c("Specific entity 1",
               "Specific entity 2"),
      lty=c(1,3),
      col=c("black","black"),
      lwd = c(3,3),bty = "n")

```



10.3 VARG

This example simulates default intensities and state variables in the VARG credit model.

Purpose: This example simulates default intensities and state variables in the VARG credit model

```
library(DTAM)
```

```
nb_periods <- 150
```

Set the dimensions of the VARG credit system.

Number of entities (hence dimension of delta):

```
n.e <- 2
```

Number of elements in y:

```
n.y <- 2 # in the context of this example: two entity-specific factors
```

Dimension of w:

```
n.w <- n.y + n.e
```

($w_t = [y_t', \delta_t']'$)

```

beta.delta.y <- .2
beta.delta   <- 1

# Calibrate the common state factors and the default indicators.
mu.y <- .004
nu.y <- .5
mu.delta <- 1

rho.y <- .8
beta.y <- rho.y/mu.y

alpha <- matrix(0,n.w,1)
mu     <- matrix(c(rep(mu.y,n.y),
                    rep(mu.delta,n.e)),n.w,1)
# mu[2] <- .75*mu[1]
nu     <- matrix(c(rep(nu.y,n.y),
                    rep(0,n.e)),n.w,1)
beta   <- matrix(0,n.w,n.w)
beta[1,1] <- beta.y
beta[2,2] <- beta.y
beta[3,1] <- beta.delta.y
beta[4,2] <- beta.delta.y
# beta[3,4] <- beta.delta
beta[4,3] <- beta.delta
model <- list(
  alpha = alpha,
  mu = mu,
  nu = nu,
  beta = beta
)

# Simulate state variables and default indicators.
w0 <- matrix(0,n.w,1) # Initial state variable

simul_w <- simul_VARG(model,
                      nb_periods=nb_periods,w0=w0)

# Plot the two entity-specific state factors.
par(mfrow=c(2,1))
par(plt=c(.1,.95,.15,.8))

plot(simul_w[1,],
     type="l",ylim=c(0,max(simul_w[1:2,])),lwd=2,
     xlab="",ylab="",

```

```

    main=expression(paste("(a) Simulated ",y[1*'],'*t'," and ",y[2*'],'*t'],sep=""),
    las=1)
grid()
lines(simul_w[2,],col="dark grey",lwd=2)

legend("topleft",
      legend=c(
        expression(paste(y[1*'],'*t'],sep="")),
        expression(paste(y[2*'],'*t'],sep=""))),
      lty=c(1,1),
      col=c("black","dark grey"),
      lwd=c(2,2),bty = "n")

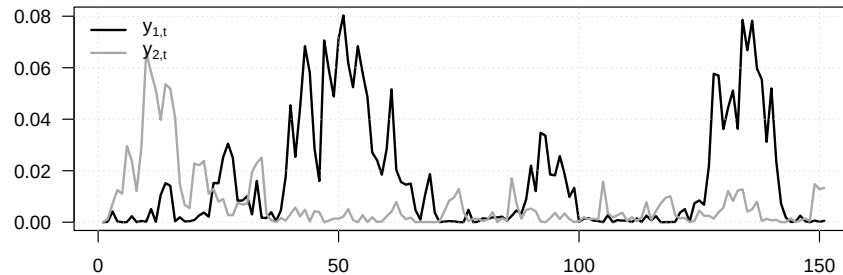
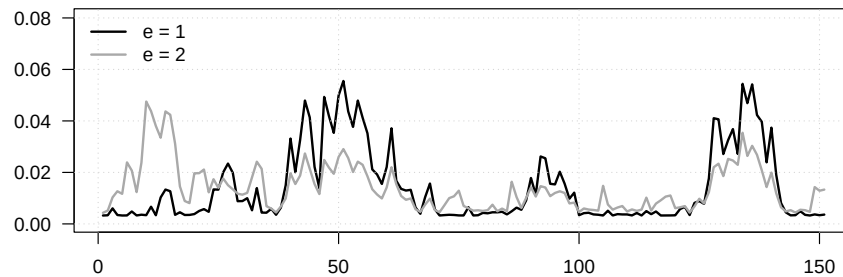
# Reset realized defaults before computing conditional default probabilities.
# Remove defaults:
simul_w[(n.y+1):n.w,] <- 0
# plot(simul_w[3,],type="l",
#       ylim=c(0,max(simul_w[3:4,])))
# lines(simul_w[4,],col="dark grey",lwd=2)

h <- 5
res_PD <- compute_proba_def_VARG(model,H=h,indic_delta1 = n.y+1,
                                X=t(simul_w))

# Plot default probabilities implied by the simulated state variables.
plot(res_PD$PD[,1,h],
     type="l",ylim=c(0,max(simul_w[1:2,])),lwd=2,
     xlab="",ylab="",
     main=expression(paste("(b) Probability of default (",h,"=5)",sep="")),
     las=1)
grid()
lines(res_PD$PD[,2,h],col="dark grey",lwd=2)

legend("topleft",
      legend=c(
        expression(paste("e = 1",sep="")),
        expression(paste("e = 2",sep=""))),
      lty=c(1,1),
      col=c("black","dark grey"),
      lwd=c(2,2),bty = "n")

```

(a) Simulated $y_{1,t}$ and $y_{2,t}$ (b) Probability of default ($h=5$)

10.3.1 CDS spreads vary with the state variables in the VARG credit model

This example shows how CDS spreads vary with the state variables in the VARG credit model.

Purpose: This example shows how CDS spreads vary with the state variables in the VARG credit model

```
H <- 20
```

Specification of the short-term rate:

```
model$xi0 <- 0
```

```
model$xi1 <- matrix(rep(0,n.w),ncol=1)
```

Specification of the recovery rate:

```
# ( RR_t = exp(- a.0 - a.1'w_t) )
```

```
model$mu_R1 <- matrix(0,n.y+n.e,n.e) # one column = one entity
```

```
model$mu_R0 <- rep(100,n.e,1)
```

SDF specification:

```
kappa <- 0.3
```

```
model$alpha_w <- matrix(c(rep(0,n.y),rep(kappa,n.e)),n.w,1)
```



```

# Determine risk-neutral dynamics:
modelQ <- make_VARG_Q(model)

# Determine values of state vector where CDS evaluated:
Ew <- compute_uncondmean_VARG(model)
w <- Ew
w[(n.y+1):n.w] <- 0 # no default
w <- cbind(w,w)
w[,2] <- 2*w[,1]

# CDS pricing -----
res_cds <- prices_CDS_RFV_VARG(modelQ,H=H,indic_delta1 = n.y+1,
                               X=t(w))
res_cds_P <- prices_CDS_RFV_VARG(model,H=H,indic_delta1 = n.y+1,
                               X=t(w))

# Add FX -----
model_FX <- model

# Specification of exchange rate:
# (increase in s = depreciation of domestic currency)
model_FX$mu_s1 <- matrix(0,n.y+n.e,1) # one column = one entity
model_FX$mu_s1[(n.y+1):n.w] <- .05
model_FX$mu_s0 <- 0

# Determine risk-neutral dynamics:
model_FXQ <- make_VARG_Q(model_FX)

res_cds_FX <- prices_CDS_RFV_VARG(model_FXQ,H=H,indic_delta1 = n.y+1,
                               X=t(w))
res_cds_FXP <- prices_CDS_RFV_VARG(model_FX,H=H,indic_delta1 = n.y+1,
                               X=t(w))

# -----

par(mfrow=c(1,2))
par(plt=c(.2,.95,.2,.85))

for(e in 1:2){
  if(e==1){
    maint <- expression(paste("(a) Entity e = 1",sep=""))
  }else{
    maint <- expression(paste("(b) Entity e = 2",sep=""))
  }
  plot(10000*res_cds_FX$CDS_spreads[1,e,],type="l",

```

```

ylim=10000*c(0,1.3*max(res_cds_FX$CDS_spreads[1,e,])),lwd=2,
xlab="maturity",ylab="CDS spread, in basis points",
main=maint)
grid()
lines(10000*res_cds_FXP$CDS_spreads[1,e,],type="l",lwd=3,lty=3)

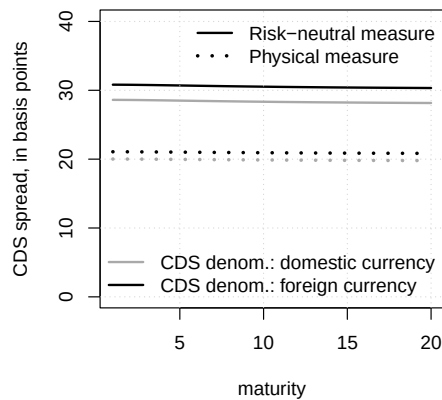
lines(10000*res_cds$CDS_spreads[1,e,],type="l",lwd=2,lty=1,col="darkgrey")
lines(10000*res_cds_P$CDS_spreads[1,e,],type="l",lwd=3,lty=3,col="darkgrey")

if(e==1){
  legend("bottomright",
    # legend=c(
    #   expression(paste(y[i*'],'*t'," = E(",y[i*'],'*t],")",sep="")),
    #   expression(paste(y[i*'],'*t'," = ",2%%E,"(",y[i*'],'*t],")",sep=""))
    # ),
    legend=c("CDS denom.: domestic currency",
             "CDS denom.: foreign currency"),
    lty=c(1,1),
    col=c("dark grey","black"),
    lwd=c(2,2),bty = "n")

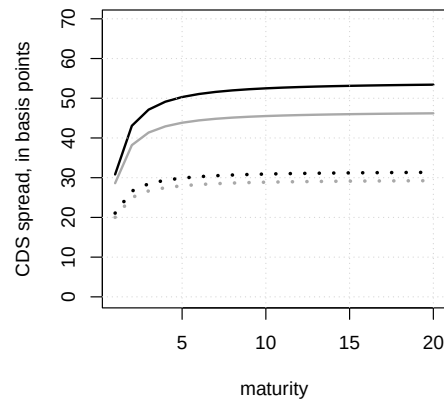
  legend("topright",
    c("Risk-neutral measure",
      "Physical measure"),
    lty=c(1,3),
    col=c("black","black"),
    lwd=c(2,3),bty = "n")
}
}

```

(a) Entity e = 1



(b) Entity e = 2



```

# for(e in 1:2){
#   if(e==1){
#     maint <- expression(paste("(c) Entity e = 1 (foreign currency)",sep=""))
#   }else{
#     maint <- expression(paste("(d) Entity e = 2 (foreign currency)",sep=""))
#   }
#   plot(10000*res_cds_FX$CDS_spreads[1,e,],type="l",
#        ylim=10000*c(0,max(res_cds_FX$CDS_spreads[,e,])),lwd=2,
#        xlab="maturity",ylab="CDS spread, in basis points",
#        main=maint)
#   grid()
#   lines(10000*res_cds_FXP$CDS_spreads[1,e,],type="l",lwd=2,lty=3)

#   lines(10000*res_cds_FX$CDS_spreads[2,e,],type="l",lwd=2,lty=1,col="dark grey")
#   lines(10000*res_cds_FXP$CDS_spreads[2,e,],type="l",lwd=2,lty=3,col="dark grey")
# }

```

10.4 Top-down approach

This example illustrates clustering and default-count dynamics in the top-down credit model.

```

# Purpose: This example illustrates clustering and default-count dynamics in the top-down credit
# model
library(DTAM)

J <- 2 # Number of segments

# Specification of y_t process:
rho <- .9
mu.y <- .002
beta.yy <- rho/mu.y
nu.y <- .25

kappa <- .2

beta_A <- 100
beta_B <- 10
zeta_A <- 0
zeta_B <- 0.45

model_no_cluster <- list(
  mu.y = mu.y,

```

```

nu.y = nu.y,
alpha.y = 0,
beta.yy = beta.yy,
beta.yn = matrix(0,1,J),
alpha.n = rep(0,J),
beta.nn = matrix(zeta_A,J,J),
beta.ny = matrix(beta_A,J,1),
n_w = J + 1,
xi0 = 0,
xi1 = matrix(c(0,rep(0,J)),J+1,1),
alpha.m = matrix(c(0,kappa,0),J+1,1),
I = rep(100,J)
)

nb_periods <- 200
sim_TopDown <- simul_TopDown(model_no_cluster,nb_periods=nb_periods)
W <- sim_TopDown$all_w

# Construct selection vectors:
e1 <- matrix(0,J+1,1)
e1[1+1] <- 1
e2 <- matrix(0,J+1,1)
e2[1+2] <- 1

H <- 5
max.N <- 20

for(indic_cluster in 0:1){
  if(indic_cluster==1){
    model_cluster <- model_no_cluster
    # Specification with clustering:
    model_cluster$beta.nn <- matrix(zeta_B,J,J)
    model_cluster$beta.ny <- matrix(beta_B,J,1)
    model <- model_cluster
  }else{
    model <- model_no_cluster
  }

  # Compute first and second moments of w_t:
  EV <- compute_expect_variance(psi_w_TopDown,model,du=1e-5)
  EVQ <- compute_expect_variance(psiQ_w_TopDown,model,du=1e-5)

  # Situation where the value of y_t differs:
  # W <- cbind(EV$Ew,
  #           EV$Ew + 1*matrix(sqrt(diag(EV$Vw)),ncol=1))

```

```

# Situation where the value of n_{2,t} differs:
W <- cbind(EV$Ew,EV$Ew)

W[2:(J+1),] <- 0 # set n to zeros.
W[J+1,2] <- 2

res <- compute_G(model,
  W = W,
  gamma = matrix(0,J+1,1),
  v = e1,
  b.matrix = t(matrix(.5+(0:max.N),max.N+1,H)),
  H = H,
  indic_Q = FALSE,
  max_x = 500,
  dx_statio = .2,
  min_dx = 10^(-5),
  nb_x1 = 500)
Probas <- cbind(res[,1,H],
  res[,2:(max.N+1),H] - res[,1:(max.N),H])

if(indic_cluster==0){
  Probas_no_cluster <- Probas
}else{
  Probas_cluster <- Probas
}
}

par(mfrow=c(1,dim(W)[2]))
par(plt=c(.2,.95,.17,.88))

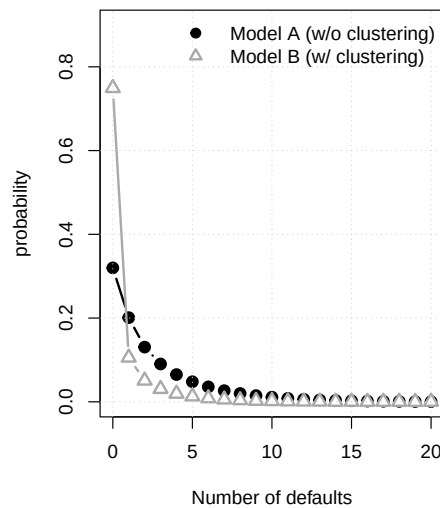
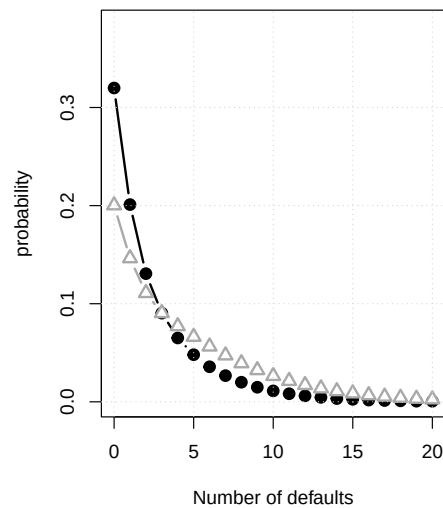
for(i in 1:dim(W)[2]){
  if(i==1){
    main.t <- expression(paste("(a) ",n[1*','*t]," = 0 and ",n[2*','*t]," = 0",sep=""))
  }else{
    main.t <- expression(paste("(b) ",n[1*','*t]," = 0 and ",n[2*','*t]," = 2",sep=""))
  }
  plot(0:max.N,Probas_no_cluster[i,],type="b",pch=19,lwd=2,cex=1.2,
    ylim=c(0,1.2*max(Probas_no_cluster[i,],
      Probas_cluster[i,])),
    main=main.t,
    xlab="Number of defaults",ylab="probability")
  lines(0:max.N,Probas_cluster[i,],type="b",pch=24,lwd=2,cex=1.2,
    bg="white",col="dark grey")
  grid()
}

```

```

if(i==1){
  legend("topright",
    legend=c(
      "Model A (w/o clustering)",
      "Model B (w/ clustering)" ),
    lty=c(NaN,NaN),
    pch = c(19,24),
    col=c("black","dark grey"),
    pt.bg = c("black","white"),
    lwd=c(2,2),bty = "n")
}
}

```

(a) $n_{1,t} = 0$ and $n_{2,t} = 0$ (b) $n_{1,t} = 0$ and $n_{2,t} = 2$ 

10.4.1 Prices CDS index spreads in the top-down model

This example prices CDS index spreads in the top-down model.

Purpose: This example prices CDS index spreads in the top-down model
`H <- 10`

```

# Compute CDS spreads without default clustering.
CDS_no_cluster <- compute_CDS_TopDown(model_no_cluster,
  H=H,
  W=W,
  RR=0)
CDS_P_no_cluster <- compute_CDS_TopDown(model_no_cluster,
  H=H,

```

```

                                W=W,
                                RR=0,
                                indic_Q = FALSE)
# Compute CDS spreads with default clustering.
CDS_cluster <- compute_CDS_TopDown(model_cluster,H=H,
                                W = W,
                                RR=0)
CDS_P_cluster <- compute_CDS_TopDown(model_cluster,H=H,
                                W = W,
                                RR=0,
                                indic_Q = FALSE)

par(mfrow=c(1,2))
par(plt=c(.2,.95,.2,.85))

# Plot both models under two initial default-count states.
for(value_of_w in 1:2){
  if(value_of_w==1){
    main.t <- expression(paste("(a) ",n[1*','*t]," = 0 and ",n[2*','*t]," = 0",sep=""))
  }else{
    main.t <- expression(paste("(b) ",n[1*','*t]," = 0 and ",n[2*','*t]," = 2",sep=""))
  }

  plot(1:H,10000*CDS_cluster[value_of_w,1,],type="l",lwd=2,
        ylim=10000*c(0,1.4*max(CDS_cluster[,,],
                                CDS_no_cluster[,,])),
        main=main.t,
        xlab="maturity",ylab="CDS spread (in bps)",las=1)
  grid()

  points(1:H,10000*CDS_P_cluster[value_of_w,1,],lwd=2,cex=1,pch=3)
  lines(1:H,10000*CDS_cluster[value_of_w,2,],lwd=2,cex=1,lty=3)

# Overlay the no-clustering benchmark in grey.
  points(1:H,10000*CDS_P_no_cluster[value_of_w,1,],lwd=2,cex=1,pch=3,
        col="dark grey")
  lines(1:H,10000*CDS_no_cluster[value_of_w,2,],lwd=2,cex=1,lty=3,
        col="dark grey")
  lines(1:H,10000*CDS_no_cluster[value_of_w,1,],lwd=2,cex=1,lty=1,
        col="dark grey")

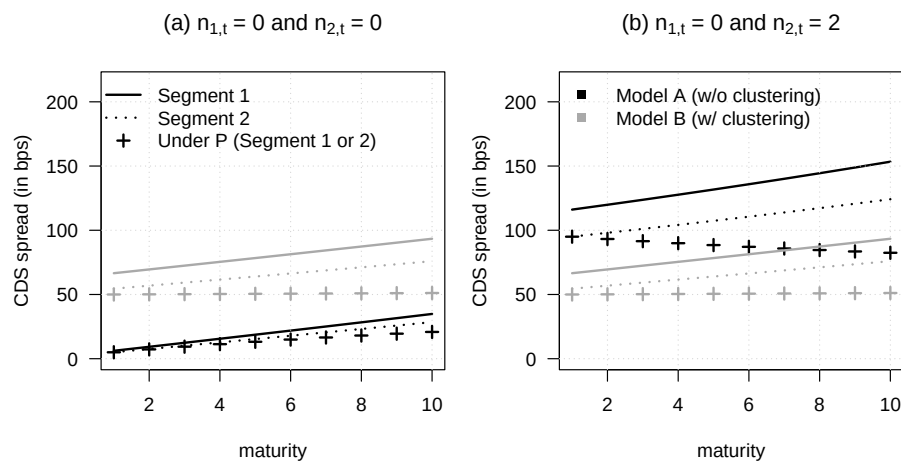
  if(value_of_w==1){
    legend("topleft",
          legend=c(
            "Segment 1",

```

```

        "Segment 2",
        "Under P (Segment 1 or 2)"),
lty=c(1,3,NaN),
pch=c(NaN,NaN,3),
col=c("black"),
lwd=c(2,2,2),bty = "n")
}else{
  legend("topleft",
    legend=c(
      "Model A (w/o clustering)",
      "Model B (w/ clustering)"),
    lty=c(NaN),
    pch=c(15),
    col=c("black","dark grey"),
    lwd=c(2),bty = "n")
}
}

```



10.4.2 Tranche pricing across attachment points in the top-down framework

This example studies tranche pricing across attachment points in the top-down framework.

```

# Purpose: This example studies tranche pricing across attachment points in the top-down
# framework

# Maximum maturity of tranche products:
H <- 10

```



```

# Segment:
j <- 1

# Under P or Q:
indic_Q <- TRUE

vector.of.a <- c(0,.03,.06)
tranche_no_cluster <- compute_tranche(model_no_cluster,
                                       W = W,
                                       RR = 0.5,
                                       indic_Q = indic_Q,
                                       j=j,H=H,
                                       vector.of.a,
                                       max_x = 500,
                                       dx_statio = .2,
                                       min_dx = 10(-5),
                                       nb_x1 = 500)
tranche_cluster <- compute_tranche(model_cluster,
                                    W = W,
                                    RR = 0.5,
                                    indic_Q = indic_Q,
                                    j=j,H=H,
                                    vector.of.a,
                                    max_x = 500,
                                    dx_statio = .2,
                                    min_dx = 10(-5),
                                    nb_x1 = 500)

par(mfrow=c(1,2))
par(plt=c(.2,.95,.2,.85))

for(value_of_w in 1:2){
  if(value_of_w==1){
    main.t <- expression(paste("(a) ",n[1*','*t]," = 0 and ",n[2*','*t]," = 0",sep=""))
  }else{
    main.t <- expression(paste("(b) ",n[1*','*t]," = 0 and ",n[2*','*t]," = 2",sep=""))
  }

  tranche <- 1
  plot(tranche_no_cluster$tranche_prices[value_of_w,tranche,],
       ylim=c(0,1.3*max(tranche_no_cluster$tranche_prices[,,],
                        tranche_cluster$tranche_prices[,,])),
       type="l",las=1,lwd=2,col="white",main=main.t,
       xlab="maturity",ylab="tranche price")

```

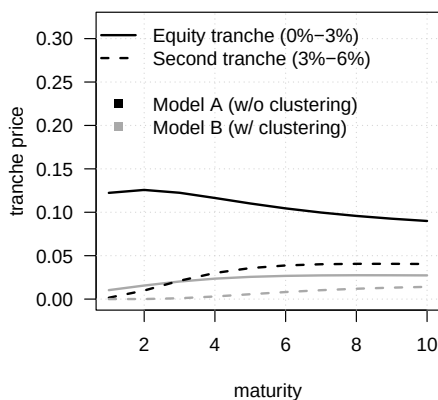
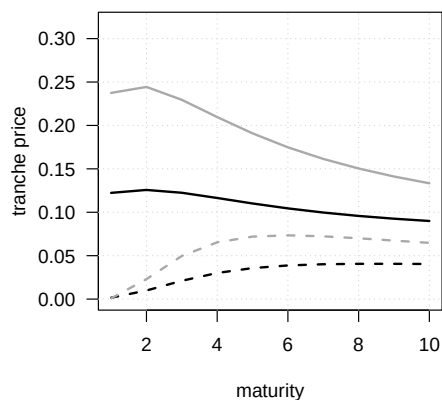
```

grid()

for(tranche in 1:2){
  lines(tranche_no_cluster$tranche_prices[value_of_w,tranche,],
        lty=tranche,col="black",lwd=2)
  lines(tranche_cluster$tranche_prices[value_of_w,tranche,],
        lty=tranche,col="dark grey",lwd=2)
}

if(value_of_w==1){
  legend("topleft",
        legend=c(
          "Equity tranche (0%-3%)",
          "Second tranche (3%-6%)",
          "",
          "Model A (w/o clustering)",
          "Model B (w/ clustering)"
        ),
        lty=c(1,2,NaN,NaN,NaN),
        pch=c(NaN,NaN,NaN,15,15),
        col=c("black","black","white","black","dark grey"),
        lwd=c(2),bty = "n")
}
}

```

(a) $n_{1,t} = 0$ and $n_{2,t} = 0$ (b) $n_{1,t} = 0$ and $n_{2,t} = 2$ 

Chapter 11

Consumption Capital Asset Pricing Model

11.1 Risk aversion and intertemporal substitution in the CCAPM

This example contrasts the roles of risk aversion and intertemporal substitution in the CCAPM.

```
# Purpose: This example contrasts the roles of risk aversion and intertemporal substitution in  
# the CCAPM  
  
# Define the two-state consumption lottery.  
C.l <- .8  
C.h <- 1.2  
cc <- c(C.l,C.h)  
  
par(mfrow=c(1,2))  
par(plt=c(.15,.95,.25,.85))  
  
C <- seq(.7,1.5,by=.01)  
  
# Plot certainty equivalents for moderate risk aversion.  
gamma <- 2  
E.u <- mean(cc^(1-gamma)/(1-gamma))  
c.CE <- (E.u*(1-gamma))^(1/(1-gamma))  
U <- C^(1-gamma)/(1-gamma)  
plot(C,U,type="l",lwd=2,las=1,  
      xlab="Consumption",ylab="Utility",
```

```

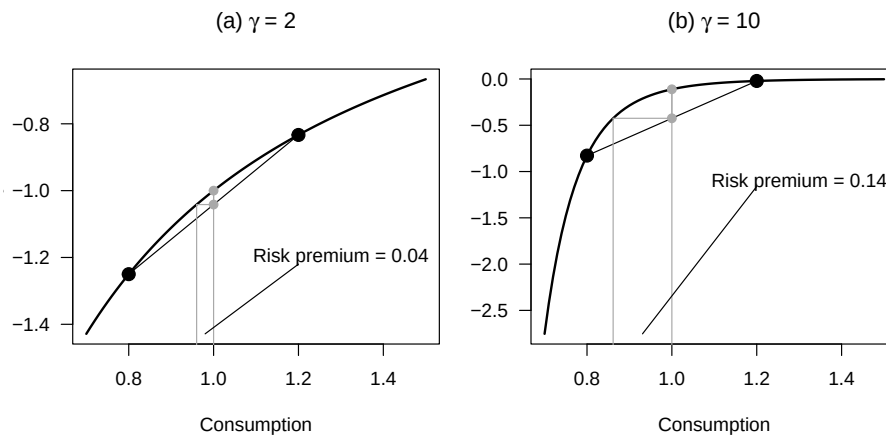
    main=expression(paste("(a) ",gamma," = 2",sep=""))
points(cc,cc^(1-gamma)/(1-gamma),pch=19,col="black",cex=1.5)
lines(cc,cc^(1-gamma)/(1-gamma),col="black")
lines(c(c.CE,mean(cc)),c(E.u,E.u),lty=1,col="darkgrey",lwd=1)
lines(c(c.CE,c.CE),c(-1000,E.u),lty=1,col="darkgrey",lwd=1)
lines(c(mean(cc),mean(cc)),c(-1000,1/(1-gamma)),lty=1,col="darkgrey",lwd=1)
points(c(1,1),c(mean(cc^(1-gamma)/(1-gamma)),1/(1-gamma)),col="darkgrey",pch=19)
lines(c(1,1),c(mean(cc^(1-gamma)/(1-gamma)),1/(1-gamma)),col="darkgrey")
rp <- - mean(cc^(1-gamma)/(1-gamma)) + 1/(1-gamma)

# Annotate the certainty equivalent and the risk-premium gap.
text(x = 1.3,y=1/(1-gamma) - .2,
     labels = paste("Risk premium = ",toString(round(mean(cc)-c.CE,2)),sep=""),
     col="black")
lines(c(mean(c(c.CE,1)),1.2),c(C[1]^(1-gamma)/(1-gamma),1/(1-gamma) - .22),col="black")

# Repeat the same construction with higher risk aversion.
gamma <- 10
E.u <- mean(cc^(1-gamma)/(1-gamma))
c.CE <- (E.u*(1-gamma))^(1/(1-gamma))
U <- C^(1-gamma)/(1-gamma)

plot(C,U,type="l",lwd=2,las=1,
     xlab="Consumption",ylab="Utility",
     main=expression(paste("(b) ",gamma," = 10",sep="")))
points(cc,cc^(1-gamma)/(1-gamma),pch=19,col="black",cex=1.5)
lines(cc,cc^(1-gamma)/(1-gamma),col="black")
lines(c(c.CE,mean(cc)),c(E.u,E.u),lty=1,col="darkgrey",lwd=1)
lines(c(c.CE,c.CE),c(-1000,E.u),lty=1,col="darkgrey",lwd=1)
lines(c(mean(cc),mean(cc)),c(-1000,1/(1-gamma)),lty=1,col="darkgrey",lwd=1)
points(c(1,1),c(mean(cc^(1-gamma)/(1-gamma)),1/(1-gamma)),col="darkgrey",pch=19)
lines(c(1,1),c(mean(cc^(1-gamma)/(1-gamma)),1/(1-gamma)),col="darkgrey")
rp <- - mean(cc^(1-gamma)/(1-gamma)) + 1/(1-gamma)
text(x = 1.3,y=1/(1-gamma) - 1,
     labels = paste("Risk premium = ",toString(round(mean(cc)-c.CE,2)),sep=""),
     col="black")
lines(c(mean(c(c.CE,1)),1.2),c(C[1]^(1-gamma)/(1-gamma),1/(1-gamma) - 1.05),col="black")

```



11.2 Asset-pricing implications of risk-aversion and EIS calibrations

This example visualizes asset-pricing implications of different risk-aversion and EIS calibrations.

Purpose: This example visualizes asset-pricing implications of different risk-aversion and EIS calibrations

Define the two-state consumption lottery.

`C.l <- .8`

`C.h <- 1.2`

`cc <- c(C.l,C.h)`

`par(mfrow=c(1,1))`

`par(plt=c(.15,.95,.25,.95))`

`C <- seq(.7,1.5,by=.01)`

Plot the certainty equivalent and the implied risk premium.

`gamma <- 5`

`E.u <- mean(cc^(1-gamma)/(1-gamma))`

`c.CE <- (E.u*(1-gamma))^(1/(1-gamma))`

`U <- C^(1-gamma)/(1-gamma)`

`plot(C,U,type="l",lwd=2,las=1,`
`xlab="Consumption",ylab="Utility")`

`grid()`

`points(cc,cc^(1-gamma)/(1-gamma),pch=19,col="black",cex=1.5)`

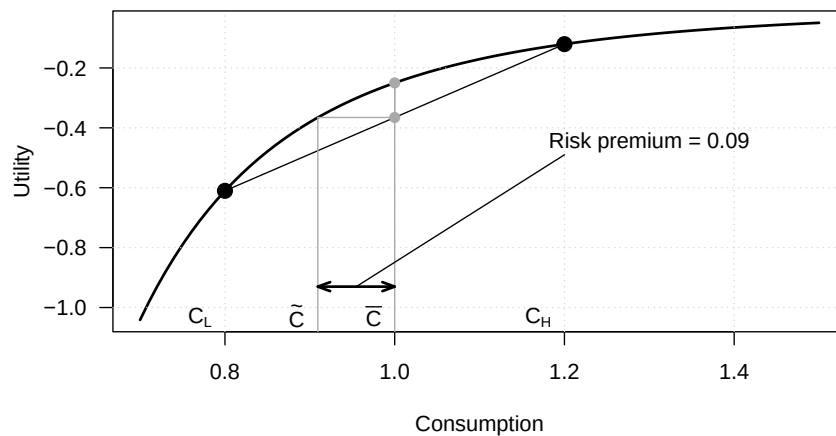
`lines(cc,cc^(1-gamma)/(1-gamma),col="black")`

```

lines(c(c.CE,mean(cc)),c(E.u,E.u),lty=1,col="darkgrey",lwd=1)
lines(c(c.CE,c.CE),c(-1000,E.u),lty=1,col="darkgrey",lwd=1)
lines(c(mean(cc),mean(cc)),c(-1000,1/(1-gamma)),lty=1,col="darkgrey",lwd=1)
points(c(1,1),c(mean(cc^(1-gamma))/(1-gamma)),1/(1-gamma)),col="darkgrey",pch=19)
lines(c(1,1),c(mean(cc^(1-gamma))/(1-gamma)),1/(1-gamma)),col="darkgrey")
rp <- - mean(cc^(1-gamma))/(1-gamma) + 1/(1-gamma)
text(x = 1.3,y=1/(1-gamma) - .2,
     labels = paste("Risk premium = ",toString(round(mean(cc)-c.CE,2)),sep=""),
     col="black")
lines(c(mean(c(c.CE,1)),1.2),c(C[3]^(1-gamma)/(1-gamma),1/(1-gamma) - .24),col="black")
text(x=C.l,y=min(U),labels = expression(C[L]),pos=2)
text(x=C.h,y=min(U),labels = expression(C[H]),pos=2)
text(x=c.CE,y=min(U),labels = expression(tilde(C)),pos=2)
text(x=1,y=min(U),labels = expression(bar(C)),pos=2)

# Mark the risk-premium distance on the consumption axis.
arrows(c.CE,C[3]^(1-gamma)/(1-gamma),1,C[3]^(1-gamma)/(1-gamma),lwd=2,
       angle = 20,code = 3,length = 0.1)

```



The following codes makes a table that illustrates the equity premium puzzle.

This table summarizes the long-run joint behavior of macro-financial variables used in the CCAPM discussion.

Purpose: This table summarizes the long-run joint behavior of macro-financial variables in the CCAPM discussion

```
library(DTAM)
```

```
lag_exp_infl <- 5 # order of moving average process used to capture inflation expectations
```

```

matrix_periods <- rbind(c(1950,2019),
                       c(1980,2019))

list.of.JST.ctries <- levels(as.factor(JSTdataset$iso))
# remove countries with no equity return data:
list.of.JST.ctries <- list.of.JST.ctries[!(list.of.JST.ctries %in% c("CAN","IRL"))]

matrix_corresp_ctries <- rbind(c("AUS","BEL","CAN","CHE","DEU","DNK","ESP",
                                "FIN","FRA","GBR","IRL","ITA","JPN","NLD","NOR",
                                "PRT","SWE","USA"),
                              c("Australia","Belgium","Canada","Switzerland","Germany",
                                "Denmark","Spain","Finland","France","U.K.",
                                "Ireland","Italy","Japan","Netherlands","Norway",
                                "Portugal","Sweden","U.S.A.))

# outputs: Sample-period,
#           Average excess return, sd(return), sd(dc), Cor(dc,er), Cov(dc,er),
#           Sharpe ratio, RRA1, RRA2
matrix_result <- array(NA,c(length(list.of.JST.ctries),9,dim(matrix_periods)[1]))

DATA <- JSTdataset[,c("year","iso","cpi","stir","ltrate","eq_tr",
                     "rconsbarro")]

for(period in 1:dim(matrix_periods)[1]){
  first.year <- matrix_periods[period,1]
  last.year <- matrix_periods[period,2]

  count_ctry <- 0
  for(ctry in list.of.JST.ctries){
    count_ctry <- count_ctry + 1
    DATA.reduced <- subset(DATA,iso==ctry)
    T <- dim(DATA.reduced)[1]
    DATA.reduced$dc <- NA
    DATA.reduced$dc <-
      c(NA,log(DATA.reduced$rconsbarro[2:T]/DATA.reduced$rconsbarro[1:(T-1)]))
    DATA.reduced$eq_tr
    DATA.reduced$infl <-
      c(NA,log(DATA.reduced$cpi[2:T]/DATA.reduced$cpi[1:(T-1)]))
    DATA.reduced$exp_infl <- 0
    for(i in 1:lag_exp_infl){
      DATA.reduced$exp_infl <- DATA.reduced$exp_infl + 1/lag_exp_infl*
        c(rep(NA,i),DATA.reduced$infl[1:(T-i)])
    }
    DATA.reduced$rstir <- DATA.reduced$stir/100 - DATA.reduced$exp_infl
    DATA.reduced$req_tr <- DATA.reduced$eq_tr - DATA.reduced$infl
  }
}

```

```

DATA.reduced <- DATA.reduced[which(DATA.reduced$year==first.year):
                                which(DATA.reduced$year==last.year),]

sample <- range(DATA.reduced$year[complete.cases(DATA.reduced)])

DATA.reduced$Excess_return <- DATA.reduced$req_tr - DATA.reduced$rstir

mean_er <- mean(DATA.reduced$Excess_return,na.rm=TRUE)
sd_er <- sd(DATA.reduced$Excess_return,na.rm=TRUE)
sd_dc <- sd(DATA.reduced$dc,na.rm=TRUE)
cor_er_dc <- cor(DATA.reduced$dc,DATA.reduced$req_tr,use="complete.obs")
cov_er_dc <- cov(DATA.reduced$dc,DATA.reduced$req_tr,use="complete.obs")
SR <- mean_er/sd_er
mean_Rf <- mean(DATA.reduced$rstir,na.rm=TRUE)
RRA1 <- mean_er/(1 + mean_Rf)/cov_er_dc
RRA2 <- mean_er/(1 + mean_Rf)/sd_er/sd_dc

matrix_result[count_ctype,,period] <-
  c(paste(sample[1],"-",sample[2],sep=""),
    round(100*mean_Rf,2),
    round(100*mean_er,2),round(100*sd_er,2),
    round(100*sd_dc,2),
    round(cor_er_dc,2),
    round(SR,2),round(RRA1,1),round(RRA2,1))
  }
}

make_entry <- function(x,format.nb){
  output <- paste("$",sprintf(paste("%. ",format.nb,"f",sep=""),x),"$",sep="")
  return(output)
}

format.nb <- 2

# Build LaTeX table
latex.table <- paste0(
  "Country&Sample&$\\widetilde{R}_f$&$\\overline{er}$&$\\sigma(er)$",
  "&$\\sigma(\\Delta c)$&$\\rho(\\Delta c,er)$&$SR$",
  "&$RRA1$&$RRA2$\\\\\\\"
)
for(period in 1:dim(matrix_periods)[1]){
  latex.table <- rbind(latex.table,
    "\\hline",
    paste("&\\multicolumn{9}{l}{\\textbf{" ,LETTERS[period] ,". Sample

```



```

matrix_periods[period,1],"-",matrix_periods[period,2],"}}\\\\",sep="
"\\hline")
count_ctype <- 0
for(ctype in list.of.JST.ctries){
  count_ctype <- count_ctype + 1
  this.line <- matrix_corresp_ctries[2,which(ctype==matrix_corresp_ctries[1,])]
  for(j in 1:dim(matrix_result)[2]){
    if(j==1){
      this.line <- paste(this.line,"&",
                        matrix_result[count_ctype,j,period],sep="")
    }else{
      num_value <- as.numeric(matrix_result[count_ctype,j,period])
      if((num_value<0)&(j>7)){
        this.line <- paste(this.line,"&$ < 0$",sep="")
      }else{
        this.line <- paste(this.line,"&",make_entry(num_value,format.nb),sep="")
      }
    }
  }
  latex.table <- rbind(latex.table,
                      paste(this.line,"\\\\",sep=""))
}
latex.table <- rbind(latex.table,
                    "\\hline")

latex.file <- paste("tables/table_EquityPuzzle.txt",sep="")
write(latex.table, file = latex.file)

```

The following code illustrates the link between IES and smoothness of the optimal consumption path.

11.3 Smoothing preferences affect consumption and pricing dynamics

This example shows how smoothing preferences affect consumption and pricing dynamics.

Purpose: This example shows how smoothing preferences affect consumption and pricing dynamics

```

par(mfrow=c(1,1))
par(plt=c(.15,.95,.25,.95))

# Set the two-period saving problem.
W <- 1

```

```

delta <- 1
R <- .05

# Compute and plot optimal consumption paths for several IES values.
gamma <- 1.5
C1 <- W * (1 + R) / ((1/delta * 1/(1+R))(-1/gamma) + (1+R))
C2 <- (W - C1) * (1 + R)
par(mgp = c(3, 0.5, 0))

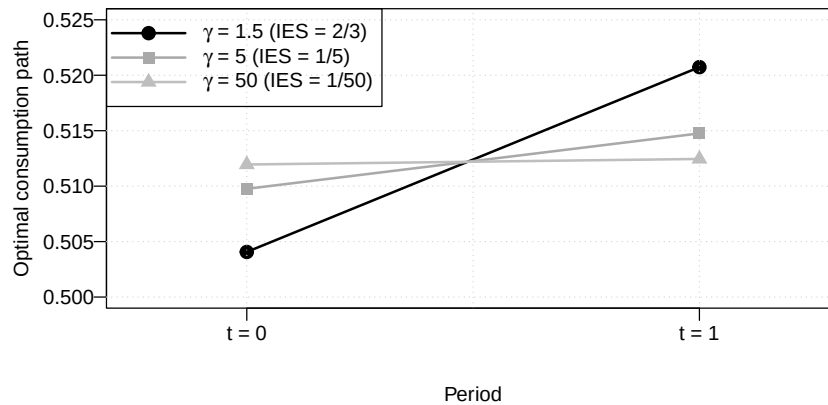
# Draw the baseline path before overlaying alternative IES cases.
plot(c(C1,C2),pch=19,xlab="Period",ylab="Optimal consumption path",ylim=c(.50,.525),
     xaxt="n",xlim=c(.75,2.25),lwd=2,cex=1.3,las=1)
grid()
axis(side=1,at=c(1,2),labels=c("t = 0", "t = 1"))
lines(c(C1,C2),lwd=2)

gamma <- 5
C1 <- W * (1 + R) / ((1/delta * 1/(1+R))(-1/gamma) + (1+R))
C2 <- (W - C1) * (1 + R)
points(c(C1,C2),pch=15,col="darkgrey",cex=1.3)
lines(c(C1,C2),col="darkgrey",lwd=2)

gamma <- 50
C1 <- W * (1 + R) / ((1/delta * 1/(1+R))(-1/gamma) + (1+R))
C2 <- (W - C1) * (1 + R)
points(c(C1,C2),pch=17,col="grey",cex=1.3)
lines(c(C1,C2),col="grey",lwd=2)

legend("topleft",
      c(expression(paste(gamma," = 1.5", " (IES = 2/3)",sep="")),
        expression(paste(gamma," = 5", " (IES = 1/5)",sep="")),
        expression(paste(gamma," = 50", " (IES = 1/50)",sep=""))
      ),
      pt.bg = c("black","darkgrey","grey"),
      lty=c(1,1), # gives the legend appropriate symbols (lines)
      lwd=c(2,2), # line width
      pch=c(19,15,17),
      pt.cex = 1.3,
      col= c("black","darkgrey","grey"), # gives the legend lines the correct color as
      seg.len = 4
    )

```



The following code shows the time-varying conditional correlation between consumption and inflation.

11.4 Empirical correlations between key macro-financial variables

This example reports empirical correlations between key macro-financial variables.

```
# Purpose: This example reports empirical correlations between key macro-financial variables
library(DTAM)
# Build consumption-growth and inflation series at a two-year horizon.
vec_dates <- as.Date(Data_Macro_US_monthly$date)
CONSO <- Data_Macro_US_monthly$RCONS
CPI <- Data_Macro_US_monthly$CPIAUCSL

h <- 24
T <- length(CPI)
conso <- 100*(12/h)*log(CONSO[(h+1):T]/CONSO[1:(T-h)])
inflation <- 100*(12/h)*log(CPI[(h+1):T]/CPI[1:(T-h)])

# Compute rolling correlations.
correla <- NaN*inflation
rwindow <- 12*7
for(t in (rwindow+1):(T-h)){
  correla[t] <- cor(conso[(t-rwindow+1):t],inflation[(t-rwindow+1):t])
}

par(mfrow=c(2,1))
par(plt=c(0.15,0.88,0.2,0.84))
layout(matrix(c(1,2)), heights=c(2, 2))
```

```

# Plot consumption growth and inflation.
plot(vec_dates[(h+1):T], conso, type='l',
      ylim=c(-8,15), col="black", lty=1, lwd=2, cex.axis=.8, cex=.8, las=1,
      ylab="in percent", xlab="", main="(a) Inflation and consumption growth", cex.main=.9)
grid()
abline(h=0, col="black", lty=3)
lines(vec_dates[(h+1):T], inflation, col="darkgrey", lwd=2)

legend("bottomleft",
      c("Consumption growth", "Inflation rate"),
      lty=c(1,1), # gives the legend appropriate symbols (lines)
      lwd=c(2,2), # line width
      col=c("black", "darkgrey"), # gives the legend lines the correct color and width
      cex=.8 # size of the text
)

# Plot the rolling correlation together with term-premium estimates.
plot(vec_dates[(h+1):T], 100*correla, col="black", type="l",
      lwd=3, ylim=c(-130,130), lty=2,
      ylab="Correlation, in percent", xlab="", cex.axis=.8, cex=.8, las=1,
      main="(b) Correlation between consumption growth and inflation", cex.main=.9)
grid()
abline(h=0, col="black", lty=3)
text(vec_dates[330], -15, labels="Supply shocks prevail", cex=.9)
text(vec_dates[330], +15, labels="Demand shocks prevail", cex=.9)

values.TP <- seq(-450,450,by=150)
location.ticks <- -values.TP/4.5
axis(4, at=-location.ticks,
      labels = paste(-values.TP, "", sep=""),
      col="darkgrey", col.axis="darkgrey", las=1, cex.axis=.9)
mtext("Term premium, in basis points", side=4, line=3, col="darkgrey")

lines(ACMTermPremium$date, -100/4.5*ACMTermPremium$ACMTP10,
      col="darkgrey", lwd=2)
lines(Data_Macro_US_monthly$date,
      -100/4.5*Data_Macro_US_monthly$THREEFYTP10, col="black", lwd=2, lty=1)

legend("topleft",
      title="Term premiums (rhs)",
      c("ACM", "KW"),
      lty=c(1,1), # gives the legend appropriate symbols (lines)
      lwd=c(2,2), # line width
      seg.len = 3,
      col=c("darkgrey", "black"), # gives the legend lines the correct color and width
      cex=.8 # size of the text
)

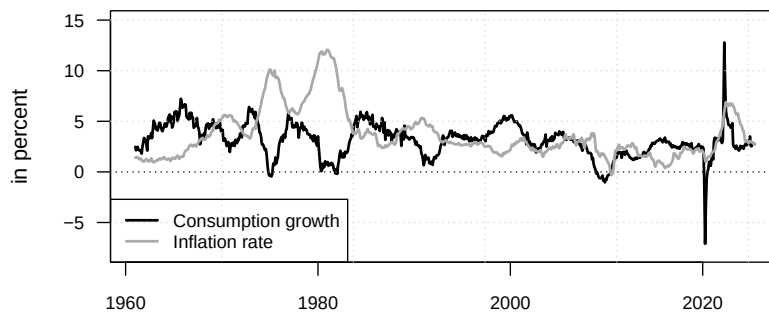
```

```

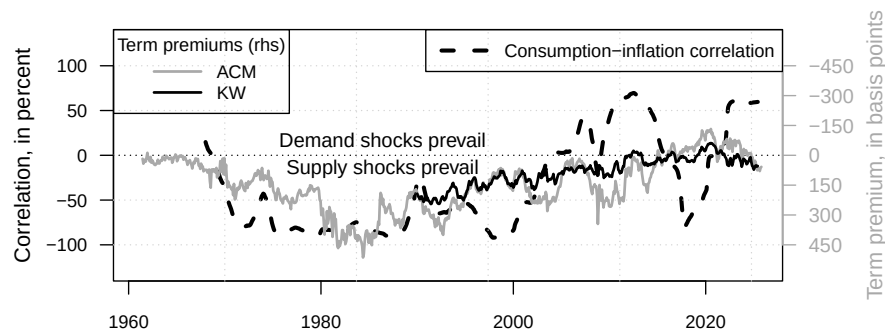
)
legend("topright",
      c("Consumption-inflation correlation"),
      lty=c(2), # gives the legend appropriate symbols (lines)
      lwd=c(3), # line width
      seg.len = 3,
      col=c("black"), # gives the legend lines the correct color and width
      cex=.8 # size of the text
)

```

(a) Inflation and consumption growth



(b) Correlation between consumption growth and inflation



The following code creates a figure that displays a simulated path for the short-term rate, and shows the term structure of interest rates at different point in times.

11.5 Term structures under a CCAPM-based pricing kernel

This example simulates term structures implied by a CCAPM-based pricing kernel.

```

# Purpose: This example simulates term structures implied by a CCAPM-based pricing kernel
library(DTAM)
set.seed(123)

phi <- 0.8
mu <- 0.005
sigma.bar <- .01 # unconditional std dev of growth
sigma2 <- sigma.bar^2 * (1 - phi^2)
sigma.c <- sqrt(sigma2)
gamma <- 10
delta <- 0.99
# Determine specification of real short-term rate:
eta.0 <- -log(delta) + mu * gamma * (1 - phi) -
  gamma^2 * sigma.c^2 / 2
eta.1 <- gamma * phi
# Specify the model:
psi.parameterization <- list(
  mu = matrix(mu * (1-phi)),
  Phi = matrix(phi),
  Sigma = matrix(sigma.c^2))
# Use reverse-order multi-horizon LT to price ZC bonds:
h = 10 # maximum maturity
u2 <- matrix(-gamma)
u1 <- matrix(-gamma)
AB <- reverse_MHLT(psi_GaussianVAR,u1,u2,H=h,psi.parameterization)
a.h <- -c(AB$A)/(1:h)
b.h <- -log(delta) + -c(AB$B)/(1:h)
# Under EH:
u2 <- matrix(- eta.1)
u1 <- matrix(0)
AB <- reverse_MHLT(psi_GaussianVAR,u1,u2,H=h,psi.parameterization)
a.h.EH <- -c(AB$A - eta.1)/(1:h)
b.h.EH <- eta.0 - c(AB$B)/(1:h)
# Simulate growth:
model <- list(mu=mu*(1-phi),
              Phi=as.matrix(phi),
              Sigma12=as.matrix(sigma.c))
T <- 100
x.sim <- simul_GaussianVAR(model,nb.sim=T)
r.t <- (eta.0 + eta.1 * x.sim)

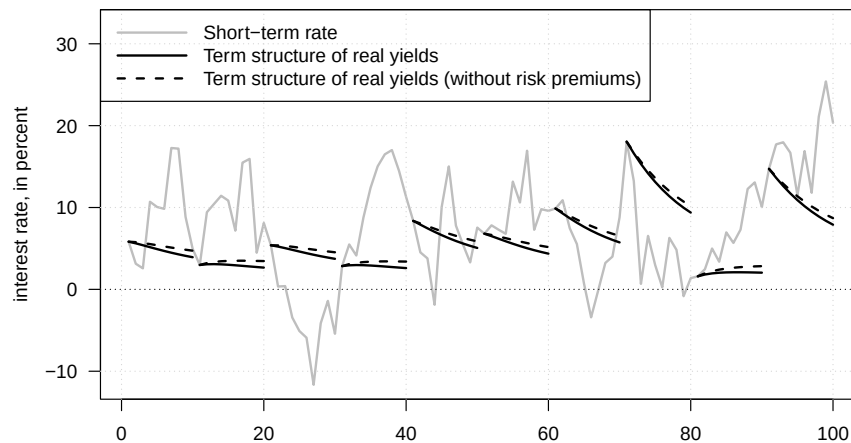
# Prepare plots:
par(mfrow=c(1,1))
par(plt=c(.15,.98,.1,.95))

```

```

plot(100*r.t,type="l",xlab="",ylab="interest rate, in percent",
     ylim=100*c(min(r.t),
                 max(r.t)+.07),las=1,
     lwd=2,col="grey")
grid()
abline(h=0,lty=3)
for(t in seq(1,T,by=10)){
  lines(t+(1:h)-1,100*(b.h      + a.h      * x.sim[t]),col="black",lwd=2)
  lines(t+(1:h)-1,100*(b.h.EH + a.h.EH * x.sim[t]),col="black",lwd=2,lty=2)}
legend("topleft",
      c("Short-term rate", "Term structure of real yields",
        "Term structure of real yields (without risk premiums)",
        lwd=c(2),lty=c(1,1,2),col=c("grey", "black", "black"),seg.len = 4)

```



The following code produces a figure illustrating the real term premium.

11.6 Decomposes the real term premium in the baseline CCAPM setting

This example decomposes the real term premium in the baseline CCAPM setting.

Purpose: This example decomposes the real term premium in the baseline CCAPM setting
`library(DTAM)`

```

# Load and merge nominal, real, and survey data.
data("SPF", package = "DTAM")
data("Data_Macro_US_quarterly", package = "DTAM")
data("YC_LW", package = "DTAM")

```

```

data <- merge(YC_LW,SPF,by="date")
data <- merge(data,DAT_GSW,by="date")

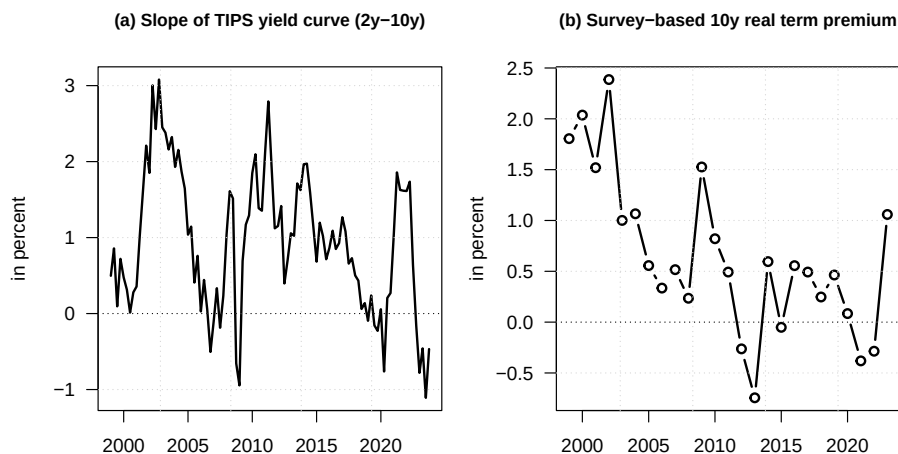
par(mfrow=c(1,2))
par(plt=c(.2,.95,.1,.85))

# Plot the slope of the real yield curve.
data_reduced <- subset(data,select=c(date,TIPSY10,TIPSY02))
data_reduced <- data_reduced[complete.cases(data_reduced),]
plot(data_reduced$date,data_reduced$TIPSY10 - data_reduced$TIPSY02,type="l",
      lwd=2,las=1,ylab="in percent",xlab="",
      main="(a) Slope of TIPS yield curve (2y-10y)",cex.main=.9)
grid()
abline(h=0,lty=3)

# Computation of model-free real term premium:
data_reduced <- subset(data,select=c(date,TIPSY10,yld_10y,CPI10,BILL10))
data_reduced <- data_reduced[complete.cases(data_reduced),]

BEIR10 <- data_reduced$yld_10y - data_reduced$TIPSY10
IRP10 <- BEIR10 - data_reduced$CPI10
NTP10 <- data_reduced$yld_10y - data_reduced$BILL10
RTP10 <- NTP10 - IRP10
plot(data_reduced$date,RTP10,type="b",
      lwd=2,las=1,ylab="in percent",xlab="",
      main="(b) Survey-based 10y real term premium",cex.main=.9)
grid()
abline(h=0,lty=3)

```



The following code produces a figure showing average real and inflation yield

curves.

11.7 Model-implied yield-curve moments with CCAPM restrictions

This example compares model-implied yield-curve moments with CCAPM restrictions.

Purpose: This example compares model-implied yield-curve moments with CCAPM restrictions
`library(DTAM)`

```
values_sigma.pi.c <- c(0,-.03)
title_plot <- c(
  expression(paste("(a) ",sigma[pi*','*c]," = 0",sep="")),
  expression(paste("(a) ",sigma[pi*','*c]," = -0.03",sep=""))
)

par(plt=c(.2,.95,.25,.85))
par(mfrow=c(1,2))

pi_bar <- .02
phi_pi <- .8
sigma.pi <- .01

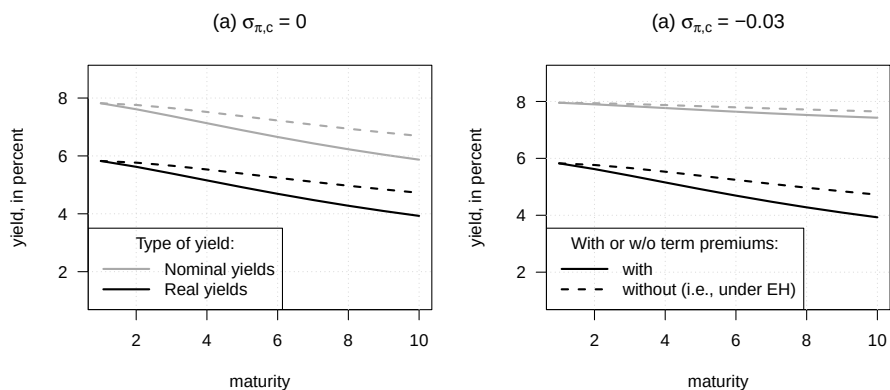
count_plot <- 0
for(sigma.pi.c in values_sigma.pi.c){
  count_plot <- count_plot + 1
  # Determine specification of nominal short-term rate:
  eta.0.i <- -log(delta) + mu * gamma * (1 - phi) + pi_bar *
    (1 - phi_pi) - (gamma*sigma.c + sigma.pi.c)^2/2 - sigma.pi^2/2
  eta.1.i <- matrix(c(gamma * phi,phi_pi),2,1)
  # Specify the model:
  psi.parameterization <- list(
    mu = matrix(c(mu*(1-phi),pi_bar*(1-phi_pi)),2,1),
    Phi = diag(c(phi,phi_pi)),
    Sigma = matrix(c(sigma.c^2,sigma.c*sigma.pi.c,sigma.c*
      sigma.pi.c,sigma.pi^2+sigma.pi.c^2),2,2))
  # Use reverse-order multi-horizon LT to price ZC bonds:
  u2 <- matrix(c(-gamma,-1),2,1);u1 <- matrix(c(-gamma,-1),2,1)
  AB <- reverse_MHLT(psi_GaussianVAR,u1,u2,H=h,psi.parameterization)
  a.h.i <- -matrix(AB$A,2,h)/t(matrix((1:h),h,2))
  b.h.i <- -log(delta) + -c(AB$B)/(1:h)
  # Under EH:
  u2 <- matrix(- eta.1.i);u1 <- matrix(0,2,1)
  AB <- reverse_MHLT(psi_GaussianVAR,u1,u2,H=h,psi.parameterization)
```

```

a.h.i.EH <- -(matrix(AB$A,2,h)-
               matrix(eta.1.i,2,h))/t(matrix((1:h),h,2))
b.h.i.EH <- eta.0.i - c(AB$B)/(1:h)
# Average yield curves:
avg.i.h <- b.h.i + matrix(c(mu,pi_bar),1,2) %*% a.h.i
avg.i.h.EH <- b.h.i.EH + matrix(c(mu,pi_bar),1,2) %*% a.h.i.EH

plot(100*c(avg.i.h),type="l",lwd=2,col="darkgrey",lty=1,
     ylim=100*c(.01,max(avg.i.h)+.01),
     main=title_plot[count_plot],
     xlab="maturity",ylab="yield, in percent",las=1);grid()
lines(100*c(avg.i.h.EH),type="l",lwd=2,col="darkgrey",lty=2)
lines(100*(b.h+a.h*mu),lwd=2,col="black")
lines(100*(b.h.EH+a.h.EH*mu),lwd=2,col="black",lty=2)
if(count_plot==1){
  legend("bottomleft",title="Type of yield:",
        c("Nominal yields",
          "Real yields"),
        lwd=c(2),lty=c(1,1),col=c("darkgrey","black"),seg.len = 3)
}
if(count_plot==2){
  legend("bottomleft",title="With or w/o term premiums:",
        c("with",
          "without (i.e., under EH)"),
        lwd=c(2),lty=c(1,2),col=c("black"),seg.len = 3)
}
}

```



11.8 Separates trend and cycle contributions to the real term premium

This example separates trend and cycle contributions to the real term premium.

Purpose: This example separates trend and cycle contributions to the real term premium

```
mu_g    <- .02
rho_g    <- .9
rho_z    <- .8
sigma_g  <- .002
sigma_z  <- .02
delta    <- 1
gamma    <- 2

h <- 8

# Specify the model:
psi.parameterization <- list(
  mu = matrix(c(mu_g*(1-rho_g),0,0),3,1),
  Phi = matrix(c(rho_g,0,0,0,rho_z,1,0,0,0),3,3),
  Sigma = diag(c(sigma_g^2,sigma_z^2,0)))

all_sigma_g <- c(0,.005,.01)
all_lty     <- c(1,2,3)
all_col     <- c("black","black","black")
names_legend <- c(expression(paste(sigma[g], ' = ',0,sep="")),
  expression(paste(sigma[g], ' = ',0.005,sep="")),
  expression(paste(sigma[g], ' = ',0.01,sep="")))

par(plt=c(.15,.95,.25,.95))
for(i in 1:length(all_sigma_g)){

  psi.parameterization$Sigma[1,1] <- all_sigma_g[i]^2

  # Use reverse-order multi-horizon LT to price ZC bonds:
  u1 <- matrix(c(-gamma,-gamma,gamma),3,1)
  AB <- reverse_MHLT(psi_GaussianVAR,u1,H=h,
    psi.parameterization=psi.parameterization)
  a.h.r <- -matrix(AB$A,3,h)/t(matrix((1:h),h,3))
  b.h.r <- -log(delta) + -c(AB$B)/(1:h)
  # Average yield curves:
  yds <- 100 * (b.h.r + matrix(c(mu_g,0,0),1,3) %*% a.h.r)

  if(i==1){
    plot(1:h,yds,type="l",col=all_col[i],lty=all_lty[i],lwd=2,
```

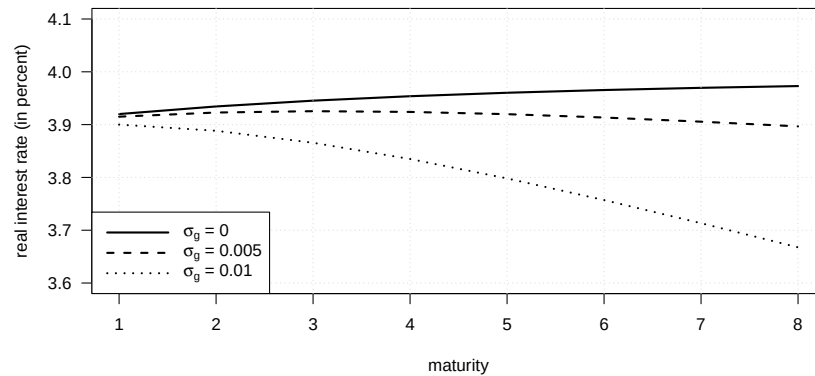
```

        xlab="maturity",ylab="real interest rate (in percent)",las=1,
        ylim=c(3.6,4.1))
    grid()
  }else{
    lines(1:h,yds,col=all_col[i],lty=all_lty[i],lwd=2)
  }
}

abline(h=0,col="grey",lty=3)

legend("bottomleft",
      names_legend,
      lty=all_lty, # gives the legend appropriate symbols (lines)
      lwd=2, # line width
      col=all_col, # gives the legend lines the correct color and width
      bg="white",
      seg.len = 4
)

```



Chapter 12

Recursive preferences

12.1 External habits amplify variation in marginal utility

This example illustrates how external habits amplify variation in marginal utility.

```
# Purpose: This example illustrates how external habits amplify variation in marginal utility

# Set the habit-preference parameters.
delta <- 1
lambda <- .49
rho <- .5
gamma <- 3

r.f <- c(rep(0.07,5),c(-.2,-.2,-.2))

# Simulate scenario A.
T <- length(r.f)
C <- 1
C.t <- C
X <- lambda * C /(1-rho)
X.t <- X
U <- delta * (C.t - X.t)^(1 - gamma)/(1 - gamma)
U.no.habit <- delta * (C.t)^(1 - gamma)/(1 - gamma)
for(t in 1:(T)){
  C.t <- (1/(1+r.f[t]))^(-1/gamma) * (C.t - X.t) + rho * X.t + lambda * C.t
  C <- c(C,C.t)
  X.t <- rho * X.t + lambda * C.t
  X <- c(X,X.t)
```

```

    U <- U + delta * (C.t - X.t)^(1 - gamma)/(1 - gamma)
    U.no.habit <- U.no.habit + delta * (C.t)^(1 - gamma)/(1 - gamma)
  }
  U1 <- U
  U1.no.habit <- U.no.habit
  C1 <- C
  X1 <- X
  R.f.1 <- r.f

  # Simulate scenario B.
  r.f <- c(rep(0.025,8))

  T <- length(r.f)
  C <- 1
  C.t <- C
  X <- lambda * C / (1-rho)
  X.t <- X
  U <- delta * (C.t - X.t)^(1 - gamma)/(1 - gamma)
  U.no.habit <- delta * (C.t)^(1 - gamma)/(1 - gamma)
  for(t in 1:(T)){
    C.t <- (1/(1+r.f[t]))^(-1/gamma) * (C.t - X.t) + rho * X.t + lambda * C.t
    C <- c(C,C.t)
    X.t <- rho * X.t + lambda * C.t
    X <- c(X,X.t)
    U <- U + delta * (C.t - X.t)^(1 - gamma)/(1 - gamma)
    U.no.habit <- U.no.habit + delta * (C.t)^(1 - gamma)/(1 - gamma)
  }
  U2 <- U
  U2.no.habit <- U.no.habit
  C2 <- C
  X2 <- X
  R.f.2 <- r.f

  # Plot interest rates, consumption, and surplus consumption.
  par(mfrow=c(1,3))
  par(plt=c(.2,.9,.2,.85))

  plot(R.f.1,col="black",lwd=2,pch=19,ylim=c(-.25,.2),las=1,
       main=expression(paste("(a) Short-term interest rate ",
                             tilde(R)[f*,"*t", sep = "")),
       xlab="time",ylab="",type="b")
  grid()
  lines(R.f.2,col="darkgrey",pch=15,cex=1.2,type="b",lwd=2)

  legend("topright",

```

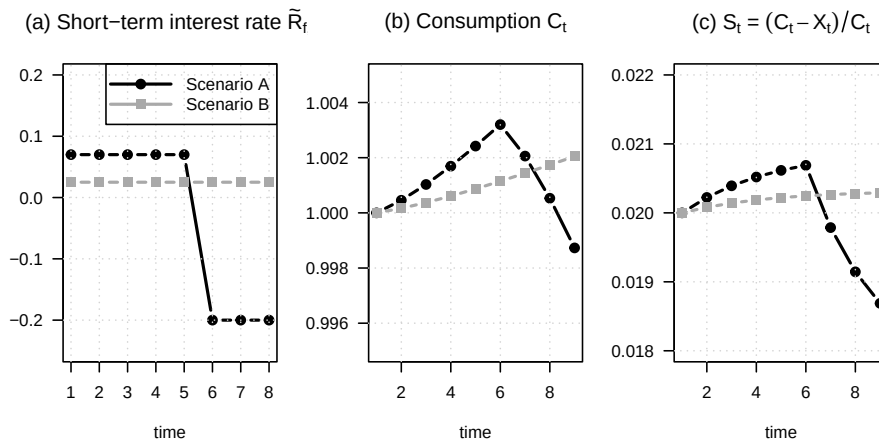
```

c("Scenario A","Scenario B"),
lty=c(1,1),
lwd=c(2,2),
pch=c(19,15),
pt.cex=c(1,1.2),
col=c("black","darkgrey"),
seg.len = 4
)

plot(C1,col="black",lwd=2,pch=19,ylim=c(.995,1.005),las=1,
     main=expression(paste("(b) Consumption ",C[t],sep="")),xlab="time",ylab="",
     type="b")
grid()
lines(C2,col="darkgrey",pch=15,cex=1.2,lwd=2,type="b")

plot((C1-X1)/C1,col="black",lwd=2,pch=19,ylim=c(0.018,0.022),las=1,
     main=expression(paste("(c) ",S[t], " = ", (C[t]-X[t])/C[t],sep="")),xlab="time",ylab="",type="b")
grid()
lines((C2-X2)/C2,col="darkgrey",pch=15,cex=1.2,lwd=2,type="b")

```



12.2 Exponentially smoothed consumption growth enters recursive preferences

This example shows how exponentially smoothed consumption growth enters recursive preferences.

```

# Purpose: This example shows how exponentially smoothed consumption growth enters recursive
# preferences
# Simulate a consumption-growth path.

```

```

mu      <- .015
sigma   <- .02
nb_sim  <- 300
p       <- .00
sigma_crisis <- .0
eta_crisis <- .1
dc <- mu + sigma * rnorm(nb_sim) +
      (runif(nb_sim)<p)*(- eta_crisis + sigma_crisis*rnorm(nb_sim))
c <- cumsum(dc)
C <- exp(c)

# Specify surplus-consumption smoothing parameters.
all_rho   <- c(.6,.97)
all_alpha <- c(1,1)
all_alpha_tilde <- c(.2,.2)

start.date <- 100
end.date   <- nb_sim

par(mfrow=c(2,1))
par(plt=c(.15,.95,.15,.83))
plot(C[start.date:end.date],type="l",lwd=2,
      ylim=c(1 - max(all_alpha_tilde),max(C)),
      main=expression(paste("(a) Consumption, ",C[t],sep="")),
      las=1,xlab="time",ylab="",col="darkgrey")
grid()

# Construct habit-adjusted consumption for each smoothing parameter.
all_S <- NULL
for(i in 1:length(all_rho)){
  lambda_t <- mu
  lambda   <- lambda_t
  for(t in 2:nb_sim){
    lambda_t <- all_rho[i] * lambda_t + (1 - all_rho[i]) * c[t-1]
    lambda   <- c(lambda,lambda_t)
  }
  S <- all_alpha_tilde[i] * exp(all_alpha[i] * (c - lambda))
  all_S <- cbind(all_S,S)
  X <- C * (1 - S)
  lines(X[start.date:end.date],lwd=2,lty=i)
}
legend("topleft",
      c(expression(C[t]),
        expression(paste(X[t], " for ",rho," = 0.60",sep="")),
        expression(paste(X[t], " for ",rho," = 0.97",sep=""))),

```



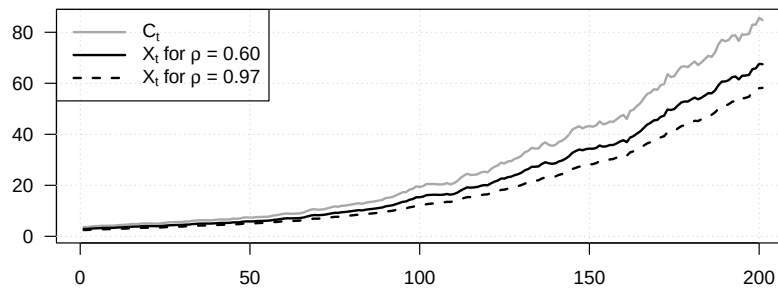
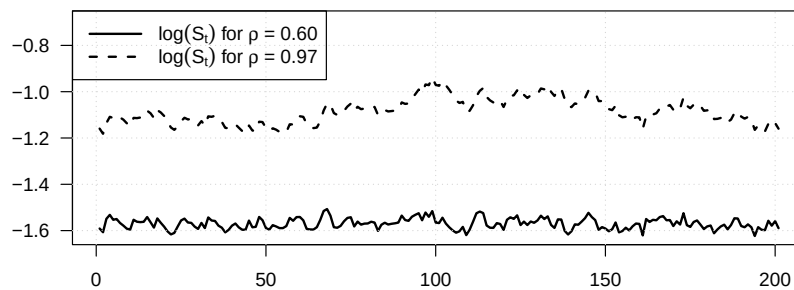
```

    lty=c(1,1:length(all_rho)),
    lwd=2, # line width
    col=c("darkgrey",rep("black",length(all_rho))),
    seg.len = 3
)

# Plot the surplus-consumption ratio.
plot(log(all_S[start.date:end.date,1]),type="l",lwd=2,
     main=expression(paste("(b) Consumption surplus, ",S[t],sep="")),
     las=1,xlab="time",ylab="",ylim=c(min(log(all_S[start.date:end.date,])),
                                     .25+max(log(all_S[start.date:end.date,])))),
     grid()
lines(log(all_S[start.date:end.date,2]),lwd=2,lty=2)

legend("topleft",
      c(expression(paste(log(S[t])," for ",rho," = 0.60",sep="")),
        expression(paste(log(S[t])," for ",rho," = 0.97",sep="))),
      lty=c(1:length(all_rho)),
      lwd=2, # line width
      col=c(rep("black",length(all_rho))),
      seg.len = 3
)

```

(a) Consumption, C_t (b) Consumption surplus, S_t 

```
# Compute state-space objects associated with the smoothing specification.
for(i in 1:length(all_rho)){
  rho   <- all_rho[i]
  alpha <- all_alpha[i]
  Phi   <- matrix(c(0,1-rho,0,rho),2,2)
  Sigma <- matrix(c(sigma,0),2,1)
  V     <- matrix(solve(diag(4)-Phi %x% Phi)%*%c(Sigma%*%t(Sigma)),2,2)

  vec_gamma_dc          <- matrix(c(-gamma,0),2,1)
  vec_gamma_dc_dlambda <- matrix(c(-gamma*(1+alpha),gamma*alpha),2,1)
}
```

12.3 The fixed-point resolution behind Epstein-Zin preferences

This example illustrates the fixed-point resolution behind Epstein-Zin preferences.

```
# Purpose: This example illustrates the fixed-point resolution behind Epstein-Zin preferences.

# Compare the fixed-point implication across elasticity parameters.
par(mfrow=c(1,3))
par(plt=c(.2,.95,.2,.8))

for(rho in c(.5,.9,1.5)){

  all.gamma.values <- seq(.1001,6,by=.1)
  all.C.minus.D.values <- NULL

  # Compute the utility difference over risk-aversion values.
  for(gamma in all.gamma.values){
    c.h <- 1.2
    c.l <- .8
    beta <- .5

    U.2.h <- (1 - beta)^(1/(1-rho)) * c.h
    U.2.l <- (1 - beta)^(1/(1-rho)) * c.l

    U.1.C.hh <- ((1-beta)*c.h^(1-rho) + beta * U.2.h^(1-rho))^(1/(1-rho))
    U.1.C.hl <- ((1-beta)*c.h^(1-rho) + beta * U.2.l^(1-rho))^(1/(1-rho))
    U.1.C.lh <- ((1-beta)*c.l^(1-rho) + beta * U.2.h^(1-rho))^(1/(1-rho))
    U.1.C.ll <- ((1-beta)*c.l^(1-rho) + beta * U.2.l^(1-rho))^(1/(1-rho))

    U.0.C <- (1 - beta)^(1/(1-rho)) * (.25 * U.1.C.hh^(1-gamma) +.25 * U.1.C.hl^(1-gamma) +.25 * U.1.C.lh^(1-gamma) +.25 * U.1.C.ll^(1-gamma))
  }
}
```

```

                                .25 * U.1.C.lh^(1-gamma) +.25 * U.1.C.ll^(1-gamma)
)^((1/(1-gamma)))

E.U2gamma <- .5 * U.2.h^(1 - gamma) + .5 * U.2.l^(1 - gamma)

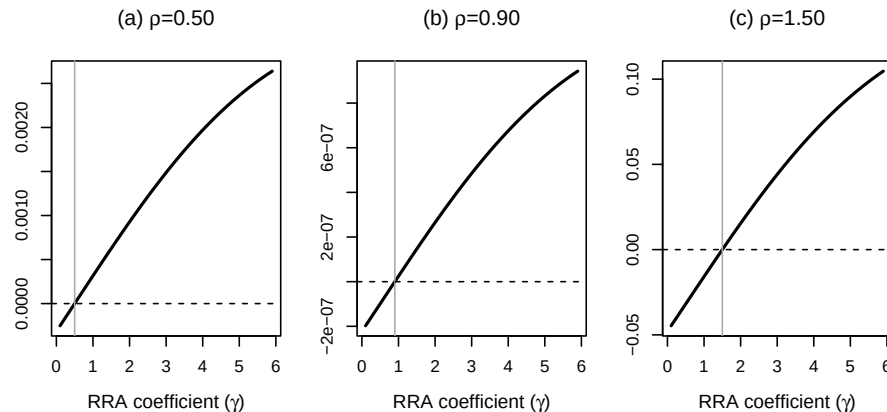
U.1.D.h <- ((1-beta)*c.h^(1-rho) + beta * E.U2gamma^((1-rho)/(1-gamma)))^(1/(1-rho))
U.1.D.l <- ((1-beta)*c.l^(1-rho) + beta * E.U2gamma^((1-rho)/(1-gamma)))^(1/(1-rho))

U.0.D <- (1 - beta)^(1/(1-rho)) * (.5 * U.1.D.h^(1-gamma) +.5 * U.1.D.l^(1-gamma))^(1/(1-gamma))

all.C.minus.D.values <- c(all.C.minus.D.values,U.0.C-U.0.D)
}

# Plot each panel for one value of rho.
if(rho==.5){
  plot(all.gamma.values,all.C.minus.D.values,type="l",lwd=2,las=0,
        xlab=expression(paste("RRA coefficient (",gamma,")",sep="")),ylab="",
        main=expression(paste("(a) ",rho,"=0.50",sep="")),cex.lab=1.1
  )
}
if(rho==.9){
  plot(all.gamma.values,all.C.minus.D.values,type="l",lwd=2,las=0,
        xlab=expression(paste("RRA coefficient (",gamma,")",sep="")),ylab="",
        main=expression(paste("(b) ",rho,"=0.90",sep="")),cex.lab=1.1
  )
}
if(rho==1.5){
  plot(all.gamma.values,all.C.minus.D.values,type="l",lwd=2,las=0,
        xlab=expression(paste("RRA coefficient (",gamma,")",sep="")),ylab="",
        main=expression(paste("(c) ",rho,"=1.50",sep="")),cex.lab=1.1
  )
}
abline(v=rho,col="darkgrey")
abline(h=0,col="black",lty=2)
}

```



12.4 Epstein-Zin pricing implications vary with preference parameters

This example shows how Epstein-Zin pricing implications vary with preference parameters.

Purpose: This example shows how Epstein-Zin pricing implications vary with preference parameters

```

beta <- .9
gamma <- seq(0.5001,10,by=.1)

par(mfrow=c(1,2))
par(plt=c(.15,.9,.25,.85))

omega <- .05
E.m.0 <- - log(0.5 * exp(beta*(1-gamma)*omega) + 0.5 * exp(-beta*(1-gamma)*omega))
P <- .5 * exp(E.m.0 - beta*(1-gamma)*omega)
P.check <- .5 * exp(E.m.0 + beta*(1-gamma)*omega) +
  .5 * exp(E.m.0 - beta*(1-gamma)*omega)
# Should be equal to 1.
plot(gamma,P,type="l",lwd=2,cex.main=1.1,las=1,
     xlab=expression(paste("RRA coefficient (",gamma,")",sep="")),ylab="",
     main=expression(paste("(a) ",omega," = 0.05",sep="")))
abline(v=1,col="darkgrey")
abline(h=0.5,col="black",lty=2)

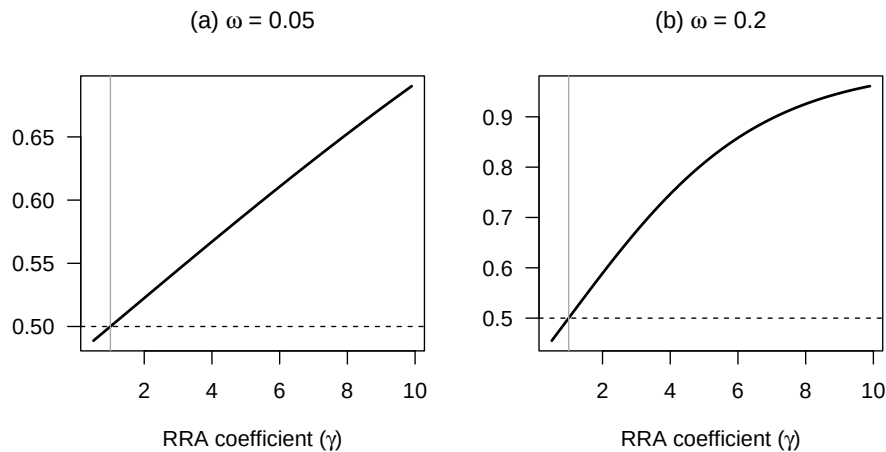
omega <- .2
E.m.0 <- - log(0.5 * exp(beta*(1-gamma)*omega) + 0.5 * exp(-beta*(1-gamma)*omega))

```

```

P <- .5 * exp(E.m.0 - beta*(1-gamma)*omega)
P.check <- .5 * exp(E.m.0 + beta*(1-gamma)*omega) +
.5 * exp(E.m.0 - beta*(1-gamma)*omega)
# Should be equal to 1.
plot(gamma,P,type="l",lwd=2,cex.main=1.1,las=1,
      xlab=expression(paste("RRA coefficient (",gamma,")",sep="")),ylab="",
      main=expression(paste("(b) ",omega," = 0.2",sep="")))
abline(v=1,col="darkgrey")
abline(h=0.5,col="black",lty=2)

```



The following code produces a figure that shows the maximum Sharpe ratios that can be achieved with affine Epstein-Zin models.

12.5 The Bansal-Yaron long-run-risk environment and its state variables

This example introduces the Bansal-Yaron long-run-risk environment and its state variables.

```

# Purpose: This example introduces the Bansal-Yaron long-run-risk environment and its state
# variables
library(DTAM)

# -----
# Bansal and Yaron (2004): Homoskedastic case
# (using psi_GaussianVAR() for Laplace transform)
# -----

# Parameterization from Bansal and Yaron (2004):
EIS <- 1.5

```

```

gamma <- 10
delta <- .998
mu <- .0015
mu_d <- .0015
phi <- .979
beta <- 3
sigma <- 0.0078
varphi_e <- .044
varphi_d <- 4.5

model <- list(
  mu = matrix(c(0,mu,mu_d),3,1),
  Phi = matrix(c(phi,1,beta,rep(0,6)),3,3),
  Sigma = diag(sigma^2*c(varphi_e,1,varphi_d)^2),
  rho = 1/EIS,
  phi = phi,
  gamma = gamma,
  delta = delta,
  sigma = sigma,
  mu_c0 = 0,
  mu_c1 = c(0,1,0),
  n_w = 3
)

model_solved <- solve_EZ_SDF(model,psi_GaussianVAR,z_bar_ini=5,
                             nb_loop_z_bar = 20,nb_loop_mu_z1 = 20)
# Check formula (compare with analytical BY formula):
A1 <- (1 - 1/EIS)/(1 - model_solved$kap1*model$phi)

all_EIS <- c(.5,.9999,1.5,NaN)
all_col <- c("black","black","black","darkgrey")
all_lty <- c(1,2,3,1)
all_gamma <- c(seq(.50001,3,length.out=10),4,6,8,10)

all_maxSR <- matrix(0,length(all_EIS),length(all_gamma))

count_EIS <- 0
for(EIS in all_EIS){
  count_EIS <- count_EIS + 1
  count_gamma <- 0
  model$rho <- 1/EIS
  for(gamma in all_gamma){
    count_gamma <- count_gamma + 1
    model$gamma <- gamma
    if(is.na(EIS)){

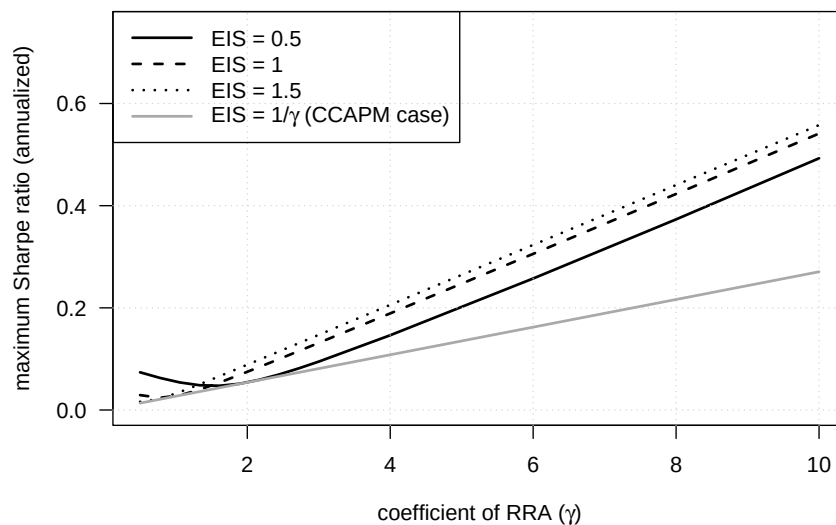
```

```

    model$rho <- model$gamma
  }
  res <- solve_EZ_SDF(model,psi_GaussianVAR,z_bar_ini=5,
                      nb_loop_z_bar = 20,nb_loop_mu_z1 = 20)
  psi_w_2alpha <- psi_GaussianVAR(2*res$alpha,model)
  psi_w_alpha <- psi_GaussianVAR(res$alpha,model)
  w <- res$Ew
  maxSR <- sqrt(exp(t(psi_w_2alpha$a - 2*psi_w_alpha$a)%%w+
                    psi_w_2alpha$b - 2*psi_w_alpha$b)-1)
  all_maxSR[count_EIS,count_gamma] <- maxSR*sqrt(12)
}
}

par(plt=c(.15,.95,.2,.99))
plot(all_gamma,(all_maxSR[1,]),ylim=c(0,.75),type="l",
     lwd=2,las=1,col=all_col[1],lty=all_lty[1],
     xlab=expression(paste("coefficient of RRA (" ,gamma,")",sep="")),
     ylab="maximum Sharpe ratio (annualized)")
grid()
for(i in 2:length(all_EIS)){
  lines(all_gamma,(all_maxSR[i,]),lwd=2,
        col=all_col[i],lty=all_lty[i])
}
legend("topleft",
      c(expression(paste("EIS = 0.5",sep="")),
        expression(paste("EIS = 1",sep="")),
        expression(paste("EIS = 1.5",sep="")),
        expression(paste("EIS = 1/",gamma," (CCAPM case)",sep="))),
      lwd=2,
      lty=all_lty,
      col=all_col,
      seg.len = 3
)

```



```
# for(EIS in all_EIS){
#   abline(v=1/EIS,lty=3)
# }
```

Bansal and Yaron (2004) as a special case of Bansal and Shaliastovich (2012):

12.6 Yields, returns, and premia in the Bansal-Yaron model

This example studies yields, returns, and premia in the Bansal-Yaron model.

```
# Purpose: This example studies yields, returns, and premia in the Bansal-Yaron model
library(DTAM)

# Parameterization from Bansal and Yaron (2004):
EIS <- 1.5
gamma <- 10
delta <- .998
mu <- .0015
mu_d <- .0015
phi <- .979
beta <- 3
sigma <- 0.0078
varphi_e <- .044
varphi_d <- 4.5
nu1 <- .987
sigma_w <- .23*10^(-5)
# sigma_w <- 0
```


12.6. YIELDS, RETURNS, AND PREMIA IN THE BANSAL-YARON MODEL 233

```

freq <- 12 # monthly frequency

# Definition of theta, using Bansal and Yaron notations:
theta <- (1 - gamma)/(1 - 1/EIS)

Phi <- matrix(0,3,3)
Phi[1,1] <- phi
Phi[2,1] <- 1
Phi[3,1] <- beta

model <- list(
  mu_c0 = mu,
  mu_c1 = matrix(c(0,1,0, # x weightings (x,x_1,eta,u)
                    0), # z weightings (sigma_t)
                  ncol=1),
  mu_d0 = mu,
  mu_d1 = matrix(c(0,0,1, # x weightings (x,x_1,eta,u)
                    0), # z weightings (sigma_t)
                  ncol=1),
  Pi     = Phi,
  # unconditional means of x's conditional variances:
  chi.0  = c(varphi_e^2*sigma^2,sigma^2,varphi_d^2*sigma^2),
  # loadings of x's conditional variances on z:
  chi.1  = matrix(c(varphi_e^2,1,varphi_d^2),nrow=1),
  nu     = as.matrix(nu1),
  sigma.w = c(sigma_w),
  gamma  = gamma,
  rho    = 1/EIS,
  delta  = delta,
  n_w    = 3+1)

model_solved <- solve_EZ_SDF(model,psi_BansalShaliastovich,z_bar_ini=5,
                             nb_loop_z_bar = 10,nb_loop_mu_z1 = 400)

# Check with formulas reported in Bansal and Yaron:
# A1 <- (1 - 1/EIS)/(1 - model_solved$kappa1*phi)
# A2 <- .5*((theta - theta/EIS)^2 + (theta*model_solved$mu_z1[1]*model_solved$kappa1*varphi_e)^2)
# (theta*(1 - model_solved$kappa1*nu1))
# model_solved$mu_z1
# c(A1,A2)

all_EIS <- c(.5,.9999,1.5,NaN)
all_col <- c("black","black","black","darkgrey")
all_lty <- c(1,2,3,1)
all_gamma <- c(seq(.50001,4,length.out=20),6,8,10)

```

```

# all_gamma <- seq(.80001,4,length.out=40)

all_maxSR      <- matrix(0,length(all_EIS),length(all_gamma))
all_Erf        <- matrix(0,length(all_EIS),length(all_gamma))
all_Ers        <- matrix(0,length(all_EIS),length(all_gamma))
all_std_dev_pd <- matrix(0,length(all_EIS),length(all_gamma))

count_EIS <- 0
for(EIS in all_EIS){
  count_EIS <- count_EIS + 1
  count_gamma <- 0
  model$rho <- 1/EIS
  for(gamma in all_gamma){
    count_gamma <- count_gamma + 1
    model$gamma <- gamma
    if(is.na(EIS)){
      model$rho <- model$gamma
    }
    model_solved <- solve_EZ_SDF(model,psi_BansalShaliastovich,
                                z_bar_ini=5,
                                nb_loop_z_bar = 10,nb_loop_mu_z1 = 200)

    # Compute maximum Sharpe ratio:
    psi_w_2alpha <- psi_BansalShaliastovich(2*model_solved$alpha,model_solved)
    psi_w_alpha <- psi_BansalShaliastovich(model_solved$alpha,model_solved)
    Ew <- model_solved$Ew
    maxSR <- sqrt(exp(t(psi_w_2alpha$a - 2*psi_w_alpha$a)%*%Ew +
                       psi_w_2alpha$b - 2*psi_w_alpha$b)-1)
    all_maxSR[count_EIS,count_gamma] <- maxSR*sqrt(freq)

    # Compute average risk-free rate:
    Erf <- model_solved$eta0 + t(model_solved$eta1)%*%Ew
    all_Erf[count_EIS,count_gamma] <- freq*Erf

    # Solve for stock prices:
    model_solved <- solve_EZ_stock_return(model_solved,psi_BansalShaliastovich,
                                           nb_loop_z_bar = 10,nb_loop_mu_z1 = 200)

    # Compute expected excess return:
    psi_mu_rs2 <- psi_BansalShaliastovich(model_solved$mu_rs2,
                                           model_solved)
    Ers <- model_solved$mu_rs0 + t(model_solved$mu_rs1)%*%Ew +
          psi_mu_rs2$b + t(psi_mu_rs2$a) %*%Ew
    all_Ers[count_EIS,count_gamma] <- freq*Ers

    # Compute variance of p-d:

```

```

    Var_pd <- t(model_solved$mu_s1) %*% model_solved$Vw %*% model_solved$mu_s1
    all_std_dev_pd[count_EIS,count_gamma] <- sqrt(Var_pd)
  }
}

par(mfrow=c(2,2))
par(plt=c(.25,.98,.25,.85))

# Average stock excess return:
all_xs <- all_Ers - all_Erf
plot(all_gamma,100*all_xs[1,],ylim=c(0,7),type="l",
     lwd=2,las=1,col=all_col[1],lty=all_lty[1],
     xlab=expression(paste("coefficient of RRA (" ,gamma,")",sep="")),
     ylab="in percent, annualized",
     main=expression(paste("(a) Expected excess return",sep="")),cex.main=1.1)
grid()
for(i in 2:length(all_EIS)){
  lines(all_gamma,100*all_xs[i,],lwd=2,
        col=all_col[i],lty=all_lty[i])
}

# Plot points where the curve 1/EIS crosses the other ones:
interpolation <- spline(all_gamma, all_xs[4,], xout = 1/all_EIS[1:3])
points(1/all_EIS[1:3],interpolation$y,pch=19)

legend("topleft",
      c(expression(paste("EIS = 0.5",sep="")),
        expression(paste("EIS = 1",sep="")),
        expression(paste("EIS = 1.5",sep="")),
        expression(paste("EIS = 1/",gamma," (CCAPM case)",sep="))),
      lwd=2,
      lty=all_lty,
      col=all_col,
      seg.len = 3
)

# Maximum Sharpe ratios:
plot(all_gamma,(all_maxSR[1,]),ylim=c(0,.75),type="l",
     lwd=2,las=1,col=all_col[1],lty=all_lty[1],
     xlab=expression(paste("coefficient of RRA (" ,gamma,")",sep="")),
     ylab="annualized",
     main=expression(paste("(b) Maximum Sharpe ratio",sep="")),cex.main=1.1)
grid()
for(i in 2:length(all_EIS)){
  lines(all_gamma,(all_maxSR[i,]),lwd=2,
        col=all_col[i],lty=all_lty[i])
}

```

```

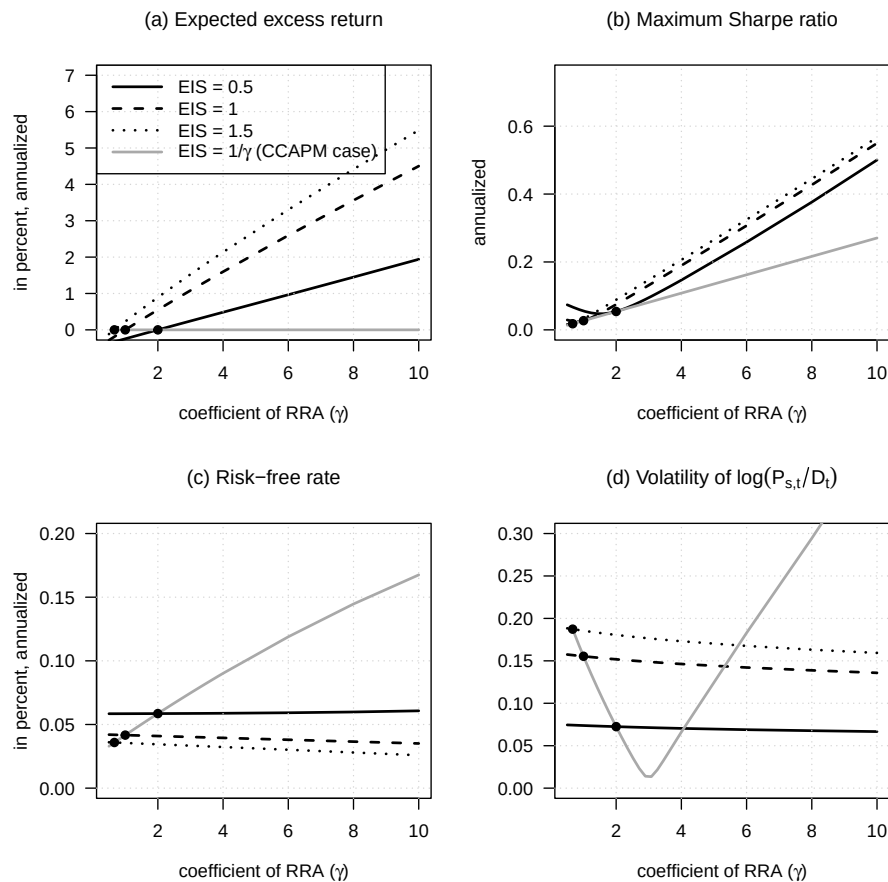
}
# Plot points where the curve 1/EIS crosses the other ones:
interpolation <- spline(all_gamma, all_maxSR[4,], xout = 1/all_EIS[1:3])
points(1/all_EIS[1:3],interpolation$y,pch=19)

# Risk-free rate:
plot(all_gamma,(all_Erf[1,]),ylim=c(0,.2),type="l",
      lwd=2,las=1,col=all_col[1],lty=all_lty[1],
      xlab=expression(paste("coefficient of RRA (",gamma,")",sep="")),
      ylab="in percent, annualized",
      main=expression(paste("(c) Risk-free rate",sep="")),cex.main=1.1)
grid()
for(i in 2:length(all_EIS)){
  lines(all_gamma,(all_Erf[i,]),lwd=2,
        col=all_col[i],lty=all_lty[i])
}
# Plot points where the curve 1/EIS crosses the other ones:
interpolation <- spline(all_gamma, all_Erf[4,], xout = 1/all_EIS[1:3])
points(1/all_EIS[1:3],interpolation$y,pch=19)

# Volatility of p-d:
plot(all_gamma,(all_std_dev_pd[1,]),ylim=c(0,.3),type="l",
      lwd=2,las=1,col=all_col[1],lty=all_lty[1],
      xlab=expression(paste("coefficient of RRA (",gamma,")",sep="")),
      ylab="",
      main=expression(paste("(d) Volatility of ",log(P[s*', '*t]/D[t]),sep="")),cex.main=1.1)
grid()
for(i in 2:length(all_EIS)){
  lines(all_gamma,(all_std_dev_pd[i,]),lwd=2,
        col=all_col[i],lty=all_lty[i])
}
# Plot points where the curve 1/EIS crosses the other ones:
interpolation <- spline(all_gamma, all_std_dev_pd[4,], xout = 1/all_EIS[1:3])
points(1/all_EIS[1:3],interpolation$y,pch=19)

```

12.6. YIELDS, RETURNS, AND PREMIA IN THE BANSAL-YARON MODEL 237



```
# Computation of mean and variance of P/D:
T <- 500
psi_PD_inf <- reverse_MHLT(psi_BansalShaliastovich,
                           u1 = model_solved$mu_s1,
                           u2 = 0*model_solved$mu_s1,
                           H = T, model_solved)
E_PD <- exp(model_solved$mu_s0 + psi_PD_inf$B[T])

psi_PD2_inf <- reverse_MHLT(psi_BansalShaliastovich,
                           u1 = 2*model_solved$mu_s1,
                           u2 = 0*model_solved$mu_s1,
                           H = T, model_solved)
Var_PD <- exp(2*model_solved$mu_s0 + psi_PD2_inf$B[T]) -
  exp(2*model_solved$mu_s0 + 2*psi_PD_inf$B[T])
Std_dev_PD <- sqrt(Var_PD)
# Compare with Gaussian approximation:
```

```
Var_pd <- t(model_solved$mu_s1) %*% model_solved$Vw %*% model_solved$mu_s1
E_PD_approx <- exp(model_solved$mu_s0 + Var_pd/2)
```

12.7 The Bansal-Shaliastovich specification for bond and currency risk premia

This example illustrates the Bansal-Shaliastovich specification for bond and currency risk premia.

```
# Purpose: This example illustrates the Bansal-Shaliastovich specification for bond and
# risk premia
library(DTAM)

max_maturity_in_years <- 10

# Parameterization of Bansal & Shaliastovich (2012):
rho.c <- 0.810
rho.cpi <- -0.047
rho.pi <- .988
mu.c <- .0049
mu.pi <- .009
sigma.c <- 4.6e-03
sigma.pi <- 5.5e-03
delta <- .994
gamma <- 20.9
psi <- 1.81
nu.c <- 0.994
nu.pi <- 0.979
sigma.xc <- 1.09e-03
sigma.xpi <- 1.11e-03
sigma.wc <- 1.85e-07
sigma.wpi <- 1.81e-07

Phi <- matrix(0,4,4)
Phi[1:2,1:2] <- c(rho.c,0,rho.cpi,rho.pi)

model <- list(
  mu_c0 = mu.c,
  mu_c1 = matrix(c(1,0,0,0, # x weightings
                  0,0), # z weightings
                ncol=1),
  Pi = Phi,
  # uncondit. means of x's conditional variances:
  chi.0 = c(sigma.xc^2,sigma.xpi^2,sigma.c^2,sigma.pi^2),
```

```

# loadings of x's conditional variances on z:
chi.1 = cbind(diag(2),0*diag(2)),
nu     = diag(c(nu.c,nu.pi)),
sigma.w = c(sigma.wc,sigma.wpi),
gamma  = gamma,
rho     = 1/psi,
delta  = delta,
n_w    = 2+2+2)

freq <- 4 # quarterly frequency

model_solved <- solve_EZ_SDF(model,psi_BansalShaliastovich,z_bar_ini=5,
                             nb_loop_z_bar = 10,nb_loop_mu_z1 = 500)

# Q log Laplace transform:
psiQ <- function(u,model_solved){
  psi1 <- psi_BansalShaliastovich(u + model_solved$alpha, model_solved)
  psi2 <- psi_BansalShaliastovich(model_solved$alpha, model_solved)
  return(list(a = psi1$a - psi2$a,
             b = psi1$b - psi2$b))
}

# Specification of real yields:
psi <- reverse_MHLT(psiQ,
                   u1 = 0*model_solved$eta1, u2 = - model_solved$eta1,
                   H=max_maturity_in_years*freq, model_solved)
Br <- - (1:(max_maturity_in_years*freq))*c(model_solved$eta0) + c(psi$B)
Ar <- - model_solved$eta1 %%% matrix(1,1,max_maturity_in_years*freq) +
  matrix(psi$a,model_solved$n_w,
         max_maturity_in_years*freq)
br <- - Br/(1:(max_maturity_in_years*freq))
ar <- - Ar/t(matrix(1:(max_maturity_in_years*freq),max_maturity_in_years*freq,
                  model_solved$n_w))

# Specification of nominal yields:
mu_pi1 = matrix(c(0,1,0,0, # x weightings
                 0,0), # z weightings
               ncol=1)
psi <- reverse_MHLT(psiQ,
                   u1 = - mu_pi1, u2 = - model_solved$eta1 - mu_pi1,
                   H=max_maturity_in_years*freq, model_solved)
Bn <- - (1:(max_maturity_in_years*freq))*(mu.pi + c(model_solved$eta0)) + c(psi$B)
An <- - model_solved$eta1 %%% matrix(1,1,max_maturity_in_years*freq) +
  matrix(psi$a,model_solved$n_w,
         max_maturity_in_years*freq)

```

```

bn <- - Bn/(1:(max_maturity_in_years*freq))
an <- - An/t(matrix(1:(max_maturity_in_years*freq),max_maturity_in_years*freq,
                    model_solved$n_w))

avg_real_yields <- freq*(br + t(ar) %%% model_solved$Ew)
avg_nomi_yields <- freq*(bn + t(an) %%% model_solved$Ew)

# Compute standard deviations of yields:
loadings_r <- freq * t(ar)
loadings_n <- freq * t(an)
vec_1 <- matrix(1,1,model_solved$n_w)
aux_r <- (loadings_r %xx vec_1)*(vec_1 %xx loadings_r) *
  (matrix(1,max_maturity_in_years*freq,1) %xx matrix(model_solved$Vw,nrow=1))
aux_n <- (loadings_n %xx vec_1)*(vec_1 %xx loadings_n) *
  (matrix(1,max_maturity_in_years*freq,1) %xx matrix(model_solved$Vw,nrow=1))
std_dev_r <- sqrt(aux_r %xx matrix(1,model_solved$n_w^2,1))
std_dev_n <- sqrt(aux_n %xx matrix(1,model_solved$n_w^2,1))

# Consider alternative values of covariance between consumption and inflation
# and recompute nominal prices.
other_avg_nomi_yields <- NULL
all_rho.cpi <- c(0,0.047)
for(rho.cpi in all_rho.cpi){
  model$Pi[1,2] <- rho.cpi

  model_solved <- solve_EZ_SDF(model,psi_BansalShaliastovich,z_bar_ini=5,
                              nb_loop_z_bar = 10,nb_loop_mu_z1 = 500)
  psi <- reverse_MHLT(psiQ,
                     u1 = - mu_pi1, u2 = - model_solved$eta1 - mu_pi1,
                     H=max_maturity_in_years*freq, model_solved)
  Bn <- - (1:(max_maturity_in_years*freq))*(mu.pi + c(model_solved$eta0)) + c(psi$B)
  An <- - model_solved$eta1 %xx matrix(1,1,max_maturity_in_years*freq) +
    matrix(psi$A,model_solved$n_w,
           max_maturity_in_years*freq)
  bn <- - Bn/(1:(max_maturity_in_years*freq))
  an <- - An/t(matrix(1:(max_maturity_in_years*freq),max_maturity_in_years*freq,
                     model_solved$n_w))
  other_avg_nomi_yields <- cbind(other_avg_nomi_yields,
                                freq*(bn + t(an) %%% model_solved$Ew))
}

# Prepare plot
par(plt=c(.12,1,.2,.95))

```



```

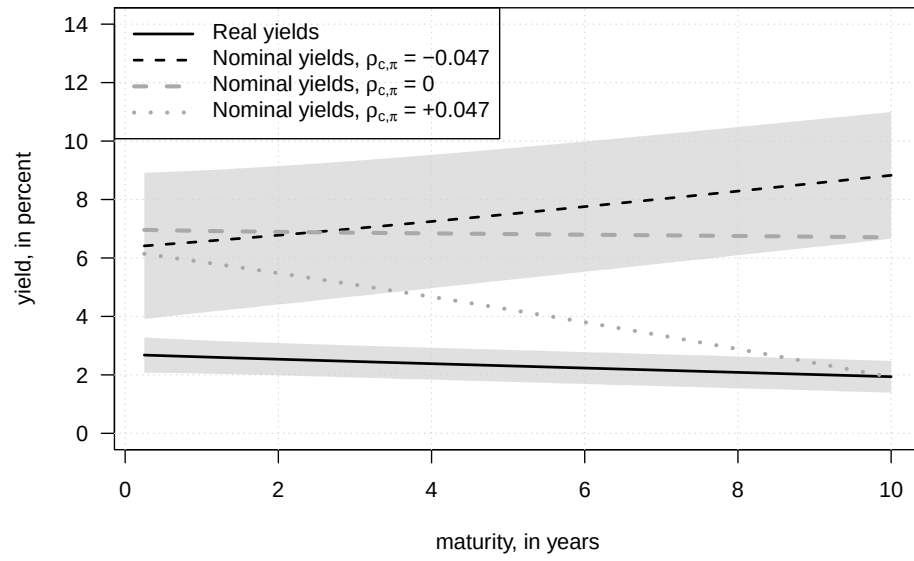
x_axis <- 1:(max_maturity_in_years*freq)/freq
plot(x_axis, 100*avg_real_yields, lwd=2, ylim=c(0, 14), type="l",
     col="white",
     las=1, xlab="maturity, in years", ylab="yield, in percent")

lower_bound <- 100*(avg_real_yields - std_dev_r)
upper_bound <- 100*(avg_real_yields + std_dev_r)
polygon(c(x_axis, rev(x_axis)),
        c(lower_bound, rev(upper_bound)), col="#66666633", border=NaN)
lower_bound <- 100*(avg_nomi_yields - std_dev_n)
upper_bound <- 100*(avg_nomi_yields + std_dev_n)
polygon(c(x_axis, rev(x_axis)),
        c(lower_bound, rev(upper_bound)), col="#66666633", border=NaN)
grid()
lines(x_axis, 100*avg_real_yields, lwd=2, lty=1)
lines(x_axis, 100*avg_nomi_yields, lwd=2, lty=2)

for(i in 1:length(all_rho.cpi)){
  lines(x_axis, 100*other_avg_nomi_yields[,i], lwd=3, lty=i+1,
        col="darkgrey")
}

legend("topleft",
      c("Real yields",
        expression(paste("Nominal yields, ", rho[c*'],'*pi], " = -0.047", sep="")),
        expression(paste("Nominal yields, ", rho[c*'],'*pi], " = 0", sep="")),
        expression(paste("Nominal yields, ", rho[c*'],'*pi], " = +0.047", sep=""))),
      lwd=c(2,2,3,3), # line width
      lty=c(1,2,2,3),
      col=c("black", "black", "darkgrey", "darkgrey"),
      seg.len = 3
)

```



Chapter 13

Preferred-habitat model

13.1 The preferred-habitat specification and the maturity-demand function

This example introduces the preferred-habitat specification and the maturity-demand function.

```
# Purpose: This example introduces the preferred-habitat specification and the maturity-demand  
# function  
library(DTAM)  
  
n <- 2 # number of factors  
H <- 30 # maximum maturity  
# Calibration: (see Table 1 in Vayanos-Vila):  
kappa_r <- .125  
sigma_r <- .0146  
kappa_beta <- .053  
a.theta <- 3155  
a.alpha <- 35.3  
delta_alpha <- .297  
delta_theta <- .307  
alpha <- 5.21  
r_bar <- 4.8  
a.theta0 <- 289  
  
# Specif of short-rate:  
a1 <- matrix(c(1,rep(0,n-1)),ncol=1) # The first factor is r_t  
b1 <- r_bar/100 # minus mean of short-term rate  
  
# Specify factors' dynamics:
```

```

mu      <- matrix(0,n,1)
Phi     <- matrix(0,n,n)
Phi[1:2,1:2] <- diag(c(1-kappa_r,1-kappa_beta))
Sigma   <- sigma_r^2*diag(n) # since sigma_beta = sigma_r

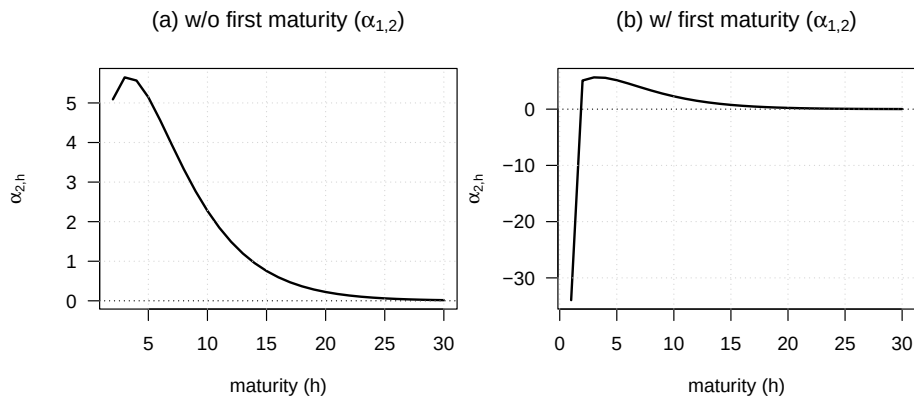
# Specify supply of bonds:
a       <- a.alpha/alpha
theta   <- a.theta/a
theta0  <- a.theta0/a

Beta    <- matrix(theta0 * (exp(-delta_alpha*(1:H)) -
                        exp(-delta_theta*(1:H))),H,1)
Alpha   <- matrix(0,H,n)
Alpha[,2] <- theta * (exp(-delta_alpha*(1:H)) -
                     exp(-delta_theta*(1:H)))
xi <- alpha * matrix(1:H * exp(-delta_alpha*(1:H)),H,1)
model_VV <- list(gamma = a,mu = mu,Phi = Phi,Sigma = Sigma,
                 alpha = Alpha,beta = Beta,xi = xi,a1 = a1,b1 = b1)

par(mfrow=c(1,2))
par(plt=c(.2,.98,.25,.85))
plot(2:H,
     las=1,
     Alpha[2:H,2],type="l",lwd=2,
     xlab=expression(paste("maturity (",h,")",sep="")),ylab=expression(alpha[2*','*h])
     main=expression(paste("(a) w/o first maturity (",alpha[1*','*2],")",sep=""))
)
grid()
abline(h=0,col="black",lty=3)

# Solve the model to get the right specification of alpha_1 and beta_1:
RES <- solve_PH_TSM(model_VV)
Alpha <- RES$alpha
Beta <- RES$beta
plot(1:H,
     las=1,
     Alpha[1:H,2],type="l",lwd=2,
     xlab=expression(paste("maturity (",h,")",sep="")),
     ylab=expression(alpha[2*','*h]),
     main=expression(paste("(b) w/ first maturity (",alpha[1*','*2],")",sep=""))
)
grid()
abline(h=0,col="black",lty=3)

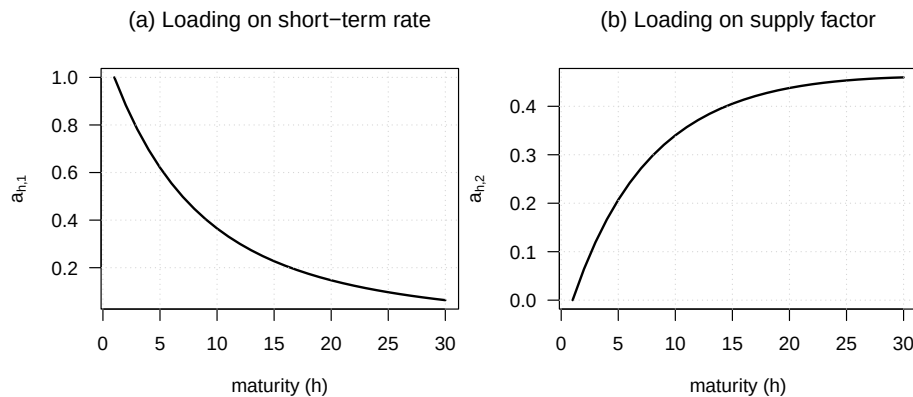
```



13.2 Factor loadings in the preferred-habitat model

This example shows the factor loadings generated by the preferred-habitat term-structure model.

```
# Purpose: This example shows the factor loadings generated by the preferred-habitat
# term-structure model
# Single yield curve:
RES <- solve_PH_TSM(model_VV)
par(mfrow=c(1,n))
par(plt=c(.2,.98,.25,.85))
for(i in 1:n){
  if(i==1){
    main = expression(paste("(a) Loading on short-term rate",sep=""))
    ylab = expression(a[h*','*1])
  }else{
    main = expression(paste("(b) Loading on supply factor",sep=""))
    ylab = expression(a[h*','*2])
  }
  plot(RES$a[,i],type="l",
       las=1,lwd=2,
       xlab=expression(paste("maturity (",h,")",sep="")),
       ylab=ylab,
       main=main)
  grid()
}
```



13.3 How bond supply shocks affect the yield curve under preferred habitat

This example studies how bond supply shocks affect the yield curve under preferred habitat.

```
# Purpose: This example studies how bond supply shocks affect the yield curve under preferred
# habitat
# Shock:
shock <- matrix(0,n,1)
shock[2] <- sigma_r # shock on the net supply factor
# baseline yield curve:
avg.y <- RES$b # average yield curve
# yield curve after shock:
shock.y <- RES$b + RES$a %*% shock
# Quantities:
avg.z <- Beta - xi * avg.y
shock.z <- Beta + Alpha %*% shock - xi * shock.y

legendtext <- c("w/o supply shock",
               "w/ supply shock")

par(mfrow=c(2,1))
par(plt=c(.15,.95,.2,.8))

plot(100*avg.y,type="l",
     ylim=c(4,8),
     lwd=2,las=1,
     xlab=expression(paste("maturity (",h,")",sep="")),
     ylab=expression(paste("yield, in percent",sep="")),
     main=expression(paste("(a) Yield curve impact",sep="")))
```

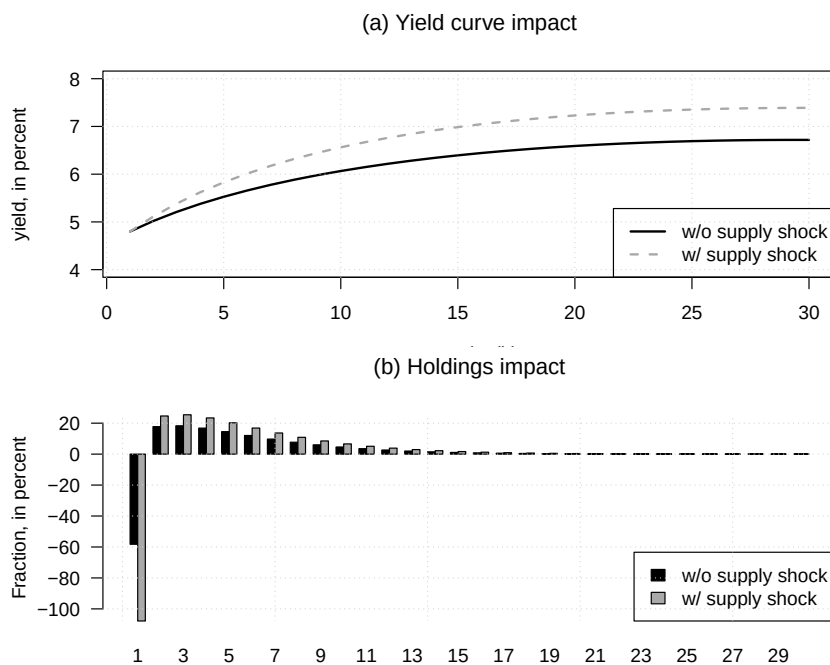
```

grid()
lines(100*shock.y,lty=2,lwd=2,col="dark grey")
legend("bottomright",
      legendtext,
      lty=c(1,2), # gives the legend appropriate symbols (lines)
      lwd=c(2,2), # line width
      col= c("black","dark grey"),
      # pch=c(19,15,17),
      # pt.cex = 1,
      # pt.bg = c("black","dark grey"),
      seg.len = 2
)

# plot(avg.z,ylim=c(min(avg.z,shock.z),1.1*max(avg.z,shock.z)),pch=19,
#       xlab="maturity",ylab="Fraction of portfolio",las=1)
# grid()
# abline(h=0,col="black",lty=3)
# points(shock.z,lty=2,col="red",lwd=2)

# Create the barplot
barplot(100*rbind(c(avg.z), c(shock.z)), beside = TRUE,
        names.arg = 1:H, col = c("black", "dark grey"), las = 1,
        legend.text = legendtext, args.legend = list(x = "bottomright",bg="white"),
        main=expression(paste("(b) Holdings impact",sep="")),
        xlab=expression(paste("maturity (",h,")",sep="")),
        ylab=expression(paste("Fraction, in percent",sep="")))
grid()

```



13.4 The excess-return implications of the preferred-habitat model

This example reports the excess-return implications of the preferred-habitat model.

```
# Purpose: This example reports the excess-return implications of the preferred-habitat
a_h <- RES$A
a_h_1 <- rbind(0,a_h[1:(H-1),])
b_h <- RES$B
b_h_1 <- matrix(rbind(0,matrix(b_h[1:(H-1)])),ncol=1)

# Conditional variance of returns:
cond_var <- ((a_h_1 %x% matrix(1,1,n)) * (matrix(1,1,n) %x% a_h_1)) %*% c(Sigma)
# Average excess return:
avg_xs_returns <- b_h_1 - b_h - b1 + 0.5*cond_var
# Average Sharpe ratio:
SR <- avg_xs_returns/sqrt(cond_var)

# Compute unconditional standard deviation of excess return:-----
# Compute matrix of variance of  $W_t = [w_t', w_{t-1}']'$ :
Var_w <- matrix(solve(diag(4) - Phi %x% Phi) %*% c(Sigma),2,2)
```



```

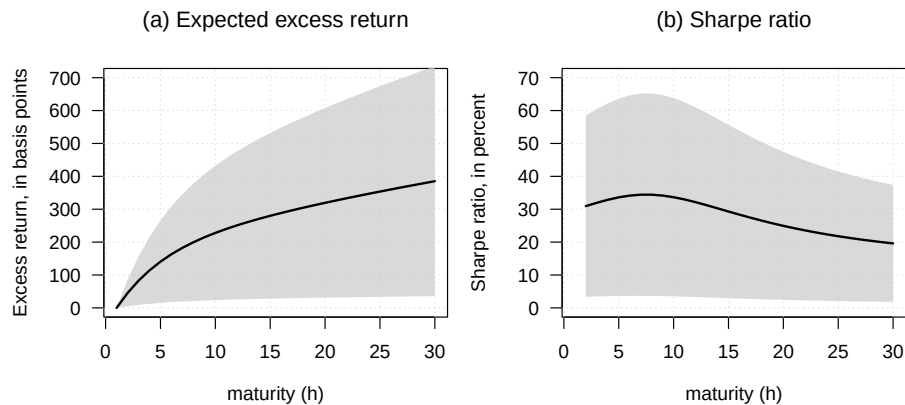
Var_w_w_1 <- cbind(rbind(Var_w, Var_w %*% t(Phi)),
                  rbind(Phi %*% Var_w, Var_w))
# Loadings of excess return on W_t:
a_h_1_a_h <- cbind(a_h_1, - a_h - matrix(1,H,1) %*% t(a1))
# Unconditional variance of excess return:
cond_var_xs <-
  ((a_h_1_a_h %x% matrix(1,1,2*n)) * (matrix(1,1,2*n) %x% a_h_1_a_h)) %*%
  c(Var_w_w_1)
# Unconditional variance of Expected excess return:
aux <- a_h_1 %*% Phi - a_h - matrix(1,H,1) %*% t(a1)
cond_var_Exs <- ((aux %x% matrix(1,1,n)) * (matrix(1,1,n) %x% aux)) %*% c(Sigma)

# Compute unconditional standard deviation of expected excess return:-----
# condi. variance of expected returns:
lower_bound_xs <- avg_xs_returns - 1.64 * sqrt(cond_var_Exs)
upper_bound_xs <- avg_xs_returns + 1.64 * sqrt(cond_var_Exs)
lower_bound_SR <- SR - 1.64 * sqrt(cond_var_Exs)/sqrt(cond_var)
upper_bound_SR <- SR + 1.64 * sqrt(cond_var_Exs)/sqrt(cond_var)
# -----

par(mfrow=c(1,2))
par(plt=c(.23,.98,.23,.85))
plot(10000*avg_xs_returns,col="white",las=1,
     xlab=expression(paste("maturity (" ,h,")",sep="")),
     ylab=expression(paste("Excess return, in basis points",sep="")),
     ylim=c(0,700),
     main=expression(paste("(a) Expected excess return",sep=")))
grid()
polygon(c(1:H,H:1),10000*c(lower_bound_xs,rev(upper_bound_xs)),
        col="#CCCCCB",border = NaN)
lines(10000*avg_xs_returns,lwd=2)

plot(100*SR,col="white",las=1,
     ylim=c(0,70),
     xlab=expression(paste("maturity (" ,h,")",sep="")),
     ylab=expression(paste("Sharpe ratio, in percent",sep="")),
     main=expression(paste("(b) Sharpe ratio",sep=")))
grid()
polygon(c(1:H,H:1),100*c(lower_bound_SR,rev(upper_bound_SR)),
        col="#CCCCCB",border = NaN)
lines(100*SR,lwd=2)

```



```
# Compute average maximum Sharpe ratio
k <- 20
quantiles <- seq(.05,.95,length.out=20)
q <- matrix(qnorm(quantiles),nrow=1)
values.w <- rbind(q %x% matrix(1,1,k),
                  matrix(1,1,k) %x% q)
values.lambda <- RES$lambda0 %** matrix(1,1,k^2) + RES$lambda1 %** values.w

lambda.Sigma.lambda <- ((t(values.lambda) %x% matrix(1,1,n)) *
                        (matrix(1,1,n) %x% t(values.lambda))) %** c(Sigma)
E_SR <- mean(sqrt(exp(lambda.Sigma.lambda)-1))
```

13.5 The preferred-habitat setup to exchange-rate implications

This example extends the preferred-habitat setup to exchange-rate implications.

```
# Purpose: This example extends the preferred-habitat setup to exchange-rate implications.
# Example where there is a foreign yield curve.
mu_2yc <- matrix(0,3,1)
Phi_2yc <- diag(c(diag(Phi),0.3))
# Phi_2yc <- diag(c(diag(Phi),Phi[1,1]))
Sigma_2yc <- diag(c(diag(Sigma),5*Sigma[1,1]))
rho <- .5
Sigma_2yc[1,3] <- rho*Sigma[1,1]
Sigma_2yc[3,1] <- rho*Sigma[1,1]
Sigma_2yc[2,2] <- 1*Sigma[2,2]

model <- list(gamma = a,
              mu = mu_2yc,Phi = Phi_2yc,Sigma = Sigma_2yc,
              alpha = cbind(.5*Alpha,0),
              beta = .5*Beta,xi = .5*xi,
```

```

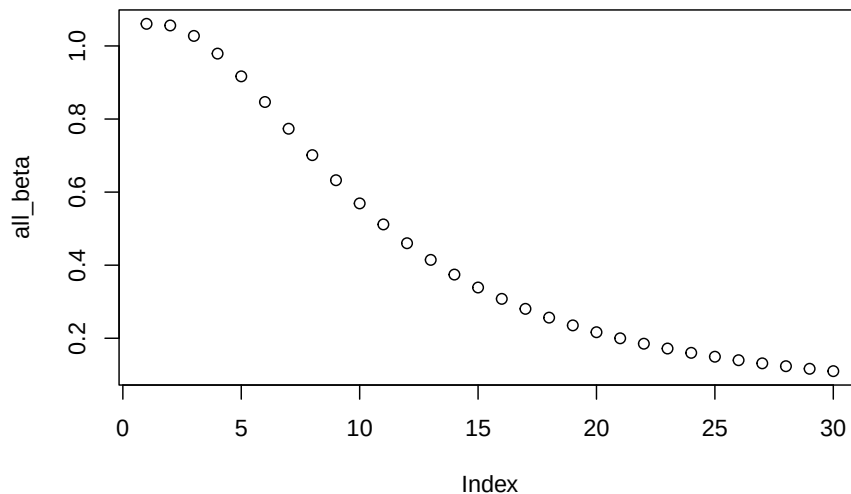
        alpha_star = cbind(0.5*Alpha,0),
        beta_star = .5*Beta,xi_star = .5*xi,
        # b1_star = b1, a1_star = matrix(c(0,0,1),n+1,1),
        mu_e0 = 0, mu_e1 = matrix(c(0,0,1),n+1,1),
        a1 = matrix(c(1,0,0),n+1,1),b1 = b1)
RES_2yc <- solve_PH_TSM(model)

# Compute beta's:
mu_e1 <- RES_2yc$mu_e1

Var_w <- matrix(solve(diag((n+1)^2) - Phi_2yc %x% Phi_2yc)%*%c(Sigma_2yc),n+1,n+1)

sum_Phi_h <- 0
all_beta <- NULL
Phi_h <- diag(n+1)
for(h in 1:H){
  a_h <- matrix(RES_2yc$a[h,],ncol=1)
  a_h_star <- matrix(RES_2yc$a_star[h,],ncol=1)
  Phi_h <- Phi_h %*% Phi_2yc
  sum_Phi_h <- sum_Phi_h + Phi_h
  Cov <- t(mu_e1) %*% (sum_Phi_h) %*% Var_w %*% (a_h - a_h_star)
  Var_x <- t(a_h - a_h_star) %*% Var_w %*% (a_h - a_h_star)
  beta_h <- Cov/Var_x/h
  all_beta <- c(all_beta,beta_h)
}
plot(all_beta)

```



13.6 Nominal and real term structures in the preferred-habitat framework

This example compares nominal and real term structures in the preferred-habitat framework.

Purpose: This example compares nominal and real term structures in the preferred-habitat framework

Model parameterization:

```
rho_r <- .8
rho_pi <- .8
sigma_r <- .01
sigma_pi <- sigma_r
gamma <- 0.1
a_pi <- 0.3
Phi <- matrix(c(rho_r, -gamma, a_pi, rho_pi), 2, 2)
Sigma_12 <- diag(c(.01, .01))
```

```
H <- 20
```

Compute unconditional variances of state variables:

```
Var_w <- matrix(solve(diag((2)^2) - Phi %x% Phi) %*% c(Sigma), 2, 2)
stdv_r <- sqrt(Var_w[1,1])
```

```

stdv_pi <- sqrt(Var_w[2,2])

# Elasticity of net supply of Inflation-linked bonds (ILBs) to inflation:
alpha_pi <- -0.3/H/stdv_pi

# Average share of ILBs:
share_ILB <- .2

avg_r <- .015 # average real short-term rate
avg_pi <- .02 # average inflation rate

# Check IRFs:
# shock <- c(1,0)
# x <- Sigma_12 %*% shock
# all_x <- matrix(x,ncol=1)
# for(h in 1:20){
#   x <- Phi %*% x
#   all_x <- cbind(all_x,x)
# }
# par(mfrow=c(1,2))
# plot(all_x[1,])
# plot(all_x[2,])

mu <- matrix(0,2,1)
Sigma <- Sigma_12 %*% t(Sigma_12)

# Create matrices defining supply functions:
Alpha <- matrix(0,H,2)
Alpha[,2] <- alpha_pi
Alpha_star <- matrix(0,H,2)
Beta <- matrix(share_ILB/H,H,1)
Beta_star <- matrix((1-share_ILB)/H,H,1)
xi <- matrix(0,H,1)

model <- list(gamma = a,
             mu = mu, Phi = Phi, Sigma = Sigma,
             alpha = Alpha,
             beta = Beta, xi = xi,
             alpha_star = Alpha_star,
             beta_star = Beta_star, xi_star = xi,
             # b1_star = b1, a1_star = matrix(c(0,0,1),n+1,1),
             mu_e0 = - avg_pi, mu_e1 = matrix(c(0,-1),2,1),
             a1 = matrix(c(1,0),2,1), b1 = avg_r)
RES_2yc <- solve_PH_TSM(model) # solve model

```

```

# Plot average real and nominal yield curves:
par(mfrow=c(1,2))
par(plt=c(.2,.98,.25,.85))
plot(100*RES_2yc$b,type="l",col="black",lwd=2,las=1,
     ylim=c(1,7),
     xlab="maturity (in years)",ylab="yield (in percent)",
     main=expression("(a) Average yield curves"),cex.main=1.1)
grid()
lines(100*RES_2yc$b_star,col="black",lwd=2,lty=2)
legend("topleft",
      c("Nominal","Real"),
      lty=c(2,1), # gives the legend appropriate symbols (lines)
      lwd=c(2,2), # line width
      col= c("black"),
      seg.len = 2)

# Compute average inflation swap rates -----
# Specify the model for it to be used in 'reverse_MHLT':
psi.parameterization.P <- list(
  mu = mu,
  Phi = Phi,
  Sigma = Sigma)
psi.parameterization.Q <- list(
  mu = RES_2yc$muQ,
  Phi = RES_2yc$PhiQ,
  Sigma = Sigma)
# Use reverse-order multi-horizon LT to price ZC bonds:
AB.P <- reverse_MHLT(psi_GaussianVAR,u1=matrix(c(0,+1),2,1),u2=NaN,
                    H=H,psi.parameterization.P)
a.h.P <- - t(matrix(AB.P$a,2,H))
b.h.P <- - model$mu_e0 - matrix(AB.P$b,ncol=1)
AB.Q <- reverse_MHLT(psi_GaussianVAR,u1=matrix(c(0,+1),2,1),u2=NaN,
                    H=H,psi.parameterization.Q)
a.h.Q <- - t(matrix(AB.Q$a,2,H))
b.h.Q <- - model$mu_e0 - matrix(AB.Q$b,ncol=1)

plot(100*(RES_2yc$b_star-RES_2yc$b),type="l",col="black",lwd=2,las=1,
     ylim=c(1.5,4),
     xlab="maturity (in years)",ylab="inflation rate (in percent)",
     main=expression("(b) Average inflation expectations"),cex.main=1.1)
grid()
points(100*b.h.Q,col="black",lwd=2,pch=3)
lines(100*b.h.P,col="dark grey",lwd=2)
legend("topleft",
      c("BEIR","Inflation swap","Expected inflation"),

```



```

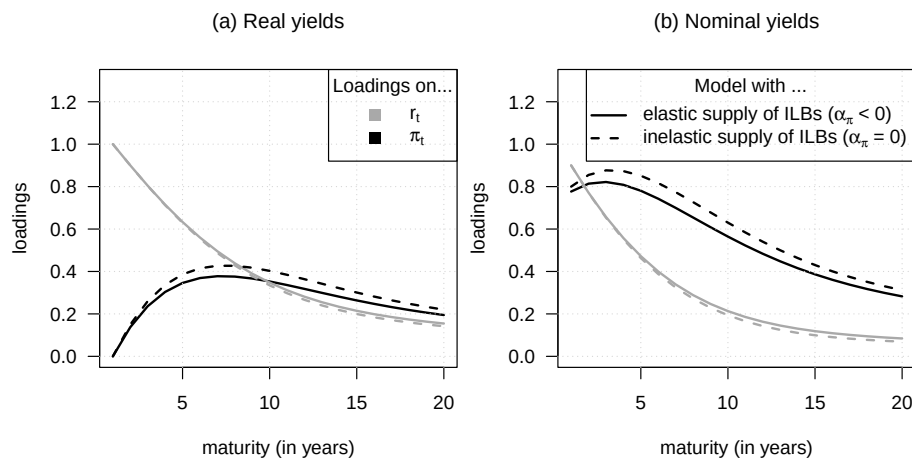
# Compare real-yield loadings with and without elastic ILB supply.
par(mfrow=c(1,2))
par(plt=c(.2,.98,.2,.85))
plot(RES_2yc$a[,2],ylim=c(0,1.3),type="l",col="black",lwd=2,
     main=expression("(a) Real yields"),
     xlab="maturity (in years)",
     ylab="loadings",
     cex.main=1.1,
     las=1)
grid()
lines(RES_2yc$a[,1],col="dark grey",lwd=2)
lines(RES_2yc_no_elast$a[,2],col="black",lty=2,lwd=2)
lines(RES_2yc_no_elast$a[,1],col="dark grey",lty=2,lwd=2)

legend("topright",
      title="Loadings on...",
      c(expression(paste(r[t],sep="")),
        expression(paste(pi[t],sep="))),
      lty=c(NaN,NaN), # gives the legend appropriate symbols (lines)
      lwd=c(2,2), # line width
      pch=c(15,15),pt.cex = 1.3,
      col= c("dark grey","black"),
      seg.len = 2)

# Compare nominal-yield loadings with and without elastic ILB supply.
plot(RES_2yc$a_star[,2],ylim=c(0,1.3),type="l",col="black",lwd=2,
     main=expression("(b) Nominal yields"),
     xlab="maturity (in years)",
     ylab="loadings",
     cex.main=1.1,
     las=1)
grid()
lines(RES_2yc$a_star[,1],col="dark grey",lwd=2)
lines(RES_2yc_no_elast$a_star[,2],col="black",lty=2,lwd=2)
lines(RES_2yc_no_elast$a_star[,1],col="dark grey",lty=2,lwd=2)

legend("topright",
      title="Model with ...",
      c(expression(paste("elastic supply of ILBs (",alpha[pi]," < 0)",sep="")),
        expression(paste("inelastic supply of ILBs (",alpha[pi]," = 0)",sep="))),
      lty=c(1,2), # gives the legend appropriate symbols (lines)
      lwd=c(2,2), # line width
      pch=c(NaN,NaN),pt.cex = 1.3,
      col= c("black","black"),
      seg.len = 2)

```

13.8 Additional nominal-real yield-curve implications of habitat demand

This example highlights additional nominal-real yield-curve implications of habitat demand.

Purpose: This example highlights additional nominal-real yield-curve implications of habitat demand

Compute factor loadings under physical and risk-neutral dynamics.

```
for(type_yield in c("real", "nominal")){
  for(type_dynamics in c("P", "Q", "Q_no_elast")){
    if(type_dynamics=="P"){
      psi.parameterization <- psi.parameterization.P
    }else if(type_dynamics=="Q"){
      psi.parameterization <- psi.parameterization.Q
    }else{
      psi.parameterization <- psi.parameterization.Q_no_elast
    }

    aux.loadings <-
      reverse_MHLT(psi_GaussianVAR,
        u1=matrix(c(+0, ifelse(type_yield=="nominal", -1, 0)), 2, 1),
        u2=matrix(c(-1, ifelse(type_yield=="nominal", -1, 0)), 2, 1),
        H=H,
        psi.parameterization)
    # Multiply by exp(-r_t):
```

```

Loadings <- matrix(c(-1,0),2,H)
Loadings[,1:H] <- Loadings[,1:H] + aux.loadings$A[,1:H]

eval(parse(text = gsub(" ", "", paste("Loadings_", type_yield, "_", type_dynamics,
                                     " <- Loadings",
                                     sep="")))))
}
}

# Convert loadings into term-premium loadings.
Loadings_real_TP <- Loadings_real_Q - Loadings_real_P
Loadings_nominal_TP <- Loadings_nominal_Q - Loadings_nominal_P

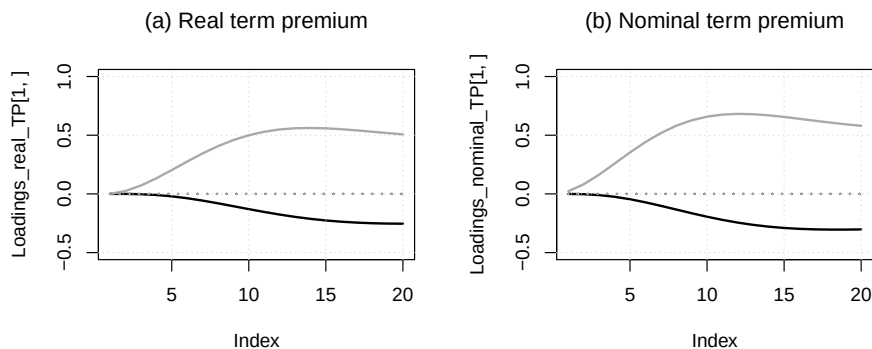
Loadings_real_TP_no_elast <- Loadings_real_Q_no_elast - Loadings_real_P
Loadings_nominal_TP_no_elast <- Loadings_nominal_Q_no_elast - Loadings_nominal_P

# Plot real and nominal term-premium loadings.
par(mfrow=c(1,2))

plot(Loadings_real_TP[1,], type="l", lwd=2, col="black", ylim=c(-.5,1),
     main=expression("(a) Real term premium"))
grid()
lines(Loadings_real_TP[2,], lwd=2, col="dark grey")
lines(Loadings_real_TP_no_elast[1,], lwd=2, col="black", lty=3)
lines(Loadings_real_TP_no_elast[2,], lwd=2, col="dark grey", lty=3)

plot(Loadings_nominal_TP[1,], type="l", lwd=2, col="black", ylim=c(-.5,1),
     main=expression("(b) Nominal term premium"))
grid()
lines(Loadings_nominal_TP[2,], lwd=2, col="dark grey")
lines(Loadings_nominal_TP_no_elast[1,], lwd=2, col="black", lty=3)
lines(Loadings_nominal_TP_no_elast[2,], lwd=2, col="dark grey", lty=3)

```



13.9 Preferred-habitat bond demand to stock-market valuation effects

This example connects preferred-habitat bond demand to stock-market valuation effects.

```
# Purpose: This example connects preferred-habitat bond demand to stock-market valuation effects
# Check model with stocks

# ---- Different models: ----
# 1. Perpetuity
# 2. Dividend medium vol (Baseline)
# 3. Dividend high vol (HV)
# 4. Search for yields SFY
# 5. omega
# 6. More stock in economy

rho_k <- .5

mu_k <- 0.02

omega <- .05
sd_k_med <- .08
sd_k_high <- .13
alpha_r <- +1
share_stocks <- .25
share_stocks_high <- .5

all_omega <- c(0,0,0,0,omega,0)
all_stdev_k <- c(0,
                sd_k_med,
                sd_k_high,
                sd_k_med,sd_k_med,sd_k_med)
all_alpha_r <- c(0,0,0,alpha_r,0,0)
all_share_stocks <- c(rep(share_stocks,5),
                     share_stocks_high)
all_mu_k <- c(0,rep(mu_k,length(all_omega)-1))

mu_K1 <- matrix(c(0,0,1),ncol=1)

beta_K <- 0
alpha_K <- matrix(c(0,0,0),ncol=1)
xi_K <- 0

# Output table (relevant results stored in "all_results"):
all_results <- matrix(0,16,length(all_omega))
```



```

    rbind(Var_w, Var_w %*% t(Phi)),
    rbind(Phi %*% Var_w, Var_w)
  )

model <- list(gamma = a, mu = mu,
             Phi = Phi,
             Sigma = Sigma,
             alpha = Alpha,
             beta = Beta,
             xi = xi,
             a1 = rbind(model_VV$a1, 0), b1 = model_VV$b1,
             omega = all_omega[i], mu_K0=all_mu_k[i], mu_K1=mu_K1,
             beta_K=all_share_stocks[i], alpha_K=alpha_K, xi_K=xi_K)

H <- dim(model$alpha)[1]

RES <- solve_PH_TSM(model, max_iter = 200)

mu_ERK0 <- RES$C_K + RES$Theta_K %*% model$mu
mu_ERK1 <- RES$D_K + RES$Theta_K %*% model$Phi
avg_ERK <- mu_ERK0 + c(mu_ERK1 %*% Ew)

# Average excess return:
all_results["$\\mathbb{E}(R_{K,t,t+1}-r_t)$", i] <- avg_ERK - b1

# Compute unconditional variance of  $R_{K,t,t+1}$ :
loadings_RK <- matrix(c(RES$Theta_K, RES$D_K), ncol=1)
Var_RK <- t(loadings_RK) %*% Var_w_w1 %*% loadings_RK
all_results["$\\sigma(R_{K,t,t+1})$", i] <- sqrt(Var_RK)

# Average Sharpe ratio (for stocks):
SR <- (avg_ERK - b1)/sqrt(RES$Theta_K %*% Sigma %*% t(RES$Theta_K))
all_results["$\\mathbb{E}(SR_{K,t})$", i] <- SR

# Average 10-year yield:
avg_r10 <- RES$b[10]
all_results["$\\mathbb{E}(r_{t,10})$", i] <- avg_r10

# Volatility of 10-year yield:
loadings_r10 <- matrix(RES$a[10,], ncol=1)
sd_r10 <- sqrt(t(loadings_r10) %*% Var_w %*% loadings_r10)
all_results["$\\sigma(r_{t,10})$", i] <- sd_r10

# SR of 10-year yield:
a_h <- RES$a
a_h_1 <- rbind(0, a_h[1:(H-1),])

```

```

b_h <- RES$B
b_h_1 <- matrix(rbind(0,matrix(b_h[1:(H-1)])),ncol=1)
# Conditional variance of returns:
cond_var <- ((a_h_1 %x% matrix(1,1,3)) * (matrix(1,1,3) %x% a_h_1)) %**% c(Sigma)
avg_xs_returns <- b_h_1 - b_h - b1 + 0.5*cond_var # Average bond xs return
SR <- avg_xs_returns/sqrt(cond_var) # Average Sharpe ratio
all_results["$\\mathbb{E}(R_{10,t,t+1}-r_t)$",i] <- avg_xs_returns[10]
all_results["$\\mathbb{E}(SR_{10,t})$",i] <- SR[10]

# Unconditional variance of bond returns:
loadings_R10 <- matrix(c(a_h_1[10,],[-a_h[10,]],ncol=1)
Var_R10 <- t(loadings_R10) %**% Var_w_w_1 %**% loadings_R10
all_results["$\\sigma(R_{10,t,t+1})$",i] <- sqrt(Var_R10)

# Correlation between R_K and r_t:
loadings_r <- matrix(c(0,0,0,1,0,0),ncol=1)
sigma_r <- sqrt(Var_w[1])
Cov <- t(loadings_RK) %**% Var_w_w_1 %**% loadings_r
all_results["$\\mathbb{C}or(R_{K,t,t+1},r_t)$",i] <- Cov/sigma_r/sqrt(Var_RK)

# Correlation between R_10 and r_t:
Cov <- t(loadings_R10) %**% Var_w_w_1 %**% loadings_r
all_results["$\\mathbb{C}or(R_{10,t,t+1},r_t)$",i] <- Cov/sigma_r/sqrt(Var_R10)

# Correlation between E_t(R_{k,t+1}) and r_t:
# loadings_r <- matrix(c(1,0,0),ncol=1)
# sigma_r <- sqrt(Var_w[1])
# mu_ERK1_r <- mu_ERK1 - matrix(c(1,0,0),nrow=1)
# stdev_ERK1_r <- sqrt(mu_ERK1_r %**% Var_w %**% t(mu_ERK1_r))
# Cov <- mu_ERK1_r %**% Var_w %**% loadings_r
# all_results["$\\mathbb{C}or(\\mathbb{E}_t[R_{K,t,t+1}],r_t)$",i] <- Cov/sigma_r/st

}

# Prepare LaTeX table: -----
make_entry <- function(x,format.nb){
  output <- paste("$",sprintf(paste("%. ",format.nb,"f",sep=""),x),"$",sep="")
  return(output)
}
format.nb <- 2

latex.table <- NULL
flag <- 0
for(i in 1:dim(all_results)[1]){

```

```

this.line <- rownames(all_results)[i]
for(j in 1:dim(all_results)[2]){
  this.line <-
    paste(this.line,"&",make_entry(multip_factors[i]*all_results[i,j],all_nb[i]),suffix[i],sep="")
}
this.line <-
  paste(this.line,"\\\\",sep="")
latex.table <- rbind(latex.table,
                     this.line)
if(i %in% indic_hline){
  flag <- flag + 1
  if(flag==1){
    latex.table <- rbind(latex.table,
                        "\\hline",
                        "{\\bf Model-implied moments:}\\\\",
                        "\\hline")
  }else{
    latex.table <- rbind(latex.table,
                        "\\hline")
  }
}
}
}
latex.table <- rbind(latex.table,
                    "\\hline")

latex.file <- paste("tables/table_PH_stocks.txt",sep="")
write(latex.table, file = latex.file)

```

This final block explores an additional calibration of the preferred-habitat model.

Purpose: Check the additional preferred-habitat calibration and perpetuity price.

Check perpetuity price:

Phi[3,3] <- 0.5

Sigma <- diag(c(diag(model_VV\$Sigma),0.04^2))

*# Sigma <- .0000001*diag(3)*

Sigma[1:2,1:2] <- model_VV\$Sigma

Sigma[2,2] <- 0

Sigma[1,1] <- 0

Sigma[3,3] <- 0.05

mu_k <- 0

```

model <- list(gamma = a,mu = mu,
             Phi = Phi,
             Sigma = Sigma,
             alpha = Alpha,
             beta = Beta,
             xi = xi,
             a1 = rbind(model_VV$a1,0),b1 = model_VV$b1,
             omega = 0, mu_K0=mu_k, mu_K1=mu_K1,
             beta_K=.3, alpha_K=alpha_K, xi_K=xi_K)

RES <- solve_PH_TSM(model,max_iter = 300)

# Compute perpetuity price "manually":
psi.parameterization.Q <- list(mu = RES$muQ,
                               Phi = RES$PhiQ,
                               Sigma = Sigma)

HH <- 400
loadings.before.t.plus.h <- mu_K1 + matrix(c(-1,0,0),3,1)
aux.loadings <-
  reverse_MHLT(psi_GaussianVAR,
              u1=mu_K1,
              u2=loadings.before.t.plus.h,
              H=HH,
              psi.parameterization.Q)
# Multiply by exp(-r_t):
Loadings <- matrix(c(-1,0,0),3,HH)
Loadings[,1:HH] <- Loadings[,1:HH] + aux.loadings$A[,1:HH]

B <- c(aux.loadings$B) + (1:HH)*(- b1 + mu_k)

# select any potential value for state vector:
X <- matrix(c(0.0,0.02,0),ncol=1)

B <- exp(B + t(Loadings) %*% X)
sum(B)

## [1] 11.74872
exp(RES$mu_zeta0 + t(RES$mu_zeta1)%*%X)

##          [,1]
## [1,] 11.42774

```


Chapter 14

Filtering

Indicate below whether the models' parameterizations are re-estimated (otherwise), use pre-estimated parameters.

```
# Purpose: Show the switch used when re-estimating the filtering examples.  
# Switch to TRUE when re-estimating the filtering examples.  
indic_reestim <- TRUE # if FALSE, then use results in folder /stored_results/
```

14.1 Modified Kalman filter

This example extracts an output gap with the Kalman filter and compares it with standard indicators.

```
# Purpose: This example extracts an output gap with the Kalman filter and compares it with  
# standard indicators  
library(DTAM)  
library(tis) # for NBER recession dates  
library(optimx)  
  
logistic <- function(theta){  
  return(exp(theta)/(1+exp(theta)))  
}  
invlogistic <- function(theta){  
  return(log(theta/(1-theta)))  
}  
  
model2param <- function(model){  
  param <- matrix(0,6,1)  
  log(model$nu) -> param[1]  
  log(model$alpha) -> param[2]
```

```

    log(model$mu) -> param[3]
    invlogistic(model$betamu) -> param[4]
    invlogistic(model$rho) -> param[5]
    log(model$kappa) -> param[6]
    log(model$sigma) -> param[7]
    return(c(param))
}

param2model <- function(param){
  nu <- exp(param[1])
  alpha <- exp(param[2])
  mu <- exp(param[3])
  betamu <- logistic(param[4])
  rho <- logistic(param[5])
  kappa <- exp(param[6])
  # sigma <- .002
  sigma <- exp(param[7])
  return(model=list(nu=nu,alpha=alpha,
                    mu=mu,betamu=betamu,
                    rho=rho,kappa=kappa,
                    sigma=sigma))
}

Qf_ARGO <- function(model,rho,t){
  nu <- model$nu
  alpha <- model$alpha
  mu <- model$mu
  betamu <- model$betamu
  sigma <- model$sigma
  aux <- mu^2*(nu+2*alpha) + 2*mu*betamu*rho[1]
  Q <- matrix(c(aux,-aux,-aux,aux+sigma^2),2,2)
  return(Q)
}

Y <- matrix(
  log(Data_Macro_US_quarterly$GDPC1/Data_Macro_US_quarterly$GDPPOT),
  ncol=1)

model <- list(
  nu = .02,
  alpha = .02,
  mu = .01,
  betamu = .8,
  rho = .6,

```

```

kappa = .03*(1-.6),
sigma = .005
)

param <- model2param(model)

run_KF <- function(model,Y){

  TT <- dim(Y)[1]

  nu_t <- t(matrix(c(model$mu*(model$nu + model$alpha),
                    -model$mu*(model$nu + model$alpha) + model$kappa),2,TT))
  H <- matrix(c(model$betamu,-model$betamu,
                0,model$rho),2,2)
  M <- matrix(0.0000001,1,1)
  rho_0 <- solve(diag(2)-H) %*% nu_t[1,]
  AUX <- Qf_ARGO(model,rho_0)
  Sigma_0 <- matrix(solve(diag(4) - H %x% H) %*% c(AUX),2,2)
  G <- matrix(c(-1,1),1,2)
  mu_t <- matrix(0,TT,1)
  N <- model
  res_KF <- Kalman_filter(Y, nu_t, H, N, mu_t, G, M, Sigma_0, rho_0,
                          indic_pos = c(1,0),
                          Qfunction = Qf_ARGO)

  return(res_KF)
}

loglik4optim <- function(param,Y){
  model <- param2model(param)
  res_KF <- run_KF(model,Y)
  return(-res_KF$loglik)
}

param_est <- model2param(model)

if(indic_reestim){

  nb_loop <- 2

  for(i in 1:nb_loop){
    print(paste("--- Loop ",i," ---",sep=""))
    # for(method in c("nlsminb")){
    for(method in c("nlsminb","Nelder-Mead")){

```

```

    res.optim <- optimx(par=param_est,
                       fn=loglik4optim,
                       Y=Y,
                       method=method,
                       control=list(trace=TRUE,
                                    maxit=ifelse(method=="Nelder-Mead",1000,20),
                                    kkt = FALSE))
    print(paste("--- Log-lik (" ,method,") : ",round(res.optim$value,2)," ---",sep=""))
    param_est <- c(as.matrix(res.optim)[1:length(param_est)])
  }
}
save(param_est,file= "stored_results/param_ARGO.Rdat")
}else{
  # In that case, use parameters estimated previously:
  load(file= "stored_results/param_ARGO.Rdat")
}

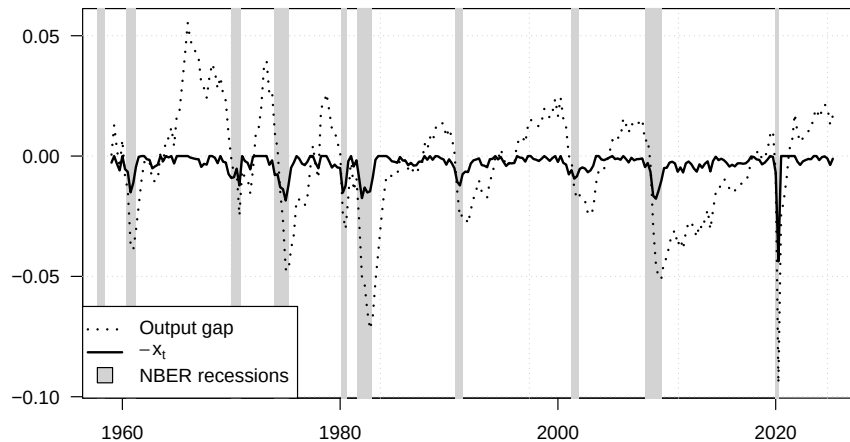
model <- param2model(param_est)
res_KF <- run_KF(model,Y)

w1 <- res_KF$r[,1]
stdv_w1 <- sqrt(res_KF$Sigma_tt[,1])
lower_bound <- -w1 - 1.64*stdv_w1
upper_bound <- -w1 + 1.64*stdv_w1

par(plt=c(.1,.95,.1,.95))
plot(Data_Macro_US_quarterly$date,Y,type="l",col="white",las=1,
     xlab="",ylab="")
grid()
make_recessions(col = "lightgrey")
# polygon(c(Data_Macro_US_quarterly$date,rev(Data_Macro_US_quarterly$date)),
#         c(lower_bound,rev(upper_bound)),col="grey",border = NaN)
lines(Data_Macro_US_quarterly$date,Y,lwd=2,col="black",lty=3)
lines(Data_Macro_US_quarterly$date,
      -w1,col="black",lwd=2)
legend("bottomleft",
      legend=c("Output gap",
               expression(-x[t]),
               "NBER recessions"),
      lty=c(3, 1, NA),          # line type
      pch=c(NA, NA, 22),        # filled squares
      pt.bg = c(NA, NA, "lightgrey"), # fill
      pt.cex = 2,
      col = c("black", "black", "black"), # border
      lwd = c(2,2,1),

```

```
bg = "white")
```



14.2 Kalman filter

The following figure illustrates the updating step of the Kalman algorithm:

14.2.1 The mechanics of the linear Kalman filter in a simple state-space model

This example illustrates the mechanics of the linear Kalman filter in a simple state-space model.

```
# Purpose: This example illustrates the mechanics of the linear Kalman filter in a simple
# state-space model
# library(DTAM)

rho <- .8
var <- .4
xhat <- NaN
sigma <- 2
Sigma <- sigma*matrix(c(1,rho,rho,1),2,2)

# Generate a bivariate cloud used to draw the filtering geometry.
library(cluster)
x <- 2*rnorm(1000)
y <- rnorm(1000) + 2*x + 10
xy <- unname(cbind(x, y))
exy <- ellipsoidhull(xy)

rho <- .85
```

```

coef <- .85
exy$loc <- c(2,2)

par(mfrow= c(2,1))
layout(matrix(c(1,2)), heights=c(2, 2))

par(plt=c(.05,.95,.1,.85))

x <- seq(-2,6,by=.05)

# Draw the predictive density before observing the new signal.
plot(0,0,col='white',
     xlim=c(-2,6),
     ylim=c(0,1.5),ylab="",xlab="",xaxt="n",yaxt="n",
     main=expression(paste("(a) Conditional distribution of ", w[t],
                           ", before and after observing ", y[t], sep = "")))

sigma <- 1
Sigma <- sigma*matrix(c(1,rho,rho,1),2,2)
sigmax <- sqrt(Sigma[1,1])
Ex <- 2
x <- seq(-2,6,by=.05)
densx <- 1/sqrt(2*pi*sigmax^2)*exp(-.5*(x-Ex)^2/sigmax^2)
lines(x,densx,col='black',lwd=2)
abline(v=exy$loc[2],lty=2)

arrows(1,.73,1.5,.36,length=.1,col="black")
text(.3,.8,expression(paste("Predictive density ",w[t],"|",y[t-1],"",y[t-2],"",...,"s

abline(v=xhat,lty=2,col="dark grey")

sigmax <- sqrt(var)
y <- 3.8
xhat <- 2 + coef*(y-2)
x <- seq(-2,6,by=.05)
densx <- 1/sqrt(2*pi*sigmax^2)*exp(-.5*(x-xhat)^2/sigmax^2)

# Add the updated density after observing the signal.
lines(x,densx,lwd=2,col="dark grey")

arrows(5,.63,4.5,.2,length=.1,col="dark grey")
text(4,.7,expression(paste("Updated density ",w[t],"|",y[t],"",y[t-1],"",...,"sep=")))

abline(v=xhat,lty=2,col="dark grey")

```

```

text(x = Ex, y = 1.5,
     labels = expression(paste(E, "(", w[t], "|",
                               y[t-1], ",", y[t-2], ",...)", sep = "")),
     pos = 2, srt = 90)
text(x = xhat, y = 1.5,
     labels = expression(paste(E, "(", w[t], "|",
                               y[t], ",", y[t-1], ",...)", sep = "")),
     pos = 2, srt = 90, col = "dark grey")

par(plt=c(.05,.95,.1,.85))

x <- seq(-2,6,by=.05)

sigma0 <- .5
Sigma <- sigma*matrix(c(1,rho,rho,1),2,2)

plot(0,0,col="white",xlim=c(-2,6),ylim = c(-2,7),lwd=2,xlab="",
     ylab="",xaxt="n",yaxt="n",
     main=expression(paste("(b) Determination of ",E,"(",w[t], "|",y[t], ",", y[t-1], ",...)",sep="")),
     mtext(expression(w[t]),side=1,line=1)
     mtext(expression(y[t]),side=2,line=1)

sigma <- 1
Sigma <- sigma*matrix(c(1,rho,rho,1),2,2)
exy$cov <- Sigma
lines(predict(exy),lwd=2)

abline(h=exy$loc[1],lty=2)
abline(v=exy$loc[2],lty=2)
sigma <- 2
Sigma <- sigma*matrix(c(1,rho,rho,1),2,2)
exy$cov <- Sigma
lines(predict(exy),type='l',lwd=2)
sigma <- 3.5
Sigma <- sigma*matrix(c(1,rho,rho,1),2,2)
exy$cov <- Sigma
lines(predict(exy),type='l',lwd=2)

arrows(0,5,1.2,3,length=.1)
text(-.7,6.2,"Contour (iso-density) of")
text(-.7,5.5,expression(paste("(",w[t], ",", y[t], ") ", "| ",y[t-1], ",", y[t-2], ",", "...",sep="")))

sigma <- 1
Sigma <- sigma*matrix(c(1,rho,rho,1),2,2)

```

```

coef <- Sigma[2,1]*(1/Sigma[1,1])
var <- Sigma[2,2] - Sigma[2,1]*(1/Sigma[1,1])*Sigma[1,2]

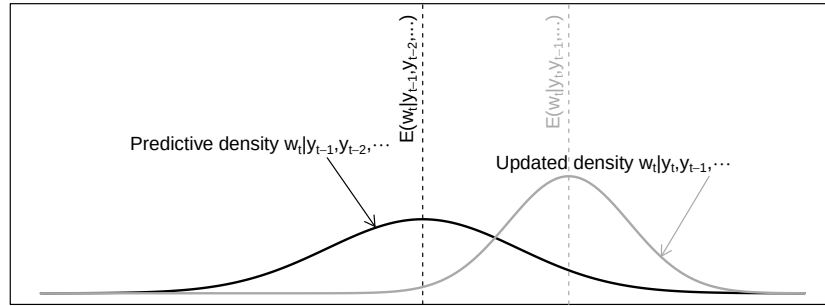
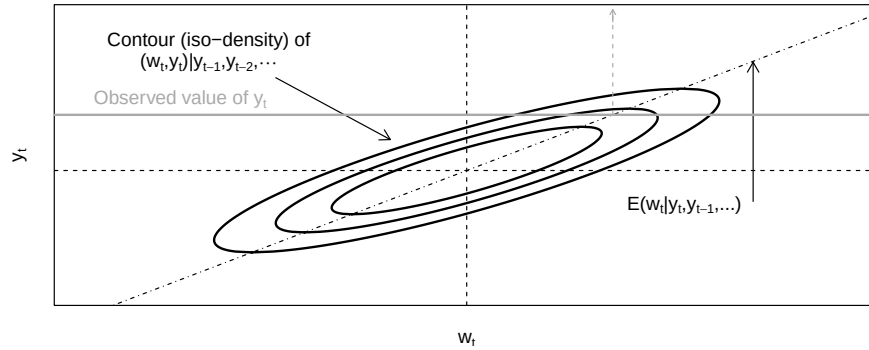
Y <- seq(-10,10,by=.05)

lines(2+coef*(Y-2),Y,col="black",lwd=1,lty=4)
arrows(5,1,5,5.5,length=.1,col="black")
text(5, .8,
      expression(paste(E, "(", w[t], "|",
                        y[t], ",", y[t-1], "...)", sep = "")),
      pos = 2, col = "black")

abline(h=y,col="dark grey",lwd=2)
text(-1,y+.5,expression(paste("Observed value of ",y[t],sep="")),col="dark grey")

lines(c(xhat,xhat),c(y,7),col="dark grey",lty=2)
arrows(xhat,6.8,xhat,7.2,col="dark grey",length=.05)

```

(a) Conditional distribution of w_t , before and after observing y_t (b) Determination of $E(w_t|y_t, y_{t-1}, ...)$ 

14.2.2 Filtered and smoothed estimates in a multivariate Gaussian system

This example shows filtered and smoothed estimates in a multivariate Gaussian state-space system.

Purpose: This example shows filtered and smoothed estimates in a multivariate Gaussian state-space system

```
library(DTAM)

set.seed(123)

# Set model specifications:
alpha1 <- 0 # if we had autoregressive y_1
alpha2 <- 0 # if we had autoregressive y_2
Alpha <- diag(c(alpha1,alpha2))
sigma1 <- 4
sigma2 <- 4
D <- matrix(c(sigma1,0,0,sigma2),2,2)
beta1 <- 1
beta2 <- 1
Gamma <- matrix(c(beta1,beta2),2,1)
phi <- .8

# Simulate model:
T <- 100
Y <- NULL;X <- NULL
Alpha.Y_1 <- NULL
y <- c(0,0);x <- 0
for(i in 1:T){
  Alpha.Y_1 <- rbind(Alpha.Y_1,c(Alpha %*% y))
  y <- Alpha %*% y + Gamma * x + D %*% rnorm(2)
  x <- phi * x + rnorm(1)
  Y <- rbind(Y,t(y));X <- rbind(X,x)
}

# Define matrices needed in the Kalman_filter procedures:
nu_t <- matrix(0,T,1)
H <- phi
G <- Gamma
mu_t <- Alpha.Y_1
N <- 1
M <- D
Sigma_0 <- 1/(1-phi^2) # unconditional variance of w
rho_0 <- 0 # unconditional mean of w
```

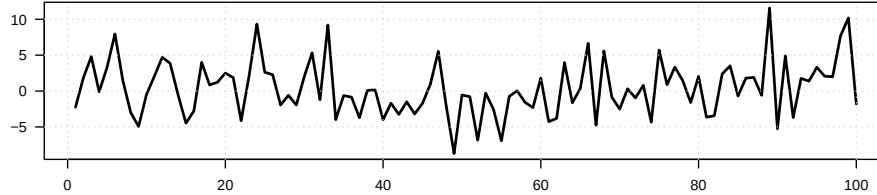
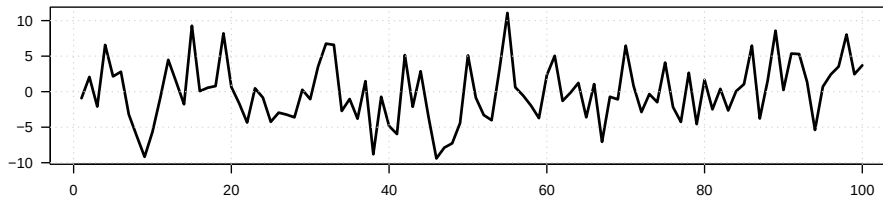
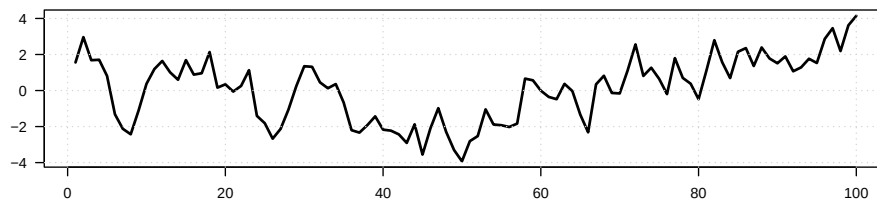
```

filter.res <- Kalman_filter(Y,nu_t,H,N,mu_t,G,M,Sigma_0,rho_0)
smoother.res <- Kalman_smoother(Y,nu_t,H,N,mu_t,G,M,Sigma_0,rho_0)

par(mfrow=c(3,1))
par(plt=c(.05,.97,.2,.8))

par(mfrow=c(3,1))
plot(Y[,1],type="l",lwd=2,xlab="",ylab="",las=1,
     main=expression(paste("(a) First observed variable (",y[1*],"*t],")",sep="")))
grid()
plot(Y[,2],type="l",lwd=2,xlab="",ylab="",las=1,
     main=expression(paste("(b) Second observed variable (",y[2*],"*t],")",sep="")))
grid()
plot(X,type="l",lwd=2,xlab="",ylab="",las=1,
     main=expression(paste("(c) Latent variable (",w[t],")",sep="")))
grid()

```

(a) First observed variable ($y_{1,t}$)(b) Second observed variable ($y_{2,t}$)(c) Latent variable (w_t)

14.2.3 Latent-state recovery under alternative measurement equations

This example studies latent-state recovery under alternative measurement equations.

```
# Purpose: This example studies latent-state recovery under alternative measurement equations
# Plot the observed signals.
par(mfrow=c(3,1))
par(plt=c(.05,.97,.2,.8))

plot(Y[,1],type="l",lwd=2,xlab="",ylab="",las=1,
      main=expression(paste("(a) Observations of ",
                             y[1*],"*t", " and ",y[2*],"*t",sep="")),
      ylim=c(-10,10))
grid()
lines(Y[,2],type="l",lwd=2,lty=3)

legend("topleft",
      c(expression(paste(y[1*],"*t",sep="")),
        expression(paste(y[2*],"*t",sep=""))),
      lwd=c(2), # line width
      lty=c(1,3),
      bg = "white")

# lower_bound <- filter.res$r - 2*sqrt(filter.res$Sigma_tt)
# upper_bound <- filter.res$r + 2*sqrt(filter.res$Sigma_tt)
lower_bound <- smoother.res$r - 2*sqrt(smoother.res$S_smooth)
upper_bound <- smoother.res$r + 2*sqrt(smoother.res$S_smooth)

# Plot true, filtered, and smoothed states with uncertainty bands.
plot(X,type="l",lwd=2,xlab="",ylab="",col="white",las=1,
      ylim=c(min(lower_bound),1.6*max(upper_bound)),
      main=expression(paste("(b) Filtered and smoothed estimates of ",w[t],sep="")))
grid()

polygon(c(1:T,T:1),c(lower_bound,rev(upper_bound)),col="#88888844",border = NaN)

lines(X,lwd=2,col="dark grey",lty=1)
lines(filter.res$r,col="black",lwd=2,lty=2)
lines(smoother.res$r,col="black",lwd=2,lty=3)

legend("topleft",
      c(expression(w[t]),
        expression(paste("Filtered values of ",w[t],sep=""))),
```

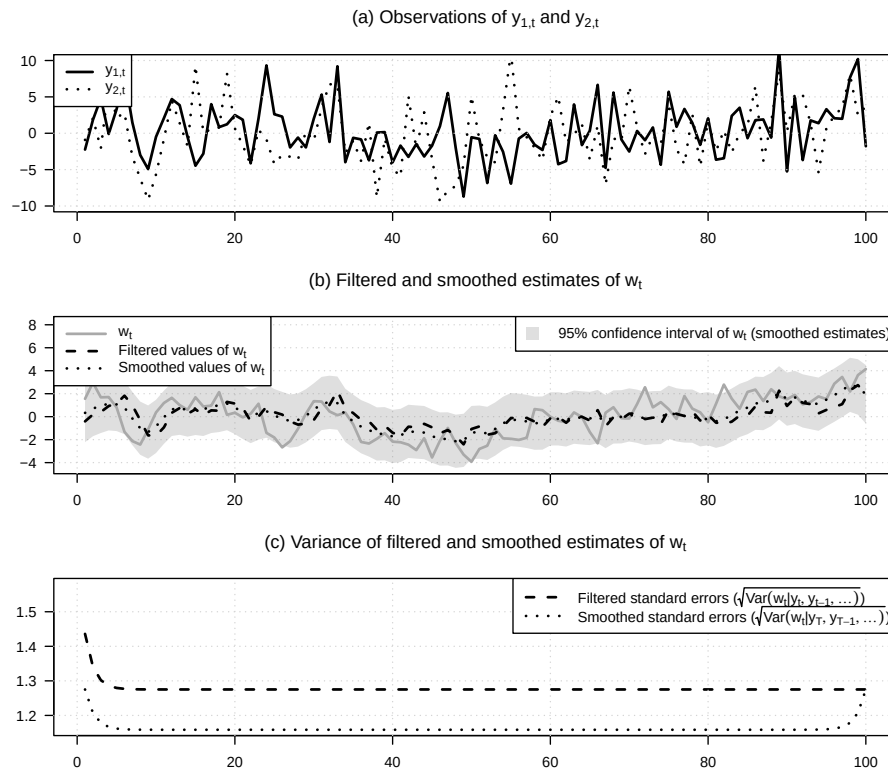
```

        expression(paste("Smoothed values of ",w[t],sep="")),
        lwd=c(2), # line width
        lty=c(1,2,3),
        col=c("dark grey","black","black"), # gives the legend lines the correct color
        seg.len = 3,
        bg = "white"
    )
    legend("topright",
        c(expression(paste("95% confidence interval of ",w[t]," (smoothed estimates)",s
        lwd=c(NaN), # line width
        lty=c(NaN),
        pch=15,
        pt.cex=2,
        col=c(col="#88888844"),
        bg = "white"
    )

# Compare filtered and smoothed standard errors.
plot(sqrt(filter.res$Sigma_tt),type="l",lty=2,
     lwd=2,xlab="",ylab="",col="black",
     ylim=c(min(sqrt(smoother.res$S_smooth)),1.1*max(sqrt(filter.res$Sigma_tt))),las=1,
     main=expression(paste("(c) Variance of filtered and smoothed estimates of ",w[t],
grid()
lines(sqrt(smoother.res$S_smooth),lwd=2,col="black",lty=3)

legend("topright",
     c(expression(paste("Filtered standard errors (",
                        sqrt(Var(w[t]*"|"*y[t], y[t-1], ...)), ")"), sep = "")),
     expression(paste("Smoothed standard errors (",
                        sqrt(Var(w[t]*"|"*y[T], y[T-1], ...)), ")"), sep = ""))
    ),
    lwd=c(2), # line width
    lty=c(2,3),
    col=c("black","black"),
    seg.len = 3,
    bg = "white"
)

```



14.2.4 A richer affine term-structure filtering exercise

This example illustrates a richer affine term-structure filtering exercise.

Purpose: This example illustrates a richer affine term-structure filtering exercise
Introduce missing observations in both measurement equations.

```
Y.modif <- Y
Y.modif[30:50,1] <- NaN
Y.modif[40:70,2] <- NaN

# Call of Kalman filter and smoother:
filter.res <- Kalman_filter(Y.modif,nu_t,H,N,mu_t,
                           G,M,Sigma_0,rho_0)
smoother.res <- Kalman_smoother(Y.modif,nu_t,H,N,mu_t,
                               G,M,Sigma_0,rho_0)
```

Plot observations, recovered states, and uncertainty under missing data.
`par(mfrow=c(3,1))`
`par(plt=c(.05,.97,.13,.8))`

```

plot(Y.modif[,1],type="l",lwd=2,xlab="",ylab="",las=1,
     main=expression(paste("(a) Observations of ",
                           y[1*,"*t"]," and ",y[2*,"*t"],sep="")),
     ylim=c(-10,10))

grid()
lines(Y.modif[,2],type="l",lwd=2,lty=3)

legend("topleft",
      c(expression(paste(y[1*,"*t"],sep="")),
        expression(paste(y[2*,"*t"],sep="))),
      lwd=c(2), # line width
      lty=c(1,3),
      bg = "white")

# lower_bound <- filter.res$r - 2*sqrt(filter.res$Sigma_tt)
# upper_bound <- filter.res$r + 2*sqrt(filter.res$Sigma_tt)
lower_bound <- smoother.res$r - 2*sqrt(smoother.res$S_smooth)
upper_bound <- smoother.res$r + 2*sqrt(smoother.res$S_smooth)

plot(X,type="l",lwd=2,xlab="",ylab="",col="white",las=1,
     ylim=c(min(lower_bound),1.6*max(upper_bound)),
     main=expression(paste("(b) Filtered and smoothed estimates of ",w[t],sep=")))
grid()

polygon(c(1:T,T:1),c(lower_bound,rev(upper_bound)),col="#88888844",border = NaN)

lines(X,lwd=2,col="dark grey",lty=1)
lines(filter.res$r,col="black",lwd=2,lty=2)
lines(smoother.res$r,col="black",lwd=2,lty=3)

legend("topleft",
      c(expression(w[t]),
        expression(paste("Filtered values of ",w[t],sep="")),
        expression(paste("Smoothed values of ",w[t],sep="))),
      lwd=c(2), # line width
      lty=c(1,2,3),
      col=c("dark grey","black","black"), # gives the legend lines the correct color
      seg.len = 3,
      bg = "white"
)
legend("topright",
      c(expression(paste("95% confidence interval of ",w[t]," (smoothed estimates)",s
      lwd=c(NaN), # line width

```

```

        lty=c(NaN),
        pch=15,
        pt.cex=2,
        col=c(col="#88888844"),
        bg = "white"
    )

plot(sqrt(filter.res$Sigma_tt),type="l",lty=2,
     lwd=2,xlab="",ylab="",col="black",
     ylim=c(min(sqrt(smoother.res$S_smooth)),1.1*max(sqrt(filter.res$Sigma_tt))),las=1,
     main=expression(paste("(c) Variance of filtered and smoothed estimates of ",w[t],sep="")))
grid()
lines(sqrt(smoother.res$S_smooth),lwd=2,col="black",lty=3)

legend("topright",
      c(expression(paste("Filtered standard errors (",
                          sqrt(Var(w[t]*"|"*y[t], y[t-1], ...)), ")", sep = "")),
        expression(paste("Smoothed standard errors (",
                          sqrt(Var(w[t]*"|"*y[T], y[T-1], ...)), ")", sep = ""))
      ),
      lwd=c(2), # line width
      lty=c(2,3),
      col=c("black","black"),
      seg.len = 3,
      bg = "white"
    )

```

The figure displays two time series, $y_{1,t}$ and $y_{2,t}$, plotted against time t from 0 to 100. The vertical axis represents the values of the series, ranging from -10 to 10. The horizontal axis represents the time index t . The solid line represents $y_{1,t}$ and the dotted line represents $y_{2,t}$. Both series exhibit high volatility and are highly correlated, with significant peaks and troughs occurring at similar times. The plot is divided into two segments by a gap in the x-axis between approximately 40 and 55.

Figure 1 is a line plot showing the standard errors of the estimated parameters of the fitted model. The x-axis represents the index of the parameter, ranging from 0 to 100. The y-axis represents the standard error, ranging from 1.2 to 1.8. Two lines are plotted: a dashed line for 'Filtered standard errors' and a dotted line for 'Smoothed standard errors'. Both lines show a similar pattern, with a peak around index 45 and a dip around index 75. The smoothed standard errors are generally lower than the filtered standard errors.

This example compares quadratic filtering with the linear Gaussian benchmark.

Purpose: This example compares quadratic filtering with the linear Gaussian benchmark

[illegible]


```

data$infl <- infl
Y_t <- matrix(data$infl[(k+1):TT],ncol=1)
vec_dates <- data$date[(k+1):TT]

make_StateSpace <- function(param){
  c <- exp(param[3])
  b <- param[2]
  a <- -5 + b^2/(4*c) - exp(param[1])
  alpha <- param[4]
  beta <- param[5]
  rho <- exp(param[6])/(1+exp(param[6]))

  A <- a
  B <- matrix(c(b,alpha),nrow=1)
  C <- array(c(c,beta,0,0),c(2,2,1))
  mu = matrix(0,2,1)
  Phi <- diag(c(rho,0))
  Sigma <- diag(2)
  M <- .00001

  return(list(A=A,B=B,C=C,mu=mu,Phi=Phi,Sigma=Sigma,M=M))
}

rho <- .8
stdv_tau <- 1/sqrt(1-rho^2)
b <- sd(Y_t)/stdv_tau
c <- 0.03
a <- -5 + b^2/(4*c) - 0.5
alpha <- .3*sd(Y_t)
beta <- 0.1
param <- c(log(-5 + b^2/(4*c) - a),b,log(c),alpha,beta,log(rho/(1-rho)))

loglik_QKF <- function(param,Y_t=Y_t){
  QStateSpace <- make_StateSpace(param)
  resQKF <- QKF(Y_t,QStateSpace)
  return(-resQKF$loglik)
}

if(indic_reestim){

  nb_loop <- 3

  for(i in 1:nb_loop){
    print(paste("--- Loop ",i," ---",sep=""))
    for(method in c("Nelder-Mead","nlminb")){

```

```

    res.optim <- optimx(par=param,
                       fn=loglik_QKF,
                       Y_t = Y_t,
                       method=method,
                       control=list(trace=TRUE,
                                    maxit=ifelse(method=="Nelder-Mead",300,30),
                                    kkt = FALSE))
    print(paste("--- Log-lik (" ,method,") : ",round(res.optim$value,2)," ---",sep=""))
    param <- c(as.matrix(res.optim)[1:length(param)])
  }
}
param_est <- param
save(param,
      file=paste("stored_results/param_QKF.Rdat",sep=""))
}else{
  load(file=paste("stored_results/param_QKF.Rdat",sep=""))
}

QStateSpace <- make_StateSpace(param)
resQKF <- QKF(Y_t,QStateSpace,indic_reconciliation = TRUE)

par(mfrow=c(2,2))
par(plt=c(.1,.95,.1,.8))
plot(vec_dates,Y_t,col="white",xlab="",ylab="",las=1,
      ylim=c(range(Y_t)-c(2,0)),
      main=expression(paste("(a) Inflation and filtered persistent component",sep="")))
grid()
lines(vec_dates,Y_t,col="grey",lwd=2)
legend("bottomleft",
      legend=c(expression(paste("Inflation (",pi[t],")",sep="")),
                expression(paste("Persistent component (a+b",tau[t], " + c",tau[t]^2,"))),
      lty=c(1,3),
      col=c("dark grey","black"),
      lwd=c(2,2),bty = "n")

n <- 2
vec_b_x_tau <- matrix(0,n+n*(n+1)/2,1)
vec_b_x_tau[1] <- resQKF$B_tilde[1]
vec_c_x_tau2 <- matrix(0,n+n*(n+1)/2,1)
vec_c_x_tau2[3] <- resQKF$B_tilde[3]
b_x_tau <- resQKF$r %*% vec_b_x_tau
c_x_tau2 <- resQKF$r %*% vec_c_x_tau2
fitted <- resQKF$A + b_x_tau + c_x_tau2
lines(vec_dates,fitted,col="black",lty=3,lwd=2)

```

```

plot(vec_dates,Y_t,col="white",xlab="",ylab="",las=1,
     ylim=range(c(resQKF$r[,1],resQKF$r[,3]),finite=TRUE),
     main=expression(paste("(b) Estimates of ",tau[t]," and ",tau[t]^2,sep="")))
grid()
stdv_b_x_tau <- sqrt(resQKF$Sigma_tt %*% (vec_b_x_tau %x% vec_b_x_tau))
stdv_c_x_tau2 <- sqrt(resQKF$Sigma_tt %*% (vec_c_x_tau2 %x% vec_c_x_tau2))

# Confidence interval for b x tau:
# lower_bound <- b_x_tau - qnorm(.975)*stdv_b_x_tau
# upper_bound <- b_x_tau + qnorm(.975)*stdv_b_x_tau
# polygon(c(vec_dates,rev(vec_dates)),c(lower_bound,rev(upper_bound)),
#         border = NA, col="#33333344")
# Confidence interval for c x tau^2:
# lower_bound <- c_x_tau2 - qnorm(.975)*stdv_c_x_tau2
# upper_bound <- c_x_tau2 + qnorm(.975)*stdv_c_x_tau2
# polygon(c(vec_dates,rev(vec_dates)),c(lower_bound,rev(upper_bound)),
#         border = NA, col="#33333344")

lines(vec_dates,resQKF$r[,1],col="black",lwd=2)
lines(vec_dates,resQKF$r[,3],col="black",lwd=2,lty=3)
legend("topleft",
      legend=c(expression(tau[t]),
                    expression(tau[t]^2)),
      lty=c(1,3),
      col=c("black","black"),
      lwd=c(2,2),bty = "n")

pi_tp1_t <- c(resQKF$A) + resQKF$r %*% t(resQKF$Phi_tilde_red) %*% t(resQKF$B_tilde)
plot(vec_dates,Y_t,type="l",col="grey",lty=3,lwd=2,xlab="",ylab="",las=1,
     main=expression(paste("(c) Forecasted inflation",sep="")))
grid()
lines(vec_dates,pi_tp1_t,col="black",lwd=2)
legend("topright",
      legend=c(expression(pi[t]),
                    expression(paste("E(",pi[t+1],"|",underline(pi[t]),")",sep="))),
      lty=c(3,1),
      col=c("darkgrey","black"),
      lwd=c(2,2),bty = "n")

# First component of conditional variance of pi_{t+1} ('epsilon' uncertainty):
P_tp1_t <- resQKF$r %*% t(resQKF$Psi_red) +
  matrix(1,dim(Y_t)[1],1) %*% t(resQKF$nu_red)
var_pi_epsilon <- apply((matrix(1,dim(Y_t)[1],1) %*% (resQKF$B_tilde %x% resQKF$B_tilde)) *
  P_tp1_t,1,sum)

```

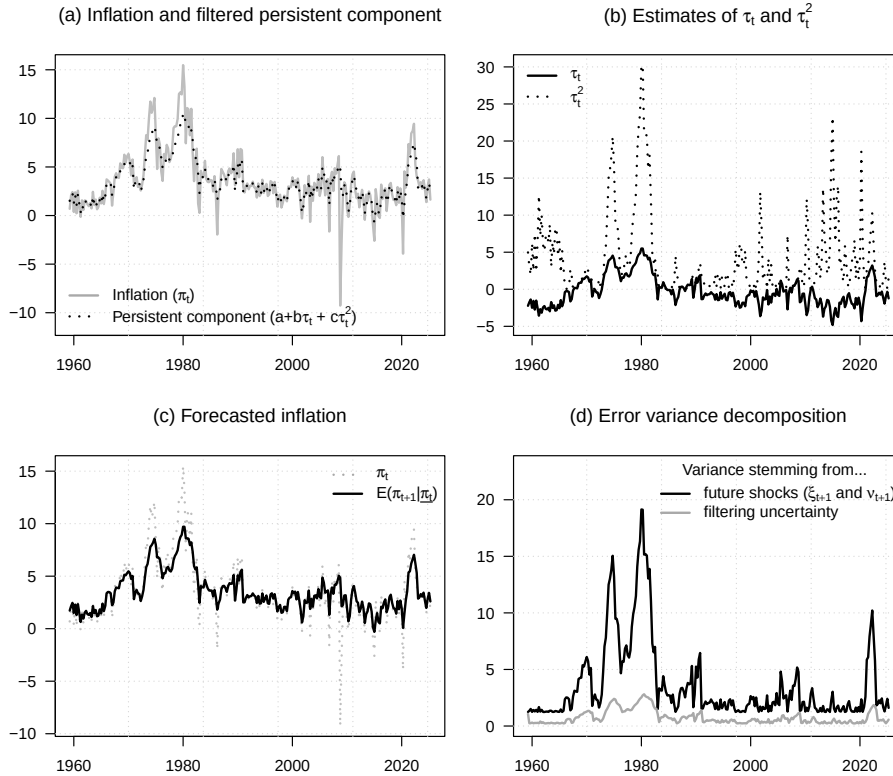
```

# Second component of conditional variance of  $\pi_{t+1}$  (filtering uncertainty)
B_Phi <- resQKF$B_tilde%%resQKF$Phi_tilde_red
var_pi_filtering_uncert <-
  apply((matrix(1,dim(Y_t)[1],1) %% (B_Phi %x% B_Phi)) * resQKF$Sigma_tt,1,sum)

plot(vec_dates,var_pi_epsilon,ylim=c(0,1.2*max(var_pi_epsilon)),
     col="white",xlab="",ylab="",las=1,
     main=expression(paste("(d) Error variance decomposition",sep="")))

grid()
lines(vec_dates,var_pi_epsilon,lwd=2)
lines(vec_dates,var_pi_filtering_uncert,col="darkgrey",lwd=2)
legend("topright",
      title="Variance stemming from...",
      legend=c(expression(paste("future shocks (",xi[t+1]," and ",nu[t+1],")",sep="")),
                expression(paste("filtering uncertainty",sep="))),
      lty=c(1,1),
      col=c("black","darkgrey"),
      lwd=c(2,2),bty = "n")

```



14.4 Kitagawa-Hamilton filter

This example reproduces the Hamilton-style regime-switching filtering exercise.

```
# Purpose: This example reproduces the Hamilton-style regime-switching filtering exercise
library(DTAM)
library(tis) # for NBER recession dates
library(optimx)

first.date <- Data_Macro_US_quarterly$date[1]
last.date  <- "2015-10-01"

# Build the GDP-growth data used in the Hamilton example.
Data_Macro_US_quarterly <- Data_Macro_US_quarterly[
  (Data_Macro_US_quarterly$date >= first.date)&
  (Data_Macro_US_quarterly$date <= last.date),]
TT <- dim(Data_Macro_US_quarterly)

logGDP <- log(Data_Macro_US_quarterly$GDPC1)
dy     <- logGDP[6:(TT-0)] - logGDP[5:(TT-1)]
dy_1   <- logGDP[5:(TT-1)] - logGDP[4:(TT-2)]
dy_2   <- logGDP[4:(TT-2)] - logGDP[3:(TT-3)]
dy_3   <- logGDP[3:(TT-3)] - logGDP[2:(TT-4)]

vec_dates <- Data_Macro_US_quarterly$date[6:TT]

Y <- 100*cbind(dy,dy_1,dy_2,dy_3)

logistic <- function(theta){
  return(exp(theta)/(1+exp(theta)))
}
invlogistic <- function(theta){
  return(log(theta/(1-theta)))
}

# Map constrained model parameters to an unconstrained optimizer vector.
model2param <- function(model){
  param <- matrix(0,8,1)
  model$phi1 -> param[1]
  model$phi2 -> param[2]
  model$phi3 -> param[3]
  model$mu2  -> param[4]
  log(model$mu2 - model$mu1) -> param[5]
  invlogistic(model$pi11) -> param[6]
  invlogistic(model$pi22) -> param[7]
  log(model$sigma) -> param[8]
```

```

    return(c(param))
}

# Map optimizer parameters back to the regime-switching model.
param2model <- function(param){
  model <- list()
  model$phi1 <- param[1]
  model$phi2 <- param[2]
  model$phi3 <- param[3]
  model$mu2 <- param[4]
  model$mu1 <- model$mu2 - exp(param[5])
  model$pi11 <- logistic(param[6])
  model$pi22 <- logistic(param[7])
  model$sigma <- exp(param[8])

  Pi <- matrix(c(model$pi11,0,0,1-model$pi22,
                 1-model$pi11,0,0,model$pi22),2,4) %x% (diag(4) %x% matrix(1,2,1))
  model$Pi <- Pi
  model$mu <- matrix(c(model$mu1,model$mu2),2,1) %x% matrix(1,8,1)

  return(model)
}

# Run the Kim-Hamilton filter, and optionally the smoother.
run_KH <- function(model,Y,indic_smoother=FALSE){

  TT <- dim(Y)[1]
  J <- 16 # number of regimes
  vec1TT <- matrix(1,TT,1)

  mu <- matrix(c(model$mu1,model$mu2),2,1)

  mu0 <- vec1TT %*% t(mu %x% matrix(1,2,1) %x% matrix(1,2,1) %x% matrix(1,2,1))
  mu1 <- vec1TT %*% t(matrix(1,2,1) %x% mu %x% matrix(1,2,1) %x% matrix(1,2,1))
  mu2 <- vec1TT %*% t(matrix(1,2,1) %x% matrix(1,2,1) %x% mu %x% matrix(1,2,1))
  mu3 <- vec1TT %*% t(matrix(1,2,1) %x% matrix(1,2,1) %x% matrix(1,2,1) %x% mu)

  y0 <- matrix(Y[,1],TT,J)
  y1 <- matrix(Y[,2],TT,J)
  y2 <- matrix(Y[,3],TT,J)
  y3 <- matrix(Y[,4],TT,J)

  phi1 <- model$phi1
  phi2 <- model$phi2
  phi3 <- model$phi3

```

```

eps <- y0 - mu0 - phi1*(y1 - mu1) - phi2*(y2 - mu2) - phi3*(y3 - mu3)

sigma <- model$sigma
Eta <- -.5*log(2*pi*sigma^2) - .5*eps^2/sigma^2

resKH <- KH_filter(model$Pi, Eta)

if(indic_smoother){
  resKH$ksi_matrix_smoothed <- KH_smoother(model$Pi, Eta)
}

return(resKH)
}

loglik4optim <- function(param,Y){
  model <- param2model(param)
  resKH <- run_KH(model,Y)
  return(-resKH$loglik)
}

# Initial param (close to Hamilton, 1989)
model <- list(mu1=-.5,mu2=.9,
             phi1=.27,phi2=.34,phi3=.13,
             sigma=.9,
             pi11=.9,pi22=.98)
param_est <- model2param(model)

param_est <- model2param(model)

if(indic_reestim){

  nb_loop <- 4

  for(i in 1:nb_loop){
    print(paste("--- Loop ",i," ---",sep=""))
    # for(method in c("nlminb")){
    for(method in c("nlminb","Nelder-Mead")){
      res.optim <- optimx(par=param_est,
                        fn=loglik4optim,
                        Y=Y,
                        method=method,
                        control=list(trace=FALSE,
                                   maxit=ifelse(method=="Nelder-Mead",1000,20),
                                   kkt = FALSE))
      print(paste("--- Log-lik (",method,"): ",round(res.optim$value,2)," ---",sep=""))
    }
  }
}

```

```

    param_est <- c(as.matrix(res.optim)[1:length(param_est)])
  }
}
save(param_est,file= "stored_results/param_Hamilton89.Rdat")
}else{
  # In that case, use parameters estimated previously:
  load(file= "stored_results/param_Hamilton89.Rdat")
}

model <- param2model(param_est)
resKH <- run_KH(model,Y,indic_smoother=TRUE)

# Compute filtered probabilities xi_tilde:
xi_tilde <- resKH$ksi_matrix %*% t(diag(2) %x% matrix(1,1,8))
xi_tilde_smoothed <- resKH$ksi_matrix_smoothed %*% t(diag(2) %x% matrix(1,1,8))

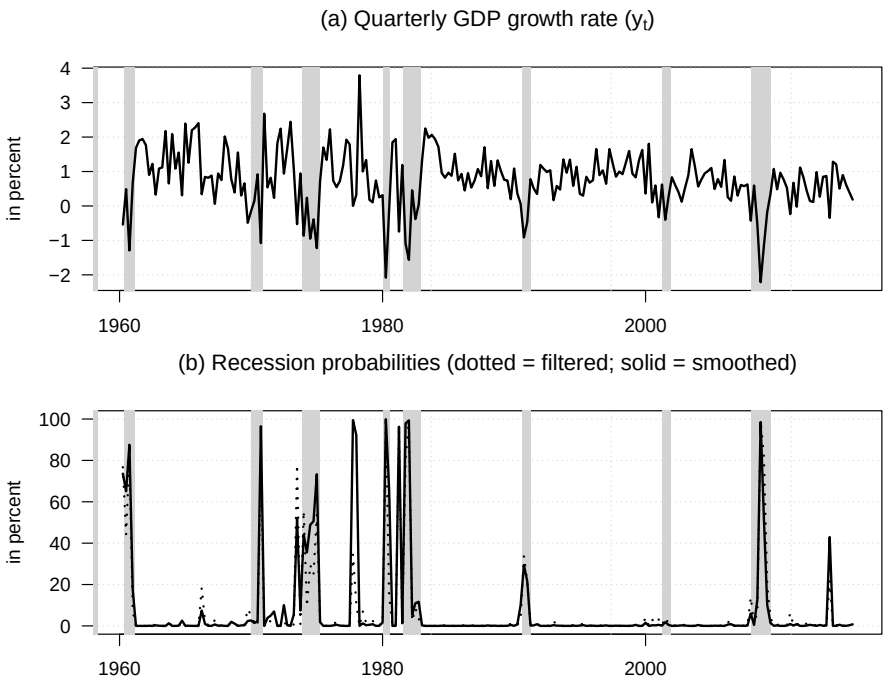
par(mfrow=c(2,1))
par(plt=c(.1,.96,.15,.8))

plot(vec_dates,Y[,1],col="white",
     xlab="",ylab="in percent",las=1,
     main=expression(paste("(a) Quarterly GDP growth rate (",y[t],")",sep="")))
grid()
make_recessions(col = "lightgrey")
lines(vec_dates,Y[,1],lwd=2)

plot(vec_dates,100*xi_tilde[,1],ylim=c(0,100),col="white",las=1,
     xlab="",ylab="in percent",
     main=expression(paste("(b) Recession probabilities",
                           " (dotted = filtered; solid = smoothed)", sep = "")))

grid()
make_recessions(col = "lightgrey")
lines(vec_dates,100*xi_tilde_smoothed[,1],lwd=2,col="black")
lines(vec_dates,100*xi_tilde[,1],lwd=2,col="black",lty=3)

```

Chapter 15

Estimation

15.1 Maximum likelihood estimation with observed variables

Indicate below whether the models' parameterizations are re-estimated (otherwise), use pre-estimated parameters.

```
# Purpose: Show the switch used when re-estimating the estimation examples.  
# Switch to TRUE when re-estimating the examples.  
indic_reestim <- TRUE # if FALSE, then use results in folder /stored_results/
```

15.1.1 The likelihood surface and basic ingredients of maximum-likelihood estimation

This example introduces the likelihood surface and basic ingredients of maximum-likelihood estimation.

```
# Purpose: This example introduces the likelihood surface and basic ingredients of  
# maximum-likelihood estimation  
library(DTAM)  
library(lubridate) # to handle dates  
library(vars)  
library(optimx)  
  
# =====  
# Prepare dataset  
# -----  
  
data("SPF", package = "DTAM")  
data("Data_Macro_US_quarterly", package = "DTAM")
```

```

data("YC_LW", package = "DTAM")

DATA <- merge(SPF,Data_Macro_US_quarterly,by="date")
DATA <- merge(DATA,YC_LW,by="date")

freq <- 4 # quarterly frequency

# Create matrix of state variables
wy <- DATA[,c("CPIAUCSL","yld_3m", # price index, 3-month rate
              "yld_1y","yld_2y","yld_5y","yld_10y", # nominal yields (LW database)
              "BILL1")] # SPF forecast, 3-month rate, 1 year ahead.
maturities_in_model_periods <- c(1,2,5,10)*freq # maturities of yields
horizons_in_model_periods <- c(1)*freq # horizon of forecast

# Compute inflation:
TT <- dim(DATA)[1]
wy$CPIAUCSL <- c(rep(NaN,4),log(DATA$CPIAUCSL[5:TT]/
                              DATA$CPIAUCSL[1:(TT-4)]))

# Make yields consistent with model frequency:
wy[,2:dim(wy)[2]] <- wy[,2:dim(wy)[2]]/(freq*100)

# Remove entries with NaNs:
date_index <- complete.cases(wy)
wy <- wy[date_index,] # remove missing observations
vec_dates <- DATA$date[date_index]

# State vector:
w <- as.matrix(wy[,1:2])
mean_r <- mean(w[, "yld_3m"]) # keep mean of short-term rate
# Demean state variables (hence no need to estimate mu):
w <- w - t(matrix(apply(w,2,mean),dim(w)[2],dim(w)[1]))

# Observed variables:
y <- as.matrix(wy[, (dim(w)[2]+1):dim(wy)[2]])

n <- dim(w)[2] # dimension of w
# =====

# Estimate VAR model (P dynamics; used as starting value)
estVAR <- VAR(w,p=1,type = "none")
Phi <- Acoef(estVAR)[[1]]
mu <- matrix(0,n,1)

```

```

Sigma <- var(residuals(estVAR)) # covariance matrix of residuals
modelP <- list(mu=mu,Phi=Phi,Sigma=Sigma)

# Specification of short-term rates:
xi0 <- mean_r
xi1 <- matrix(0,n,1)
xi1[which(colnames(w)=="yld_3m")] <- 1

# Initial specif of prices of risk:
lambda0 <- matrix(0,n,1)
lambda1 <- matrix(0,n,n)

# =====
# Specifiy functions used in the estimation:
# -----

param2model_GATSM <- function(param,n,
                               modelP = NaN){
  # If modelP is not NULL, then it is not updated.

  mu <- matrix(0,n)

  if(is.na(modelP[1])){
    Phi <- matrix(param[1:n^2],n,n)
    nb <- n^2
    Sigma12 <- matrix(0,n,n)
    Sigma12[!upper.tri(Sigma12)] <- param[(nb+1):(nb+n*(n+1)/2)]
    Sigma <- Sigma12 %*% t(Sigma12)
    nb <- nb + n*(n+1)/2
  }else{
    Phi <- modelP$Phi
    Sigma <- modelP$Sigma
    nb <- 0
  }
  lambda0 <- matrix(param[(nb+1):(nb+n)],ncol=1)
  lambda1 <- matrix(param[(nb+n+1):(nb+n+n^2)],n,n)
  nb <- nb + n + n^2
  Sigma12 <- t(chol(Sigma))
  muQ <- mu + Sigma12 %*% lambda0
  PhiQ <- Phi + Sigma12 %*% lambda1

  modelP <- list(mu=mu,Phi=Phi,Sigma=Sigma)
  modelQ <- list(mu=muQ,Phi=PhiQ,Sigma=Sigma)

  # Same standard deviation for yields and forecasts:

```

```

std_dev <- param[nb+1]

return(list(modelP=modelP,
            modelQ=modelQ,
            lambda0 = lambda0, lambda1 = lambda1,
            std_dev=std_dev))
}

make_StateSpace_GATSM <- function(models,
                                   xi0,xi1,
                                   maturities_in_model_periods,
                                   horizons_in_model_periods=NaN){
  # This function builds the state-space model associated with
# a Gaussian Affine Term Structure models.

  modelP <- models$modelP
  modelQ <- models$modelQ

  resAB <- compute_AB_classical(xi0,xi1,
                                H=max(maturities_in_model_periods), # maximum maturity,
                                psi=psi_GaussianVAR,
                                psi.parameterization=modelQ)
  # Affine representation of yields, for selected maturities:
  A <- resAB$a[,1,maturities_in_model_periods]
  B <- resAB$b[,1,maturities_in_model_periods]

  # Short-term rate forecasts:
  resAB_P <- compute_AB_classical(xi0,xi1,
                                   H=max(maturities_in_model_periods,
                                           horizons_in_model_periods), # maximum horizon,
                                   psi=psi_GaussianVAR,
                                   psi.parameterization=modelP)

  if(!is.na(horizons_in_model_periods[1])){
    # Affine representation of short-term rate forecasts, for selected maturities:
    A_BILL <- resAB_P$a[,1,horizons_in_model_periods]
    B_BILL <- resAB_P$b[,1,horizons_in_model_periods]
  }else{
    A_BILL <- NaN
    B_BILL <- NaN
  }

  # Affine representation of yields under P, for selected maturities:
  A_P <- resAB_P$a[,1,maturities_in_model_periods]
  B_P <- resAB_P$b[,1,maturities_in_model_periods]

```

```

return(list(A = A, B = B,
           A_P = A_P, B_P = B_P,
           A_BILL = A_BILL, B_BILL = B_BILL,
           resAB = resAB,
           resAB_P = resAB_P)))}

loglik_GATSM_observed_factors <- function(param,y,w,
                                          maturities_in_model_periods,
                                          horizons_in_model_periods,
                                          modelP = NaN,
                                          max_lambdalambda_squared = NaN){
  # This function computes the log-likelihood associated with a Gaussian
# Affine Term Structure model with observed factors (w)
# y contains observations of yields and forecasts of the short-term yield.
# If modelP not NaN, then do not estimate P dynamics

  n <- dim(w)[2]
  TT <- dim(w)[1]

  models <- param2model_GATSM(param,n,modelP)

  SS <- make_StateSpace_GATSM(models,
                              xi0,xi1,
                              maturities_in_model_periods,
                              horizons_in_model_periods)

  nb_mat <- length(maturities_in_model_periods)
  nb_horiz <- length(horizons_in_model_periods)

  yds_model <- w %*% SS$A + t(matrix(SS$B,nb_mat,TT))
  BILL_model <- w %*% SS$A_BILL + t(matrix(SS$B_BILL,nb_horiz,TT))

  # Measurement errors:
  eta <- cbind(yds_model,BILL_model) - y # pricing errors

  # w's innovations:
  epsilon <- w[2:TT,] - w[1:(TT-1),] %*% t(models$modelP$Phi)

  logL_measur <- log_mvdnorm(eta, diag(rep(models$std_dev^2,nb_mat+nb_horiz)))
  logL_transit <- log_mvdnorm(epsilon, Sigma)

  if(is.na(max_lambdalambda_squared)){
    penalty <- 0
  }else{
    lambda0 <- models$lambda0
  }
}

```

```

lambda1 <- models$lambda1
Phi <- models$modelP$Phi
Var_w <- matrix(
  solve(diag(n^2) - Phi %x% Phi) %*% c(models$modelP$Sigma),
  n,n)
sum_lambdalambda <-
  sum(models$lambda0^2) + sum(diag(lambda1 %*% Var_w %*% t(lambda1)))
penalty <- 10*(sum_lambdalambda > max_lambdalambda_squared) *
  (sum_lambdalambda - max_lambdalambda_squared)^2
}

return(-(logL_measur + logL_transit) + penalty)
}
# =====

```

The following chunk estimates the model while imposing restrictions on λ_0 and λ_1 (which disciplines the estimation).

15.1.2 Initialization affects likelihood-based estimation results

This example shows how initialization affects likelihood-based estimation results.

```

# Purpose: This example shows how initialization affects likelihood-based estimation results

max_lambdalambda_squared <- 1 # If NaN, no restrictions imposed on lambda_t
# =====

# =====
# Estimation by MLE
# -----

if(indic_reestim){
  std_dev <- sd(y)
  for(indic_update_P in c(FALSE,TRUE)){

    if(indic_update_P){
      modelP <- NaN
      Sigma12 <- t(chol(Sigma))
      param_est <- c(Phi,Sigma12[!upper.tri(Sigma12)])
    }else{
      modelP <- list(mu = matrix(0,dim(w)[2]),
                    Phi = Phi, Sigma = Sigma)
      param_est <- NULL
    }
  }
}

```



```

param_est <- c(param_est,lambda0,lambda1,std_dev)

if(!indic_update_P){
  nb_loop <- 2
}else{
  nb_loop <- 5
}

for(i in 1:nb_loop){
  print(paste("--- Loop ",i," ---",sep=""))
  for(method in c("Nelder-Mead","nlminb")){
    res.optim <- optimx(par=param_est,
                       fn=loglik_GATSM_observed_factors,
                       y=y,w=w,
                       maturities_in_model_periods=maturities_in_model_periods,
                       horizons_in_model_periods=horizons_in_model_periods,
                       modelP = modelP,
                       max_lambdalambda_squared = max_lambdalambda_squared,
                       method=method,
                       control=list(trace=FALSE,
                                   maxit=ifelse(method=="Nelder-Mead",4000,20),
                                   kkt = FALSE))
    print(paste("--- Log-lik (",method,"): ",round(res.optim$value,2)," ---",sep=""))
    param_est <- c(as.matrix(res.optim)[1:length(param_est)])
  }
}

models <- param2model_GATSM(param_est,n,
                             modelP = modelP)

lambda0 <- models$lambda0
lambda1 <- models$lambda1
std_dev <- models$std_dev
}
save(param_est,
      file=
        paste("stored_results/param_GATSM_obs_varia_lambda_",
              max_lambdalambda_squared,".Rdat",sep=""))
}else{
  load(file=paste("stored_results/param_GATSM_obs_varia_lambda_",
                  max_lambdalambda_squared,".Rdat",sep=""))
}
# =====

```

```

# =====
# Prepare plots
# -----

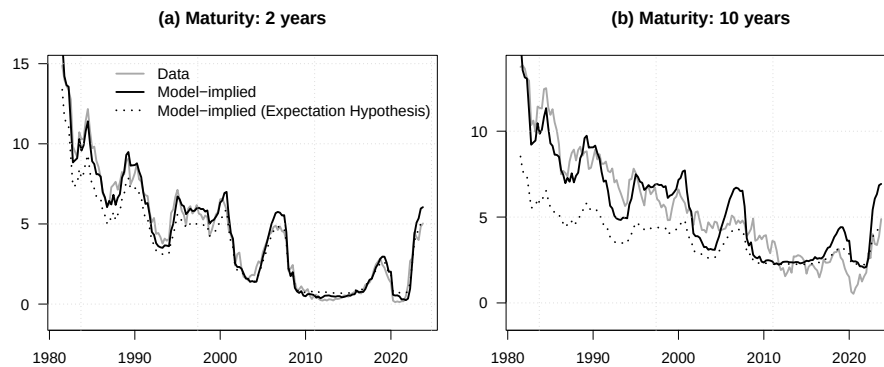
n <- dim(w)[2]
TT <- dim(w)[1]
models <- param2model_GATSM(param_est,n)
SS <- make_StateSpace_GATSM(models,
                             xi0,xi1,
                             maturities_in_model_periods,
                             horizons_in_model_periods)
nb_mat <- length(maturities_in_model_periods)
nb_horiz <- length(horizons_in_model_periods)
yds_model <- w %%% SS$A + t(matrix(SS$B,nb_mat,TT))
BILL_model <- w %%% SS$A_BILL + t(matrix(SS$B_BILL,nb_horiz,TT))
yds_model_P <- w %%% SS$A_P + t(matrix(SS$B_P,nb_mat,TT))

par(mfrow=c(1,2))
par(plt=c(.1,.95,.1,.85))
count <- 0
for(maturity in c(2,10)){
  count <- count + 1
  main.t <- paste("(",letters[count],") Maturity: ",maturity," years",sep="")

  m <- which(maturities_in_model_periods==freq*maturity)
  vari <- paste("yld_",maturity,"y",sep="")
  plot(vec_dates,freq*100*y[,vari],type="l",lty=1,lwd=2,col="dark grey",
       ylim=c(-1,max(freq*100*y[,vari])),
       xlab="",ylab="",las=1,main=main.t)
  lines(vec_dates,freq*100*yds_model[,m],lty=1,lwd=2)
  lines(vec_dates,freq*100*yds_model_P[,m],lty=3,lwd=2)
  grid()

  if(count == 1){
    legend("topright",
          legend=c("Data","Model-implied","Model-implied (Expectation Hypothesis)"),
          lty=c(1,1,3),
          col=c("dark grey","black","black"),
          lwd=c(2,2,2),bty = "n")
  }
}

```



In the following chunk, we remove the restrictions on λ_0 and λ_1 .

15.1.3 Observed variables with their fitted counterparts under the estimated model

This example compares observed variables with their fitted counterparts under the estimated model.

Purpose: This example compares observed variables with their fitted counterparts under the estimated model

```
# =====
max_lambdalambda_squared <- NaN # If NaN, no restrictions imposed on lambda_t
# =====

# =====
# Estimation by MLE
# -----

if(indic_reestim){
  nb_loop <- 20

  for(i in 1:nb_loop){
    print(paste("--- Loop ",i," ---",sep=""))
    for(method in c("Nelder-Mead","nlminb")){
      res.optim <- optimx(par=param_est,
                          fn=loglik_GATSM_observed_factors,
                          y=y,w=w,
                          maturities_in_model_periods=maturities_in_model_periods,
                          horizons_in_model_periods=horizons_in_model_periods,
                          modelP = modelP,
                          max_lambdalambda_squared = max_lambdalambda_squared,
                          method=method,
                          control=list(trace=FALSE,
```

```

maxit=ifelse(method=="Nelder-Mead",4000,20),
kkt = FALSE))
print(paste("--- Log-lik (" ,method,"): ",round(res.optim$value,2)," ---",sep=""))
param_est <- c(as.matrix(res.optim)[1:length(param_est)])
}
}

save(param_est,
file=
paste("stored_results/param_GATSM_obs_varia_lambda_",
max_lambdalambda_squared,".Rdat",sep=""))
}else{
load(file=paste("stored_results/param_GATSM_obs_varia_lambda_",
max_lambdalambda_squared,".Rdat",sep=""))
}
# =====

# =====
# Prepare plots
# -----

n <- dim(w)[2]
TT <- dim(w)[1]
models <- param2model_GATSM(param_est,n)
SS <- make_StateSpace_GATSM(models,
xi0,xi1,
maturities_in_model_periods,
horizons_in_model_periods)
nb_mat <- length(maturities_in_model_periods)
nb_horiz <- length(horizons_in_model_periods)
yds_model <- w %%% SS$A + t(matrix(SS$B,nb_mat,TT))
BILL_model <- w %%% SS$A_BILL + t(matrix(SS$B_BILL,nb_horiz,TT))
yds_model_P <- w %%% SS$A_P + t(matrix(SS$B_P,nb_mat,TT))

par(mfrow=c(1,2))
par(plt=c(.1,.95,.1,.85))
count <- 0
for(maturity in c(2,10)){
count <- count + 1
main.t <- paste("(",letters[count],") Maturity: ",maturity," years",sep="")

m <- which(maturities_in_model_periods==freq*maturity)
vari <- paste("yld_",maturity,"y",sep="")

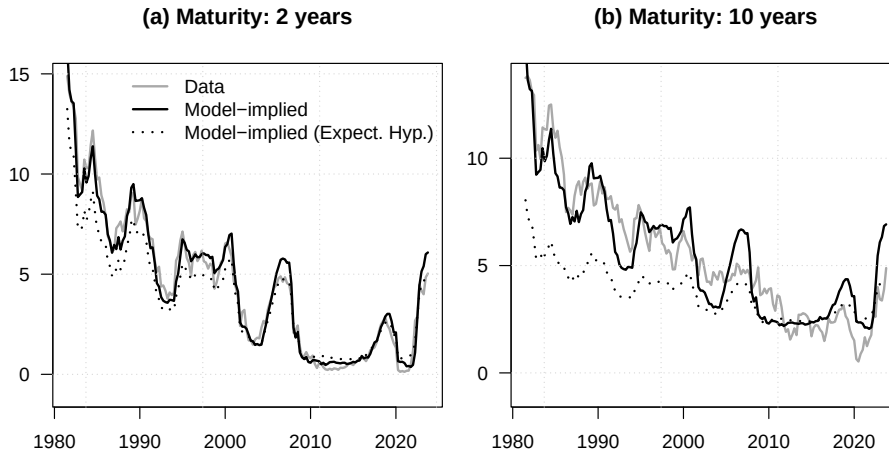
```

```

plot(vec_dates,freq*100*y[,vari],type="l",lty=1,lwd=2,col="dark grey",
     ylim=c(-1,max(freq*100*y[,vari])),
     xlab="",ylab="",las=1,main=main.t)
lines(vec_dates,freq*100*yds_model[,m],lty=1,lwd=2)
lines(vec_dates,freq*100*yds_model_P[,m],lty=3,lwd=2)
grid()

if(count == 1){
  legend("topright",
        legend=c("Data","Model-implied","Model-implied (Expect. Hyp.)"),
        lty=c(1,1,3),
        col=c("dark grey","black","black"),
        lwd=c(2,2,2),bty = "n")
}
}

```



15.2 Maximum Likelihood with the Kalman filter

In this section, we take the same data and estimate a model with latent state vector, using the Kalman filter.

In this example, we employ the results of Example ?? in the context described by Christensen, Diebold, and Rudebusch (2009). Specifically, we consider a three factor model following a Gaussian VAR:

$$w_t = \begin{bmatrix} X_{1,t} \\ X_{2,t} \\ X_{3,t} \end{bmatrix} = \begin{bmatrix} \lambda_1 & 0 & 0 \\ 0 & 1 - \lambda_2 & \lambda_2 \\ 0 & 0 & 1 - \lambda_2 \end{bmatrix} \begin{bmatrix} X_{1,t-1} \\ X_{2,t-1} \\ X_{3,t-1} \end{bmatrix} + \begin{bmatrix} \sigma_{11} & 0 & 0 \\ \sigma_{21} & \sigma_{22} & 0 \\ \sigma_{31} & \sigma_{32} & \sigma_{33} \end{bmatrix} \begin{bmatrix} \varepsilon_{1,t} \\ \varepsilon_{2,t} \\ \varepsilon_{3,t} \end{bmatrix},$$

where $\begin{bmatrix} \varepsilon_{1,t} \\ \varepsilon_{2,t} \\ \varepsilon_{3,t} \end{bmatrix} \sim i.i.d. \mathcal{N}(0, Id)$.

The nominal short-term rate is given by $i_t = X_{1,t} + X_{2,t}$.

Christensen, Diebold, and Rudebusch (2009) show that such specifications allow to approximate the widely-used Nelson-Siegel yield curve specification.¹

15.2.1 Estimated latent factors from the maximum-likelihood exercise

This example reports estimated latent factors from the maximum-likelihood exercise.

```
# Purpose: This example reports estimated latent factors from the maximum-likelihood exercise.

# Dataset -----

# Create matrix of observables
observables <- DATA[,c("yld_3m", "yld_1y", "yld_2y", "yld_5y", "yld_10y", # nominal yields
                        "BILL1", "BILL10")] # SPF forecast, 3-month rate, 1 and 10 years
maturities_in_model_periods <- c(.25, 1, 2, 5, 10)*freq # maturities of yields
horizons_in_model_periods <- c(1, 10)*freq # horizon of forecast

# Make yields consistent with model frequency:
observables <- as.matrix(observables)/(freq*100)

# vector of dates:
vec_dates <- DATA$date

# Estimation -----
# Constraint on unreasonable Sharpe Ratios:
max_lambda_lambda_squared <- NaN

# Number of factors
n <- 3
TT <- dim(observables)[1]

param2model_AFNS <- function(param){
  # Parameterize the three-factor model of Christensen, Diebold, and Rudebusch (2009).
  # This model is called Arbitrage-Free Nelson-Siegel (AFNS)

  n <- 3 # three factor model
  mu <- matrix(param[1:n], n)
```

¹In the context of the Nelson-Siegel model, the zero-coupon yield of maturity h is given by $\beta_0 + \beta_1 \left(\frac{1 - \exp(-\lambda h)}{\lambda h} \right) + \beta_2 \left(\frac{1 - \exp(-\lambda h)}{\lambda h} - \exp(-\lambda h) \right)$, where β_0 , β_1 , β_2 , and λ are parameters.

```

nb <- n

lambda1P <- exp(param[nb+1])/(1+exp(param[nb+1]))
lambda2P <- exp(param[nb+2])/(1+exp(param[nb+2]))
Phi <- diag(c(1-lambda1P,1-lambda2P,1-lambda2P))
Phi[2,3] <- lambda2P
nb <- nb + 2

# Phi <- matrix(param[(nb+1):(nb+n^2)],n,n)
# nb <- nb + n^2

Sigma12 <- matrix(0,n,n)
Sigma12[!upper.tri(Sigma12)] <- param[(nb+1):(nb+n*(n+1)/2)]
Sigma <- Sigma12 %*% t(Sigma12)
nb <- nb + n*(n+1)/2

muQ <- matrix(0,n,1)
lambda1Q <- exp(param[nb+1])/(1+exp(param[nb+1]))
lambda2Q <- exp(param[nb+2])/(1+exp(param[nb+2]))
PhiQ <- diag(c(1-lambda1Q,1-lambda2Q,1-lambda2Q))
PhiQ[2,3] <- lambda2Q
nb <- nb + 2

lambda0 <- solve(Sigma12) %*% (muQ - mu)
lambda1 <- solve(Sigma12) %*% (PhiQ - Phi)

modelP <- list(mu=mu,Phi=Phi,Sigma=Sigma,Sigma12=Sigma12)
modelQ <- list(mu=muQ,Phi=PhiQ,Sigma=Sigma,Sigma12=Sigma12)

xi0 <- param[nb+1]
xi1 <- matrix(c(1,1,0),n,1)
nb <- nb + 1

# Same standard deviation for yields and forecasts:
std_dev <- max(exp(param[nb+1]),.0001)

return(list(modelP=modelP,
            modelQ=modelQ,
            xi0 = xi0, xi1 = xi1,
            lambda0 = lambda0,lambda1 = lambda1,
            std_dev=std_dev))
}

loglik_AFNS <- function(param,observables,

```

```

        maturities_in_model_periods,
        horizons_in_model_periods,
        max_lambdalambda_squared = NaN){
# This function computes the log-likelihood associated with a Gaussian
# Affine Term Structure model with observed factors (w)
# y contains observations of yields and forecasts of the short-term yield.
# If modelP not NaN, then do not estimate P dynamics

models <- param2model_AFNS(param)
n <- dim(models$modelP$Phi)[2]

# Unconditional variance of w:
Phi <- models$modelP$Phi
Var_w <- matrix(
  solve(diag(n^2) - Phi %x% Phi) %*% c(models$modelP$Sigma),
  n,n)

SS <- make_StateSpace_GATSM(models,
                             models$xi0,models$xi1,
                             maturities_in_model_periods,
                             horizons_in_model_periods)

nb_mat <- length(maturities_in_model_periods)
nb_horiz <- length(horizons_in_model_periods)

# Define matrices needed in the Kalman_filter procedures:
nu_t <- t(matrix(models$modelP$mu,n,TT))
H <- models$modelP$Phi
G <- t(cbind(SS$A,SS$A_BILL))
mu_t <- cbind(t(matrix(SS$B,nb_mat,TT)),
              t(matrix(SS$B_BILL,nb_horiz,TT)))
N <- models$modelP$Sigma12
M <- diag(rep(models$std_dev,dim(observables)[2]))
Sigma_0 <- Var_w # unconditional variance of w
rho_0 <- matrix(0,n,1) # unconditional mean of w

filter.res <- Kalman_filter(observables,nu_t,H,N,mu_t,G,M,Sigma_0,rho_0)

if(is.na(max_lambdalambda_squared)){
  penalty <- 0
}else{
  lambda0 <- models$lambda0
  lambda1 <- models$lambda1
  sum_lambdalambda <-
    sum(models$lambda0^2) + sum(diag(lambda1 %*% Var_w %*% t(lambda1)))

```



```

    penalty <- 10*(sum_lambdalambda > max_lambdalambda_squared) *
      (sum_lambdalambda - max_lambdalambda_squared)^2
  }

  penalty <- penalty + 100*sum(
    (abs(param)>20)*(abs(param)-20)
  )

  loss <- - sum(filter.res$loglik.vector) + penalty

  return(loss)
}

mu <- matrix(0,n,1)
lambda1P <- .1
lambda2P <- .1
lambda1Q <- .1
lambda2Q <- .1
Sigma <- .01^2 * diag(n)
Sigma12 <- t(chol(Sigma))
param_est <- c(mu,
  log(lambda1P/(1-lambda1P)),log(lambda2P/(1-lambda2P)),
  Sigma12[!upper.tri(Sigma12)],
  log(lambda1Q/(1-lambda1Q)),log(lambda2Q/(1-lambda2Q)),
  .06/freq,#r_infty
  log(.05))# std_dev

# Initialization:
param <- param_est

if(indic_reestim){
  nb_loop <- 3

  for(i in 1:nb_loop){
    print(paste("--- Loop ",i," ---",sep=""))
    for(method in c("nlminb","Nelder-Mead")){
      # for(method in c("nlminb")){
      res.optim <- optimx(par=param_est,
        fn=loglik_AFNS,
        observables=observables,
        maturities_in_model_periods=maturities_in_model_periods,
        horizons_in_model_periods=horizons_in_model_periods,
        max_lambdalambda_squared = max_lambdalambda_squared,
        method=method,
        control=list(trace=TRUE,

```

```

maxit=ifelse(method=="Nelder-Mead",2000,100),
kkt = FALSE)

)
print(paste("--- Log-lik: ",round(res.optim$value,2)," ---",sep=""))
param_est <- c(as.matrix(res.optim)[1:length(param_est)])
}
}

save(param_est,
file=
paste("stored_results/param_AFNS_lambda_",
max_lambdalambda_squared,".Rdat",sep=""))
}else{
load(file=paste("stored_results/param_AFNS_lambda_",
max_lambdalambda_squared,".Rdat",sep=""))
}
# =====

# =====
# Prepare plots
# -----

TT <- dim(observables)[1]

models <- param2model_AFNS(param_est)
n <- dim(models$modelP$Phi)[2]

# Unconditional variance of w:
Phi <- models$modelP$Phi
Var_w <- matrix(
solve(diag(n^2) - Phi %x% Phi) %*% c(models$modelP$Sigma),
n,n)

SS <- make_StateSpace_GATSM(models,
models$xi0,models$xi1,
maturities_in_model_periods,
horizons_in_model_periods)

nb_mat <- length(maturities_in_model_periods)
nb_horiz <- length(horizons_in_model_periods)

# Define matrices needed in the Kalman_filter procedures:
nu_t <- t(matrix(models$modelP$mu,n,TT))
H <- models$modelP$Phi

```

```

G <- t(cbind(SS$A,SS$A_BILL))
mu_t <- cbind(t(matrix(SS$B,nb_mat,TT)),
              t(matrix(SS$B_BILL,nb_horiz,TT)))
N <- models$modelP$Sigma12
M <- diag(rep(models$std_dev,dim(observables)[2]))
Sigma_0 <- Var_w # unconditional variance of w
rho_0 <- matrix(0,n,1) # unconditional mean of w

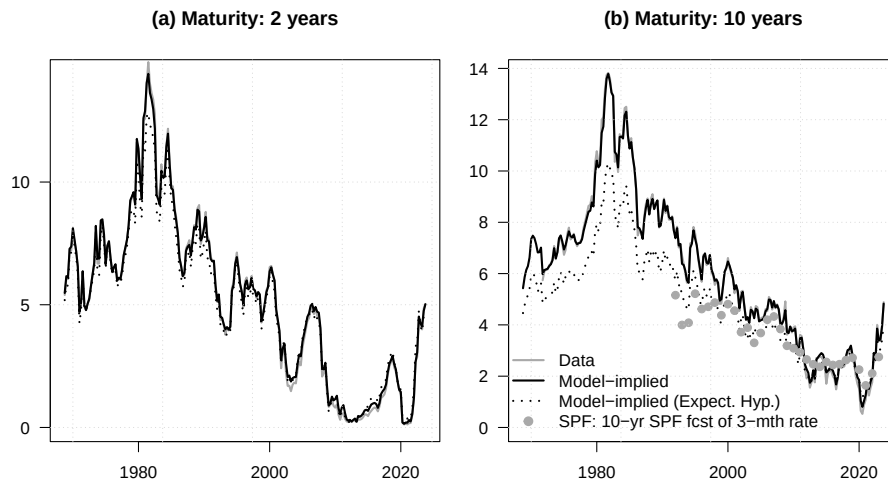
filter.res <- Kalman_filter(observables,nu_t,H,N,mu_t,G,M,Sigma_0,rho_0)
ww <- filter.res$r
yds_model <- ww %%% SS$A + t(matrix(SS$B,nb_mat,TT))
BILL_model <- ww %%% SS$A_BILL + t(matrix(SS$B_BILL,nb_horiz,TT))
yds_model_P <- ww %%% SS$A_P + t(matrix(SS$B_P,nb_mat,TT))

par(mfrow=c(1,2))
par(plt=c(.1,.95,.1,.85))
count <- 0
for(maturity in c(2,10)){
  count <- count + 1
  main.t <- paste("(",letters[count],") Maturity: ",maturity," years",sep="")

  m <- which(maturities_in_model_periods==freq*maturity)
  plot(vec_dates,freq*100*observables[,m],type="l",lty=1,lwd=2,col="dark grey",
       ylim=c(0,max(freq*100*yds_model[,m])),
       xlab="",ylab="",las=1,main=main.t)
  lines(vec_dates,freq*100*yds_model[,m],lty=1,lwd=2)
  lines(vec_dates,freq*100*yds_model_P[,m],lty=3,lwd=2)
  if(maturity==10){
    points(vec_dates,DATA$BILL10,pch=19,lwd=1,col="dark grey")
  }
  grid()

  if(count == 2){
    legend("bottomleft",
          legend=c("Data","Model-implied","Model-implied (Expect. Hyp.)",
                  "SPF: 10-yr SPF fcst of 3-mth rate"),
          lty=c(1,1,3,NaN),
          pch=c(NaN,NaN,NaN,19),
          col=c("dark grey","black","black","dark grey"),
          lwd=c(2,2,2),bty = "n")
  }
}
}

```



15.3 Kitagawa-Hamilton

This example estimates a regime-switching stock-return model and visualizes the inferred states.

Purpose: This example estimates a regime-switching stock-return model and visualizes the inferred states

```
library(DTAM)
library(expm)
library(mvtnorm)
library(optimx)
library(tis) # for NBER recession dates

param2model <- function(param,J=2){
  nb <- 0 # number of parameters already used
  # matrix of transition probabilities:
  Pi <- matrix(0,J,J)

  diag(Pi) <- exp(param[1:J])/(1+exp(param[1:J]))
  Pi[1,2] <- 1- Pi[1,1]
  Pi[J,J-1] <- 1- Pi[J,J]
  nb <- nb + J

  if(J>2){
    par_aux <- param[(nb+1):(nb+J-2)]
    par_aux <- exp(par_aux)/(1+exp(par_aux))
    if(length(par_aux)==1){# in this case, J=3
```

```

    aux <- par_aux*(1-Pi[2,2])
  }else{
    aux <- diag(c(par_aux)*(1-diag(Pi[2:(J-1),2:(J-1)])))
  }
  Pi[2:(J-1),1:(J-2)] <- Pi[2:(J-1),1:(J-2)] + aux

  sum_rows <- apply(matrix(Pi[2:(J-1),],J-2,J),1,sum)
  if(length(sum_rows)==1){
    aux <- 1 - sum_rows
  }else{
    aux <- diag(J-2) - diag(c(sum_rows))
  }
  Pi[2:(J-1),3:J] <- Pi[2:(J-1),3:J] + aux

  nb <- nb + J - 2
}

phi <- .999*exp(param[nb+1])/(1+exp(param[nb+1]))
nb <- nb + 1

mu_r <- matrix(param[(nb+1):(nb+J)],ncol=1)/100
nb <- nb + J

sigma_r <- matrix(param[(nb+1):(nb+J)],ncol=1)/100
nb <- nb + J

mu_s2 <- matrix(param[(nb+1):(nb+3*J+1)],ncol=1)/100
nb <- nb + 3*J + 1

lambda <- matrix(param[(nb+1):(nb+3*J+1)],ncol=1)/100
nb <- nb + 3*J + 1

model <- list(Pi = Pi,
              phi = phi,
              mu_r = mu_r,
              sigma_r = sigma_r,
              mu_s2 = mu_s2,
              lambda = lambda)
return(model)
}

model2param <- function(model){

  Pi <- model$Pi

```

```

J <- dim(Pi)[1]

param <- log(diag(Pi)/(1-diag(Pi)))

if(J>2){
  if(J==3){
    aux <- Pi[2,1]/(1-Pi[2,2])
  }else{
    aux <- diag(Pi[2:(J-1),1:(J-2)]/(1-diag(Pi)[2:(J-1)]))
  }
  param <- c(param,
             log(aux/.999/(1-aux/.999)))
}

param <- c(param,
           log(model$phi/(1-model$phi)),
           100*model$mu_r,
           100*model$sigma_r,
           100*model$mu_s2,
           100*model$lambda)
return(param)
}

solve_model_RS_stocks <- function(model){
  model$mu_s0 <- 0
  J <- dim(model$Pi)[1]
  e_1 <- matrix(c(1,rep(0,3*J)),1+3*J,1)
  model$mu_s1 <- e_1 + psi_RS_stock(model$lambda,model)$a -
    psi_RS_stock(model$lambda + model$mu_s2,model)$a
  return(model)
}

psi_RS_stock <- function(u,model){
  # Laplace transform of the state vector w_t, with
  # w_t = (r_t, z_t', eps_{r,t}z_t', eps_{s,t}z_t')'
  # u is n x k, where n = 1+3*J. Indeed, dimension of J is 1+3*J
  # k is the number of vectors on which one wants to evaluate the LT of w

  J <- (dim(u)[1]-1)/3
  k <- dim(u)[2]

  Pi <- model$Pi
  phi <- model$phi

  mu_r <- model$mu_r

```

```

sigma_r <- model$sigma_r
mu_s2   <- model$mu_s2
lambda  <- model$lambda

u_r <- matrix(u[1,],nrow=1)
U_z <- matrix(u[(1+0*J+1):(1+1*J)],,J,k)
U_r <- matrix(u[(1+1*J+1):(1+2*J)],,J,k)
U_s <- matrix(u[(1+2*J+1):(1+3*J)],,J,k)

a <- rbind(
  u_r*phi,
  log(Pi %% exp((matrix(1,J,1)%%u_r)*(mu_r %% matrix(1,1,k)) + U_z +
    .5*((matrix(1,J,1)%%u_r) * sigma_r %% matrix(1,1,k) + U_r)^2 +
    .5*U_s^2)),
  matrix(0,2*J,k)
)
return(list(a=a,b=0))
}

simul_model_RS_stocks <- function(model,TT){
  # This function simulates the regime-switching stock-return model.

  J <- dim(model$Pi)[1]

  z <- simul_RS(model$Pi,TT)

  # Compute unconditional distribution of z:
  Paux <- t(model$Pi)
  for(i in 1:10){
    Paux <- Paux %>% 2
  }
  Ez <- matrix(Paux[,1],ncol=1)

  # mean STR:
  Emu_r <- t(model$mu_r) %>% Ez
  Er    <- Emu_r/(1-model$phi)

  epsilon_r <- matrix(rnorm(TT),ncol=1)
  epsilon_s <- matrix(rnorm(TT),ncol=1)

  w <- cbind(0,z,
    z*(epsilon_r%%matrix(1,1,J)),
    z*(epsilon_s%%matrix(1,1,J)))

  r <- Er

```

```

for(t in 1:TT){
  r <- z[t,] %*% model$mu_r + phi*r + (z[t,] %*% model$sigma_r)*epsilon_r[t]
  w[t,1] <- r
}

w_1 <- rbind(NaN,w[1:(TT-1),])

rs <- model_solved$mu_s0 + w_1%*%model_solved$mu_s1 + w%*%model_solved$mu_s2

return(list(w=w,z=z,
            rs=rs,r=w[,1]))
}

filter_RS_stocks <- function(model_solved,Y,indic_smooth=FALSE){
  # Y contains the dataset:
  # - 1st column: short-term rate r_t
  # - 2nd column: stock return rs_t

  TT <- dim(Y)[1]
  J <- dim(model_solved$Pi)[1]

  mu_r_extended <- model_solved$mu_r %x% matrix(1,J,1)
  sigma_r_extended <- model_solved$sigma_r %x% matrix(1,J,1)

  # Compute the matrix of transition probabilities of z_{t} %x% z_{t-1}:
  PI <- make_Pi_z_kron_z(model_solved$Pi)

  # Compute epsilon_r as a function of z_{t} %x% z_{t-1} -----
  r <- Y[2:TT,1] %*% matrix(1,1,J*J)
  r_1 <- Y[1:(TT-1),1] %*% matrix(1,1,J*J)

  sigma_r_epsilon_r <- r - model_solved$phi * r_1 - matrix(1,TT-1,1) %*% t(mu_r_extended)
  epsilon_r <- sigma_r_epsilon_r/(matrix(1,TT-1,1) %*% t(sigma_r_extended))

  # Compute epsilon_s as a function of z_{t} %x% z_{t-1} -----
  rs <- Y[2:TT,2] %*% matrix(1,1,J*J)

  mu_s1_z_extended <- matrix(1,J,1) %x% matrix(model_solved$mu_s1[2:(J+1)],ncol=1)
  mu_s2_z_extended <- matrix(model_solved$mu_s2[2:(J+1)],ncol=1) %x% matrix(1,J,1)

  sigma_sr_extended <- matrix(model_solved$mu_s2[(1+1*J+1):(1+2*J)],ncol=1) %x% matrix
  sigma_ss_extended <- matrix(model_solved$mu_s2[(1+2*J+1):(1+3*J)],ncol=1) %x% matrix

  sigma_s_epsilon_s <-
    rs - model_solved$mu_s0 -

```



```

model_solved$mu_s1[1] * r_1 -
matrix(1,TT-1,1) %*% t(mu_s1_z_extended) -
model_solved$mu_s2[1] * r -
matrix(1,TT-1,1) %*% t(mu_s2_z_extended) -
(matrix(1,TT-1,1) %*% t(sigma_sr_extended))*epsilon_r
# epsilon_s <- sigma_s_epsilon_s/(matrix(1,TT-1,1) %*% t(sigma_ss_extended))

Eta <- matrix(NA,TT-1,J*J)
for(i in 1:(J*J)){
  sigma <- diag(c(sigma_sr_extended[i]^2,sigma_ss_extended[i]^2))
  sigma_times_epsilon <- cbind(sigma_r_epsilon_r[,i],sigma_s_epsilon_s[,i])
  # Eta[,i] <- dmnorm(sigma_times_epsilon,sigma=sigma,log = TRUE)
  Eta[,i] <- -
    log(sqrt(2*pi*sigma_sr_extended[i]^2))-0.5*sigma_r_epsilon_r[,i]^2/sigma_sr_extended[i]^2-
    log(sqrt(2*pi*sigma_ss_extended[i]^2))-0.5*sigma_s_epsilon_s[,i]^2/sigma_ss_extended[i]^2
}

res_KH <- KH_filter(PI,Eta)

res_KH$ksi_matrix <- res_KH$ksi_matrix %*% (diag(J) %x% matrix(1,J,1))

if(indic_smooth){
  res_KH$ksi_tT_matrix <- KH_smoother(PI,Eta) %*% (diag(J) %x% matrix(1,J,1))
}

return(res_KH)
}

loglik4optim <- function(param,Y,J=2){
  model <- param2model(param,J=J)
  model_solved <- solve_model_RS_stocks(model)
  res_KH <- filter_RS_stocks(model_solved,Y)
  return(-res_KH$loglik)
}

# Prepare dataset:

P <- Shiller$RealTotReturnPrice
TT <- dim(Shiller)[1]
Shiller$RealTotReturn <- c(NA,log(P[2:TT]/P[1:(TT-1)]))

DATA <- YC_LW[,c("date","yld_1m")]
DATA$yld_1m <- DATA$yld_1m/1200
names(DATA)[1] <- "Date"
DATA <- merge(DATA,Shiller,by="Date")

```

```

TT <- dim(DATA)[1]

Y <- cbind(DATA$yld_1m, DATA$RealTotReturn)
Y <- Y[complete.cases(Y),]
TT <- dim(Y)[1]

# Make initial model:
Pi <- matrix(c(.95, .15, .05, .85), 2, 2)
mu_s2 <- matrix(c(0,
                  .01, -.03,
                  .01, .01,
                  .01, .03), ncol=1)
lambda <- matrix(c(0,
                  .01, .02,
                  .01, .01,
                  .01, .01), ncol=1)
model <- list(Pi=Pi,
              phi = .9,
              mu_r = matrix(c(.001, -.001), ncol=1),
              sigma_r = matrix(c(.0025, .005), ncol=1),
              mu_s2 = mu_s2,
              lambda = lambda)

param_est <- model2param(model)

if(indic_reestim){
  nb_loop <- 20
  for(i in 1:nb_loop){
    print(paste("--- Loop ", i, " ---", sep=""))
    for(method in c("Nelder-Mead", "nlminb")){
      res.optim <- optimx(par=param_est,
                          fn=loglik4optim,
                          Y=Y, J=2,
                          method=method,
                          control=list(trace=TRUE,
                                       maxit=ifelse(method=="Nelder-Mead", 3000, 20),
                                       kkt = FALSE))
      print(paste("--- Log-lik (", method, "): ", round(res.optim$value, 2), " ---", sep=""))
      param_est_tempo <- c(as.matrix(res.optim)[1:length(param_est)])
      if(sum(is.na(param_est_tempo))==0){
        param_est <- param_est_tempo
      }else{
        print("problem")
      }
    }
  }
}

```

```

}
  save(param_est,file="stored_results/param_RS_stocks.Rdat")
}else{
  load(file="stored_results/param_RS_stocks.Rdat")
}

J <- 2
model <- param2model(param_est,J=J)
model_solved <- solve_model_RS_stocks(model)
res_KH <- filter_RS_stocks(model_solved,Y,indic_smooth = TRUE)
res_KH_smoothed <- 1
par(mfrow=c(1,1))
regime <- 1

vec_dates <- DATA$Date[2:dim(DATA)[1]]

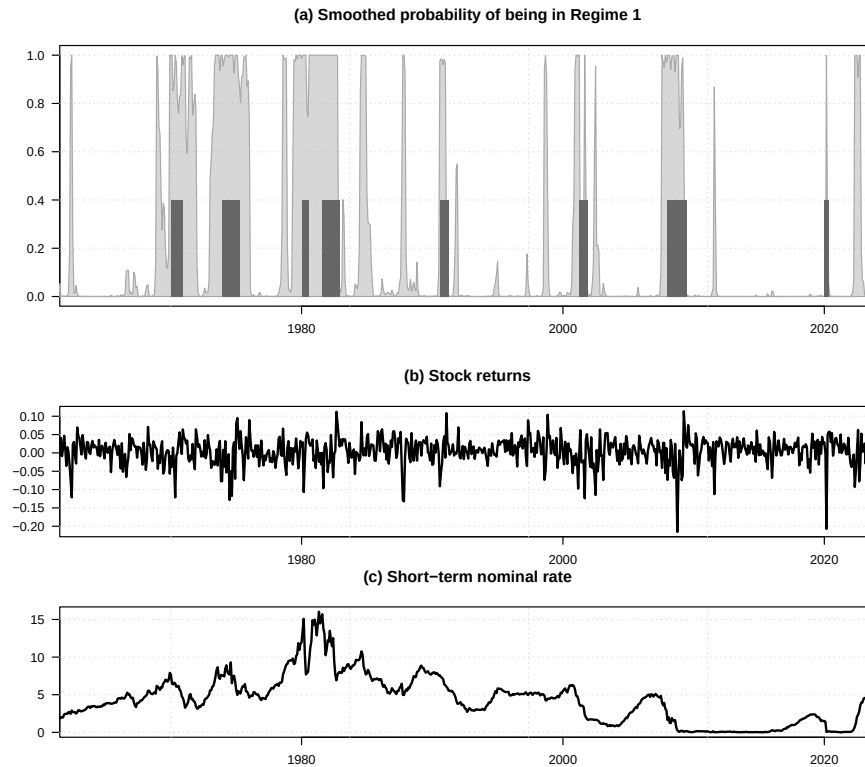
par(mfrow=c(3,1))
par(plt=c(.08,.97,.15,.8))

layout(matrix(1:3, nrow = 3, byrow = TRUE),
       heights = c(2, 1, 1)) # relative heights

proba <- res_KH$ksi_tT_matrix[,regime]
plot(vec_dates,proba,type="l",lwd=2,col="white",
     xaxs = "i",xlab="",ylab="",
     main="(a) Smoothed probability of being in Regime 1",las=1)
grid()
# lines(DATA$Date[2:dim(DATA)[1]],res_KH$ksi_matrix[,regime],col="red",lwd=2)
polygon(c(vec_dates,rev(vec_dates)),c(proba,0*proba),
       border="darkgrey",col="#AAAAAA77")
invisible(make_recessions(col = "#666666FF",MIN=0,MAX=.4))

plot(vec_dates,Y[2:TT,2],type="l",col="white",las=1,
     xaxs = "i",xlab="",ylab="",
     main="(b) Stock returns")
grid()
lines(vec_dates,Y[2:TT,2],lwd=2)
plot(vec_dates,1200*Y[2:TT,1],type="l",col="white",las=1,
     xaxs = "i",xlab="",ylab="",
     main="(c) Short-term nominal rate")
grid()
lines(vec_dates,1200*Y[2:TT,1],lwd=2)

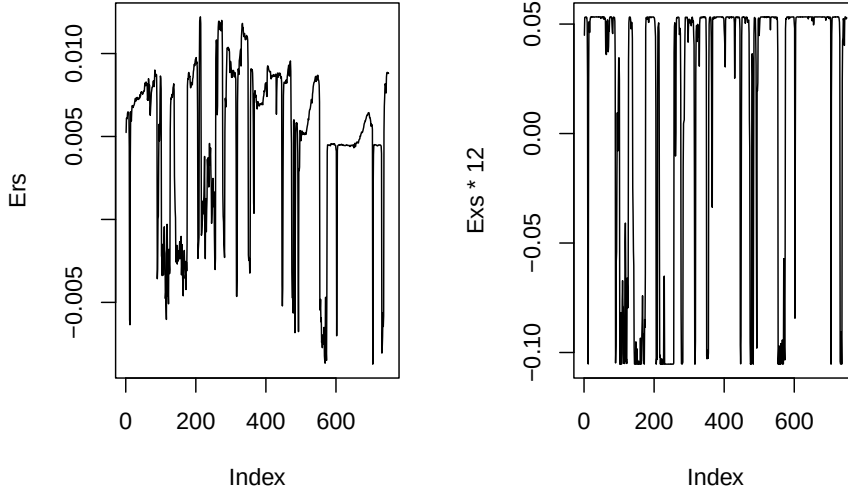
```



15.3.1 Additional diagnostics for the regime-switching stock model

This example reports additional diagnostics for the regime-switching stock model.

```
# Purpose: This example reports additional diagnostics for the regime-switching stock model
# Compute expected stock returns
model_solved$n_w <- length(model_solved$mu_s1)
aux <- compute_expect_variance(psi_RS_stock,model_solved)
r <- Y[2:TT,1]
wt <- cbind(r,res_KH$ksi_tT_matrix,matrix(0,TT-1,2*J))
Ewt <- matrix(1,TT-1,1) %*% t(aux$mu) + wt %*% t(aux$Phi)
Ers <- model_solved$mu_s0 + wt %*% model_solved$mu_s1 + Ewt %*% model_solved$mu_s2
Exs <- Ers - r
par(mfrow=c(1,2))
plot(Ers,type="l")
plot(Exs*12,type="l")
```



15.4 Inversion technique plus regime switching

The following example is set at a relatively high frequency (weekly). At this frequency, recursive formulas that involve loops can slow down computations. In the present example, bond yields decompose into two independent components: one driven by a time-homogeneous Markov chain, and the other by a VAR(1) process. Both processes are stable under temporal aggregation, which allows their multi-horizon Laplace transforms to be computed more efficiently by lowering the frequency. The following propositions form the basis of the functions `compute_LT_RS()` (for fast computation of the Markov-switching-based multi-horizon Laplace transforms) and `compute_Gaussian_fast()` (for the Gaussian VAR model).

Theorem 15.1 (VAR(1): Fast computation of multi-horizon conditional distribution). *If*

$$\mathbf{x}_{t+1} = \mu + \Phi \mathbf{x}_t + \Sigma^{\frac{1}{2}} \varepsilon_{t+1}, \quad \varepsilon_t \sim \mathcal{N}(\mathbf{0}, \mathbf{Id}),$$

then

$$\mathbf{x}_{t+h} + \dots + \mathbf{x}_{t+1} \mid \underline{\mathbf{x}}_t \sim \mathcal{N}(\mu_h + \Phi_h \mathbf{x}_t, \Sigma_h),$$

with

$$\begin{aligned}
\mu_h &= (\mathbf{Id} - \Phi)^{-1}(h\mathbf{Id} - \Phi\mathbf{K}_h)\mu, \\
\Phi_h &= \Phi\mathbf{K}_h, \\
(\text{vec})(\Sigma_h) &= h(\text{vec})(\Sigma^*) - (\text{vec})(\Phi\mathbf{K}_h\Sigma^*) - (\text{vec})(\Sigma^*\mathbf{K}_h'\Phi') + (\text{vec})(\mathcal{S}_h),
\end{aligned}$$

where $\Sigma^* = (\mathbf{Id} - \Phi)^{-1}\Sigma(\mathbf{Id} - \Phi')^{-1}$,
 $\mathbf{K}_h = (\mathbf{Id} - \Phi)^{-1}(\mathbf{Id} - \Phi^h)$, and

$$(\text{vec})(\mathcal{S}_h) = (\Phi \otimes \Phi)(\mathbf{Id} - \Phi \otimes \Phi)^{-1} \left[\mathbf{Id} - (\Phi \otimes \Phi)^h \right] (\text{vec})(\Sigma^*).$$

Proof. We have

$$\mathbf{x}_{t+h} = (\mathbf{Id} + \Phi + \dots + \Phi^{h-1})\mu + \Phi^h\mathbf{x}_t + \Sigma^{\frac{1}{2}}\varepsilon_{t+h} + \Phi\Sigma^{\frac{1}{2}}\varepsilon_{t+h-1} + \dots + \Phi^{h-1}\Sigma^{\frac{1}{2}}\varepsilon_{t+1}.$$

The variance of $\mathbf{x}_{t+h} + \dots + \mathbf{x}_{t+1}$, conditional on $\underline{\mathbf{x}}_t$, is

$$\begin{aligned}
&\Sigma + (\mathbf{Id} + \Phi)\Sigma(\mathbf{Id} + \Phi)' + \dots + (\mathbf{Id} + \Phi + \dots + \Phi^{h-1})\Sigma(\mathbf{Id} + \Phi + \dots + \Phi^{h-1})' \\
&= \sum_{i=1}^h (\mathbf{Id} - \Phi)^{-1}(\mathbf{Id} - \Phi^i)\Sigma[(\mathbf{Id} - \Phi)^{-1}(\mathbf{Id} - \Phi^i)]' \\
&= \sum_{i=1}^h (\mathbf{Id} - \Phi^i)\Sigma^*(\mathbf{Id} - \Phi^i)' \\
&= \sum_{i=1}^h \Sigma^* - \Phi^i\Sigma^* - \Sigma^*(\Phi^i)' + \Phi^i\Sigma(\Phi^i)' \\
&= h\Sigma^* - \Phi(\mathbf{Id} - \Phi)^{-1}(\mathbf{Id} - \Phi^h)\Sigma^* - \Sigma^*(\mathbf{Id} - \Phi^h)'(\mathbf{Id} - \Phi')^{-1}\Phi' + \mathcal{S}_h,
\end{aligned}$$

which leads to the result. $\square$

Theorem 15.2 (Fast computation of multi-horizon LT of a Markov chain).
Consider an exogenous Markov chain $\mathbf{z}_t$, valued in the columns of the identity matrix of dimension $J \times J$. Its dynamics is characterized by a time-homogeneous matrix of transition probabilities denoted by Π (whose rows sum to one).

We have:

$$\mathbb{E} \left[\exp(\mathbf{u}'\{\mathbf{z}_{t+1} + \dots + \mathbf{z}_{t+h}\}) \mid \underline{\mathbf{z}}_t \right] = (1 \quad 1 \quad \dots \quad 1) (\mathbf{D}\Pi')^h \mathbf{z}_t, \quad (15.1)$$

where $\mathbf{D} = \text{diag}[\exp(\mathbf{u})]$.

Proof. Equation (15.1) is satisfied for $h = 1$ since

$$\mathbb{E} [\exp(\mathbf{u}'\mathbf{z}_{t+1}) | \underline{\mathbf{z}}_t] = \mathbb{E} [\exp(\mathbf{u})'\mathbf{z}_{t+1} | \underline{\mathbf{z}}_t] = \exp(\mathbf{u})'\Pi'\mathbf{z}_t.$$

Assume it is valid for a given $h > 0$, we have:

$$\begin{aligned} & \mathbb{E} [\exp(\mathbf{u}'\{\mathbf{z}_{t+1} + \dots + \mathbf{z}_{t+h} + \mathbf{z}_{t+h+1}\}) | \underline{\mathbf{z}}_t] \\ &= \mathbb{E} [\mathbb{E} [\exp(\mathbf{u}'\{\mathbf{z}_{t+1} + \dots + \mathbf{z}_{t+h} + \mathbf{z}_{t+h+1}\}) | \underline{\mathbf{z}}_{t+1}] | \underline{\mathbf{z}}_t] \\ &= \mathbb{E} [(1 \ 1 \ \dots \ 1) (\mathbf{D}\Pi')^h \mathbf{z}_{t+1} \exp(\mathbf{u}'\mathbf{z}_{t+1}) | \underline{\mathbf{z}}_t] \\ &= (1 \ 1 \ \dots \ 1) (\mathbf{D}\Pi')^h \mathbb{E} [\mathbf{z}_{t+1} \exp(\mathbf{u}'\mathbf{z}_{t+1}) | \underline{\mathbf{z}}_t] \\ &= (1 \ 1 \ \dots \ 1) (\mathbf{D}\Pi')^h \text{diag}[\exp(\mathbf{u})] \Pi \mathbf{z}_t, \end{aligned}$$

which leads to the result. $\square$

A direct implication of Theorem 15.2 is:

Theorem 15.3 (Fast pricing with Markov chains). *We consider a scalar process x_t that is such that*

$$\mathbb{E} [\exp(x_t) | \underline{\mathbf{z}}_t, \underline{x}_{t-1}] = \exp(\alpha)'\mathbf{z}_t,$$

where $\exp(\alpha)$ is a J -dimensional vector whose i^{th} entry is $\exp(\alpha_i)$ (α_i being the i^{th} entry of vector α). If $\mathbf{z}_t$ is exogenous to x_t , we have:

$$\mathbb{E} [\exp(x_{t+1} + \dots + x_{t+h}) | \underline{\mathbf{z}}_t, \underline{x}_t] = (1 \ 1 \ \dots \ 1) \{\text{diag}[\exp(\alpha)]\Pi'\}^h \mathbf{z}_t.$$

15.4.1 Kitagawa-Hamilton inversion in a regime-switching environment

This example illustrates Kitagawa-Hamilton inversion in a regime-switching environment.

```
# Purpose: This example illustrates Kitagawa-Hamilton inversion in a regime-switching
# environment
library(DTAM)
library(expm)
library(mvtnorm)
library(optimx)

bond_prices_RS <- function(model, Maturities_decompo){
  # This function computes the loadings of yields on pricing factors
  # (Markov chain and Gaussian factors).
  # Modeled yields are given by:
  # R_t = A_yds_z' * z_t + B_yds_s' + A_yds_s' * s_t
  # -----
```

```

# Maturities_decompo defines the maturities of the yields considered.
# Denote by k the number of these yields.
# -----

mu_r <- model$mu_r
n_r <- length(mu_r)

# Construct matrix of transition probabilities:
J <- 3
Pi <- matrix(0, J*n_r, J*n_r)

p_ES <- logistic(model$alpha_ES + model$beta_ES*mu_r)
p_TS <- logistic(model$alpha_TS + model$beta_TS*mu_r)
p_SS <- logistic(model$alpha_SS + model$beta_SS*mu_r)
p_ST <- logistic(model$alpha_ST + model$beta_ST*mu_r)*(1 - p_SS)

p_EE <- 1-p_ES
p_TT <- 1-p_TS
p_SE <- 1-p_SS-p_ST

p_u <- logistic(model$alpha_u + model$beta_u*mu_r)
p_d <- logistic(model$alpha_d + model$beta_d*mu_r)

# If current regime is Easing -----
Pi[1:n_r, 1:n_r] <- diag(p_EE*(1-p_d))
# Rate cut (25 bps:)
Pi[2:n_r, 1:(n_r-1)] <- Pi[2:n_r, 1:(n_r-1)] +
  diag(p_EE[2:n_r]*p_d[2:n_r]*(1-model$ kappa_d))
# Rate cut (50 bps:)
Pi[3:n_r, 1:(n_r-2)] <- Pi[3:n_r, 1:(n_r-2)] +
  diag(p_EE[3:n_r]*p_d[3:n_r]*model$ kappa_d)
# Correction for impossibility to have rate cut when the rate is low:
Pi[1,1] <- p_EE[1]
Pi[2,1] <- p_EE[2] - Pi[2,2]
# Change in regime towards S:
Pi[1:n_r, n_r + (1:n_r)] <- diag(p_ES)

# If current regime is Status Quo -----
Pi[n_r + (1:n_r), 1*n_r + (1:n_r)] <- diag(p_SS)
Pi[n_r + (1:n_r), 0*n_r + (1:n_r)] <- diag(p_SE)
Pi[n_r + (1:n_r), 2*n_r + (1:n_r)] <- diag(p_ST)

# If current regime is Tightening -----
Pi[2*n_r + (1:n_r), 2*n_r + (1:n_r)] <- diag(p_TT*(1-p_u))
# Rate hike (25 bps:)

```



```

Pi[2*n_r + (1:(n_r-1)),2*n_r + (2:n_r)] <-
  Pi[2*n_r + (1:(n_r-1)),2*n_r + (2:n_r)] +
  diag(p_TT[1:(n_r-1)]*p_u[1:(n_r-1)]*(1-model$kappa_u))
# Rate hike (50 bps:)
Pi[2*n_r + (1:(n_r-2)),2*n_r + (3:n_r)] <-
  Pi[2*n_r + (1:(n_r-2)),2*n_r + (3:n_r)] +
  diag(p_TT[1:(n_r-2)]*p_u[1:(n_r-2)]*model$kappa_u)
# Correction for impossibility to have rate hikes when the rate is high:
Pi[J*n_r,J*n_r] <- p_TT[n_r]
Pi[J*n_r-1,J*n_r] <- p_TT[n_r-1] - Pi[J*n_r-1,J*n_r-1]
# Change in regime towards S:
Pi[2*n_r + (1:n_r),n_r + (1:n_r)] <- diag(p_TS)

res_pricing <- compute_LT_RS(alpha = matrix(1,J,1) %x% matrix(-mu_r,ncol=1),Pi = Pi,
                             Maturities_decompo = Maturities_decompo,indic_add_current = TRUE)

A_yds_z <- -log(res_pricing$A)/(matrix(1,J*n_r,1) %*% matrix(res_pricing$H+1,nrow=1))

# Process Gaussian part:
res_AB_Gauss <- compute_Gaussian_fast(model,Maturities_decompo)
nb_mat <- dim(Maturities_decompo)[1]
H <- res_AB_Gauss$H + 1
# Add current date:
res_AB_Gauss$PHI <- res_AB_Gauss$PHI + array(diag(2),c(2,2,nb_mat))
A_yds_s <- matrix(NaN,2,nb_mat)
B_yds_s <- matrix(NaN,1,nb_mat)
for(i in 1:nb_mat){
  A_yds_s[,i] <- t(res_AB_Gauss$PHI[,i]) %*% (model$delta1) / H[i]
  B_yds_s[i] <- (t(res_AB_Gauss$MU[,1,i]) %*% (model$delta1) -
                 t(model$delta1) %*% res_AB_Gauss$SIGMA[,i] %*% model$delta1)/2 / H[i]
}
return(list(Pi = Pi,
            H = res_pricing$H,
            A_yds_z = A_yds_z,
            A_yds_s = A_yds_s,
            B_yds_s = B_yds_s))
}

filter_model_RS <- function(model,Y,Maturities_decompo,
                             vec_std_dev,indic_smoother=FALSE){
  # This function estimates the probabilities of being in the different
  # monetary policy regimes depending on the sample Y.
  # Note that one date (the first one) is lost due to the auto-regressive
  # nature of s_t.
  # There are nine regimes, as the extended Markovian vector is

```

```

#       $Z_t = z_{\{m,t\}} \times z_{\{m,t-1\}}$ .
#      Hence, since  $z_{\{m,t\}}$  is (1) Easing, (2) Status Quo, (3) Tightening,
#      We have:  $Z_t = e_1 \Rightarrow z_{\{m,t\}} = \text{Easing}, z_{\{m,t-1\}} = \text{Easing}.$ 
#       $Z_t = e_2 \Rightarrow z_{\{m,t\}} = \text{Easing}, z_{\{m,t-1\}} = \text{Status Quo}$ 
#       $Z_t = e_3 \Rightarrow z_{\{m,t\}} = \text{Easing}, z_{\{m,t-1\}} = \text{Tightening}$ 
#       $Z_t = e_4 \Rightarrow z_{\{m,t\}} = \text{Status Quo}, z_{\{m,t-1\}} = \text{Easing}$ 
#      etc.
# Maturities_decompo defines the maturities of the yields considered.
#      Denote by  $k$  the number of these yields.
# vec_std_dev is of dimension  $k \times 1$ ; it defines the standard deviations of the
#      measurement errors. Some of them have to be equal
#      to zero (number = number of Gaussian factors).
# Y contains  $1+k$  yields (where  $k$  is the dimension of Maturities_decompo).
#      The first column of Y is the target rate (which defines  $z_{\{r,t\}}$ )
# vec_std_dev is a vector of 0 and 1. The zero indicates the position of the
#      yields measured without pricing error.

Y <- as.matrix(Y)
n_r <- length(mu_r)
TT <- dim(Y)[1]
J <- 3
nb_mat <- dim(Maturities_decompo)[1]

mu_r <- model$mu_r

res_pricing <- bond_prices_RS(model, Maturities_decompo)
Pi <- res_pricing$Pi
A_yds_z <- res_pricing$A_yds_z
A_yds_s <- res_pricing$A_yds_s
B_yds_s <- res_pricing$B_yds_s

p_ES <- logistic(model$alpha_ES + model$beta_ES*mu_r)
p_SS <- logistic(model$alpha_SS + model$beta_SS*mu_r)
p_TS <- logistic(model$alpha_TS + model$beta_TS*mu_r)
p_ST <- logistic(model$alpha_ST + model$beta_ST*mu_r)*(1 - p_SS)

p_EE <- 1-p_ES
p_TT <- 1-p_TS
p_SE <- 1-p_SS-p_ST

# Determine matrix of transition probabilities of  $z_{\{m,t\}} \times z_{\{m,t-1\}}$  in the
# data (this depends on the prevailing target rate.)
# step 1: Determine index of prevailing target rate:
step <- model$mu_r[2] - model$mu_r[1]
index_rates <- round(Y[, "rbar"] / step + 1)

```

```

# step 2:
PI <- array(0,c(3,3,TT))
PI[1,1,] <- p_EE[index_rates]
PI[1,2,] <- p_ES[index_rates]
PI[2,1,] <- p_SE[index_rates]
PI[2,2,] <- p_SS[index_rates]
PI[2,3,] <- p_ST[index_rates]
PI[3,2,] <- p_TS[index_rates]
PI[3,3,] <- p_TT[index_rates]
# Convert
PI_9x9 <- make_Pi_z_kron_z(PI)

# Compute values of Gaussian factor, depending on regime. -----
# The regime variable is  $Z_t = z_{\{m,t\}} \times z_{\{m,t-1\}}$ . Hence 9 regimes.
matur_Oerror <- which(vec_std_dev==0)
nb_matur_Oerror <- length(matur_Oerror)
if(nb_matur_Oerror != dim(model$Phi)[1]){
  print("Inconsistency between number of yields perfectly observed and dimension of Phi.")
  return(0)
}
B_s_Oerror <- B_yds_s[1,matur_Oerror]
A_s_Oerror <- A_yds_s[,matur_Oerror]
# Determine model-implied yields, for each of the 3 regimes, on each date t:
A_Z_t <- array(0,c(TT,nb_mat,J))
for(j in 1:J){
  A_yds_z_aux <- A_yds_z[((j-1)*n_r+1):(j*n_r),]
  A_Z_t[,j] <- A_yds_z_aux[index_rates,]
}
# For each regime, compute the values of the Gaussian factors:
S_t <- array(0,c(TT,nb_matur_Oerror,J))
A1_1 <- solve(A_s_Oerror)
for(j in 1:J){
  S_t[,j] <- (Y[,matur_Oerror+1] - A_Z_t[,matur_Oerror,j] -
    t(matrix(B_s_Oerror,nb_matur_Oerror,TT))) %*% A1_1
}

# For each of the J*J regimes, compute epsilon_s (and log-likelihood): -----
loglik_epsilon_s <- matrix(0,TT,J*J)
for(zm in 1:J){
  s_t <- S_t[,zm]
  for(zm_1 in 1:J){
    s_t_1 <- rbind(NaN,S_t[1:(TT-1),,zm_1])
    epsilon_s <- s_t -
      t(matrix(model$mu,nb_matur_Oerror,TT)) - s_t_1 %*% t(model$Phi)
    loglik_epsilon_s[, (zm-1)*J+zm_1] <-

```

```

    dmvnorm(epsilon_s, sigma = model$Sigma, log = TRUE)
  }
}

# For each of the J*J regimes, compute pricing errors (and log-likelihood):-----

Modeled_yields <- array(0, c(TT, J, nb_mat))
matur_with_error <- which(vec_std_dev != 0)

loglik_pricing_error <- matrix(0, TT, J*J)
for(j in 1:J){
  modeled_yields <- t(matrix(B_yds_s, nb_mat, TT)) + S_t[, j] %*% A_yds_s + A_Z_t[, j]
  pricing_error <- Y[, matur_with_error+1] - modeled_yields[, matur_with_error]
  # Note in previous line: "+1" because the first column of Y is the target.
  loglik_pricing_error[, (J*(j-1)+1):(J*J)] <-
    dmvnorm(pricing_error,
             sigma = diag(c(model$std_pricing_error*vec_std_dev[matur_with_error]^2))
             log = TRUE)
  # Store modeled yields (for final outputs):
  Modeled_yields[, j, ] <- modeled_yields
}

# Compute part of log-likelihood associated with dynamics of z_{r,t}: -----
index_rates_1 <- c(NaN, index_rates[1:(TT-1)])

# Some dates correspond to an "impossible" change in rate (larger than 50 bps),
# proceed as if change = 50 bps:
index_rates_modif <- index_rates
large_positive_move <- which(index_rates - index_rates_1 > 2)
index_rates_modif[large_positive_move] <- index_rates_1[large_positive_move] + 2
large_negative_move <- which(index_rates - index_rates_1 < -2)
index_rates_modif[large_negative_move] <- index_rates_1[large_negative_move] - 2

loglik_dynamics_z_r <- matrix(0, TT, J*J)
z_r <- matrix(0, TT, n_r)
z_r[index_rates]
for(zm in 1:J){
  for(zm_1 in 1:J){
    Pi_sub <- Pi[(n_r*(zm_1-1)+1):(n_r*z_m), (n_r*(zm-1)+1):(n_r*z_m)]
    # Proba_numer is the vector of proba to have (z_{r,t}, z_{m,t})
    # given (z_{r,t-1}, z_{m,t-1})
    Proba_numer <- Pi_sub[(index_rates_modif-1)*n_r + index_rates_1]
    # Prob_denom is the vector of proba to have z_{m,t} given (z_{r,t-1}, z_{m,t-1})

```

```

    Pi_sub_sum <- apply(Pi_sub,1,sum)
    Proba_denom <- Pi_sub_sum[index_rates_1]
    Proba_denom[Proba_denom==0] <- 1 # Just to avoid NaN
    # Compute ratio (this is  $P(z_{\{r,t\}}/z_{\{r,t-1\}},z_{\{m,t\}},z_{\{m,t-1\}})$ ):
    loglik_dynamics_z_r[, (zm-1)*J+zm_1] <- log(Proba_numer/Proba_denom)
    loglik_dynamics_z_r[is.infinite(loglik_dynamics_z_r)] <- -100000
  }
}

# -----
# Compute resulting log-likelihood, conditional on  $Z_t$ , adding the determinant
# of the Jacobian matrix associated with the relationship between
# yields observed without errors, and Gaussian factors  $s_t$ .
loglik_per_regime <- log(abs(det(A1_1))) +
  loglik_epsilon_s + loglik_pricing_error + loglik_dynamics_z_r

# Use the Kitagawa-Hamilton filter: -----
res_KH <- KH_filter(PI_9x9,loglik_per_regime[2:TT,])
ksi_matrix <- res_KH$ksi_matrix
loglik_vec <- res_KH$loglik_vec
loglik <- res_KH$loglik
# Probabilities associated with  $z_{\{m,t\}}$  (and not  $z_{\{m,t\}} \times z_{\{m,t-1\}}$ ):
ksi_tt <- ksi_matrix %*% (diag(J) %x% matrix(1,J,1))

# Compute estimates of Gaussian factors: -----
S_t_hat <- matrix(0,TT-1,dim(S_t)[2])
for(j in 1:J){
  S_t_hat <- S_t_hat + S_t[2:TT,,j] *
    (apply(ksi_matrix[, (J*(j-1)+1):(j*J)],1,sum) %*% matrix(1,1,dim(S_t)[2]))
}

# If requested, use the Kitagawa-Hamilton smoother: -----
if(indic_smoother){
  ksi_tT <- KH_smoother(PI_9x9,loglik_per_regime[2:TT,])
  # Probabilities associated with  $z_{\{m,t\}}$  (and not  $z_{\{m,t\}} \times z_{\{m,t-1\}}$ ):
  ksi_tT <- ksi_tT %*% (diag(J) %x% matrix(1,J,1))
}else{
  ksi_tT <- NaN
}

return(list(A_yds_z = A_yds_z,
  A_yds_s = A_yds_s,
  B_yds_s = B_yds_s,
  A_Z_t = A_Z_t,
  Pi = Pi,

```

```

        H = res_pricing$H+1,
        PI_9x9 = PI_9x9,
        loglik_vec = loglik_vec,
        loglik = loglik,
        index_rates = index_rates,
        S_t_hat = S_t_hat,
        ksi_tT = ksi_tT,
        Modeled_yields = Modeled_yields
    ))
}

# Prepare dataset: -----

# Compute approximate policy rate:
YC_US_weekly$rbar <- YC_US_weekly$DFEDTAR

indic_where_use_DW <- which(is.na(YC_US_weekly$rbar)&
                           (!is.na(YC_US_weekly$DFEDTARU))&
                           (!is.na(YC_US_weekly$DFEDTARL)))
YC_US_weekly$rbar[indic_where_use_DW] <-
  0.5*YC_US_weekly$DFEDTARU[indic_where_use_DW] +
  0.5*YC_US_weekly$DFEDTARL[indic_where_use_DW]
YC_US_weekly$rbar <- round(4*YC_US_weekly$rbar)/4

Maturities_decompo <- matrix(0,6,6)
Maturities_decompo[1,] <- c(3, 0, 0, 0, 0, 0)
Maturities_decompo[2,] <- c(0, 4, 0, 0, 0, 0)
Maturities_decompo[3,] <- c(1, 0, 2, 0, 0, 0)
Maturities_decompo[4,] <- c(1, 0, 0, 2, 0, 0)
Maturities_decompo[5,] <- c(1, 0, 0, 0, 2, 0)
Maturities_decompo[6,] <- c(2, 0, 0, 0, 1, 2)

Y <- YC_US_weekly[,c("rbar", "DTB3", "DTB6", "THREEFY1", "THREEFY2", "THREEFY3")]
Maturities_decompo <- Maturities_decompo[1:5,1:5]
vec_std_dev <- c(1,0,1,1,0)

Maturities_decompo <- matrix(0,4,4)
Maturities_decompo[1,] <- c(12, 0, 0, 0)
Maturities_decompo[2,] <- c(1, 2, 0, 0)
Maturities_decompo[3,] <- c(1, 0, 2, 0)
Maturities_decompo[4,] <- c(1, 0, 0, 2)

Y <- YC_US_weekly[,c("rbar", "DTB3", "DTB6", "THREEFY1", "THREEFY2")]

```

```

vec_std_dev <- c(1,0,1,0)

vec_dates <- YC_US_weekly$date[complete.cases(Y)]

Y <- Y[complete.cases(Y),]/52/100 # weekly returns
TT <- dim(Y)[1]

logistic <- function(theta){
  return(exp(theta)/(1+exp(theta)))
}
invlogistic <- function(theta){
  return(log(theta/(1-theta)))
}

rmax <- .10
step <- .0025
mu_r <- seq(0,rmax,by=step)/52 # weekly return
n_r <- length(mu_r)

# Specify initial model (used as starting point during estimation) -----

alpha_d <- -2
beta_d <- 245
alpha_u <- -2
beta_u <- -380

alpha_ES <- -1
beta_ES <- -900
alpha_TS <- -5
beta_TS <- 2400

alpha_SS <- 2.3
beta_SS <- -420
p_SS <- logistic(alpha_SS + beta_SS*mu_r)
alpha_ST <- 1.4
beta_ST <- -800
p_ST <- (1-p_SS)*logistic(alpha_ST + beta_ST*mu_r)
p_SE <- 1 - p_SS - p_ST

kappa_u <- .12
kappa_d <- .08

```

```

# Gaussian factors:
lambda <- .1
Phi <- diag(2)
Phi[1,1] <- .975
Phi[2,2] <- .986
Phi[1,2] <- .05
Sigma <- diag(2)
rho <- 0.000000001
Sigma[1,2] <- rho
Sigma[2,1] <- rho

Vs <- matrix(solve(diag(4)- Phi %x% Phi) %*% c(Sigma),2,2)
std_s <- sqrt(diag(Vs))

std_dev_by_s <- .5*sd(Y[, "rbar"])

model <- list(
  alpha_d = alpha_d,
  beta_d = beta_d,
  alpha_u = alpha_u,
  beta_u = beta_u,
  alpha_ES = alpha_ES,
  beta_ES = beta_ES,
  alpha_SS = alpha_SS,
  beta_SS = beta_SS,
  alpha_TS = alpha_TS,
  beta_TS = beta_TS,
  alpha_ST = alpha_ST,
  beta_ST = beta_ST,
  kappa_u = kappa_u,
  kappa_d = kappa_d,
  mu_r = mu_r,
  mu = matrix(0,2,1),
  Phi = Phi,
  Sigma = Sigma,
  delta0 = mean(Y[, "DTB3"]-Y[, "rbar"]),
  delta1 = matrix(c(std_dev_by_s/std_s[1],0),2,1),
  std_pricing_error = .25/52/100
)

param2model <- function(param,model_ini){
  # construct matrix of transition probabilities

  model <- model_ini

```



```

model$Phi <- diag(2)
model$Phi[1,1] <- logistic(param[1])
model$Phi[2,2] <- logistic(param[2])
model$Phi[1,2] <- exp(param[3])

model$alpha_d = param[4]
model$beta_d = param[5]*1000
model$alpha_u = param[6]
model$beta_u = param[7]*1000
model$alpha_ES = param[8]
model$beta_ES = param[9]*1000
model$alpha_SS = param[10]
model$beta_SS = param[11]*1000
model$alpha_TS = param[12]
model$beta_TS = param[13]*1000
model$alpha_ST = param[14]
model$beta_ST = param[15]*1000
model$kappa_u = logistic(param[16])
model$kappa_d = logistic(param[17])

model$std_pricing_error <- exp(param[18])

Vs <- matrix(solve(diag(4)- model$Phi %x% model$Phi) %*% c(model$Sigma),2,2)
std_s <- sqrt(diag(Vs))
model$delta1 <- matrix(c(std_dev_by_s/std_s[1],0),2,1)

return(model)
}

model2param <- function(model){

  param <- matrix(NA,18,1)

  param[1] <- invlogistic(model$Phi[1,1])
  param[2] <- invlogistic(model$Phi[2,2])
  param[3] <- log(model$Phi[1,2])

  model$alpha_d -> param[4]
  model$beta_d/1000 -> param[5]
  model$alpha_u -> param[6]
  model$beta_u/1000 -> param[7]
  model$alpha_ES -> param[8]
  model$beta_ES/1000 -> param[9]
  model$alpha_SS -> param[10]
  model$beta_SS/1000 -> param[11]

```

```

model$alpha_TS -> param[12]
model$beta_TS/1000 -> param[13]
model$alpha_ST -> param[14]
model$beta_ST/1000 -> param[15]

invlogistic(model$kappa_u) -> param[16]
invlogistic(model$kappa_d) -> param[17]

log(model$std_pricing_error) -> param[18]

Vs <- matrix(solve(diag(4)- model$Phi %x% model$Phi) %*% c(model$Sigma),2,2)
std_s <- sqrt(diag(Vs))

# invlogistic(model$delta1*std_s/(1/52/100)) -> param[17:18]

return(param)
}

loglik4optim <- function(param,Y,Maturities_decompo,
                          vec_std_dev,model_ini){
  model <- param2model(param,model_ini = model_ini)
  res <- filter_model_RS(model,Y,Maturities_decompo,
                          vec_std_dev)

  return(-res$loglik)
}

model_ini <- model
param_est <- c(model2param(model))

if(indic_reestim){

  nb_loop <- 5

  for(i in 1:nb_loop){
    print(paste("--- Loop ",i," ---",sep=""))
    # for(method in c("nlsminb")){
    for(method in c("nlsminb","Nelder-Mead")){
      res.optim <- optimx(par=param_est,
                          fn=loglik4optim,
                          Y=Y,Maturities_decompo=Maturities_decompo,
                          vec_std_dev=vec_std_dev,
                          model_ini = model_ini,
                          method=method,
                          control=list(trace=TRUE,

```

```

                                maxit=ifelse(method=="Nelder-Mead",1000,20),
                                kkt = FALSE))
    print(paste("--- Log-lik (" ,method,"): ",round(res.optim$value,2)," ---",sep=""))
    param_est <- c(as.matrix(res.optim)[1:length(param_est)])
  }
}
save(param_est,file= "stored_results/param_RS_inversion.Rdat")
}else{
  # In that case, use paraemters estimated previously:
  load(file= "stored_results/param_RS_inversion.Rdat")
}

# Prepare plot -----
# Estimated model:
model <- param2model(param_est,model_ini = model_ini)
res <- filter_model_RS(model,Y,Maturities_decompo,
                      vec_std_dev,indic_smoother = TRUE)

layout_matrix <- matrix(c(1, 1,
                          2, 3),
                        nrow = 2, byrow = TRUE)

layout(layout_matrix)

p_E <- res$ksi_tT[,1]
p_T <- res$ksi_tT[,3]
p_S <- 1 - p_E - p_T

Modeled_yields <- res$Modeled_yields[2:TT,1,] * (matrix(p_E,ncol=1) %%%
                                                matrix(1,1,dim(res$Modeled_yields)[3])) +
  res$Modeled_yields[2:TT,2,] * (matrix(p_S,ncol=1) %%%
                                matrix(1,1,dim(res$Modeled_yields)[3])) +
  res$Modeled_yields[2:TT,3,] * (matrix(p_T,ncol=1) %%%
                                matrix(1,1,dim(res$Modeled_yields)[3]))

par(plt=c(.075,.95,.1,.85))
max_y <- 10
plot(vec_dates[2:TT],rep(0,TT-1),ylim=c(0,max_y),col="white",
     xlab="",ylab="",las=1,main = "(a) Target rate and monetary-policy regime",
     yaxt="n")
axis(side=2, at=seq(0,10,by=2),
     labels=paste(seq(0,10,by=2),"%",sep=""),
     col.axis="black",las=1)

```

```

polygon(c(vec_dates[2:TT],rev(vec_dates[2:TT])),max_y*c(p_E,0*rev(p_E)),col="darkgrey",
        border=NaN)
polygon(c(vec_dates[2:TT],rev(vec_dates[2:TT])),max_y*c(p_E,rev(1-p_T)),col="lightgrey",
        border=NaN)
lines(vec_dates[2:TT],52*100*Y[2:TT,"rbar"],lwd=2,ylim=c(0,max_y))

scale_factor <- max_y
axis(side=4, at=pretty(c(0,max_y)),
     labels=pretty(c(0,max_y)) / scale_factor,
     col.axis="black",las=1)

legend("topright",
      legend=c("Target rate (lhs)",
               "Proba. of Easing regime (rhs)",
               "Proba. of Status-Quo regime (rhs)",
               "Proba. of Tightening regime (rhs)"),
      lty=c(1, NA, NA, NA),      # line type
      pch=c(NA, 22, 22, 22),    # filled squares
      pt.bg = c(NA, "darkgrey", "lightgrey", "white"), # fill
      pt.cex = 1.8,
      col = c("black", "black", "black", "black"),    # border
      lwd = c(2,1,1,1),
      bg = "white")

# Compute model-implied yields for the different regimes:
nb_mat <- length(vec_std_dev)
TT <- dim(Y)[1]
modeled_yields_E <- t(matrix(res$B_yds_s,nb_mat,TT-1)) + res$S_t_hat %*% res$A_yds_s +
  res$A_Z_t[2:TT,,1]
modeled_yields_S <- t(matrix(res$B_yds_s,nb_mat,TT-1)) + res$S_t_hat %*% res$A_yds_s +
  res$A_Z_t[2:TT,,2]
modeled_yields_T <- t(matrix(res$B_yds_s,nb_mat,TT-1)) + res$S_t_hat %*% res$A_yds_s +
  res$A_Z_t[2:TT,,3]

par(plt=c(.15,.95,.2,.85))

count <- 0
for(t in c(580,1450)){
  count <- count + 1
  ylim <- c(
    min(modeled_yields_E[t,],modeled_yields_S[t,],modeled_yields_T[t,],
        as.matrix(Y[t,2:dim(Y)[2]])) - .15/52/100,
    max(modeled_yields_E[t,],modeled_yields_S[t,],modeled_yields_T[t,],
        as.matrix(Y[t,2:dim(Y)[2]])) + .5/52/100
  )

```

```

)

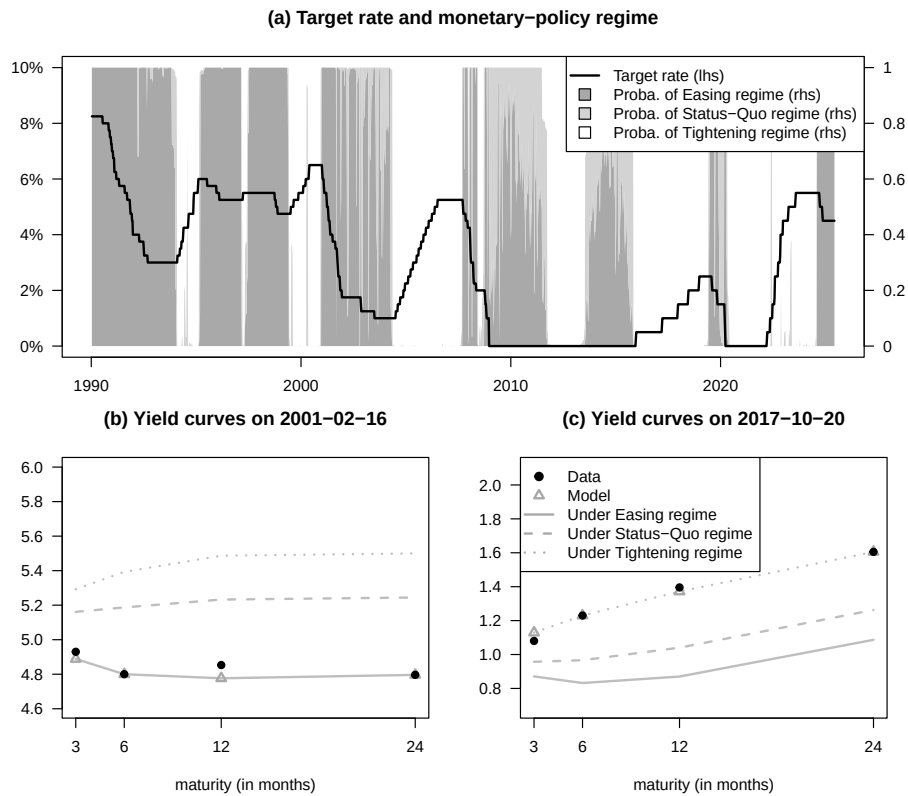
plot(res$H*12/52,0*res$H,ylim=52*100*ylim,col="white",las=1,
      xlab="maturity (in months)",ylab="",
      main=paste("(",letters[count+1],") Yield curves on ",vec_dates[t+1],sep=""),
      xaxt="n")
axis(side=1, at=res$H*12/52,
      labels=res$H*12/52,
      col.axis="black",las=1)

lines(res$H*12/52,52*100*modeled_yields_E[t,],lwd=2,col="grey",lty=1)
lines(res$H*12/52,52*100*modeled_yields_S[t,],lwd=2,col="grey",lty=2)
lines(res$H*12/52,52*100*modeled_yields_T[t,],lwd=2,col="grey",lty=3)

points(res$H*12/52,52*100*Modeled_yields[t,],pch=2,lwd=2,col="darkgrey")
points(res$H*12/52,52*100*Y[t+1,2:dim(Y)[2]],pch=19)

if(count == 2){
  legend("topleft",
        legend=c("Data","Model",
                  "Under Easing regime","Under Status-Quo regime","Under Tightening regime"),
        lty=c(NaN,NaN,1:3),
        pch=c(19,2,NaN,NaN,NaN),
        col=c("black","dark grey","dark grey","dark grey","dark grey"),
        lwd=2,bg = "white")
}
}

```



15.4.2 Inversion output and state probabilities

This example shows the inversion output and state probabilities under the estimated model.

Purpose: This example shows the inversion output and state probabilities under the estimated model.

Compute state-transition probabilities implied by the estimated model.

```
p_ES <- logistic(model$alpha_ES + model$beta_ES*mu_r)
p_SS <- logistic(model$alpha_SS + model$beta_SS*mu_r)
p_TS <- logistic(model$alpha_TS + model$beta_TS*mu_r)
p_ST <- logistic(model$alpha_ST + model$beta_ST*mu_r)*(1 - p_SS)
```

```
p_EE <- 1-p_ES
p_TT <- 1-p_TS
p_SE <- 1-p_SS-p_ST
```

```
p_u <- logistic(model$alpha_u + model$beta_u*mu_r)
p_d <- logistic(model$alpha_d + model$beta_d*mu_r)
```

```

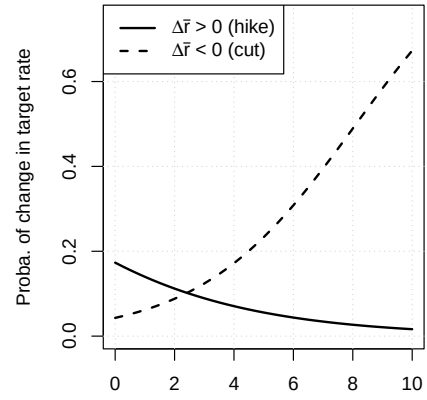
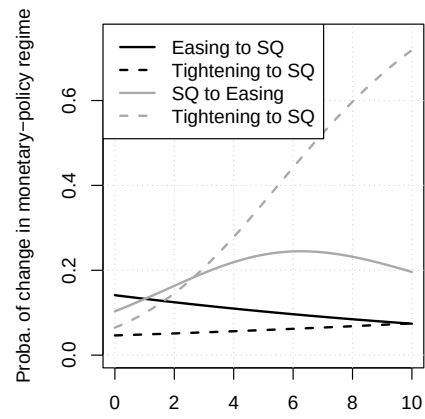
# The first panel summarizes policy-regime changes.
# Plot regime-switching and target-rate transition probabilities.
par(mfrow=c(1,2))
par(plt=c(.25,.95,.15,.9))
plot(model$mu_r*52*100,p_ES,type="l",ylim=c(0,.75),col="white",
      xlab=expression(paste(bar(r)," in percent",sep="")),
      ylab="Proba. of change in monetary-policy regime")
grid()
lines(model$mu_r*52*100,p_ES,col="black",lwd=2,lty=1)
lines(model$mu_r*52*100,p_TS,col="black",lwd=2,lty=2)
lines(model$mu_r*52*100,p_SE,col="darkgrey",lwd=2,lty=1)
lines(model$mu_r*52*100,p_ST,col="darkgrey",lwd=2,lty=2)

legend("topleft",
      legend=c("Easing to SQ","Tightening to SQ",
               "SQ to Easing","Tightening to SQ"),
      lty=c(1,2,1,2),
      col=c("black","black","darkgrey","darkgrey"),
      lwd=2,bg = "white")

# The second panel summarizes target-rate hikes and cuts.
plot(model$mu_r*52*100,p_ES,type="l",ylim=c(0,.75),col="white",
      xlab=expression(paste(bar(r)," in percent",sep="")),
      ylab="Proba. of change in target rate")
grid()
lines(model$mu_r*52*100,p_u,col="black",lwd=2,lty=1)
lines(model$mu_r*52*100,p_d,col="black",lwd=2,lty=2)

legend("topleft",
      legend=c(expression(paste(Delta,bar(r)," > 0 (hike)",sep="")),
               expression(paste(Delta,bar(r)," < 0 (cut)",sep="))),
      lty=c(1,2,1,2),
      col=c("black","black","darkgrey","darkgrey"),
      lwd=2,bg = "white")

```



Chapter 16

Imperfect information and rational expectations

16.1 Imperfect information

This example introduces learning dynamics under imperfect information.

Purpose: This example introduces learning dynamics under imperfect information
`library(DTAM)`

```
set.seed(123)
```

Bansal and Yaron (2004) specification:

```
mu_c <- 0 #.0015
```

```
phi <- .979
```

```
sigma <- 0.0078
```

```
varphi_e <- .044
```

```
mu <- matrix(c(0,mu_c),2,1)
```

```
Phi <- matrix(0,2,2)
```

```
Phi[1,1] <- phi
```

```
Phi[2,1] <- 1
```

```
Sigma12 <- matrix(0,2,2)
```

```
Sigma12[1,1] <- varphi_e*sigma
```

```
Sigma12[2,2] <- sigma
```

```
A <- matrix(c(0,1),1,2)
```

```
B <- 0
```

```
Omega12 <- 0.0000001
```

```
model <- list(mu=mu,Phi=Phi,Sigma12=Sigma12,
```

```

        A=A,B=B,Omega12=Omega12,
        Lambda0=0*Phi,Lambda1=0*Phi)
res <- solve_learning(model = model)
# Should be equivalent given no Lambda0 and Lambda1:
# res <- make_stationary_filter(mu,Phi,Sigma12,A,B,Omega12)

# Check analytical formula:
# p11 <- sigma^2/2*(sqrt((1 - phi^2 - varphi_e^2)^2+4*varphi_e^2) - (1 - phi^2 - varph
# cbind(p11,P[1,1])
# P_check <- matrix(c(p11,0,0,0),2,2)
# Pstar_check <- Phi %%% P_check %%% t(Phi) + Sigma12 %%% t(Sigma12)

model_ww <- list(mu=res$mu_ww,Phi=res$Phi_ww,Sigma12=res$Sigma12_ww)

TT <- 200
ww <- simul_GaussianVAR(model_ww,nb.sim = TT)

dc <- ww[,1:2] %%% t(A)

stdv_w <- sqrt(diag(res$P))
lower_bound <- ww[,3] - 2*stdv_w[1]
upper_bound <- ww[,3] + 2*stdv_w[1]

# Prepare plot:

par(plt=c(.1,.95,.1,.95))

col_dc <- "#11111177"
col_CI <- "#77777744"
col_wt <- "black"
col_wtt <- "black"

plot(dc,col="white",
      ylab="",xlab="",
      las=1,ylim=c(min(dc),1.0*max(dc)))
polygon(c(1:TT,TT:1),c(lower_bound,rev(upper_bound)),
        col=col_CI,border=NaN)

grid()
lines(dc,lwd=1,col=col_dc)
lines(ww[,1],lwd=2,col=col_wt)
lines(ww[,3],lty=3,lwd=3,col=col_wtt)

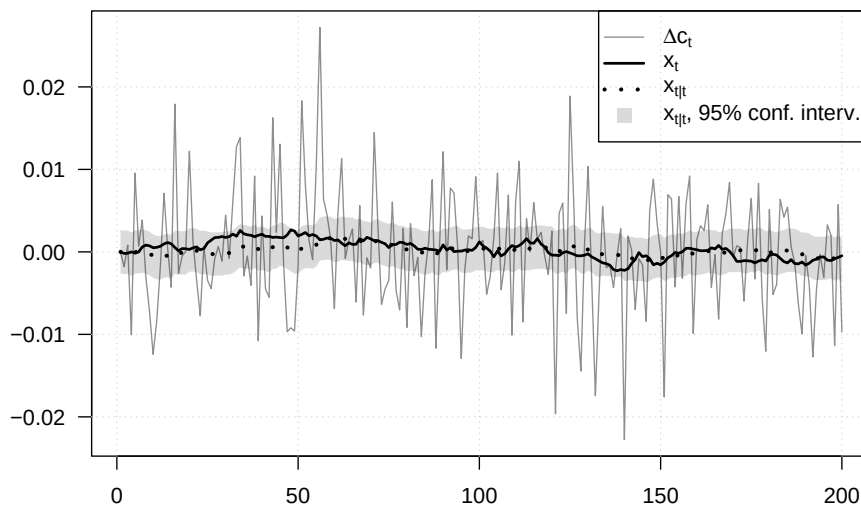
legend("topright",

```

```

c(expression(paste(Delta,c[t],sep="")),
  expression(x[t]),
  expression(x[t*'|'*t]),
  expression(paste(x[t*'|'*t],", 95% conf. interv.",sep="))),
lwd=c(1,2,3,NaN),
lty=c(1,1,3,NaN),
pch=c(NaN,NaN,NaN,15),
pt.cex=c(NaN,NaN,NaN,1.5),
col=c(col_dc,col_wt,col_wtt,col_CI),
seg.len = 2)

```



```

# Compare with KF:
# Y_t <- matrix(dc,ncol=1)
# nu_t <- matrix(0,TT,2)
# H <- Phi
# N <- Sigma12
# mu_t <- matrix(0,TT,1)
# G <- A
# M <- matrix(Omega12)
# Sigma_0 <- diag(2)
# rho_0 <- matrix(0,2,1)
# res_KF <- Kalman_filter(Y_t,nu_t,H,N,mu_t,G,M,Sigma_0,rho_0)
# lines(res_KF$r[,1],col="red",lwd=2,lty=3)

```

This short block computes auxiliary objects used in the imperfect-information examples that follow.

```

# Purpose: Compute the low-frequency contribution to consumption-growth variance.
# Computation of share of variance of dc accounted for by low-frequency component:

```

```
n <- length(model$mu)
Vww <- matrix(solve(diag(4*4) - model_ww$Phi %x% model_ww$Phi) %*%
               c(model_ww$Sigma12 %*% t(model_ww$Sigma12)),4,4)
print(Vww[1,1]/Vww[2,2])
```

```
## [1] 0.04451086
```

16.1.1 Filtered beliefs and true states under imperfect information

This example compares filtered beliefs and true states under imperfect information.

Purpose: This example compares filtered beliefs and true states under imperfect information.

```
H <- 60 # max horizon
```

Prepare IRF in the context of imperfectly observed long-run risk component

```
Phi_ww <- res$Phi_ww
Sigma_ww <- res$Sigma12_ww
```

```
par(mfrow=c(1,2))
par(plt=c(.2,.95,.2,.8))
```

```
for(i in 1:2){
```

```
  x <- matrix(Sigma_ww[,i],ncol=1)
  IRF <- x
  for(t in 1:H){
    x <- Phi_ww %*% x
    IRF <- cbind(IRF,x)
  }
  IRF <- 100*IRF
```

```
  if(i==1){
    main.t <- expression(paste("(a) Shock on ",epsilon[x*','*t],sep=""))
  }else{
    main.t <- expression(paste("(b) Shock on ",epsilon[c*','*t],sep=""))
  }
```

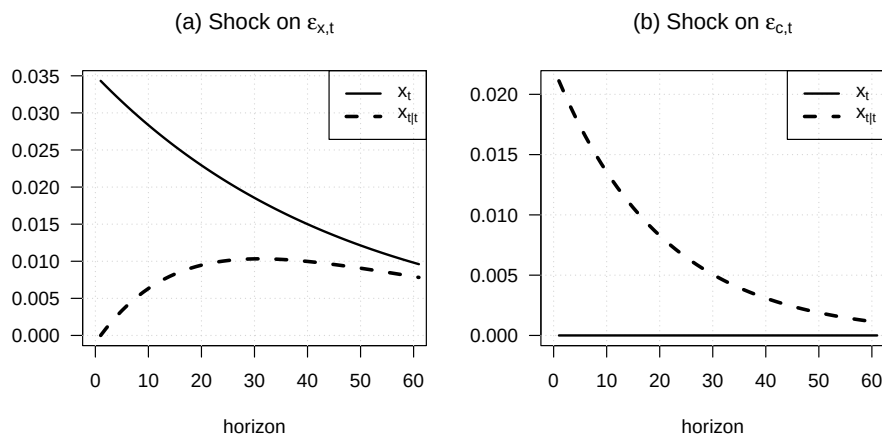
```
  plot(0:H,IRF[1,],col="white",las=1,
       xlab="horizon",ylim=c(0,max(IRF[c(1,3),])),
       ylab="",
```

```

    main=main.t)
  grid()
  lines(IRF[1,],lwd=2,col=col_wt)
  # lines(IRF[2,],lwd=1,col=col_dc)
  lines(IRF[3,],lwd=3,col=col_wtt,lty=2)

  legend("topright",
    c(
      # expression(paste(Delta,c[t],sep="")),
      expression(x[t]),
      expression(x[t*'|'*t])),
    lwd=c(2,3),
    lty=c(1,2),
    col=c(col_wt,col_wtt),
    seg.len = 2)
}

```



16.1.2 Imperfect information in a long-run-risk environment

This example embeds imperfect information in a long-run-risk environment.

Purpose: This example embeds imperfect information in a long-run-risk environment

```
library(DTAM)
```

```

# -----
# Bansal and Yaron (2004): Homoskedastic case
# (using psi_GaussianVAR() for Laplace transform)

```

```

# -----

# Parameterization from Bansal and Yaron (2004):
EIS    <- 1.5
gamma  <- 10
delta  <- .998
mu     <- .0015
mu_d   <- .0015
phi    <- .979
beta   <- 3
sigma  <- 0.0078
varphi_e <- .044
varphi_d <- 4.5

model_basic <- list(
  mu = matrix(c(0,mu,mu_d),3,1),
  Phi = matrix(c(phi,1,beta,rep(0,6)),3,3),
  Sigma = diag(sigma^2*c(varphi_e,1,varphi_d)^2),
  rho = 1/EIS,
  phi = phi,
  gamma = gamma,
  delta = delta,
  sigma = sigma,
  mu_c0 = 0,
  mu_c1 = c(0,1,0),
  n_w = 3
)

# all_EIS <- c(.5,.999,1.5,NaN)
all_EIS <- c(1,2,NaN)
all_col <- c("black","black","black","darkgrey","darkgrey","darkgrey")
all_lty <- c(1,2,3,1,2,3)
all_lwd <- c(2,2,1,2,2,1)

all_gamma <- c(seq(.10001,1.3,length.out=40),1.5,2,3)

all_maxSR <- matrix(0,2*length(all_EIS),length(all_gamma))

count_EIS <- 0

for(std_measurement_noise in c(.0000001,100)){

  # Modify model to take into account imperfect observation -----
  Omega12 <- diag(c(.0000001,std_measurement_noise))

```

```

Omega <- Omega12 %*% t(Omega12)
A <- matrix(c(model_basic$mu_c1,c(1,0,0)),nrow=2,byrow = TRUE)
B <- c(model_basic$mu_c0,0)
VAR_ImperfectInfo <-
  make_stationary_filter(model_basic$mu,
                        model_basic$Phi,
                        t(chol(model_basic$Sigma)),A,B,Omega12,nb_iter = 200)
P <- VAR_ImperfectInfo$P
Pstar <- VAR_ImperfectInfo$Pstar
K <- VAR_ImperfectInfo$K
Sigma_wtt <- K %*% (A %*% Pstar %*% t(A) + Omega) %*% t(K)

model <- model_basic
model$Sigma <- Sigma_wtt
# -----

model_solved <- solve_EZ_SDF(model,psi_GaussianVAR,z_bar_ini=1,
                           nb_loop_z_bar = 20,nb_loop_mu_z1 = 20)
# Check formula (compare with analytical BY formula):
# A1 <- (1 - 1/EIS)/(1 - model_solved$kappa1*model$phi)

for(EIS in all_EIS){
  count_EIS <- count_EIS + 1
  count_gamma <- 0
  model$rho <- 1/EIS
  for(gamma in all_gamma){
    count_gamma <- count_gamma + 1
    model$gamma <- gamma
    if(is.na(EIS)){
      model$rho <- model$gamma
    }

    if((!is.na(EIS)) & (EIS==1)){
      res <- solve_EZ_UnitEIS(model,psi = psi_GaussianVAR,Phi = model$Phi)
    }
    if((!is.na(EIS)) & (EIS!=1)){
      res <- solve_EZ_SDF(model,psi_GaussianVAR,z_bar_ini=5,
                        nb_loop_z_bar = 20,nb_loop_mu_z1 = 20)
    }
    if(is.na(EIS)){
      res <- solve_CRRa(model,psi_GaussianVAR)
    }

    psi_w_2alpha <- psi_GaussianVAR(2*res$alpha,model)
  }
}

```

```

psi_w_alpha <- psi_GaussianVAR(res$alpha,model)
w <- res$Ew
maxSR <- sqrt(exp(t(psi_w_2alpha$a - 2*psi_w_alpha$a) %*% w +
                    psi_w_2alpha$b - 2*psi_w_alpha$b)-1)
all_maxSR[count_EIS,count_gamma] <- maxSR*sqrt(12)
}
}
}

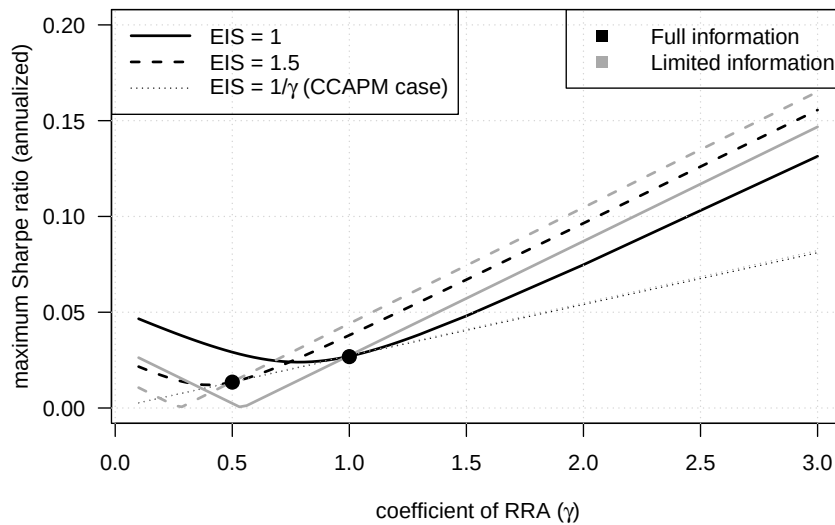
par(plt=c(.15,.95,.2,.99))
plot(all_gamma,(all_maxSR[1,]),ylim=c(0,.2),type="l",
     lwd=all_lwd[1],las=1,col=all_col[1],lty=all_lty[1],
     xlab=expression(paste("coefficient of RRA (" ,gamma,")",sep="")),
     ylab="maximum Sharpe ratio (annualized)")
grid()
for(i in 2:dim(all_maxSR)[1]){
  lines(all_gamma,(all_maxSR[i,]),lwd=all_lwd[i],
        col=all_col[i],lty=all_lty[i])
}

for(i in 1:(length(all_EIS)-1)){
  j <- which.min(abs(all_gamma - 1/all_EIS[i]))
  y <- all_maxSR[length(all_EIS),j]
  points(1/all_EIS[i],y,pch=19,cex=1.4)
}

legend("topleft",
      c(expression(paste("EIS = 1",sep="")),
        expression(paste("EIS = 1.5",sep="")),
        expression(paste("EIS = 1/",gamma," (CCAPM case)",sep="))),
      lwd=all_lwd,
      lty=all_lty,
      col=all_col,
      seg.len = 3
)

legend("topright",
      c(expression(paste("Full information",sep="")),
        expression(paste("Limited information",sep="))),
      lwd=2,
      pch=15,pt.cex = 1.2,
      lty=c(NA,NA),
      col=c("black","darkgrey"),
      seg.len = 3
)

```

16.1.3 How optimistic beliefs affect asset prices and macro dynamics

This example studies how optimistic beliefs affect asset prices and macro dynamics.

Purpose: This example studies how optimistic beliefs affect asset prices and macro dynamics

```
library(DTAM)

max_h <- 30

# -----
# Parameter values (set yours here)
# -----
phi      <- 0.8      # persistence of output gap
sigma_x  <- 0.01
rho      <- 0.9
xi       <- 0.4
mu_g     <- 0.02
sigma_g  <- 0.005
sigma_nu <- 0.01

all_xi <- c(0,.3)

col_xi <- c("grey","black")
lwd_xi <- c(3,2)
types_info <- c(1,2)
```

```

# -----
# State-space objects
# -----
# State vector  $w_t = (g_t, x_t, x_{t-1})'$ 
n <- 3

names_var <- c(expression(g[t]),
               expression(x[t]),
               expression(paste("log(", C[t], ")", sep="")))
names_shock <- c(
  expression(epsilon[g*', '*t]),
  expression(epsilon[x*', '*t]),
  expression(eta[t])
)
nb_var <- length(names_var)-1
nb_shocks <- length(names_shock)

all_IRFs <- array(NA, c(2*n, 1+max_h, nb_shocks, length(all_xi)))

count_xi <- 0
for(xi in all_xi){

  count_xi <- count_xi + 1

  # Lambda_0 = 0
  Lambda0 <- matrix(0, nrow = n, ncol = n)

  # Lambda_1
  Lambda1 <- matrix(0, nrow = n, ncol = n)
  Lambda1[1,1] <- xi # feedback from  $g_{t-1/t-1}$ 

  # mu vector
  mu <- matrix(c((1 - rho)*mu_g, 0, 0), ncol = 1)

  # Phi matrix
  Phi <- matrix(c(
    rho - xi, 0, 0,
    0, phi, 0,
    0, 1, 0
  ), nrow = n, byrow = TRUE)

  #  $\Sigma^{(1/2)}$ 
  Sigma12 <- matrix(c(
    sigma_g, 0,
    0, sigma_x,

```

```

0,      0), nrow = n, byrow = TRUE)

# Full Sigma (if needed)
Sigma <- Sigma12 %*% t(Sigma12)

# -----
# Measurement equation  $y_t = B + A w_t + \Omega^{(1/2)} \eta_t$ 
# -----
A <- t(matrix(c(1, 1, -1,
                1, 0, 0), n, 2))      #  $\Delta c_t = g_t + x_t - x_{t-1}$ 
B <- matrix(0, 2, 1)
Omega12 <- matrix(c(0, sigma_nu), 2, 1)
Omega <- Omega12^2

# -----
# Bundle into a model list (optional)
# -----
model <- list(
  phi = phi,
  rho = rho,
  xi = xi,
  mu_g = mu_g,
  sigma_x = sigma_x,
  sigma_g = sigma_g,
  Lambda0 = Lambda0,
  Lambda1 = Lambda1,
  mu = mu,
  Phi = Phi,
  Sigma12 = Sigma12,
  Sigma = Sigma,
  A = A,
  B = B,
  Omega12 = Omega12,
  Omega = Omega
)

res <- solve_learning(model)
# res <- make_stationary_filter(model$mu, model$Phi, model$Sigma12,
#                               model$A, model$B, model$Omega12)

for(shock in 1:nb_shocks){
  x <- c(res$Sigma12_ww[, shock])
  allx <- matrix(x, length(x), 1)
  for(t in 1:max_h){

```

```

    x <- res$Phi_ww %*% x
    allx <- cbind(allx,x)
  }
  all_IRFs[, ,shock,count_xi] <- allx
}
}

par(plt=c(.15,.95,.1,.7))

par(mfrow=c(nb_var+1,nb_shocks))

for(i_variable in 1:(nb_var+1)){
  for(i_shock in 1:nb_shocks){
    if(i_variable<=nb_var){

      main.t <- bquote("(" * .(letters[i_shock]) * "." * .(i_variable) * ") "
                      * "Response of " * .(names_var[[i_variable]])
                      * " to shock " * .(names_shock[[i_shock]]))

      plot(0:max_h,rep(0,max_h+1),
           xlab="time after shock",ylab="",
           type="l",lwd=2,las=1,col="white",
           ylim=c(min(all_IRFs[i_variable,,i_shock,],
                      all_IRFs[n+i_variable,,i_shock,]),
                  max(all_IRFs[i_variable,,i_shock,],
                      all_IRFs[n+i_variable,,i_shock,])),
           main=main.t)
      grid()
      for(i_xi in 1:length(all_xi)){
        lines(0:max_h,all_IRFs[i_variable,,i_shock,i_xi],
              lty=types_info[1],
              lwd=lwd_xi[i_xi],
              col=col_xi[i_xi])
        lines(0:max_h,all_IRFs[n+i_variable,,i_shock,i_xi],
              lwd=lwd_xi[i_xi],
              lty=types_info[2],
              col=col_xi[i_xi])
      }
    }else{
      # Add effect on cumulated consumption
      vector <- matrix(c(1,1,-1,0,0,0),nrow=1)
      dc_xi1 <- vector %*% all_IRFs[, ,i_shock,1]
      dc_xi2 <- vector %*% all_IRFs[, ,i_shock,2]
      cum_dc_xi1 <- cumsum(dc_xi1)
    }
  }
}

```

```

cum_dc_xi2 <- cumsum(dc_xi2)

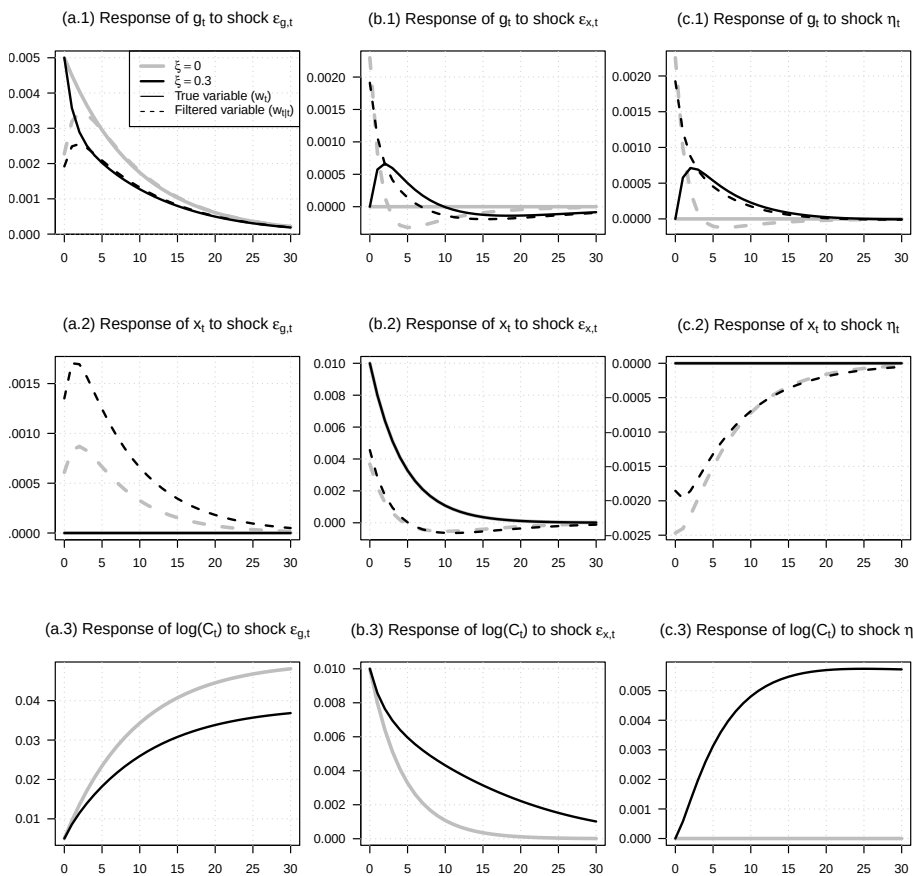
main.t <- bquote("(" * .(letters[i_shock]) * "." * .(i_variable) * ") "
               * "Response of " * .(names_var[[i_variable]])
               * " to shock " * .(names_shock[[i_shock]]))

plot(0:max_h,rep(0,max_h+1),
     xlab="time after shock",ylab="",
     type="l",lwd=2,las=1,col="white",
     ylim=c(min(cum_dc_xi1,cum_dc_xi2),
             max(cum_dc_xi1,cum_dc_xi2)),
     main=main.t)
grid()
lines(0:max_h,cum_dc_xi1,
      lty=1,
      lwd=lwd_xi[1],
      col=col_xi[1])
lines(0:max_h,cum_dc_xi2,
      lty=1,
      lwd=lwd_xi[2],
      col=col_xi[2])
}

legend_labels <- sapply(all_xi, function(x)
  bquote(xi == .(round(x, 2))))

if((i_shock==1)&(i_variable==1)){
  legend("topright",
        legend = c(legend_labels,
                    expression(paste("True variable (",w[t],")",sep="")),
                    expression(paste("Filtered variable (",w[t*'|'*t],")",sep="))),
        lwd=c(lwd_xi,1,1),
        lty=c(1,1,types_info),
        col=c(col_xi,"black","black"),
        seg.len = 2)
}
}
}

```



16.2 Rational Expectations

This example presents the benchmark BCM imperfect-information term-structure model.

Purpose: This example presents the benchmark BCM imperfect-information term-structure

Model of Bekaert, Cho, and Moreno (2010, JMCB): -----

Parameter values from Table 1

```

zeta <- 0.611      # in BCM
kappa <- 0.064     #
gamma <- 3.156     # utility curvature
mu <- 0.424        # μ
rho <- 0.723       # (policy smoothing)
beta_pi <- 1.525   # (in policy rule, response to inflation)

```

```

beta_y <- 0.001      # (in policy rule, response to ygap)
phi_yn <- 0.958
omega <- 0.877      #
phi <- 0.134
phi_1 <- 0.500      #
phi_2 <- 0.470      #
phi_3 <- 0.010 #1 - phi_1 - phi_2 # = 1 - 0.008
sigma_AS <- 1.249/100
sigma_IS <- 0.671/100
sigma_MP <- 2.177/100
sigma_yn <- 1.380/100
sigma_pistar <- 0.730/100

alpha_IS <- 0.01
r_bar <- .03
alpha_MP <- (1-rho)*r_bar

n <- 5

# Build matrices
Gamma0 <- matrix(0, 5, 5)
Gamma0[1, ] <- c(1, -kappa, 0, kappa, 0)
Gamma0[2, ] <- c(0, 1, phi, 0, 0)
Gamma0[3, ] <- c(0, -(1-rho)*beta_y, 1, (1-rho)*beta_y, (1-rho)*beta_pi)
Gamma0[4, ] <- c(0, 0, 0, 1, 0)
Gamma0[5, ] <- c(-phi_3, 0, 0, 0, 1)

Gamma1 <- matrix(0, 5, 5)
Gamma1[1,1] <- 1 - zeta
Gamma1[2,2] <- 1 - mu
Gamma1[3,3] <- rho
Gamma1[4,4] <- phi_yn      # your varphi_{y~n}
Gamma1[5,5] <- phi_2

C <- matrix(c(0, alpha_IS, alpha_MP, 0, 0), 5, 1)

Psi0 <- matrix(0, 5, 5)
Psi0[1,1] <- zeta
Psi0[2,1] <- phi
Psi0[2,2] <- mu
Psi0[3,1] <- (1-rho)*beta_pi
Psi0[5,5] <- phi_1

Psi1 <- 0*Psi0

```

```

Pi <- diag(c(sigma_AS, sigma_IS, sigma_MP, sigma_yn, sigma_pistar))

# Solve
# Your solver:
model <- list(mu = solve(Gamma0) %*% C,
              Phi = solve(Gamma0) %*% Gamma1,
              Psi0 = solve(Gamma0) %*% Psi0,
              Psi1 = solve(Gamma0) %*% Psi1,
              Sigma12 = solve(Gamma0) %*% Pi)
res <- solve_RE(model)

# Unconditional mean of w:
Ew <- solve(diag(n) - res$Phi_tilde) %*% res$mu_tilde

# Prepare IRFs: -----

max_h <- 40

nb_shocks <- dim(Pi)[2]

par(mfrow=c(n,nb_shocks))
par(plt=c(.33,.98,.15,.8))

names_var <- c(expression(pi[t]),
               expression(y[t]),
               expression(r[t]),
               expression(y[t]^n),
               expression(pi[t]^'*'))
names_shock <- c(expression(epsilon[AS*','*t]),
                expression(epsilon[IS*','*t]),
                expression(epsilon[MP*','*t]),
                expression(epsilon[y^n*','*t]),
                expression(epsilon[pi^"*"*','*t]))

ylab_IRF <- function(i_var){
  bquote("Response of " * .(names_var[[i_var]]) * "...")
}

title_IRF <- function(i_shock) {
  bquote("... to " * .(names_shock[[i_shock]]))
}

for(shock in 1:nb_shocks){
  x <- c(res$Sigma12_tilde[,shock])
  allx <- matrix(x,n,1)

```



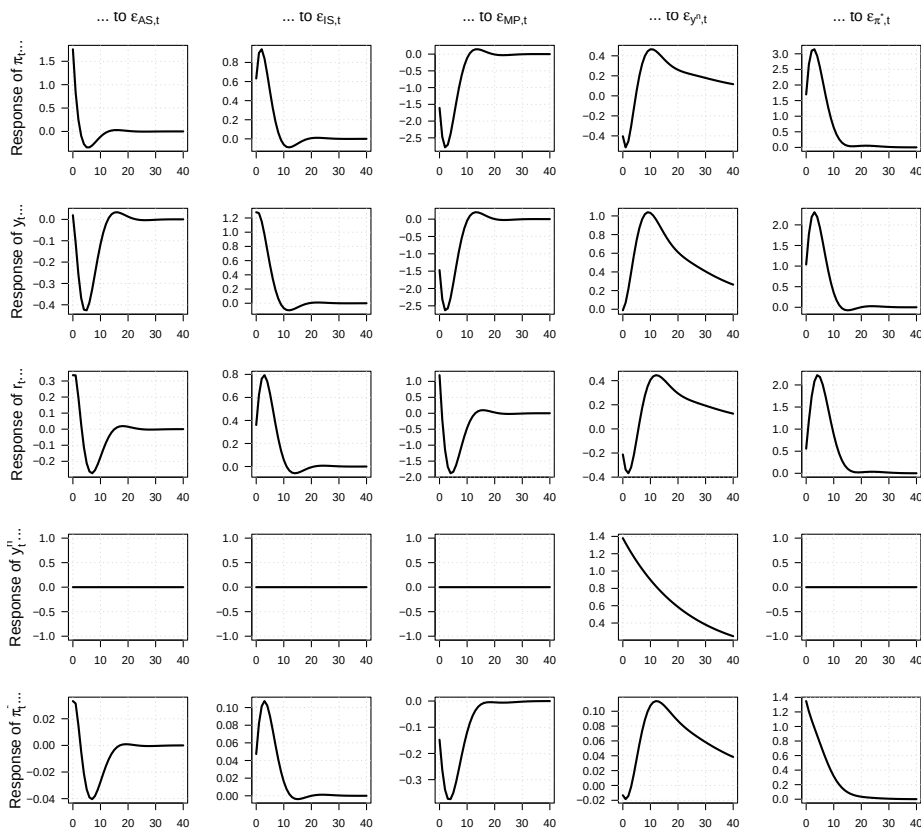
```

for(t in 1:max_h){
  x <- res$Phi_tilde %*% x
  allx <- cbind(allx,x)
}
for(i in 1:n){
  par(mfg=c(i,shock))

  ylab_val <- if (shock == 1) ylab_IRF(i) else ""
  main_val <- if (i == 1) title_IRF(shock) else ""

  plot(0:max_h,100*allx[i,],type="l",lwd=2,las=1,col="white",
       xlab="",ylab=ylab_val,main=main_val,
       cex.lab = 1.4,cex.main=1.4)
  grid()
  lines(0:max_h,100*allx[i,],lwd=2)
}
}

```



16.2.1 Additional implications of the BCM imperfect-information model

This example reports additional implications of the BCM imperfect-information model.

```
# Purpose: This example reports additional implications of the BCM imperfect-informati

# Build the risk-neutral dynamics implied by the SDF loading.
lambda0 <- solve(res$Sigma12_tilde) %*%
  matrix(c(0, sigma_IS/phi, 0, 0, 0), n, 1) -
  matrix(c(1, gamma, 0, 0, 0), n, 1)

model_P <- list(mu = res$mu_tilde,
  Phi = res$Phi_tilde,
  Sigma = res$Sigma12_tilde %*% t(res$Sigma12_tilde))
model_Q <- model_P
model_Q$mu <- res$mu_tilde + res$Sigma12_tilde %*% lambda0

resAB <- compute_AB_classical(xi0=0,xi1=matrix(c(0,0,1,0,0),n,1),
  H=40, # maximum maturity
  psi=psi_GaussianVAR,
  psi.parameterization=model_Q)

# Compute level, slope, and curvature loadings.
loadings <- matrix(NA,n,3,n)
loadings[1,] <- resAB$a[,1,c(1,4,20)] %*% matrix(rep(1/3,3),3,1)
loadings[2,] <- resAB$a[,1,20] - resAB$a[,1,1]
loadings[3,] <- resAB$a[,1,20] + resAB$a[,1,1] - 2*resAB$a[,1,4]

names_var <- c(expression(paste("level",sep="")),
  expression(paste("slope",sep="")),
  expression(paste("curvature",sep="")))

ylab_IRF <- function(i_var){
  bquote("Response of " * .(names_var[[i_var]]) * "...")
}

title_IRF <- function(i_shock) {
  bquote("... to " * .(names_shock[[i_shock]]))
}

par(mfrow=c(length(names_var),nb_shocks))
par(plt=c(.33,.98,.15,.8))

# Plot impulse responses of yield-curve factors to each structural shock.
```

```

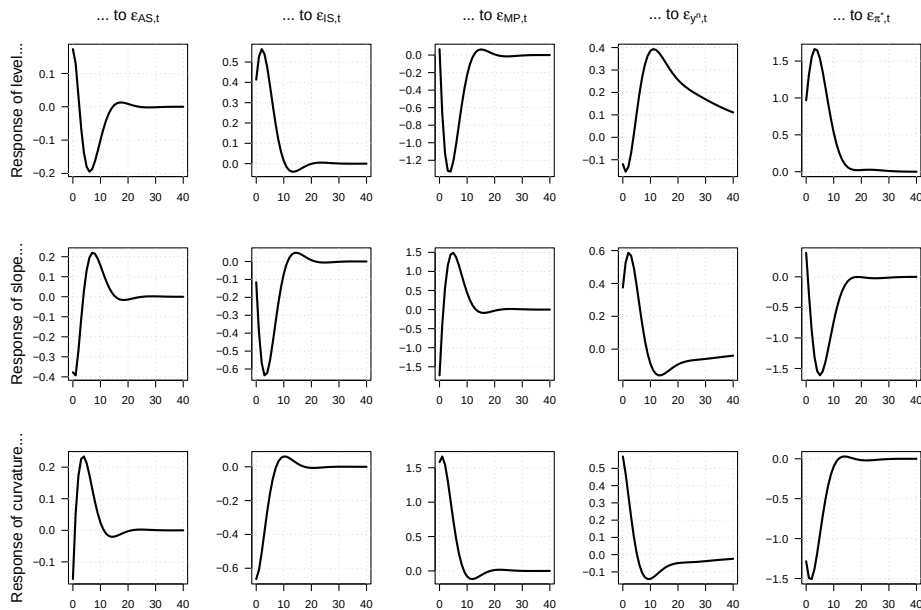
for(shock in 1:nb_shocks){
  x <- c(res$Sigma12_tilde[,shock])
  allx <- matrix(x,n,1)
  for(t in 1:max_h){
    x <- res$Phi_tilde %*% x
    allx <- cbind(allx,x)
  }
  allx <- loadings %*% allx

  for(i in 1:dim(allx)[1]){
    par(mfg=c(i,shock))

    ylab_val <- if (shock == 1) ylab_IRF(i) else ""
    main_val <- if (i == 1) title_IRF(shock) else ""

    plot(0:max_h,100*allx[i,],type="l",lwd=2,las=1,col="white",
         xlab="",ylab=ylab_val,main=main_val,
         cex.lab = 1.4,cex.main=1.4)
    grid()
    lines(0:max_h,100*allx[i,],lwd=2)
  }
}

```



16.2.2 Rational Expectation framework with longer horizon

We consider a system in which the n -dimensional state vector $\mathbf{w}_t$ evolves according to

$$\mathbf{w}_t = \mu + \Phi \mathbf{w}_{t-1} + \Psi_0 \mathbb{E}(\mathbf{w}_{t+2} \mid \underline{\mathbf{w}}_t) + \xi_t, \quad \xi_t \sim i.i.d. \mathcal{N}(\mathbf{0}, \Sigma_\xi).$$

There are no one-period-ahead expectations and no contemporaneous expectational terms:

$$\Psi_1 = \mathbf{0}, \quad \mathbb{E}(\mathbf{w}_{t+1} \mid \underline{\mathbf{w}}_t) \text{ does not enter the equation.}$$

To cast the problem into the standard first-order form, we introduce the auxiliary forward-looking variable

$$\widetilde{\mathbf{w}}_t = \mathbb{E}(\mathbf{w}_{t+1} \mid \underline{\mathbf{w}}_t),$$

and define the augmented state vector

$$\mathbf{W}_t = (\mathbf{w}'_t, \widetilde{\mathbf{w}}'_t)'$$

Using the law of iterated expectations,

$$\mathbb{E}(\widetilde{\mathbf{w}}_{t+1} \mid \underline{\mathbf{w}}_t) = \mathbb{E}(\mathbf{w}_{t+2} \mid \underline{\mathbf{w}}_t),$$

the system becomes

$$\mathbf{W}_t = \begin{pmatrix} \mu \\ \mathbf{0} \end{pmatrix} + \begin{pmatrix} \Phi & \mathbf{0} \\ \mathbf{0} & \mathbf{0} \end{pmatrix} \mathbf{W}_{t-1} + \begin{pmatrix} \mathbf{0} & \Psi_0 \\ \mathbf{Id}_n & \mathbf{0} \end{pmatrix} \mathbb{E}(\mathbf{W}_{t+1} \mid \underline{\mathbf{W}}_t) + \begin{pmatrix} \xi_t \\ \mathbf{0} \end{pmatrix}.$$

This representation has a standard structure involving conditional expectations involving a single horizon.

This block computes a two-period-ahead rational-expectations benchmark for comparison.

Purpose: Build the two-period-ahead rational-expectations benchmark.

```
# =====
# Original model dimensions
# =====
n <- 2 # dimension of w_t
mu <- matrix(c(0,0), n, 1)

Phi <- matrix(c(0.5,0.1,
               0.0,0.6), n, n)

Psi0 <- matrix(c(0.2,0,
```

```

                                0, 0.1), n, n)

Sigma12 <- diag(n)

# =====
# Augment the system
# =====
# Augmented state dimension: 2n
N <- 2 * n

# Augmented  $\tilde{m}$ 
mu_tilde <- rbind(mu, matrix(0, n, 1))

# Augmented  $\tilde{\Phi}$ 
Phi_tilde <- rbind(
  cbind(Phi, matrix(0, n, n)),
  cbind(matrix(0, n, n), matrix(0, n, n))
)

# Augmented  $\tilde{\Psi}_0$ 
Psi0_tilde <- rbind(
  cbind(matrix(0, n, n), Psi0),
  cbind(diag(n), matrix(0, n, n))
)

# No  $\tilde{\Psi}_1$  term
Psi1_tilde <- matrix(0, N, N)

# Shock matrix  $\Sigma\% = [\Sigma; 0]$ 
Sigma12_tilde <- rbind(Sigma12, matrix(0, n, n))

# Build model list for solve_RE
model_aug <- list(
  mu      = mu_tilde,
  Phi     = Phi_tilde,
  Psi0    = Psi0_tilde,
  Psi1    = Psi1_tilde,
  Sigma12 = Sigma12_tilde
)

# Solve
sol <- solve_RE(model_aug)

# Check

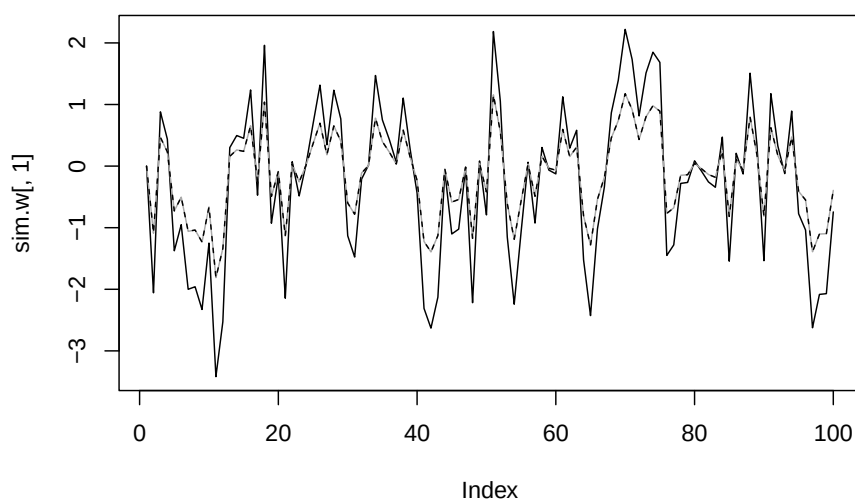
```

```

sim.w <- simul_GaussianVAR(sol,nb.sim=100)
E.w <- sim.w %*% t(sol$Phi_tilde)

plot(sim.w[,1],type="l")
lines(sim.w[,3],col="darkgrey")
lines(E.w[,1],col="black",lty=2)

```



16.2.3 The imperfect-information framework to an Ireland-style New Keynesian model

This example applies the imperfect-information framework to an Ireland-style New Keynesian model.

```

# Purpose: This example applies the imperfect-information framework to an Ireland-style
# Keynesian model

# Ireland (2004) -----
library(geigen)
library(DTAM)

# R code for Ireland (2004) system:
# w_t = (a_t, e_t, g_t, r_t, x_t, pi_t)'
# Γ0 w_t = Γ1 w_{t-1} + Ψ0 E_t w_{t+1} + Π_t

```

```

# Parameters (symbolic placeholders) -----

rho_a  <- 0.9470
rho_e  <- 0.9625
rho_g  <- 0.2536
rho_x  <- 0.0347
rho_pi <- 0.3597

omega  <- 0.0617
alpha_x <- 0.0836
alpha_pi <- 0.0000

psi    <- 0.1
beta   <- 0.99

sigma_a <- 0.0405
sigma_e <- 0.0012
sigma_z <- 0.0109
sigma_r <- 0.0031

# ----- BUILD MATRICES -----

n <- 6

Gamma0 <- matrix(0, n, n)
Gamma0[1, ] <- c(1,           0,           0,           0,           0,           0)
Gamma0[2, ] <- c(0,           1,           0,           0,           0,           0)
Gamma0[3, ] <- c(-omega,      0,           1,           0,          -1,           0)
Gamma0[4, ] <- c(0,           0,          -rho_g,      1,          -rho_x,      -rho_pi)
Gamma0[5, ] <- c(-(1-omega)*(1-rho_a), 0,           0,           1,           1,           0)
Gamma0[6, ] <- c(0,           1,           0,           0,          -psi,           1)

Gamma1 <- matrix(0, n, n)
Gamma1[1, ] <- c(rho_a, 0,           0,           0,           0,           0)
Gamma1[2, ] <- c(0,      rho_e,      0,           0,           0,           0)
Gamma1[3, ] <- c(-omega, 0,           0,           0,          -1,           0)
Gamma1[4, ] <- c(0,      0,           0,           1,           0,           0)
Gamma1[5, ] <- c(0,      0,           0,           0,      alpha_x,           0)
Gamma1[6, ] <- c(0,      0,           0,           0,           0,      beta*alpha_pi)

Psi0 <- matrix(0, n, n)
Psi0[5, ] <- c(0, 0, 0, 0, (1-alpha_x), 1)
Psi0[6, ] <- c(0, 0, 0, 0, 0, beta*(1-alpha_pi))

```

```

Psi1 <- 0*Psi0

Pi <- matrix(0, n, 4)
Pi[1,1] <- sigma_a
Pi[2,2] <- sigma_e
Pi[3,3] <- sigma_z
Pi[4,4] <- sigma_r
# rows 5-6 remain zero

C <- matrix(0, 6, 1)

# Return as list
model_matrices <- list(
  Gamma0 = Gamma0,
  Gamma1 = Gamma1,
  Psi0    = Psi0,
  C       = C,
  Pi      = Pi
)

# Solve
# Your solver:
model <- list(mu = solve(Gamma0) %*% C,
              Phi = solve(Gamma0) %*% Gamma1,
              Psi0 = solve(Gamma0) %*% Psi0,
              Psi1 = solve(Gamma0) %*% Psi1,
              Sigma12 = solve(Gamma0) %*% Pi)
res <- solve_RE(model)

# Prepare IRFs:

# Check
# Gamma0 %*% res$Phi_tilde - Gamma1 - Psi0 %*% res$Phi_tilde %*% res$Phi_tilde

max_h <- 40

nb_shocks <- dim(Pi)[2]

par(mfrow=c(nb_shocks,n))

par(plt=c(.15,.95,.1,.7))

names_var <- c("a", "e", "g", "r", "x", "pi")

```

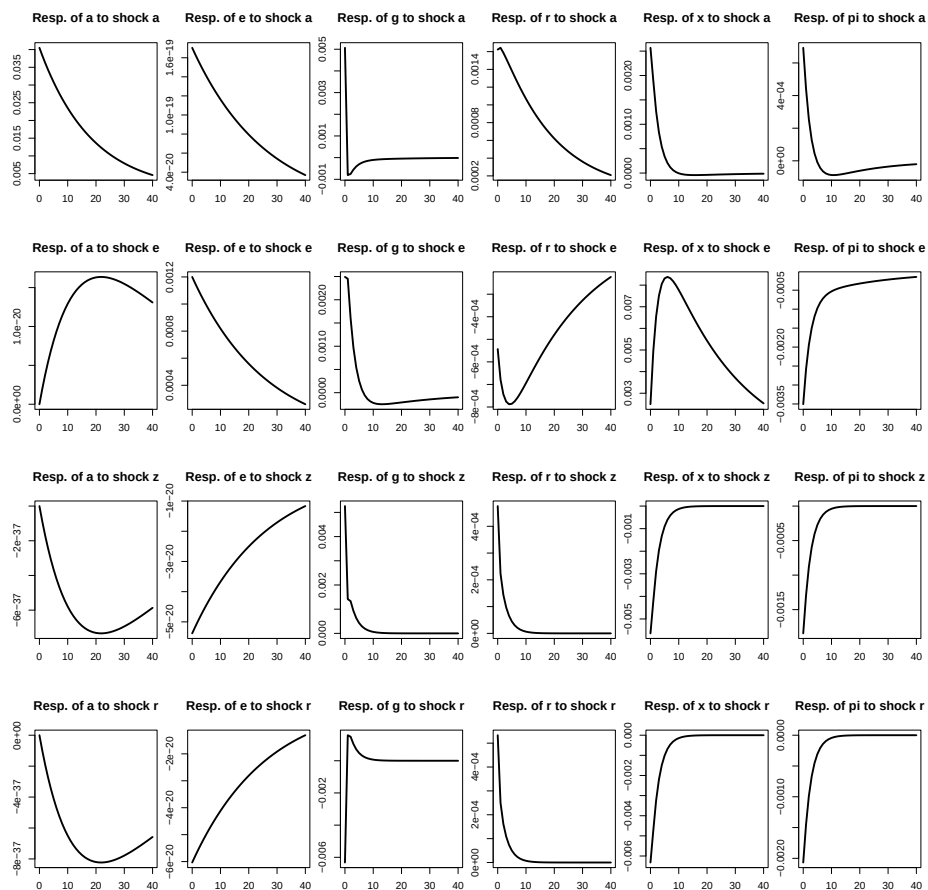


```

names_shock <- c("a","e","z","r")

for(shock in 1:nb_shocks){
  x <- c(res$Sigma12_tilde[,shock])
  allx <- matrix(x,n,1)
  for(t in 1:max_h){
    x <- res$Phi_tilde %*% x
    allx <- cbind(allx,x)
  }
  for(i in 1:n){
    plot(0:max_h,allx[i,],type="l",lwd=2,
        main=paste("Resp. of ",names_var[i],
                    " to shock ",names_shock[shock],sep=""))
  }
}

```



16.3 Imperfect information and rational expectations

This example studies another imperfect-information New Keynesian specification and its term-structure implications.

Purpose: This example studies another imperfect-information New Keynesian specification and its term-structure implications

```
library(DTAM)
```

```
max_h <- 30
```

```
col_info <- c("grey", "black")
```

```
lwd_info <- c(4, 2)
```

```
types_t_or_tt <- c(1, 2)
```

```
# --- Parameters -----
```

```
zeta <- .5
```

```
kappa <- .2 #
```

```
gamma <- 3 #
```

```
rho <- .8 #
```

```
phi_x <- .2 # _x
```

```
phi_pi <- 1.5 # _
```

```
rho_rstar <- .95 # _r_star
```

```
rho_pistar <- .95 # _pi_star
```

```
rbar_star <- .02 # r_bar_star
```

```
pibar_star <- .02 # pi_bar_star
```

```
sigma_AS <- .01 # _AS
```

```
sigma_IS <- .01 # _IS
```

```
sigma_MP <- .01 # _MP
```

```
sigma_rstar <- .01 * sqrt(1-rho_rstar^2) # _r_star
```

```
sigma_pistar <- .01 * sqrt(1-rho_pistar^2) # _pi_star
```

```
sigma_nu <- .02
```

```
# --- Define  $\Theta$  -----
```

```
Theta <- rbind(
```

```
  c(1, -kappa, 0, 0, 0),
```

```
  c(0, 1, 1/gamma, -1/gamma, 0),
```

```

c(0, -(1-rho)*phi_x, 1, -(1-rho), -(1-rho)*(1-phi_pi)),
c(0, 0, 0, 1, 0),
c(0, 0, 0, 0, 1)
)

```

```
Theta_inv <- solve(Theta)
```

```
# --- -----
```

```
mu_vec <- c(0,
            0,
            0,
            (1 - rho_rstar) * rbar_star,
            (1 - rho_pistar) * pibar_star)
```

```
mu <- Theta_inv %*% matrix(mu_vec, ncol = 1)
```

```
# ---  $\Phi$  -----
```

```
Phi_raw <- rbind(
  c(zeta,0,0,0,0),
  c(0,0,0,0,0),
  c(0,0,rho,0,0),
  c(0,0,0,rho_rstar,0),
  c(0,0,0,0,rho_pistar)
)
```

```
Phi <- Theta_inv %*% Phi_raw
```

```
# ---  $\Gamma$  -----
```

```
Gamma0_raw <- rbind(
  c((1-zeta), 0, 0, 0, 0),
  c(1/gamma, 1, 0, 0, 0),
  c(0, 0, 0, 0, 0),
  c(0, 0, 0, 0, 0),
  c(0, 0, 0, 0, 0)
)
```

```
Gamma0 <- Theta_inv %*% Gamma0_raw
```

```
# ---  $\Psi$  -----
```

```
Psi0_raw <- rbind(
  c(0,0,0,0,0),

```

```

    c(0,0,0,0,0),
    c((1-rho)*phi_pi, 0, 0, 0, 0),
    c(0,0,0,0,0),
    c(0,0,0,0,0)
  )

Psi0 <- Theta_inv %*% Psi0_raw

# ---  $\Sigma^{1/2}$  -----

Sigma_half_raw <- diag(c(sigma_AS,
                        sigma_IS,
                        sigma_MP,
                        sigma_rstar,
                        sigma_pistar))

Sigma_half <- Theta_inv %*% Sigma_half_raw

# Lambda0, Lambda1, Psi1, and Gamma1 are zero matrices -----

Lambda0 <- matrix(0, 5, 5)
Lambda1 <- matrix(0, 5, 5)
Psi1    <- matrix(0, 5, 5)
Gamma1  <- matrix(0, 5, 5)

# --- Measurement-equation matrices for imperfect information
A <- matrix(c(1,0,0,0,0,
             0,1,0,0,0,
             0,0,1,0,0),
           nrow = 3, byrow = TRUE)

B <- matrix(0, nrow = 3, ncol = 1)

Omega12 <- matrix(0, nrow = 3, ncol = 1)
Omega12[2] <- sigma_nu

# --- Final output -----

# Specify model with imperfect information and rational expectations (IIRE):
model_IIRE <- list(
  Theta      = Theta,
  mu         = mu,
  Phi        = Phi,
  Gamma0     = Gamma0,
  Psi0       = Psi0,

```

```

Sigma12      = Sigma_half,
Lambda0      = Lambda0,
Lambda1      = Lambda1,
Psi1         = Psi1,
Gamma1       = Gamma1,
Omega12      = Omega12,
A            = A,
B            = B)

n <- dim(Phi)[1]
nb_shocks <- n

names_var <- c(expression(pi[t]),
               expression(x[t]),
               expression(r[t]),
               expression(r[t]*'*'),
               expression(pi[t]*'*'))
names_shock <- c(
  expression(epsilon[AS*','*t]),
  expression(epsilon[IS*','*t]),
  expression(epsilon[MP*','*t]),
  expression(epsilon[r*'*'*','*t]),
  expression(epsilon[pi*'*'*','*t]))

all_indic_FI <- c(TRUE,FALSE)

all_IRFs <- array(NaN,c(2*n,1+max_h,nb_shocks,length(all_indic_FI)))

count_info_type <- 0
for(indic_FI in all_indic_FI){
  count_info_type <- count_info_type + 1

  if(!indic_FI){
    # Imperfect information
    model <- model_IIRE
    model$Psi0 <- model$Psi0 + model$Gamma0
    model$Gamma0 <- 0*model$Gamma0
  }else{
    # Full information
    model <- model_IIRE
    model$A <- diag(n)
    model$B <- matrix(0,n,1)
    model$Omega12 <- matrix(0,n,1)
  }
}

```

```

res <- solve_Learning_RE(model)

for(shock in 1:nb_shocks){
  x <- c(res$Sigma12_ww[,shock])
  allx <- matrix(x,length(x),1)
  for(t in 1:max_h){
    x <- res$Phi_ww %*% x
    allx <- cbind(allx,x)
  }
  all_IRFs[,shock,count_info_type] <- allx
}

all_IRFs <- 100*all_IRFs

par(plt=c(.22,1,.15,.75))

par(mfrow=c(n,nb_shocks))

for(i_variable in 1:n){
  for(i_shock in 1:nb_shocks){

    main.t <- bquote("(" * .(letters[i_shock]) * "." * .(i_variable) * ") "
                  * "Resp. of " * .(names_var[[i_variable]])
                  * " to " * .(names_shock[[i_shock]]))

    ylim <- c(min(all_IRFs[i_variable,,i_shock,],
                  all_IRFs[n+i_variable,,i_shock,]),
              max(all_IRFs[i_variable,,i_shock,],
                  all_IRFs[n+i_variable,,i_shock,]))
    ylim[1] <- min(ylim[1],-.001)
    ylim[2] <- max(ylim[2],.001)

    plot(0:max_h,rep(0,max_h+1),
         xlab="time after shock",ylab="",
         type="l",lwd=2,las=1,col="white",
         ylim=ylim,
         main=main.t,cex.main=1.3)
    grid()
    for(i_indic_FI in 1:length(all_indic_FI)){
      lines(0:max_h,all_IRFs[i_variable,,i_shock,i_indic_FI],
            lty=types_t_or_tt[1],
            lwd=lwd_info[i_indic_FI],
            col=col_info[i_indic_FI])
    }
  }
}

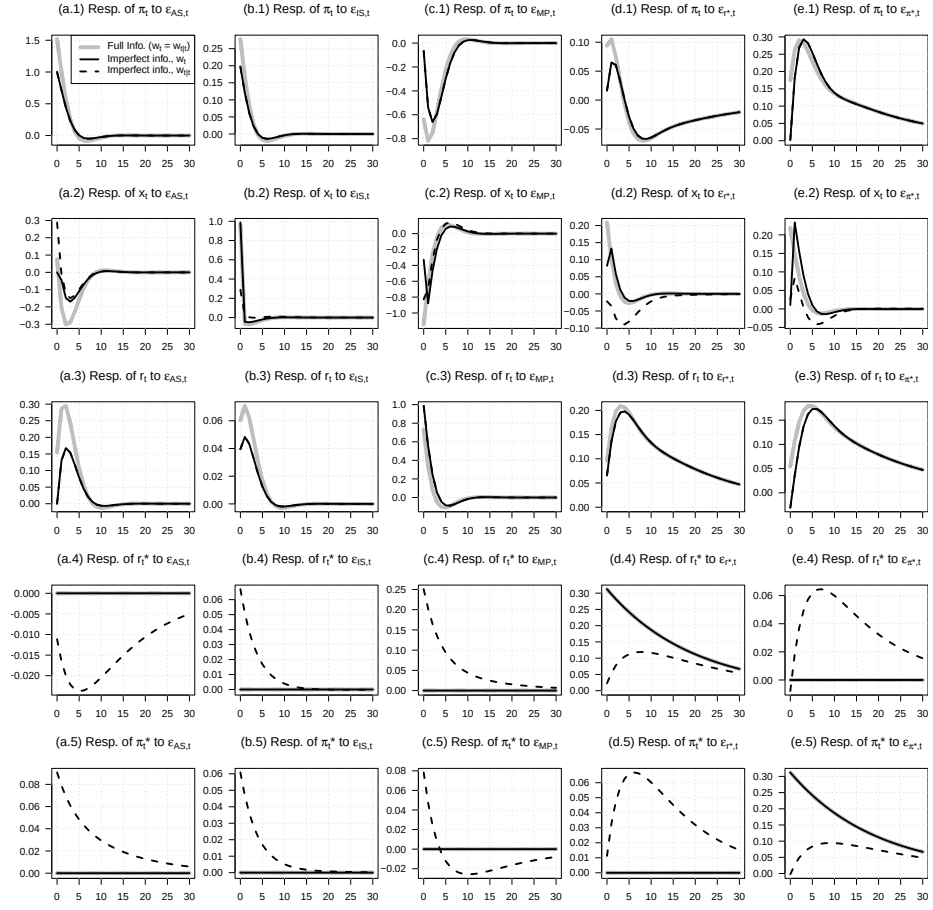
```

```

    lines(0:max_h,all_IRFs[n+i_variable,,i_shock,i_indic_FI],
          lwd=lwd_info[i_indic_FI],
          lty=types_t_or_tt[2],
          col=col_info[i_indic_FI])
  }

  if((i_shock==1)&(i_variable==1)){
    legend("topright",
          legend = c(expression(paste("Full Info. (",
                                     w[t], " = ",w[t*'|'*t],")",sep="")),
                    expression(paste("Imperfect info. ",
                                     w[t],sep="")),
                    expression(paste("Imperfect info. ",
                                     w[t*'|'*t],sep="))),
          lwd=c(lwd_info[1],lwd_info[2],lwd_info[2]),
          lty=c(1,types_t_or_tt[1],types_t_or_tt[2]),
          col=c(col_info[1],col_info[2],col_info[2]),
          seg.len = 2)
  }
}

```



Bibliography

Yihui Xie. *Dynamic Documents with R and knitr*. Chapman and Hall/CRC, Boca Raton, Florida, 2nd edition, 2015. URL <http://yihui.org/knitr/>. ISBN 978-1498716963.

Yihui Xie. *bookdown: Authoring Books and Technical Documents with R Markdown*, 2025. URL <https://github.com/rstudio/bookdown>. R package version 0.43.